

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



CAPÍTULO 9

Identificando los puntos de estrés de nuestro sistema de votación

Introducción

Este capítulo examina los cuellos de botella o puntos de estrés del sistema de votación. Nos referimos a aquellas partes del sistema que no cumplen con nuestras expectativas de rendimiento bajo una carga elevada antes de que sus módulos adyacentes fallen.

Para identificar estos puntos débiles, también es necesario determinar qué condiciones los llevan a su punto crítico. Para cada uno de estos cuellos de botella, realizaremos un análisis de efectos: identificaremos la falla específica cuando se superan los umbrales críticos, es decir, ¿qué falla? Al comprender las causas fundamentales y el impacto, podemos planificar la detección preventiva y las medidas de mitigación para las fallas que, aunque esperamos que no ocurran, normalmente ocurrirán a medida que nuestro sistema de votación se someta a la prueba del uso popular y confiable.

Estructura

En este capítulo se tratarán los siguientes temas:

- Tipos de fallos
- Evaluación del impacto

- Monitoreo, detección y notificación
- Respuesta a incidentes
- Matriz de modos de fallo y medidas apropiadas
- Implementación y monitoreo del laboratorio de código de la API de Vote Ingester

Objetivos

Un sistema puede fallar de diversas maneras, con diferentes causas raíz, impactos y enfoques de mitigación. Este capítulo guiará al lector a través de cada una de estas dimensiones de una posible interrupción, describiendo las técnicas e indicadores utilizados para categorizar un incidente específico. Al finalizar este capítulo, el lector contará con una metodología para categorizar estos modos de fallo y medidas de mitigación, aplicables a cualquier sistema distribuido de alta disponibilidad.

Tipos de fallos

Antes de analizar posibles puntos específicos de falla en nuestro sistema de votación, es útil aclarar exactamente qué queremos decir con el término "fallo" en estas discusiones y dar cuerpo al concepto mediante ejemplos.

No todas las fallas merecen la misma atención: en un ejemplo que debería ser bastante evidente, si la luz indicadora de un automóvil que avisa cuando una puerta está abierta falla debido a una lámpara rota, no recibiría la misma atención que la falla intermitente de la luz indicadora que indica que el motor se está sobrecalentando, y ninguna de estas obtendría la respuesta inmediata que debería recibir la falla del pedal del freno.

Así como la gravedad de las fallas en los sistemas de un automóvil varía, clasificamos las fallas en un sistema de software según dos aspectos: alcance y progresión. Comprender estos dos aspectos de una posible interrupción o falla de nuestra aplicación es crucial para identificar estrategias para prevenirlas, seleccionar técnicas para detectarlas en producción y planificar medidas eficaces para mitigar rápidamente sus efectos en nuestros usuarios.

Alcance de un fallo

El alcance de una falla es el alcance del problema, a menudo denominado

Radio de explosión del fallo. En una aplicación distribuida, con límites regionales de tiempo de ejecución claros, como nuestra aplicación de votación, es probable que el alcance esté limitado a uno de los siguientes:

- Una falla local afecta a un grupo de usuarios con cierta afinidad con un aspecto particular del usuario, como una versión de host del cliente, un determinado tipo de conexión o un navegador específico.
- Una falla regional afecta a un conjunto de regiones que probablemente comparten algún recurso en la nube.
- Una falla global afecta a todos los usuarios de la aplicación.

Aunque una falla local pueda parecer el menor de los males, lo más probable es que sea indicio de una condición crónica: una falla recurrente, que se convierte en objeto de teorías conspirativas y la base de memes. En la aplicación de votación, si un grupo específico de usuarios no puede votar, esto se interpretará como perjudicial para dichos usuarios o para cualquier interés común de los usuarios afectados, como un artista en particular.

Algunos ejemplos de fallas locales son un error específico de una aplicación y plataforma cliente en particular, por ejemplo, una aplicación móvil iOS en un iPhone mini. 13

Otra afinidad específica del cliente que podría provocar un fallo local es que todos los usuarios dependan de conexiones 3G en una región donde las interfaces experimentan picos de demanda y están en proceso de escalamiento horizontal. Clasificamos un fallo como local si no afecta uniformemente a los usuarios en una amplia zona geográfica o topológica, sino que se asocia a grupos de usuarios. En ese caso, presumimos que el desencadenante o la causa subyacente del fallo también es local, específico de un usuario.

Centrémonos en las fallas regionales, aquellas que afectan a los usuarios dentro de un límite geográfico o topológico. La distinción entre geografía y topología, tal como se utiliza aquí, radica en que la geografía se refiere a los vecinos en una región del espacio físico, mientras que la topología también podría incluir a los vecinos en una red, partes que comparten un punto de presencia, un par de red, un balanceador de carga u otro punto en el ciberespacio, no en el espacio físico.

Al examinar un fallo experimentado por los usuarios de nuestra aplicación de votación en un lugar, mientras que los usuarios de otros lugares pueden votar sin problemas, llegamos a una suposición lógica: el problema no está en el usuario individual. Podemos aplicar un argumento similar desde el otro lado, el...

El hecho de que los usuarios en otros lugares puedan votar nos dice que la aplicación es esencialmente capaz de funcionar como está previsto en algún lugar, por lo que no es el sistema en general el que está roto.

Si la culpa no recae ni en el individuo ni en el sistema en su conjunto, existe una causa intermedia. Un fallo regional suele tener una causa regional.

El tercer dominio de fallo, el más obvio y potencialmente catastrófico, es el de un fallo global o una interrupción total del sistema. Si nadie puede votar, el fallo no se debe a ninguna diferencia entre regiones o usuarios. El fallo se debe a algo compartido por todos los usuarios en todas partes. Una constante global del sistema es la responsable de la interrupción. Esta constante global podría ser un código de programa compartido, un único recurso de ejecución compartido, como un almacén de datos global, o un artefacto o valor de configuración común.

Progresión. El alcance

de una interrupción del sistema puede variar con el tiempo y afectar a más usuarios de una función determinada o a más usuarios debido a la falla de otras funciones. Si se grafica el número de usuarios que fallan en la aplicación a lo largo del tiempo, generalmente se ajustará a uno de los siguientes plazos:

- Una falla puede manifestarse como un conjunto constante de usuarios afectados que no pueden emitir su voto.
- Es posible que haya un número cada vez mayor de usuarios que experimenten fallas en la votación a medida que pasa el tiempo.

Primero, describamos el estado estable o el fallo de alcance constante. En este modo de fallo, el flujo de fallos que observamos inicialmente se mantiene estadísticamente constante durante los siguientes segundos, minutos y horas. En términos absolutos, si aproximadamente el 2% de los usuarios de iOS experimentan un error, o si todos los usuarios del noreste de EE. UU. no pueden votar, sea cual sea el alcance del fallo, este se mantiene mientras persista la interrupción. Se trata de un flujo constante de fallos.

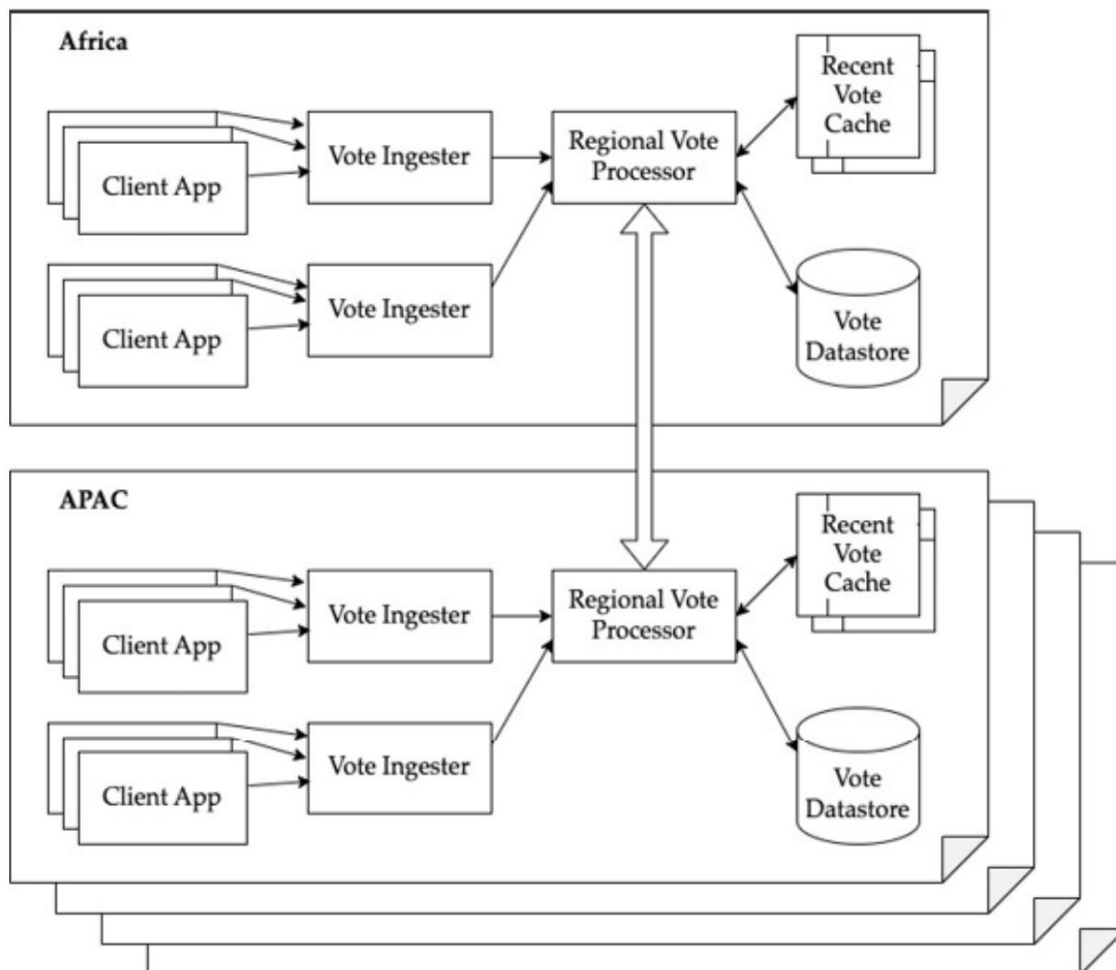
Una falla con un alcance progresivo o expansivo es aquella en la que el conjunto de usuarios afectados aumenta a medida que persiste la interrupción. Por ejemplo, quizás el 2% de los usuarios de iOS reciben inicialmente un error, pero después de cinco minutos, esa cifra ha aumentado al 25%. Otro ejemplo de una falla con

La expansión del alcance es aquella en la que los usuarios del noreste de EE. UU. no pueden votar, pero 10 minutos después, los usuarios del sureste de EE. UU. también encuentran errores.

Una falla progresiva puede ser una falla en cascada en la que los efectos de la falla se extienden a regiones adyacentes, o puede ser una falla cuya causa subyacente se está propagando por todo el sistema.

Un ejemplo de fallo en cascada es aquel en el que el tráfico se desborda hacia regiones adyacentes e inadecuadas. Seguiremos la progresión de un fallo en cascada simple en los siguientes diagramas y texto.

Como precursor de cualquier fracaso, comenzamos con un momento en el que la demanda en África y APAC está en sus picos esperados y todo está bien:



Cifra 9.1 Precursor de un fracaso regional en cascada

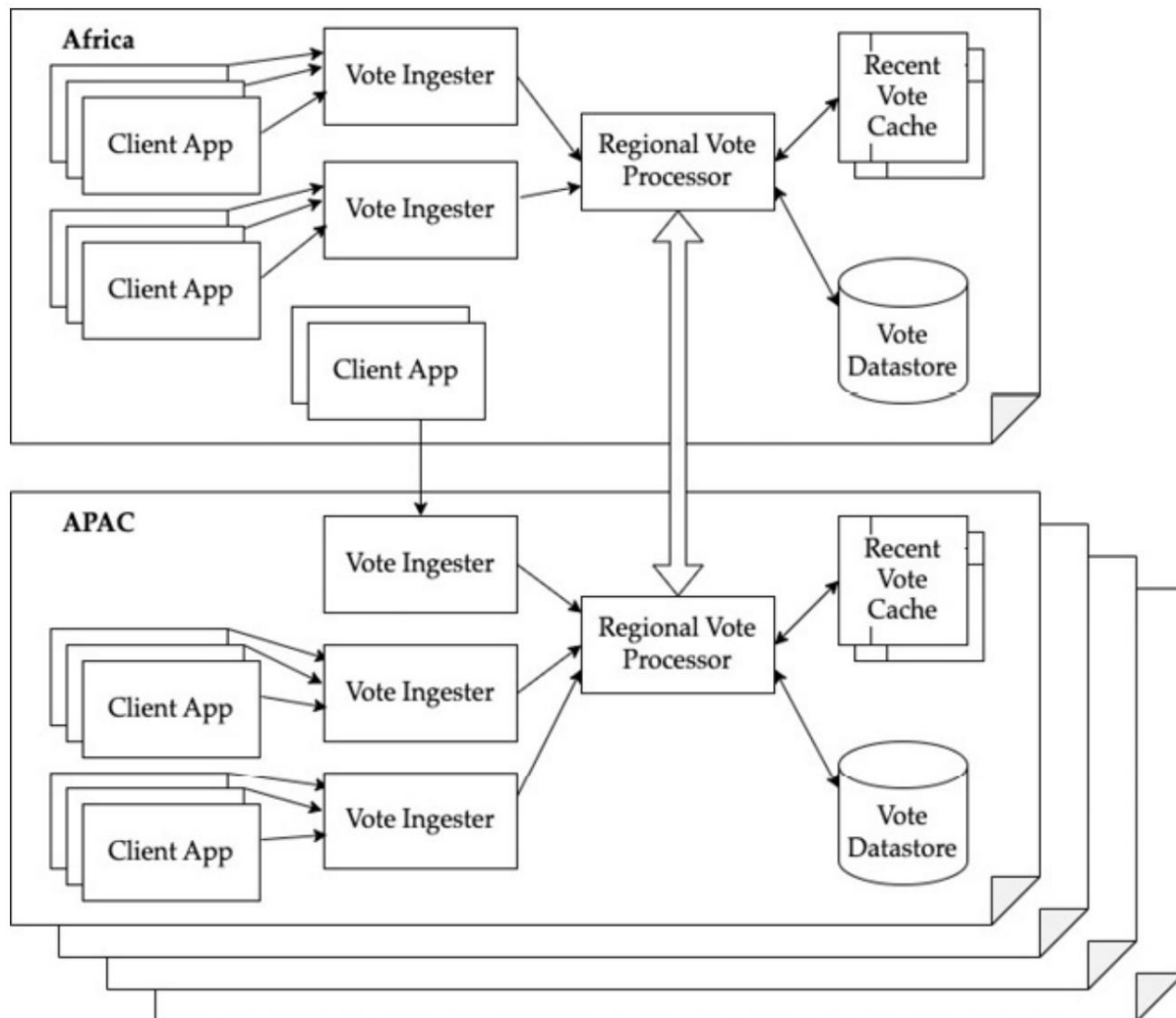
Si bien el sistema está funcionando bien y los usuarios tanto de África como de APAC

Están votando sin incidentes, pero todo está a punto de cambiar. El mundo es complejo y, a veces, presenta situaciones imprevistas que provocan un comportamiento inesperado del sistema.

En este caso, presentaremos la situación hipotética de que un artista africano local haya llegado a las semifinales y actúe junto a un artista asiático popular. La sincronización de estas actuaciones garantiza que la máxima demanda en sus continentes de origen coincida.

Una consideración adicional es que Google Cloud Platform (GCP) no cuenta con los recursos necesarios para instanciar un número excepcional de nuestros frontends de ingesta de votos. Los balanceadores de carga de nuestras regiones de red africanas están configurados para enviar solicitudes a los ingestadores de votos en APAC si no hay frontends disponibles en África. Existen recursos en las regiones de APAC para iniciar frontends adicionales de ingesta de votos si es necesario.

Esta etapa de nuestro escenario de falla se ilustra en la siguiente figura:



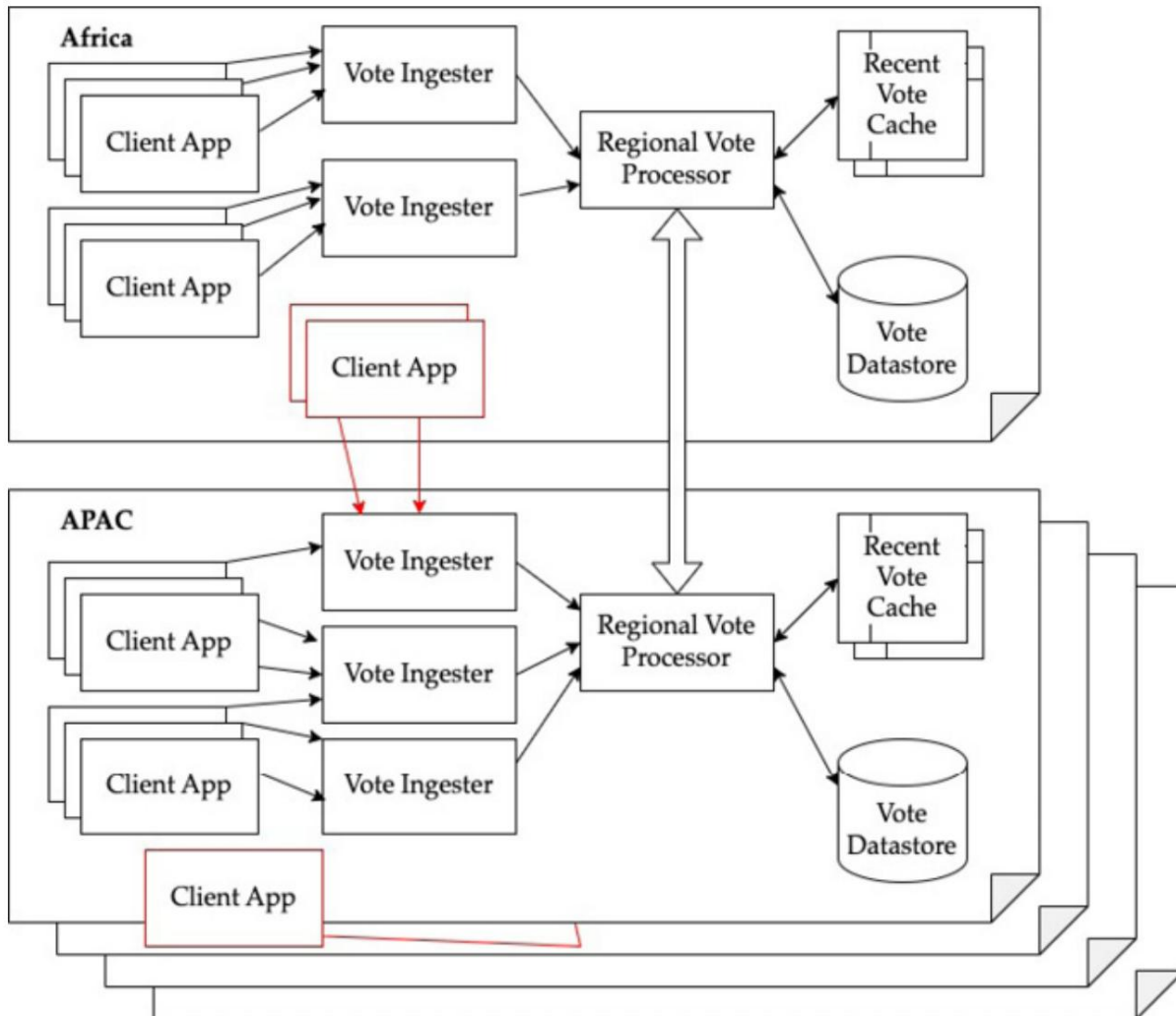
Cifra 9.2 Desbordamiento del tráfico de clientes al continente adyacente

En un principio, el enfoque elegido parece funcionar; disponíamos de recursos en otros lugares y canalizamos las solicitudes para utilizarlos. Hemos probado nuestra aplicación cliente con solicitudes interregionales y funcionó, por lo que creemos que este escenario estaba contemplado en nuestro plan de implementación.

Sin embargo, existe una condición externa con la que no pudimos realizar pruebas. A medida que nuestras solicitudes interregionales llegan a las interfaces de APAC, experimentan una alta demanda por parte de los clientes locales, quienes votan por su artista favorito. Mientras se lleva a cabo esta votación, la red subyacente en la región APAC distribuye video a todos esos millones de espectadores.

La red GCP es robusta y capaz, pero no es infinita. Las demandas competitivas de ancho de banda resultan en una mayor latencia, en algunos casos.

casos que superan los límites de tiempo de espera para las solicitudes de nuestros clientes a los frontends. Los clientes en África, enrutados a procesadores de votos en APAC, comienzan a experimentar una combinación de respuestas de alta latencia y tiempos de espera de solicitud. Esta etapa final en la evolución de un fallo en cascada se muestra en la siguiente figura:



Cifra 9.3 Comienzo de una serie de fallos en cascada

Los clientes ilustrados en la parte inferior de la región de África están conectados a un procesador de votos en APAC, cruzando varios límites de red para enviar solicitudes y recibir respuestas. Se representan en rojo para indicar el estado de fallo que experimentan. Este no es el final del fallo; el término "en cascada" indica que forma parte de una serie o secuencia, representada por los clientes rojos en la parte inferior de la región de APAC.

La incorporación de clientes en África ha saturado por completo nuestros frontends locales de ingesta de votos, y en este momento de máxima demanda, más votantes en APAC están votando por su artista favorito. La misma lógica de propagación se aplica tanto a la demanda en APAC como en África: cuando no se pueden instanciar más frontends en APAC, las solicitudes se envían preferentemente a la región vecina más cercana, en este caso, África.

Por supuesto, África ya tiene un exceso de demanda, lo que provocó el problema en APAC. Sin recursos en África, nuestra lógica de desbordamiento envía a estos clientes de APAC a regiones europeas. Esto no es óptimo en las mejores circunstancias, y estos clientes de APAC comienzan a experimentar errores similares a los de sus homólogos africanos que cruzan el continente. Existe la posibilidad de que la misma situación se produzca en las regiones europeas; de hecho, un fallo como este suele propagarse a casi todo el mundo.

Causas de un fallo

Hemos identificado, codificado y descrito los dos aspectos de una falla que usaremos para categorizarla: su forma en el espacio y en el tiempo. Estos describen cómo la falla persiste entre nuestra comunidad de usuarios una vez manifestada, pero falta una característica de una interrupción que debemos considerar al final: su desencadenante.

El origen de una falla es la semilla o el detonante del problema. Toda interrupción del servicio comienza en algún lugar y en algún momento, y esos valores son atributos obvios del origen de la falla. Cuándo y dónde ocurre una falla son evidentes. Lo que es más básico, y a veces menos obvio, es por qué una falla se manifestó en ese lugar y momento.

Aquí enumeraremos las categorías de orígenes de fallas que son el tercer y último aspecto de una interrupción que se considerará en la prevención, detección y mitigación:

- Fallos latentes ocasionados por el paso del tiempo: la obsolescencia de los recursos, la obsolescencia de componentes o dependencias y la falta de atención a la capacidad en comparación con el uso actual esperado son variantes de esto. La degradación de bits es un ejemplo de este tipo de fallo, propio de los sistemas de almacenamiento.
- Eventos de fuerza mayor: Significa fuerza mayor y se refiere a un evento por el cual ninguna de las partes puede ser considerada responsable. Un evento impredecible

Sin embargo, siempre es posible un acto de Dios o de la naturaleza, como un terremoto, Las inundaciones o tormentas solares pueden causar fallas en los proyectos mejor planificados. sistemas.

- Fallos causados por ignorancia o malevolencia: Este tipo de Las fallas tienen su origen en deficiencias del proceso: ya sean operativas procesos o procesos de seguridad respectivamente.
- Fallos debidos al uso en una secuencia de operación no prevista o Volumen: Estos ocurren porque los usuarios actuaron de maneras que no eran no previstos por los arquitectos y operadores del sistema. Estos suelen ser el resultado de deficiencias de planificación.
- Fallos resultantes de fallas en los sistemas o sus componentes: Si bien la presencia de algunos defectos se acepta como inevitable, los errores y la falta de aislamiento o de medios para contenerlos puede conducir a Fallos de tiempo de ejecución que tienen un impacto mayor del necesario. Las deficiencias en los procesos de garantía de calidad y liberación conducen a aquellos que tienen un impacto significativo en el usuario o en las operaciones.

Categorización de fallos

En esta sección hemos identificado tres facetas de un fallo: alcance, progresión y desencadenante o causa. Un fallo determinado tendrá al menos un valor para cada uno de estos tres atributos, por lo que se pueden detectar los fallos potenciales. visualizado como una matriz, que se muestra en la siguiente figura:

Cause	Scope	Progression
Introduced over/by time	Local	Constant
Force majeure		
Ignorance or malevolence	Regional	Creeping or expanding
Unanticipated usage		
Defects	Global	

Cifra 9.4 Matriz de categorización de fallas

De acuerdo con esta matriz, se obtiene una visión integral de los riesgos para nuestra aplicación debe tener en cuenta hasta 36 modos de fallo diferentes. Un Un plan eficaz para evaluar y garantizar la robustez del sistema debe incluir

ya sea un medio para detectar y mitigar cada uno de estos posibles modos de falla o reconocer la presencia de riesgos no abordados.

La probabilidad y el impacto de estos diferentes modos de fallo también deben cuantificarse y considerarse de alguna manera. Independientemente de la magnitud del efecto de un fallo determinado, debemos reconocer ese coste y utilizarlo para priorizar la asignación de recursos para mejorar el rendimiento y la fiabilidad de nuestra aplicación de votación. El coste proyectado de un fallo y su impacto es el factor principal a considerar al priorizar la solución de ese fallo entre los demás que prevemos.

Evaluación del impacto

El impacto final de un fracaso es su costo para el negocio a lo largo del tiempo.

Tenga en cuenta la parte temporal de esa oración. Una falla en cualquier sistema tiene costos a corto y largo plazo, algunos directos y otros indirectos. El impacto de una falla en una empresa es la suma de todos estos. Este ejercicio de cálculo de costos se complica porque cuanto más lejano en el tiempo o menos directo sea un efecto, más se utilizarán conjeturas y estimaciones para determinar su costo. Por lo tanto, el costo puede no ser cuantificable para cada modo de falla, pero sí debería ser posible determinar el orden de magnitud o, como mínimo, una clasificación de los diferentes modos de falla potenciales según su costo, lo cual podría ser suficiente para orientar los esfuerzos de confiabilidad del propietario del sistema.

Determinación del costo a corto plazo de una interrupción.

Supongamos que se produce una falla regional debido a un aprovisionamiento insuficiente de instancias para un servicio determinado. La mitigación es obvia y de efecto inmediato: podemos aumentar nuestro límite de escalado, cuota y límites de facturación para el componente afectado.

El coste para la empresa también es sencillo y puede determinarse rápidamente mediante varios factores. La siguiente lista no pretende ser exhaustiva ni de aplicación universal, pero presenta un conjunto razonable de factores en nuestro escenario de fallo de la interfaz de votación:

- El número de solicitudes fallidas antes de que se mitigara la situación.
Cada una de estas solicitudes implica algún coste de infraestructura. Una falla

Una solicitud suele exigir más recursos que una exitosa; por ejemplo, una solicitud que expira ata un hilo de cliente y una conexión durante el mayor tiempo posible, mientras que la solicitud promedio que tiene éxito libera esos recursos en menos tiempo.

- Si el período de tiempo de la falla es significativo, se extenderá el período de demanda máxima. Esto genera cargos por uso para los productos que se escalan durante la carga máxima.
- GCP cobrará tarifas de recursos adicionales por las instancias adicionales.
- Se incurre en algunos gastos para controlar los daños (redactar y publicar comunicaciones que expliquen la interrupción).
- Cualquier compensación o reembolso a anunciantes o patrocinadores según lo especificado contractualmente.

El costo a largo plazo de una interrupción. Supongamos

que se produce un tipo diferente de interrupción: un error de software que provoca la pérdida de datos, en nuestro caso, votos. ¿Qué sucedería si descubrimos que una parte significativa de los votos emitidos no se registró ni contabilizó? Esta falla genera un costo a largo plazo para la empresa que no se percibirá en su totalidad hasta mucho después de que se resuelva la situación.

El principal impacto de esta interrupción a largo plazo radica en la pérdida de reputación. Como resultado, se prevé que menos personas sintonizarán o participarán en nuestra competencia.

Esto afecta nuestros resultados a medio y largo plazo, ya que las tarifas de patrocinio y colocación de productos en futuras competiciones se reducirán y nuestro programa resultará menos atractivo para los anunciantes de primer nivel. Si bien la interrupción breve generó algunos gastos en su momento, esta falla tuvo efectos persistentes que redujeron los ingresos futuros. Si bien este impacto es difícil de cuantificar, tiene el potencial inminente de cambiar el futuro de nuestro negocio, mientras que el costo de una breve interrupción del servicio puede conocerse y contabilizarse poco después de que finalice el periodo de incidencia.

Ninguna interrupción es buena, pero la dificultad para determinar y contener el impacto aumenta a medida que persisten los efectos de la interrupción. Un costo desconocido o indefinido presenta un gran riesgo. Al decidir qué falla...

En cuanto a los modos de prevención, aquellos cuyos efectos no se pueden revertir rápidamente son aquellos en los que los propietarios de un sistema deben centrar su atención y esfuerzo. El razonamiento se resume en un dicho popular: más vale malo conocido que bueno por conocer. Esto significa que es mejor afrontar una situación difícil conocida que una nueva que podría ser peor.

Monitoreo, detección y notificación

Es tan importante comprender por qué se debe monitorizar un sistema distribuido como qué aspectos del sistema deben monitorizarse. Abordaremos primero el porqué antes de analizar qué y cómo se debe aplicar la monitorización a una aplicación.

Observar primero y luego controlar En prácticas empresariales

y de sistemas más generales, anteriores al software distribuido, el principio de que no se puede controlar lo que no se monitoriza ha sido aceptado durante muchas décadas. En un ejemplo muy fácil de entender, tomemos el caso de un médico cuyo objetivo es mantener la presión arterial de un paciente dentro de un rango saludable y esperado de valores. El médico puede ejercer control prescribiendo tratamientos que tienen uno de dos efectos en el paciente: elevar o disminuir la presión arterial. El médico puede controlar la presión arterial del paciente, pero ningún médico responsable prescribiría ninguno de los dos tratamientos sin establecer primero una rutina de medición y registro de la presión arterial del paciente a lo largo del tiempo.

Una vez que el médico sabe cuál es la presión arterial del paciente, y si tal vez varía con el tiempo, puede ejercer control sobre ella con confianza, dirigiéndola hacia el límite del rango deseado.

Si el control del médico fuera absoluto: llevar la presión a 130 sobre 80, sería posible omitir la monitorización prerrequisito. Sin embargo, en este ejemplo, como en la mayoría de los ciclos de retroalimentación, el control no es absoluto; no se establece un valor, sino que se lo dirige. En lugar de un tratamiento que fije la presión arterial del paciente a un valor específico, los tratamientos disponibles aumentan o disminuyen la presión arterial según un valor predicho esperanzador. Se requiere observación continua para determinar el efecto real del tratamiento y

Aconsejar una mayor aplicación del control.

Terminología de monitorización

El campo de la monitorización es amplio y complejo; antes de profundizar en él, definiremos algunos de los términos más comunes en el vocabulario del dominio:

- **Monitoreo:** Recopilación, agregación y visualización de datos cuantitativos sobre un sistema en tiempo real. Algunos ejemplos de los elementos de datos que podemos incluir en el monitoreo de un sistema informático distribuido son el número de consultas, los tiempos de procesamiento y el número de instancias del servidor.
- **Métrica:** Una métrica es un valor que se recopila del sistema monitoreado y se utiliza para evaluar o medir el sistema y su rendimiento. Ejemplos de métricas son el tiempo de procesamiento o latencia de las solicitudes, el número de errores y las solicitudes por segundo.
- **Panel de control:** Aplicación que proporciona un resumen de las métricas de un sistema. Las métricas son los datos recopilados del sistema o los datos derivados de estos. Un ejemplo de valor derivado es el número máximo de instancias de servidor observadas durante las últimas 24 horas.
- **Alerta:** Notificación dirigida y destinada a la atención y el consumo humano. Una alerta es la forma en que el sistema informa a un humano sobre alguna condición específica que ha surgido en el sistema. Las alertas suelen enviarse a los humanos mediante uno o más canales, según su urgencia. Los tickets o errores, el correo electrónico y las páginas son los canales de entrega más comunes en orden ascendente de urgencia.
- **Causa raíz:** Un defecto en el software o los procesos cuya eliminación evitaría la repetición de un fallo consecuente. Dicho de otro modo, una causa raíz es lo que se debe solucionar para evitar que un problema se repita. Un fallo puede tener múltiples causas raíz y solo se produjo cuando cada una de ellas estaba presente. Un ejemplo de una causa raíz compuesta de este tipo es un aumento de la demanda debido a un fallo en la base de datos regional y una cuenta de facturación utilizada para ampliar recursos adicionales, vinculada a una tarjeta de crédito vencida.

- **Nodo:** Un nodo se usa genéricamente para referirse a una instancia individual de algo que se está monitoreando. Por ejemplo, si monitoreamos el servicio de ingesta de votos, un nodo se refiere a un servidor o tarea específica de ingesta de votos.
- **Push:** Un cambio en el software de un sistema en ejecución se denomina push. Esto está relacionado con las notificaciones push descritas en el [capítulo "Frontends en la frontera del cliente"](#), ya que se generan desde una fuente externa al destinatario y se envían forzosamente a este en un momento imprevisto. En este contexto, el software puede incluir código, archivos de configuración o ambos. Una sustitución del código que ejecuta un servicio de App Engine por una nueva versión de la aplicación es un push de código típico.

[Tipos de métricas.](#) Métrica

es un término con una definición vaga: un valor que se recopila del sistema que se monitorea. Antes de aplicar el término en la práctica, proporcionaremos definiciones y clases de métricas más concretas. Las métricas utilizadas para monitorear el rendimiento de un sistema distribuido son de uno de los siguientes tipos:

- **Contadores:** Un contador es un valor que solo puede aumentar con el tiempo a medida que el sistema opera. Algunos ejemplos son la velocidad por segundo y el delta en una ventana de tiempo.
- **Calibres:** Un calibre es un valor derivado que puede aumentar o disminuir con el tiempo. Algunos ejemplos de calibres son el máximo a lo largo del tiempo y las conexiones abiertas.
- **Resúmenes:** Los resúmenes son contadores absolutos acumulados.

Los ejemplos de métricas de resumen son el recuento de solicitudes y la cantidad total de bytes transmitidos.

[Métricas clave de confiabilidad.](#)

Existen numerosas métricas que pueden ser reveladas y monitoreadas por un sistema distribuido. Cualquier conjunto de métricas está asociado con un aspecto específico del servicio: la generación de ingresos, la confiabilidad y el uso específico de recursos son objetivos potenciales de un servicio. Las más comunes...

El objetivo del servicio que abordarán este capítulo y los siguientes es la confiabilidad, que incluye la disponibilidad del sistema.

Existen cuatro métricas de confiabilidad que sirven para indicar la salud del sistema en observación. Las identificaremos y describiremos brevemente a continuación:

- **Errores:** la tasa de solicitudes que fallan, ya sea explícitamente porque la aplicación emite una respuesta de falla intencional, como HTTP 500, o implícitamente, como en el caso de un tiempo de espera.
- **Latencia:** el tiempo transcurrido que lleva atender una solicitud.
Se calcula a partir del tiempo transcurrido entre que un usuario realiza una solicitud (al hacer clic en un botón de envío) y la respuesta de la aplicación con el resultado de la solicitud. La latencia de las solicitudes fallidas y las exitosas debe monitorizarse por separado.
- **Saturación:** La parte de los recursos de su sistema que están en uso.
La saturación es el efecto agregado de diferentes medidas. La cantidad de espacio en disco, memoria, ancho de banda de red, cuota de instancia y cualquier otro elemento que consuma su sistema y para el cual tenga un presupuesto asignado está sujeta a saturación.
- **Tráfico:** Es la demanda que los usuarios realizan sobre su sistema.
La medición del tráfico se basa en lo que ofrece su servicio. Un servicio web mide el tráfico en solicitudes por segundo; un sistema de almacenamiento mide el tráfico en bytes almacenados y recuperados por segundo.

Monitoreo de sistemas distribuidos. Aplicando estos

conceptos a nuestro caso práctico inmediato, un sistema de votación distribuida basado en GCP, podemos visualizar la velocidad a la que los frontends de los procesadores de votos procesan los votos emitidos. Nuestro objetivo es procesar los votos a la misma velocidad a la que los emiten los usuarios. Nos referiremos a esa velocidad como votos emitidos por segundo. Para replantear esto, si en un momento dado los usuarios emiten 500 votos en un segundo, queremos que el sistema ingiera esos 500 votos en ese mismo segundo, de modo que el conjunto de procesadores de votos esté disponible para procesar los 500 votos del siguiente segundo. Deseamos alcanzar un estado estable en el que la oferta o la capacidad se igualen a la demanda. Como el conjunto limitado de...

Controles: un arquitecto de sistemas no puede establecer la latencia absoluta ni la velocidad de procesamiento de una aplicación. Lo que se puede hacer es escalar horizontal y verticalmente varios componentes del flujo de procesamiento para que la velocidad del flujo converja al valor deseado.

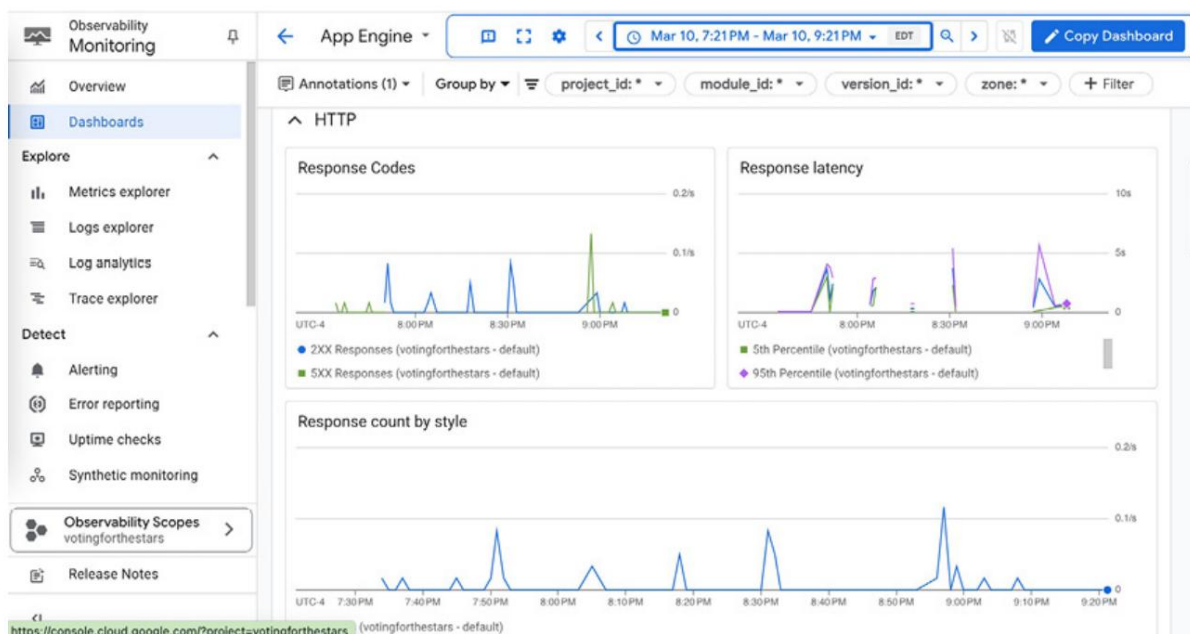
Monitoreo de aplicaciones de GCP. Google Cloud

captura automáticamente métricas de numerosos servicios de Google Cloud y las publica en paneles predefinidos. Se pueden configurar notificaciones para escalar una condición definida como posible indicador de un problema, como que el uso del espacio en disco de Cloud Storage esté al 90 % de la capacidad configurada o que la facturación del servicio en la nube de su proyecto haya alcanzado los \$1000 en el período de facturación actual.

GCP incluye una suite de operaciones, productos que ofrecen un conjunto integrado de herramientas, incluyendo herramientas de observabilidad que facilitan la monitorización de aplicaciones de GCP. Los tres servicios principales de esta suite son Cloud Monitoring, Cloud Logging y Cloud Trace. Esta sección describirá brevemente cada uno de estos productos y presentará sus aplicaciones en nuestro sistema de votación.

Cloud Monitoring proporciona visibilidad sobre la disponibilidad y el estado de su aplicación en la nube. La recopilación automática de métricas se refleja en paneles de control preconfigurados. Además de los paneles de control incluidos, el lenguaje de consulta de Cloud Monitoring permite investigar e identificar tendencias más profundas y obtener datos de rendimiento. Se pueden definir alertas para enviar notificaciones basadas en políticas de alerta con umbrales especificados por el usuario, como enviar una notificación al canal principal de guardia cuando la latencia de solicitud del percentil 90 supera los 200 milisegundos durante 5 minutos.

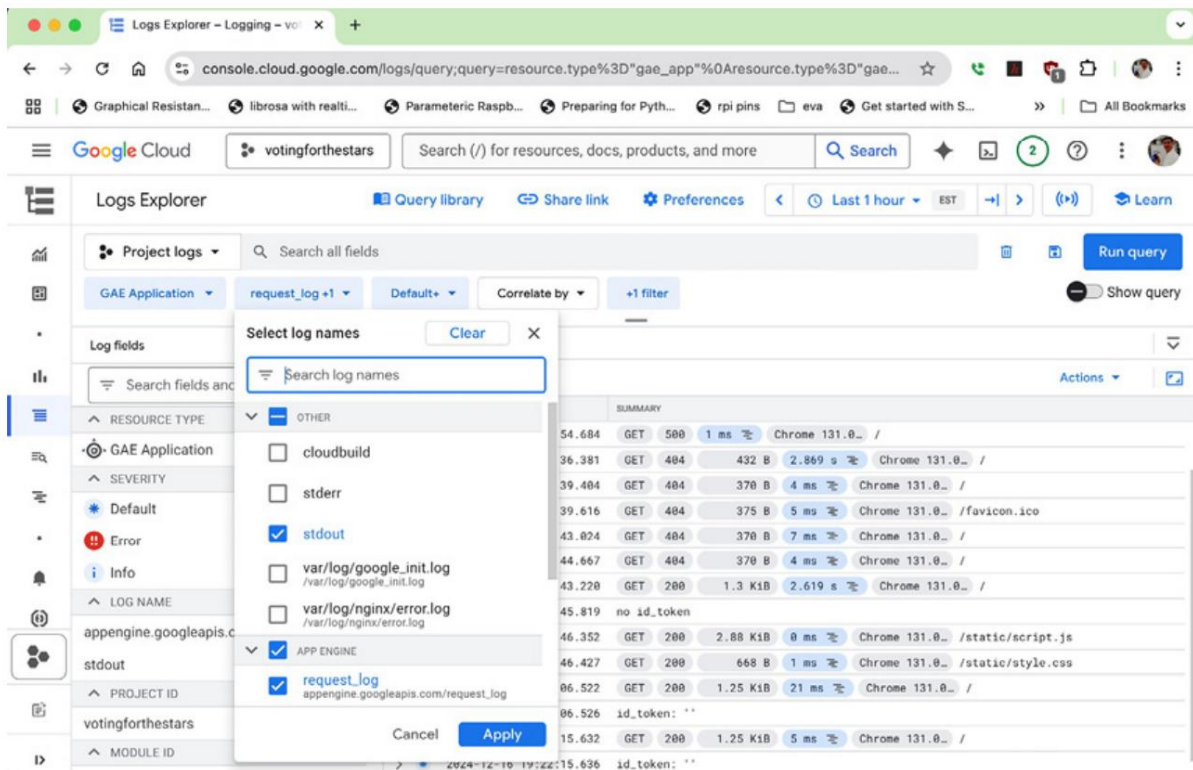
Cuando se añade un servicio de GCP a un proyecto, se añade un panel de monitorización predeterminado específico para ese servicio a la consola de Cloud del proyecto. Por ejemplo, al implementar el servicio de App Engine para el frontend del servicio de votación, este panel se añadió a la consola de Google Cloud del propietario del proyecto.



Cifra 9.5 Panel de control de supervisión del servicio de App Engine predeterminado

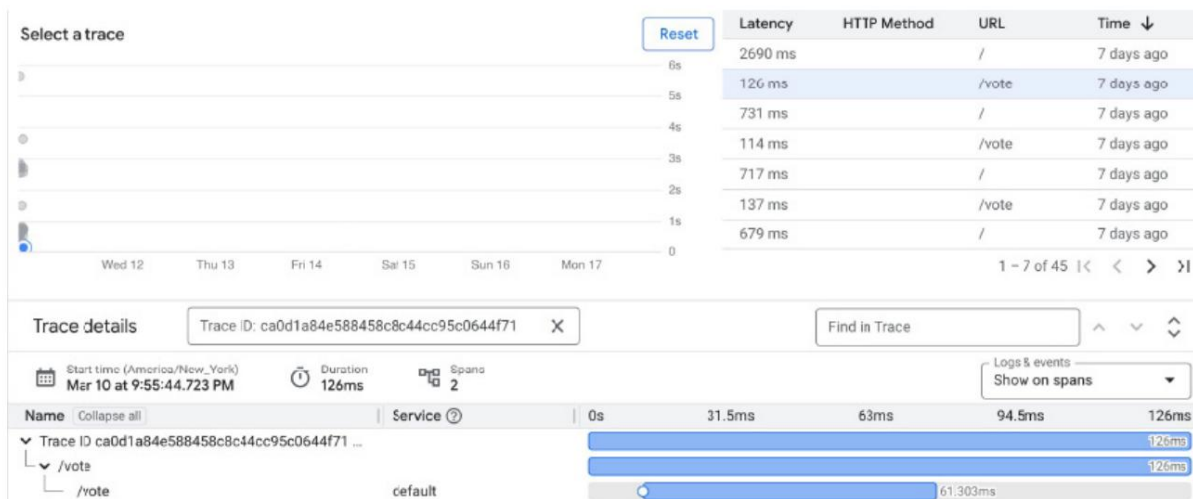
Además de los paneles predeterminados, un desarrollador puede configurar paneles personalizados utilizando una biblioteca de componentes suministrados por GCP e integrar el monitoreo de Prometheus, un popular proveedor externo de monitoreo de servicios.

Cloud Logging permite administrar y analizar registros de aplicaciones e infraestructura para la resolución de problemas y la auditoría. Logs Explorer permite ver y analizar entradas de registros. Log Analytics integra datos de registros con BigQuery para permitir la integración de registros con otros datos empresariales. Log Analytics permite procesar y analizar mayores volúmenes de datos detallados, como registros de telemetría de red, de lo que resulta práctico con otras herramientas. Esto ayuda a solucionar problemas generalizados o aquellos que existen con distintos grados de separación de los componentes de una aplicación. Log Analytics también permite obtener información empresarial del uso de aplicaciones en todo el sistema. Log Explorer y Log Analytics están disponibles en la consola de Cloud junto con el panel de supervisión y las alertas de un proyecto. A continuación, se muestra un ejemplo de Log Explorer con algunos de los filtros disponibles:



Cifra 9.6 Explorador de registros de GCP con filtros

Cloud Trace es un sistema de rastreo distribuido. Recopila datos de latencia de Aplicaciones de GCP y las muestra en paneles predefinidos en tiempo real. Cloud Trace es invaluable para proporcionar un medio adicional para monitorear el sistema. rendimiento durante un impulso de componentes o un pico de demanda anticipado. Se puede acceder al sistema Cloud Trace en la consola Cloud a través de El explorador de seguimiento predeterminado, como se muestra en esta figura:



Puntos de observación del rendimiento en la aplicación de votación. La aplicación de votación cuenta con un conjunto bien definido de funciones visibles para el usuario. Enumeraremos las funciones esenciales para la función principal de emitir un voto para un acto seleccionado. Para estas funciones, identificaremos las métricas de fiabilidad que deben supervisarse para garantizar que esta funcionalidad sea fiable y tenga el rendimiento necesario para satisfacer las expectativas del usuario.

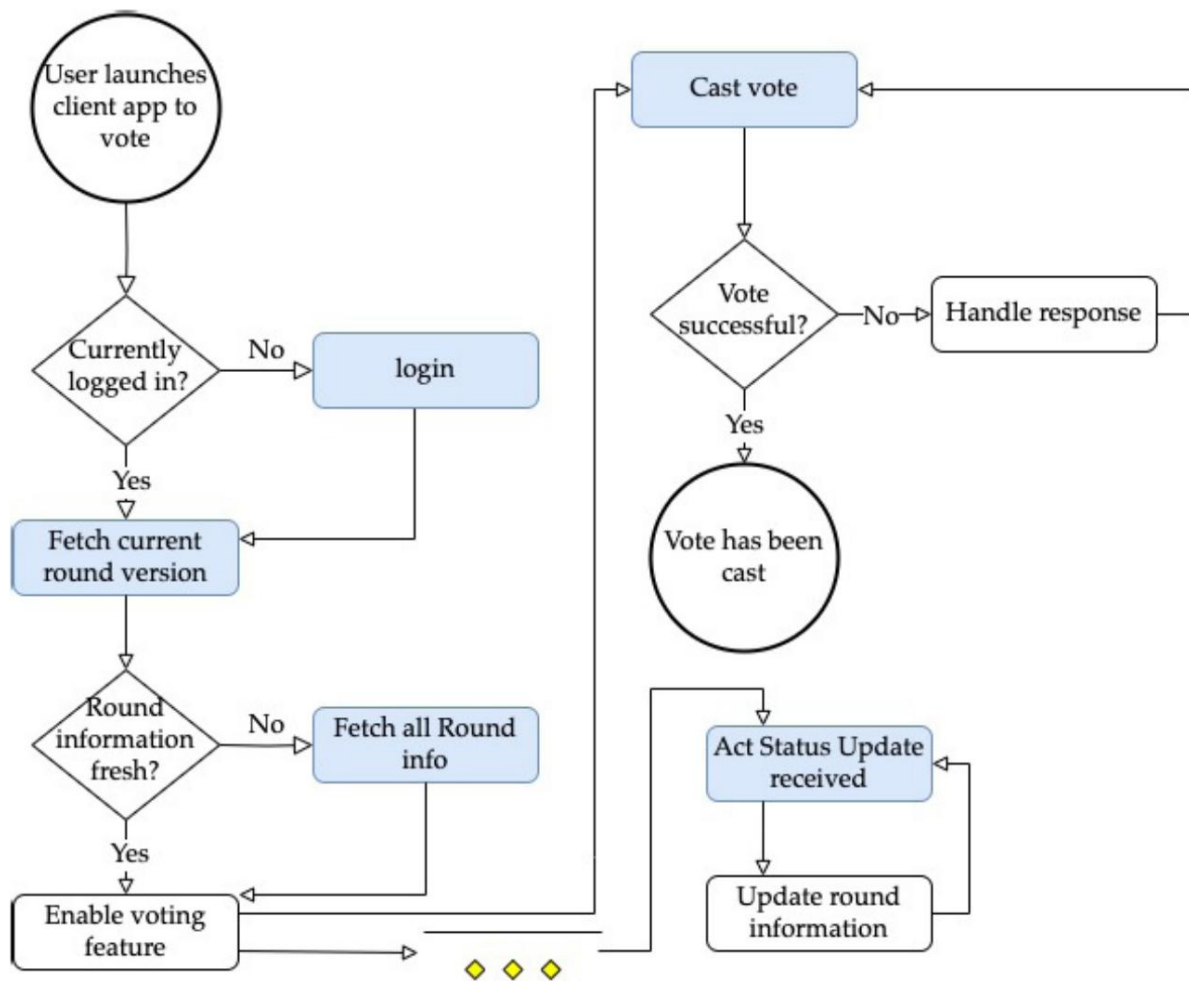
La emisión de votos se realiza en tiempo real y se considera la función más importante de la aplicación. Sin embargo, esta función depende de otras funciones prerequisite y requiere cierta actividad de seguimiento. La correcta ejecución del botón de voto del usuario no puede darse sin la correcta integración de todas estas funciones.

La siguiente es la secuencia de eventos que da como resultado un usuario satisfecho de nuestra

- **aplicación de votación: Autenticación del usuario:** La autenticación es la clave que desbloquea el resto de las funciones de nuestra aplicación. El usuario debe iniciar sesión antes de que pueda comenzar la votación. Si bien un usuario puede haber iniciado sesión tras autenticarse en una sesión anterior y puede proceder directamente a emitir su voto, existen algunas situaciones en las que un votante entusiasta se verá redirigido a iniciar sesión. Estas situaciones que redirigirán al usuario son una sesión anterior que ha expirado, una inserción de un componente de la aplicación que reemplazó o invalidó las credenciales previamente adquiridas, o simplemente no haberse autenticado previamente en la aplicación cliente actual y el dispositivo específico. Esto supondrá cierta carga en el sistema en el momento de mayor demanda. Para un número, n , de personas que deseen emitir un voto en un momento dado, podemos asumir que habrá n inicios de sesión durante los primeros actos de esa ronda, ya que cada votante ve la primera actuación por la que desea votar.
- **Obtención de la elegibilidad actual:** Para votar en una ronda, es necesario obtener información sobre los participantes. Al iniciar la aplicación, se mostrará un mensaje rápido para determinar si el cliente tiene la información actualizada de los participantes de la ronda.

- Estado de actualización: algunos usuarios conectados tendrán información obsoleta sobre los actos de la ronda y luego obtendrán la lista actual de actos y sus estados de la nube.
- Votación: La emisión de un voto es el factor impulsor de esta cadena de eventos y es la única característica que se garantiza que será ejercida por cada usuario para cada acto por el que desee votar.
- Reflejar cambios en la elegibilidad para votar: Finalmente, a medida que los actos se vuelvan elegibles para votación, se enviarán notificaciones push a cada usuario actualizando el estado de dichos actos. La votación también puede cerrarse para los actos en algún momento después de su finalización, y también se enviarán notificaciones push para estos cambios.

Se presenta un diagrama de flujo de las funciones anteriores, que pueden ejecutarse al emitirse un voto, para ayudar a identificar qué puede representar un cuello de botella en momentos de máxima demanda. Las funciones con fondo sombreado incluyen el tráfico entre el cliente y las plataformas en la nube:



Cifra 9.8 Diagrama de flujo de la función de votación

Al analizar las posibles rutas en este diagrama de flujo, se observa que las funciones que se ejecutan con mayor frecuencia y que dependen de la nube son la obtención de la versión actual de la ronda y la emisión de un voto. La emisión de un voto se ejecuta cada vez que un usuario decide votar por un acto, y la obtención de la versión actual de la ronda se ejecuta cada vez que un usuario inicia la aplicación e inicia sesión. Dado que estas funciones son las más ejecutadas, su monitorización y el escalado rápido de los servicios en la nube subyacentes son la máxima prioridad para garantizar la disponibilidad de nuestra aplicación de votación en los momentos de mayor demanda. Si bien algunos usuarios necesitan estas funciones al momento de votar, todas las demás las necesitan al momento de iniciar la aplicación.

Respuesta a incidentes

El objetivo de mecanismos como el escalado automático de recursos, los servidores redundantes y la conmutación por error automática es prevenir incidentes (el término "incidente" se refiere a la degradación o interrupción del servicio). El uso adecuado de estas técnicas y herramientas reducirá el impacto de los eventos y situaciones que afectan la disponibilidad continua y los niveles de rendimiento deseados de un sistema; se reducen, pero no se previenen por completo. Las cosas pueden salir mal.

Una monitorización exhaustiva y prácticas de alerta permitirán a los operadores de un sistema de aplicaciones detectar estas situaciones emergentes. Si una métrica predice que un usuario se enfrentará a una interrupción, en lugar de indicarla, una alerta y una notificación rápida pueden evitar que un problema afecte a los usuarios. Considere una alerta como la que indica que el uso del disco alcanza el 80 % del espacio disponible y está aumentando, en contraste con una alerta similar que indica que se está agotando el espacio en disco.

Responder a lo primero debería evitar que lo segundo se materialice y evitar cualquier impacto en el usuario.

En cualquier caso, independientemente de si se produce una interrupción, la monitorización y la notificación deben ir acompañadas de un plan de respuesta a incidentes documentado, comunicado y ensayado. Este plan consta de varios componentes que enumeraremos a continuación:

- Una rotación de guardia: En cualquier momento, alguien debe ser designado como el primer respondedor o punto de contacto tanto para los canales de notificación automatizados como para las partes interesadas humanas internas.
- Niveles de escalada de guardia: Las personas son falibles, los trenes se averían y los teléfonos tienen zonas muertas. Si el primer interviniente no responde, siempre se debe designar un segundo refuerzo listo para actuar.
- Manuales de estrategias: Saber qué hacer bajo presión es imprudente, ineficiente y propenso a errores. Establezca y mantenga un conjunto de documentos de fácil consulta que describan cómo diagnosticar e identificar un problema y cómo mitigarlo.
- Empoderamiento de los comandantes de incidentes: salvar al usuario debe ser la acción más importante que realiza un equipo de soporte de productos.
Los encargados de responder ante incidentes deben tener tanto el poder organizacional como los permisos del sistema necesarios para actuar con el menor retraso posible.
- Prueba de respuesta a incidentes: No pruebe un proceso cuando su

El negocio está en juego. Ensaye todo el proceso: Introduzca una condición, verifique que se active la monitorización prevista, que se envíen las notificaciones y que su rotación de guardia se alcance y sepa cómo responder. Google realiza pruebas periódicas de todo el entorno de producción, conocidas como pruebas de recuperación ante desastres (DiRT).

- **Análisis post mortem:** Durante un incidente, tome notas detalladas. Una vez superado el problema y todos hayan descansado, realice un análisis post mortem de lo que salió mal, lo que salió bien y lo que se podría hacer de otra manera.
- **Cambio inducido por la autopsia:** El resultado de una autopsia no es una culpa, sino una mejora. Las lecciones aprendidas deben conducir a cambios en el sistema para reducir futuros incidentes y a cambios en los procesos de respuesta para aumentar la eficacia de las respuestas.

Se recomienda leer más sobre estos temas; existen muchos recursos de aprendizaje sobre los principios de SRE publicados por Google y otros.

Matriz de modos de fallo y medidas apropiadas

Para cada interrupción, existe una mitigación más eficaz, que se ajusta a uno de varios patrones conocidos. La respuesta óptima puede determinarse en función de aspectos específicos de la interrupción; ciertas situaciones requieren respuestas específicas. Desde una perspectiva general, la causa principal de una interrupción del servicio puede clasificarse como una de varios tipos conocidos.

Anteriormente en este capítulo, [la figura 9.4](#) relacionó el modo de fallo con el alcance y la progresión. Con base en esa tabla, finalizaremos esta sección sobre respuesta a incidentes con una tabla que asocia la categoría de fallo con el tipo de respuesta más comúnmente eficaz. La siguiente figura muestra la tabla:

Cause	Response
Introduced over/by time	Scale resources
Force majeure	Drain and failover
Ignorance or malevolence	Rollback
	Temporary unavailability
Unanticipated usage	Scale resources
	Drain and failover
Defects	Rollback

Cifra 9.9 Matriz de causa raíz y respuesta

Las respuestas o mitigaciones generales de [la Figura 9.9](#) se describen brevemente aquí. El tema de la respuesta a incidentes está ampliamente cubierto en la literatura de la industria y en la comunidad sobre la confiabilidad del sitio:

- **Reversión:** Si una falla se debe a un defecto, es probable que este no estuviera presente en el último estado operativo conocido del sistema. Por lo tanto, revierte o deshaz los cambios que se correlacionen con la interrupción temporal o que afectaron la funcionalidad defectuosa. No pierdas tiempo intentando corregir el error sobre la marcha; primero restaura el sistema a un estado operativo y luego analiza, diseña e implementa una versión corregida del cambio revertido.

Nota: Estas son respuestas inmediatas, se esperan medidas de mitigación más complejas y específicas para el incidente una vez que el sistema esté fuera de peligro inmediato.

- **Drenaje y conmutación por error:** Si un recurso o componente no funciona, pero sus componentes hermanos o equivalentes sí, redirija la carga de los componentes que fallan a los que sí funcionan. Drenar un recurso significa desviar las solicitudes ascendentes de ese recurso, desactivándolo por completo (y eliminando así la posibilidad de causar daños).
- **Escalar recursos:** si un trabajador es lento o una base de datos distribuida no puede seguir el ritmo, escale el componente sobrecargado de manera vertical u horizontal, o ambas.
- **Indisponibilidad temporal:** La respuesta menos deseable es que el sistema no esté disponible durante un período. Esto puede resultar sorprendente y parecer drástico, pero informar a los usuarios de que el sistema no está disponible puede ser preferible a presentar una aplicación que no funciona o que...

cuyos datos de los usuarios posiblemente estén comprometidos.

Implementación del laboratorio de código de la API de ingesta de votos

La implementación de un servicio, como el servicio de votación que hemos descrito, puede considerarse como una tarea de dos partes: la implementación del despliegue y la interacción entre los distintos componentes, y la implementación de la lógica de negocio específica. La primera, que define la forma del sistema, es la que está más estrechamente vinculada a la infraestructura sobre la que se construye. Por ello, en este laboratorio de código, primero implementaremos un cliente de servicio de votación y un servicio en la nube que ofrezcan las características principales identificadas para el servicio, sin la funcionalidad; es decir, el usuario podrá votar por una ley; el servicio de votación aceptará los votos, pero aún no se realizará el recuento real de los votos emitidos.

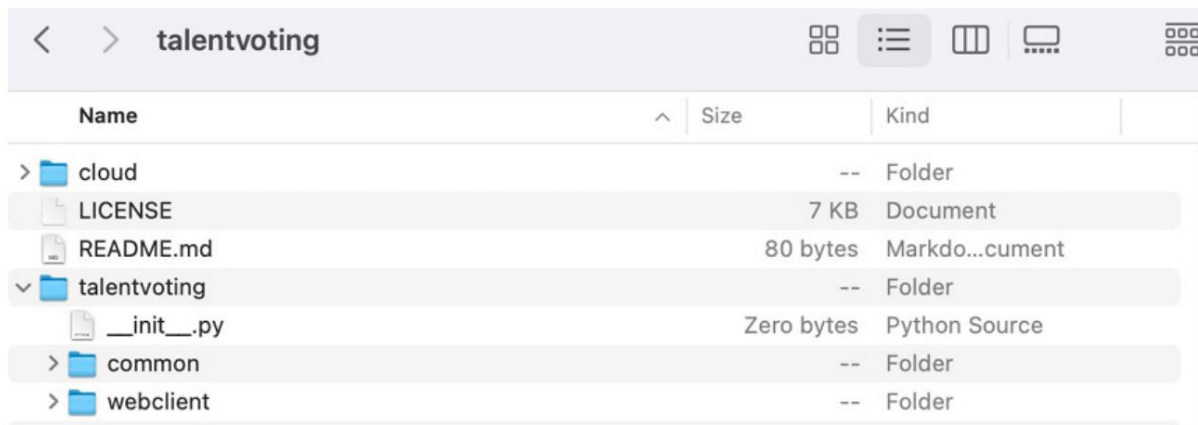
Parte de la definición del servicio es la configuración de los proyectos de GCP y Firebase en los que implementaremos nuestro servicio. El laboratorio de código describirá la definición de estos proyectos. Finalmente, identificará la monitorización de GCP para el servicio en la nube.

Consideraciones del proyecto para la configuración del repositorio. Primero, revisaremos la estructura de directorios de la aplicación, que modificamos ligeramente respecto a la estructura lógica de paquetes presentada en el capítulo anterior para que se adapte mejor a nuestras opciones de implementación de un cliente web JavaScript con Firebase Auth y un servicio de Google App Engine escrito en Python. Toda la aplicación se almacenará en un único repositorio de GitHub, lo que también se reflejará en la estructura de directorios y archivos que se desarrolle. Nuestro repositorio para esta etapa del sistema de votación se encuentra en https://www.github.com/raygeeknyc/gcp_9. Directorios de nivel superior: webclient, cloud y common.

`:// . . / / _9`

En el nivel más alto, separamos el código en tres particiones: código basado en la nube, código del cliente y código común a todo el sistema de votación.

La estructura del directorio de nivel superior del repositorio de votación de talentos se ve así:



Name	Size	Kind
> cloud	--	Folder
LICENSE	7 KB	Document
README.md	80 bytes	Markdo...cument
▼ talentvoting	--	Folder
__init__.py	Zero bytes	Python Source
> common	--	Folder
> webclient	--	Folder

Cifra **9.10** Estructura de directorio de nivel superior del repositorio

Describiremos cada una de las carpetas de alto nivel y su estructura interna en las siguientes secciones:

Cliente web y código común

El cliente web se compone de HTML y los tipos de archivos que lo acompañan, como archivos .js y contenido estático. Es posible que en la implementación final se incluyan otros tipos, como archivos Python, pero en esta definición inicial, utilizamos los tipos de archivos nativos del navegador. Además, la elección de Firebase para alojar el cliente implica la inclusión de algunos archivos específicos de Firebase.

Las aplicaciones de cliente móviles planificadas serán hermanas del cliente web, por lo que colocamos el paquete común en este mismo nivel, debajo de un paquete de votación de talentos principal para facilitar compartir el código común en todos los paquetes de cliente.

Usar la autorización de cliente de Firebase implica incluir información privada en nuestra aplicación web. Subir estos secretos de autorización a GitHub supondría una vulnerabilidad de seguridad fácilmente explotable, por lo que creamos un archivo .gitignore que impedirá que se suban a nuestro repositorio.

Los directorios de paquetes de cliente web y comunes completamente expandidos contienen los siguientes archivos y carpetas:

talentvoting							
Name	Size	Kind					
__init__.py	Zero bytes	Python Source					
common	--	Folder					
__init__.py	Zero bytes	Python Source					
acts.py	670 bytes	Python Source					
interfaces	--	Folder					
responses.py	1 KB	Python Source					
servicelocations.py	246 bytes	Python Source					
voteingester.py	285 bytes	Python Source					
policy	--	Folder					
__init__.py	Zero bytes	Python Source					
votingpolicyengine.py	358 bytes	Python Source					
webclient	--	Folder					
.firebase	--	Folder					
.firebasearc	59 bytes	Document					
.gitignore	1 KB	Document					
firebase.json	343 bytes	JSON Document					
public	--	Folder					
404.html	2 KB	HTML text					
config.js	519 bytes	JavaScr...t source					
favicon.ico	3 KB	Window...n image					
firebase-auth.js	1 KB	JavaScr...t source					
firebase-logo.png	2 KB	PNG image					
index.html	9 KB	HTML text					

Cifra 9.11 Estructura de carpetas de paquetes comunes y de cliente

Nuestro cliente web inicial utilizará la autorización de Firebase únicamente con el proveedor de identidad de Google, mediante una ventana emergente. El código para ello se encuentra, en gran parte, en el archivo `firebase-auth.js`. Los secretos mencionados, copiados desde la consola de desarrollo de Firebase a nuestra aplicación, se aíslan en el archivo `config.js`. Este aislamiento de los secretos permite que nuestro archivo `.gitignore` omita la menor cantidad de código necesaria del repositorio de GitHub.

Como se mencionó, inicialmente implementaremos un servicio que no incluye un backend completo. Para permitir las pruebas y demostraciones del servicio, incluimos un conjunto de actos codificados en el módulo `acts.py` del paquete común. Este archivo contiene el siguiente código Python:

```
desde escribir importar Lista, Dict
```

```
# Por ahora, un Act es un objeto JSON sin una definición más estricta impuesta
```

```
Act = Dict[str, cualquier]
```

```
# A menudo se hace referencia a cualquier número de actos como un grupo
```

```
Hechos = Lista[Acto]
```

```
# Este es un código auxiliar que se utilizará hasta que definamos servicios de almacenamiento de respaldo para Hechos
```

```
def ejemplo_actos() -> Actos:
```

```
    acto1 = {"temporada": 1, "acto": "S01A001", "nombre": "Rock loco y bueno", "elegible para votar":
```

```
        Verdadero} acto2 = {"temporada":
```

```
    1, "acto": "S01A002", "nombre": "Hechicería seria", "elegible para votar": Verdadero} acto3 =
```

```
        {"temporada": 1, "acto": "S01A004",
```

```
        "nombre": "Comedia acrobática", "elegible para votar": Falso}
```

```
    actos =
```

```
        lista([acto1, acto2, acto3] )
```

```
    actos de retorno
```

Código basado en la nube

La interfaz de votación se implementa como un servicio de App Engine escrito en Python 3. Las herramientas de desarrollo de Google Cloud colocan los archivos de recursos y configuración de App Engine necesarios en el directorio de inicio de una aplicación de App Engine, además de imponer una estructura de directorio esperada para la aplicación.

Al igual que los secretos de autorización de Firebase, nuestro servicio de App Engine incluye una clave de cuenta de servicio, que debe mantenerse privada. En tiempo de ejecución, esta se protege mediante un directorio privado en la aplicación.

Jerarquía, pero también debemos asegurarnos de que esta clave de cuenta no se revele en GitHub. El archivo `.gitignore` en el directorio superior impide que este archivo se suba a GitHub con la última línea de las siguientes especificaciones: `talentvoting/webclient/public/config.js talentvoting/webclient/private/** */serviceAccountKey.json`. La

jerarquía de carpetas y archivos del paquete `votingfrontend` se muestra en la siguiente figura:

Name	Size	Kind
> .git	--	Folder
.gitignore	100 bytes	Document
▼ cloud	--	Folder
__init__.py	Zero bytes	Python Source
▼ votingfrontend	--	Folder
__init__.py	Zero bytes	Python Source
.gcloudignore	562 bytes	Document
app.yaml	1 KB	YAML Document
▼ private	--	Folder
serviceAccountKey.json	2 KB	JSON Document
requirements.txt	36 bytes	Plain Text
> static	--	Folder
talentvoting	18 bytes	Alias
votingfrontend.py	4 KB	Python Source
LICENSE	7 KB	Document
README.md	80 bytes	Markdo...cument

Cifra 9.12 Estructura de carpetas y archivos de los paquetes en la nube

Hay un problema con la estructura de directorios de la aplicación al implementar una aplicación de App Engine. El comando `gcloud deploy`, que se ilustrará más adelante, solo funciona con carpetas y archivos ubicados en el directorio base de nuestra aplicación: en este caso, la carpeta `cloud:votingfrontend`. Aunque queremos incluir archivos en nuestra carpeta `talentvoting:common`, que es hermana de la carpeta `cloud`, la herramienta `gcloud` no los implementaría en App Engine, lo que generaría errores como el siguiente en nuestro servicio de App Engine: from

```
import Act, Acts ModuleNotFoundError: No module named 'talentvoting'
```

```
· · ·
:
```

Este problema es bastante común y tiene una solución fácil de describir y ampliamente utilizada. Observe la presencia y el tipo de la carpeta de votación de talentos , resaltada en este detalle de [la Figura. 9.13](#):

requirements.txt	36 bytes	Plain Text
> static	--	Folder
talentvoting	18 bytes	Alias
votingfrontend.py	4 KB	Python Source

Cifra **9.13** Detalle de la carpeta en la nube

En la carpeta base de nuestra implementación de App Engine, la carpeta `cloud:votingfrontend` , hemos creado un alias o enlace simbólico a la carpeta del paquete `talentvoting` que abarca el paquete común .

En Linux o MacOS, esto se hace con el siguiente comando `ln` desde el directorio `votingfrontend` : `$ ln -s ../../`

`talentvoting/talentvoting` El comando

`gcloud deploy` verá este Alias en la carpeta base y recorrerá y cargará los archivos incluidos en la carpeta de origen original al cargarlos en App Engine.

Configuración del proyecto de Google Cloud

El servidor frontend de votación que se ejecuta en App Engine debe estar asociado a un proyecto de Google Cloud. Esta es la entidad para la que se configuran los permisos, la facturación y los servicios disponibles. Los detalles para navegar a páginas web específicas varían y el lector puede buscarlos en Google cuando esté listo. Sin embargo, si se puede navegar, a continuación se indican los términos necesarios y las entidades que deben configurarse para implementar los servicios de GCP:

- **Consola para desarrolladores de Google Cloud:** Esta página sirve como página de inicio o base para una cuenta de desarrollador de Google Cloud. Desde aquí, se pueden definir proyectos, habilitar API, configurar cuentas de facturación y acceder a paneles de monitorización.
- **Configuración del proyecto:** Todos los servicios en tiempo de ejecución, como un servidor de App Engine o una tabla de base de datos, forman parte de un proyecto. Los roles y permisos se pueden definir y administrar en este nivel, y luego los heredan los servicios asociados al proyecto. La cuenta de facturación que se utilizará para pagar los cargos generados por los servicios.

El proyecto está asociado con él. Se pueden definir cuotas y límites de uso para un proyecto, a fin de aislar la contabilidad de costos de un proyecto de otros proyectos.

Una parte importante de la configuración del proyecto es la definición de las cuentas de servicio y la creación de sus claves. Las cuentas de servicio representan la identidad de un servicio de Google Cloud, a diferencia de un usuario conectado. Una clave de cuenta de servicio es una clave privada que se utiliza para autenticarse en las API de Google Cloud y establecer la identidad del componente de tiempo de ejecución como una cuenta de servicio conocida y confiable.

Una clave de cuenta de servicio se almacena como JSON y tiene el formato de este ejemplo redactado:

```
{ "type": "service_account",
  "project_id": "PROJECT_ID",
  "private_key_id": "KEY_ID",
  "private_key": "-----BEGIN PRIVATE KEY-----\nPRIVATE_KEY\n----- END PRIVATE KEY-----\n", "client_email":
  "SERVICE_ACCOUNT_MAIL", "client_id": "CLIENT_ID",
  "auth_uri": "https://
accounts.google.com/o/oauth2/auth", "token_uri": "https://
accounts.google.com/o/oauth2/token", "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs", "client_x509_cert_url": "https://
www.googleapis.com/robot/v1/metadata/x509/CORREO_DE_CUENTA_DE_SERVICIO"
```

Dado que la clave de la cuenta de servicio contiene la parte privada de un par de claves RSA pública/privada, debe protegerse y mantenerse en secreto para todos los espectadores innecesarios.

Nota: No cargue una clave de cuenta de servicio en GitHub y utilice funciones específicas del servicio, como una carpeta privada en App Engine, para evitar que los navegadores accedan a sus contenidos. .

Si se carga una clave privada en un repositorio público de GitHub, las funciones de seguridad cooperativa en GitHub y GCP informarán al propietario y deshabilitarán la cuenta de servicio para evitar que otros la exploten.

Configuración del proyecto de Firebase

El alojamiento de Firebase proporciona un sitio confiable y global donde un desarrollador puede alojar aplicaciones web. Ofrece gestión de versiones, subdominios estándar para su aplicación, compatibilidad con dominios personalizados, gestión de identidades y monitoreo de uso.

Los métodos de inicio de sesión, como la autenticación de Google, el inicio de sesión de Facebook, el correo electrónico/contraseña y el inicio de sesión por teléfono, están habilitados en el nivel de alojamiento, y la actividad del usuario también se rastrea allí.

Una aplicación de Firebase es una aplicación específica asociada a un sitio de alojamiento de Firebase. Las aplicaciones de Firebase pueden ser de Android, iOS, web, Unity o Flutter. Estas aplicaciones utilizan los recursos de Firebase, como la autenticación, asociados a su sitio de alojamiento, lo que facilita la autenticación del mismo usuario en varias plataformas cliente para su aplicación.

De forma similar a la información de la cuenta de servicio para los proyectos de Google Cloud, descrita en la sección anterior, Firebase genera información de configuración que se utiliza para identificar y autenticar a los usuarios de Firebase para que operen como una cuenta de servicio de Firebase. A continuación, se muestra un fragmento de JavaScript de configuración de ejemplo para una aplicación web

```
(redactado): // Importa las funciones que necesitas de los SDK que
necesitas. import { initializeApp } from "firebase/
app"; // TODO: Agrega los SDK para los productos de Firebase que quieres
usar. // https://firebase.google.com/docs/web/setup#available-libraries

// La configuración de Firebase de tu aplicación
web const firebaseConfig =
{ apiKey: "XXXXXXXXXXXXXXXXXXXXXXXXXXXX", authDomain:
"your-hosting-site.firebaseio.com",
```

```

projectId: "su-sitio-de-alojamiento",
storageBucket: "su-sitio-de-alojamiento.firebaseio.com",
messagingSenderId: "#####", appId:
"1:#####:web:XXXXXXXXXXXXXXXXXXXX" };

```

```
// Inicializar Firebase
```

```
const app = initializeApp(firebaseConfig); Esta
```

información debe estar oculta a la observación directa de los usuarios: los archivos .gitignore de GitHub y las funciones de ignorar y reescribir de Firebase pueden usarse para evitar que se expongan en el repositorio y que se distribuyan en solicitudes HTTP. El elemento del archivo de configuración firebase.json impide que Firebase Hosting distribuya un archivo, como se indica a continuación:

```

"hosting":
{ "public": "público",
"sitio": "votandoporlasestrellas",
"ignorar":
[ "firebase.json",
"**/.*",
"**/node_modules/**", "**/
config.js" ] },

```

Monitoreo de GCP

En la página base de la consola para desarrolladores de Google Cloud hay un enlace al panel del usuario. Este práctico enlace lleva al usuario a un panel general con paneles para resúmenes de uso de los servicios implementados, como App Engine, y desgloses completos por servicio y API. Estos resúmenes de uso enlazan a información detallada específica del servicio.

Paneles con filtrado por ventana temporal, desglose por URI y tipo de respuesta, como errores de aplicación y servidor. La jerarquía de paneles que se presenta aquí la genera GCP automáticamente y no requiere configuración ni solicitud por parte del desarrollador.

Conclusión

La escalabilidad y la confiabilidad de un sistema distribuido deben planificarse en su diseño. Si bien es posible corregir un sistema inherentemente poco confiable o impredecible, será mucho más fácil y directo ajustar un sistema diseñado teniendo en cuenta la confiabilidad.

Comprender los tipos de fallas potenciales y su impacto es fundamental para prevenirlas y mitigarlas. El tamaño o el radio de explosión de una falla y su progresión o avance en el tiempo son aspectos clave a considerar.

El impacto de una falla es útil para determinar el nivel de respuesta adecuado. Cuanto más costosa sea una interrupción, más urgente será su solución.

Se deben definir medios apropiados de notificación y respuesta a incidentes antes de que ocurra cualquier incidente; seguir un plan bajo estrés es mucho mejor que escribir un plan bajo estrés.

También se abordó la observabilidad, ya que es difícil controlar un sistema cuando su estado no se puede determinar fácilmente. Este principio de observar y luego controlar tiene sus raíces en prácticas del mundo real, como la medicina, y en prácticas de desarrollo de software como el desarrollo basado en pruebas.

El capítulo concluyó con un laboratorio de código que describió la implementación inicial de una función de votación: un cliente de aplicación web y un servicio de App Engine. Se presentó la configuración de los servicios de alojamiento y consejos para comprender el contenido del repositorio de GitHub que contiene esta implementación inicial de la aplicación.

En el próximo capítulo, la aplicación de votación se extenderá al siguiente nivel: el procesamiento asíncrono que proporcionará escalabilidad y confiabilidad de muchas solicitudes de votación simultáneas.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com>.



CAPÍTULO 10

Asegurar todos los votos de los usuarios

Introducción

Una aplicación distribuida está sujeta a una demanda que presenta cambios repentinos y pronunciados en la tasa de votación. En nuestro escenario de votación, muchos usuarios responden a los mismos eventos en poco tiempo y cada uno espera que el sistema acepte y reconozca (ACK) su voto rápidamente. Los usuarios no deberían verse afectados por la actividad de otros usuarios. Una experiencia predecible y consistente genera confianza en el sistema, lo que les permite seguir la competencia y participar.

Uno de los objetivos de una capa de infraestructura capaz es proporcionar servicios y herramientas que permitan a los sistemas gestionar fácilmente cargas de trabajo que cambian rápidamente.

Estructura

En este capítulo se tratarán los siguientes temas:

- Modelado de la demanda
- Equilibrio de carga
- Uso de colas de mensajes para agrupar solicitudes
- Procesamiento asíncrono de solicitudes

- Envío de solicitudes de usuarios a colas de trabajo asincrónicas a través de

Laboratorio de código de Pub/Sub

Objetivos

Google Cloud Platform (GCP) puede escalar automáticamente recursos en diferentes dimensiones según los cambios en el tráfico de usuarios. En este capítulo, el lector aprenderá las ventajas y desventajas de las alternativas disponibles para el escalado adaptativo de una aplicación.

Además del escalado de recursos, GCP incluye productos que permiten reducir el impacto de los aumentos repentinos de la demanda al desviar la latencia del usuario. El lector aprenderá patrones de diseño para utilizar estos productos y cómo aplicar diferentes tipos de escalado automático para adaptar la capacidad del sistema a la demanda.

Modelado de la demanda

La demanda es producto del efecto acumulado de varios ciclos. El final de una función desencadena una avalancha de votos, que se desvanecen y quizás se solapan con el siguiente acto, lo que genera un ciclo similar.

La conclusión de un acto puede generar votos para actos anteriores que se habían retenido hasta que se vieran actos posteriores. Se trata de un conjunto complejo de comportamientos, y si bien podemos pronosticar el número máximo teórico de votos emitidos, solo podemos predecir la forma general de los múltiples factores de demanda.

Ciclos independientes de demanda

Los factores que contribuyen a la demanda total del usuario son aquellos que impulsan a un usuario a realizar una solicitud, en nuestro caso específico, emitir un voto, sin que exista una dependencia directa entre ellos. Por dependencia directa se entiende que una solicitud específica no se genera a partir de una solicitud anterior; cualquiera de las dos solicitudes puede emitirse independientemente de la otra.

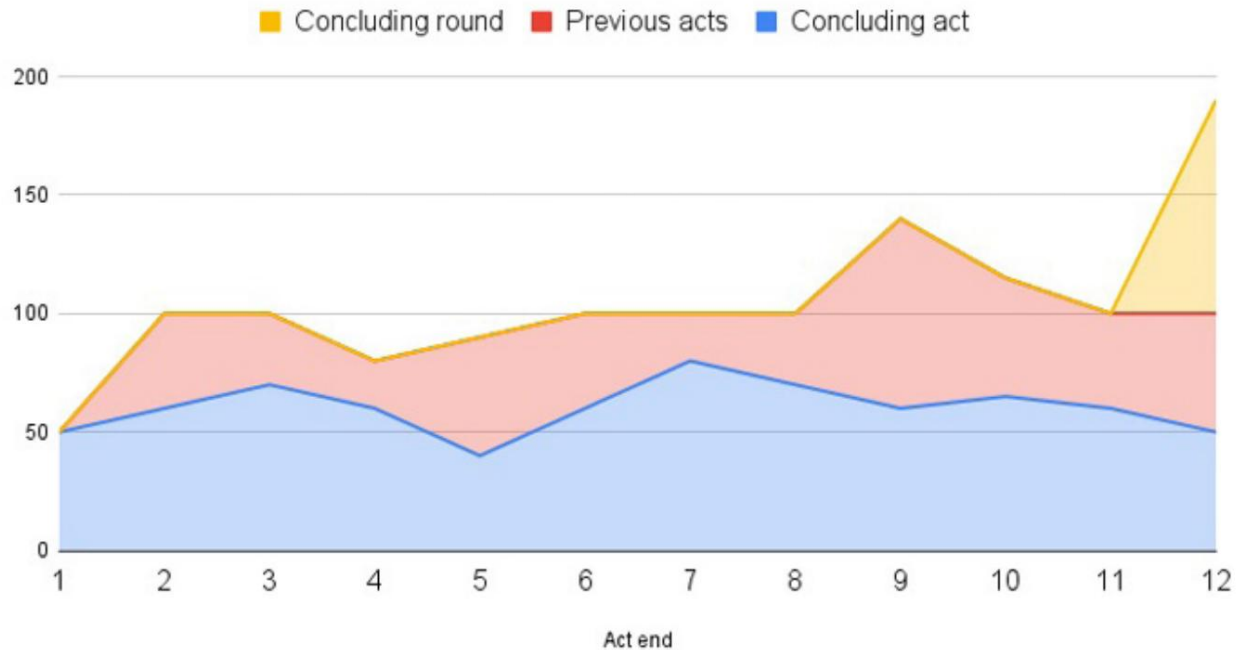
En primer lugar, enumeraremos los factores que obligan a un usuario a emitir un voto por una ley:

- Un acto concluye, o casi concluye, y genera una respuesta entusiasta en el espectador. El espectador ve un acto e inmediatamente desea que gane. Este es el ciclo de demanda más simple y directo: los votos aumentan a medida que se transmite un acto popular.
- Concluye un acto que no cumplió con alguna expectativa o aspecto previsto. Esto activa la votación para otros actos vistos previamente por parte de los usuarios que están conservando su cuota de votos; recuerda que un usuario solo puede votar por un número menor de actos que los que aparecen en cada ronda.
- La conclusión de una ronda liberará votaciones que quedaron en suspenso por razones similares al punto anterior. 35 espectadores retienen su votación hasta haber visto todos los actos, para no agotar su conteo de votos presupuestado mientras aún quedan actos desconocidos por ver.

Cada uno de estos tres factores estará presente y podrá modelarse en nuestro pronóstico de demanda para una ronda de nuestra competencia. Lo que no podemos predecir es la magnitud real de cada uno de estos efectos: cuántos votos se emitirán debido a cada uno de estos tres factores, en cada caso. Pronosticar con precisión ese grado de especificidad sería prácticamente imposible predecir el resultado de la competencia.

El número total de votos emitidos durante la duración de una ronda, desglosado en las tres categorías de voto contribuyentes para una ronda hipotética de 12 actos, se muestra en la siguiente figura:

Vote factors, aggregated



Cifra 10.1 3 ciclos de demanda vs cronograma circular

El número exacto de votos mostrados anteriormente, y los ups individuales y las caídas de los recuentos son solo hipotéticas. Sin embargo, hay una Algunos puntos clave que se deben extraer de este diagrama con el propósito de previsión de la demanda.

Lo primero es que no hay votaciones para actos anteriores hasta que se apruebe el segundo acto. en marcha, y que no hay votaciones desencadenadas por la conclusión de una ronda hasta que haya comenzado el acto final de la ronda. Estos son evidentes y son inherentes a la definición de esos factores.

Un patrón menos obvio es que un acto popular, mostrado en el alto número de votos al concluir los actos 3 y 7, va seguida de una caída en el Número de votos emitidos que habían sido retenidos durante los siguientes iteraciones. Esto se sostiene tras algunas consideraciones: si un acto generó votación entusiasta inmediata, hubo menos votantes que estaban conteniéndose, cubriendo sus apuestas para ese acto.

Un patrón de contraparte en la relación entre actos impopulares y Existe la retención de votos. Una ley relativamente impopular libera a los votantes para emitieron sus votos que habían estado reteniendo hasta que vieron

Más competencia. Este patrón se muestra en [la figura](#) de la conclusión [10.1](#) al de los actos 5 y 9.

El ejercicio de modelar la demanda de esta manera proporciona información valiosa, obligando a los arquitectos de aplicaciones a analizar las fuerzas que impulsan la demanda del usuario y puede disipar algunas suposiciones erróneas sobre la forma del tráfico. En particular, el hecho de que los votos emitidos tras actos populares e impopulares resulten en cantidades de tráfico similares puede ser sorprendente; una suposición ingenua sería que un acto popular genera una demanda claramente mayor en el sistema que uno impopular.

Equilibrio de carga

El balanceo de carga es el proceso de distribuir el trabajo entre múltiples servidores para optimizar el rendimiento del sistema en términos de tiempo total y otros recursos dedicados a completar dicho trabajo. Garantiza que ningún servidor se sobrecargue en comparación con sus pares y que ninguna solicitud compita injustamente por los recursos.

Las siguientes secciones presentan brevemente algunos conceptos y nomenclatura de equilibrio de carga antes de analizar los servicios específicos de GCP y luego se centran en la aplicación del equilibrio de carga a la aplicación de votación.

Tipos de balanceadores de carga

En términos generales, existen tres tipos de balanceadores de carga:

- Los balanceadores de carga de hardware son dispositivos físicos dedicados que sirven como destinatarios iniciales de las solicitudes de los usuarios y que luego redirigen ese tráfico a los servicios reales.
- Los balanceadores de carga de software son software que se ejecutan en los servidores existentes como otros componentes, dividiendo y distribuyendo el tráfico entrante.
- Los balanceadores de carga en la nube son servicios en la nube similares a los balanceadores de carga de hardware, pero que se ejecutan en el hardware existente en la nube. Son más escalables que los balanceadores de carga de hardware y están más aislados de los efectos secundarios de otro software que de la carga de software.

equilibradores.

Algoritmos y consideraciones de equilibrio de carga

El equilibrio de carga se puede realizar mediante varias técnicas, cada una de las cuales es una opción con mejor o peor rendimiento según varias consideraciones específicas de la aplicación:

- **Tamaño de las tareas:** Más allá del tiempo exacto de ejecución de cada tarea, conocer la variabilidad de los tiempos de ejecución ayuda a alcanzar una configuración de carga óptima. Cuanto mayor sea la desviación media respecto al tiempo de ejecución promedio, más impredecible será el tiempo de ejecución de una tarea.
- **Dependencias:** si algunas de las tareas dependen unas de otras, o si los grupos de tareas comparten dependencias comunes, la distribución de dichos grupos afectará el rendimiento general.

Cuanta más información se conozca sobre las tareas que se equilibran, mejor se podrá elegir el algoritmo de equilibrio de carga. Los algoritmos de equilibrio de carga son de dos clases: algoritmos estáticos, ideales para equilibrar trabajo relativamente estable o uniforme; y algoritmos dinámicos, más eficaces para equilibrar tareas impredecibles o con un consumo de recursos muy variable.

Si el tiempo de ejecución y la distribución de la demanda de tareas no se conocen de antemano, siempre es posible la distribución estática de la carga. El algoritmo de balanceo de carga más común y sencillo es el estático, conocido como programación round-robin.

En un balanceador de carga round-robin, hay una cierta cantidad, n , de servidores disponibles, cada uno de los cuales recibe la misma consideración a lo largo del tiempo. La primera solicitud se envía al primer servidor, la siguiente al segundo, y así sucesivamente. Una vez enviado el trabajo a cada servidor, la solicitud (n) se asigna al primer servidor y la siguiente al segundo, en un ciclo que se repite constantemente.

La simplicidad de un algoritmo round-robin permite modificarlo para adaptarse a diversas situaciones. Por ejemplo, si algunos servidores tienen más capacidad que otros, se puede ponderar su capacidad para recibir una mayor cantidad de...

solicitudes o recibirlas primero.

Un balanceador de carga dinámico requiere comunicación entre los servidores y los balanceadores de carga sobre el estado de estos. Uno de los algoritmos de balanceo de carga dinámico más comunes y sencillos es el algoritmo maestro-esclavo. En este esquema, cada servidor envía un ping al balanceador de carga cuando tiene capacidad para aceptar una solicitud, y la siguiente solicitud pendiente se dirige a ese servidor. Este tipo de algoritmo funciona bien para garantizar que todas las solicitudes enviadas se ejecuten eficientemente y que ningún servidor se sobrecargue, pero introduce la posibilidad de que las solicitudes se interrumpan cuando ningún servidor solicita su disponibilidad para asumir más trabajo. Para abordar esta vulnerabilidad, se recomienda utilizar un balanceador de carga maestro-esclavo junto con la capacidad de escalar horizontalmente el grupo de servidores a medida que crece la cola de solicitudes pendientes o la latencia en la frecuencia de las llamadas al servidor.

Estos son solo dos de los muchos algoritmos de equilibrio de carga que existen y están destinados a brindar al lector una introducción y una apreciación de las muchas consideraciones y elecciones que realiza GCP al administrar el tráfico de un servicio en ejecución.

[El balanceo de carga en Google Cloud Platform \(GCP\)](#) ofrece un conjunto de opciones de balanceo de carga que encapsulan combinaciones de las diversas decisiones que se deben tomar para el balanceo de carga: tipo, estrategia y algoritmo. La oferta se adapta a la naturaleza de cada aplicación de GCP y a los aspectos relevantes específicos de la arquitectura y red en la nube de Google. El balanceo de carga de Google Cloud puede escalar para admitir y distribuir más de un millón de consultas por segundo y puede configurarse para escalar automáticamente el conjunto de servidores frontend de la aplicación según las tendencias de la demanda.

En el nivel más alto de clasificación, GCP proporciona dos tipos de balanceadores de carga: balanceadores de carga de aplicaciones, que operan con solicitudes y solicitudes de protocolo de transferencia de hipertexto/ protocolo seguro de transferencia de hipertexto (HTTP/HTTPS) , y balanceadores de carga de red, que funcionan con cualquier protocolo de control de transmisión (TCP), protocolo universal de datagramas (UDP) u otro protocolo de Internet (IP).

tráfico de red.

Otro atributo que distingue a los balanceadores de carga de GCP es el del dominio:

- Los balanceadores de carga regionales distribuyen las solicitudes que se originan dentro de una región de GCP entre servidores de la misma región.
- Los balanceadores de carga entre regiones pueden aceptar solicitudes desde una región determinada y distribuirlas entre servidores ubicados en un conjunto de múltiples regiones.
- Los balanceadores de carga globales pueden aceptar solicitudes desde una región determinada y distribuirlas entre servidores ubicados en todas las regiones en las que el servidor de destino está configurado para tener presencia.

Una tercera característica distintiva de los balanceadores de carga de GCP es el origen del tráfico. Los balanceadores de carga internos distribuyen las solicitudes provenientes de clientes dentro de su red en la nube, mientras que los balanceadores de carga externos pueden gestionar solicitudes provenientes de cualquier lugar de internet.

Otra consideración de diferente naturaleza que determina la elección de un balanceador de carga es el nivel de servicio o el costo. GCP ofrece diferentes niveles de servicio de red: Premium, Estándar y una combinación específica de tipo de balanceador y esquema de balanceo de carga podría no estar disponible en todos los niveles.

Revisar todas las opciones de balanceadores de carga en GCP es una tarea extensa, que excede el alcance o la necesidad de este capítulo. Google publica una tabla resumen en <https://cloud.google.com/load-balancing/docs/choosing-load-balancer-summary-gclb>.

Esta tabla enumera 14 balanceadores de carga diferentes y, para una exploración exhaustiva, se recomienda al lector consultar la documentación completa de esa página.

Selección de balanceadores de carga del sistema de votación

En este capítulo, elegiremos un balanceador de carga que sea adecuado para nuestra aplicación, en función de los requisitos de escalabilidad que se describieron en [el Capítulo 8](#), Poner a disposición un sistema de votación y Confiable y las preocupaciones de confiabilidad identificadas en [el Capítulo 9](#), Identificación

Los puntos de estrés de nuestro sistema de votación. La aplicación de votación solo funciona con solicitudes y respuestas HTTPS, por lo que la primera opción (Aplicación vs. Balanceador de Carga de Red) es simple. Un Balanceador de Carga de Aplicación cubrirá los requisitos de nuestro sistema y es más fácil de configurar y operar.

La siguiente opción, el dominio, puede determinarse consultando los requisitos que se muestran en [la Figura 3.5](#) sobre el desbordamiento entre continentes. Debido a la posibilidad de desajustes entre la demanda y la capacidad relativas en regiones específicas, planeamos enviar tráfico de desbordamiento entre regiones; por lo tanto, se utilizará un balanceador de carga interregional para el sistema de votación.

La identificación del origen del tráfico requiere una exploración más profunda del modelo de red de Google. Comprender el concepto de una Nube Privada Virtual (VPC) de Google es necesario para identificar plenamente los límites de los balanceadores de carga internos. Si bien una VPC puede utilizarse para ampliar el dominio aplicable de un balanceador de carga interno, sería relativamente costoso y complejo aplicarlo a nuestros frontends de votación, que deben ser accesibles para clientes de diversos tipos que utilizan diversos tipos de conexiones de red y proveedores. Por lo tanto, un balanceador de carga externo es la mejor opción para el sistema de votación.

Para reafirmar la elección del balanceador de carga a la que se llegó en la sección anterior, los frontends de votación serán administrados por un balanceador de carga de aplicaciones externo y entre regiones.

Uso de colas de mensajes para agrupar solicitudes

Las interfaces de votación tienen la responsabilidad principal de aceptar las solicitudes de los usuarios, en este caso, emitir un voto, e ingresarlo al sistema de la forma más rápida y fiable posible. Los dos requisitos de velocidad y durabilidad, que en este caso es la expresión de la fiabilidad, compiten desde el punto de vista de la ingeniería: la persistencia y la verificación de una actualización del recuento de votos es una operación lenta. Como exploramos en [el capítulo «Determinación de la infraestructura sobre la que se construirán las funciones»](#) e ilustramos en [la figura 3.5](#), podemos desvincular la persistencia de un voto de su aplicación a los totales de cada acto mediante

el uso de un mecanismo asincrónico duradero: una cola de mensajes.

Definiremos un tema de Pub/Sub para la entrega de cada voto desde los frontends de votación al procesamiento de votos posterior. El frontend de votación considerará que un voto está completo cuando lo haya publicado correctamente en este tema, confiando en que la infraestructura funcionará como se espera. El gestor de votos habrá cumplido su función una vez que el voto esté en el sistema de forma permanente.

Creación de recursos / para votos. El uso de Pub/Sub requiere la definición de un tema y al menos una suscripción. Definiremos un tema llamado votos y una suscripción llamada votes-sub en la consola de Google Cloud. Los siguientes pasos definirán estos recursos de Pub/Sub:

1. Primero, asegúrate de que Pub/Sub esté habilitado para tu proyecto:
 - a. Abre la consola de Google Cloud e inicia sesión.
 - b. Selecciona tu proyecto de Google Cloud.
 - c. Haz clic en la pestaña Panel.
 - d. En el panel "Introducción", haga clic en "Explorar y habilitar API".
 - e. Haga clic en "+Habilitar API y servicios".
 - f. En el cuadro de búsqueda, escriba `pubsub.googleapis.com` y presione Intro.
 - g. En la lista de resultados, haga clic en "API de Cloud Pub/Sub".
 - h. Si el estado no indica "API habilitada", haga clic en "HABILITAR" botón.
2. Ahora, crea el tema Pub/Sub llamado votes y un complemento Suscripción:
 - a. En el menú de navegación de la esquina superior izquierda de la consola, seleccione Descripción general de la nube | Panel de control.
 - b. En el cuadro de búsqueda de la parte superior del panel de control, escriba Nube Pub/Sub y haga clic en el botón Buscar.
 - c. Haga clic en Cloud Pub/Sub en la lista de resultados, no en Cloud Pub/Sub API.
 - d. Haga clic en el botón Ir a Cloud Pub/Sub.

- e. Haga clic en Temas en la barra de navegación
- izquierda. f. Haga clic en el botón Crear
- tema. g. Escriba votos en el campo ID del
- tema. h. Asegúrese de que la opción Agregar una suscripción predeterminada esté
- marcada. i. Deje todas las demás opciones predeterminadas y haga clic en el botón Crear.

Debería ver un mensaje que confirma que se ha creado un nuevo tema y una nueva suscripción.

Publicación de votos en [Con un tema](#)

creado, el siguiente paso es añadir la publicación de mensajes. Esto se realizará en el controlador del método /vote del módulo votingfrontend.py .

/

Actualmente, el método de voto simplemente comprueba la presencia de un campo de solicitud llamado voteAct y acepta su valor como un ID de acto. Añadiremos una validación al método como comprobación redundante para cualquier validación del lado del cliente. Si el ID de acto no es válido, se generará una respuesta de error.

Aquí es donde entra en juego Pub/Sub. Una vez que tenemos un ID de acto válido y un usuario válido, lo cual obtenemos del código de autenticación presentado en el laboratorio de código del capítulo anterior, tenemos toda la información necesaria para registrar una votación. Para ello, el frontend construirá un objeto JSON que encapsula la votación y lo publicará como un mensaje Pub/Sub en el tema de la votación.

Los votos se escriben en el tema de Pub/Sub y su aceptación se confirma mediante una respuesta exitosa de la biblioteca cliente de Pub/Sub.

Este proceso consta de dos pasos, que se muestran en los siguientes fragmentos de código de ejemplo. El código completo se incluirá en el laboratorio de código más adelante en este capítulo:

Primero, configuramos un publicador y una dirección o ruta de tema en nuestro código Python. Cuando se recibe un voto, creamos un objeto JSON de voto y lo publicamos como un mensaje a este tema: `# import JSON support`

```
importar json

# Importar la biblioteca cliente de Python para Pub/
Sub desde google.cloud import pubsub_v1

project_id = "votingforthestars". # El ID del proyecto que se muestra en la consola
de Cloud.

topic_id = "votos". # Este es el nombre usado al crear el tema

# Crear un editor
editor = pubsub_v1.PublisherClient()

# Crea una ruta al tema de votaciones
topic_path = publisher.topic_path(project_id, topic_id)

# Crea el objeto JSON
vote_json = {"vote": {"act": actId, "user": userName}}

# Los mensajes de Pub/Sub deben ser una cadena de bytes, por lo que
serializamos y codificamos
el objeto JSON vote_str =
json.dumps(vote_json) message_data = vote_str.encode("utf-8")

# Cuando publicas un mensaje, el cliente devuelve un futuro future
= publisher.publish(topic_path, message_data)
# El método de resultado se bloquea hasta que el mensaje se haya publicado
correctamente
o # genere una excepción
try:
    print(future.result())
except Exception as e:
    print("vote for: {} failed with: {}".format(vote_str, str(e)))
```

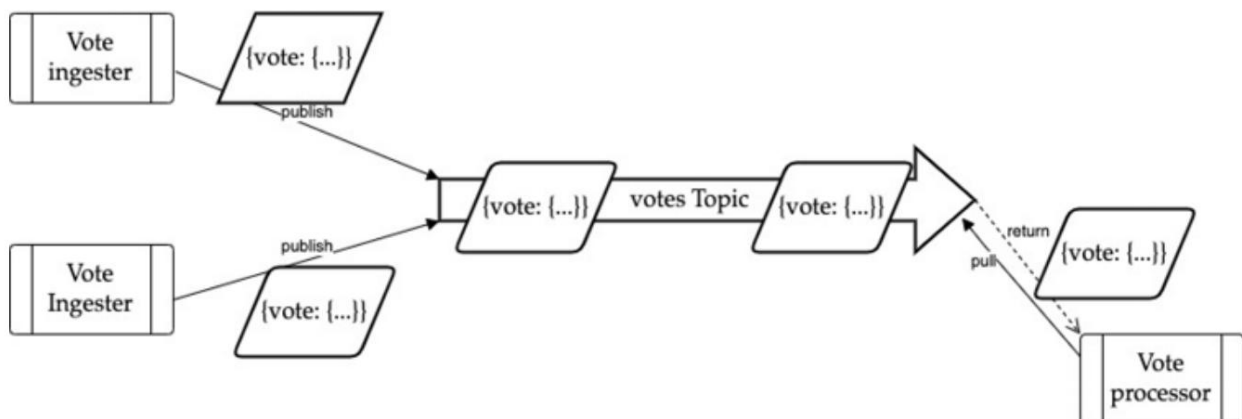
aumentar

Procesamiento asincrónico de solicitudes

Se completa el traslado de votos de acciones de usuario a mensajes en una cola
Una parte de nuestro objetivo: Hemos dissociado al usuario de la
procesamiento de votos. Para lograr nuestro objetivo, debemos implementar el
la parte restante del requisito es procesar efectivamente los votos.

Procesamiento de votos publicados en /

Recuerde la discusión sobre una cola de mensajes en [el Capítulo Elección](#),
Entre las Opciones de GCP, sección Manejo de mensajes asincrónicos.
Con base en esa descripción, la siguiente figura ilustra nuestra intención
Uso de Pub/Sub en la aplicación de votación:



Cifra **10.2** Uso de Pub/Sub en votación

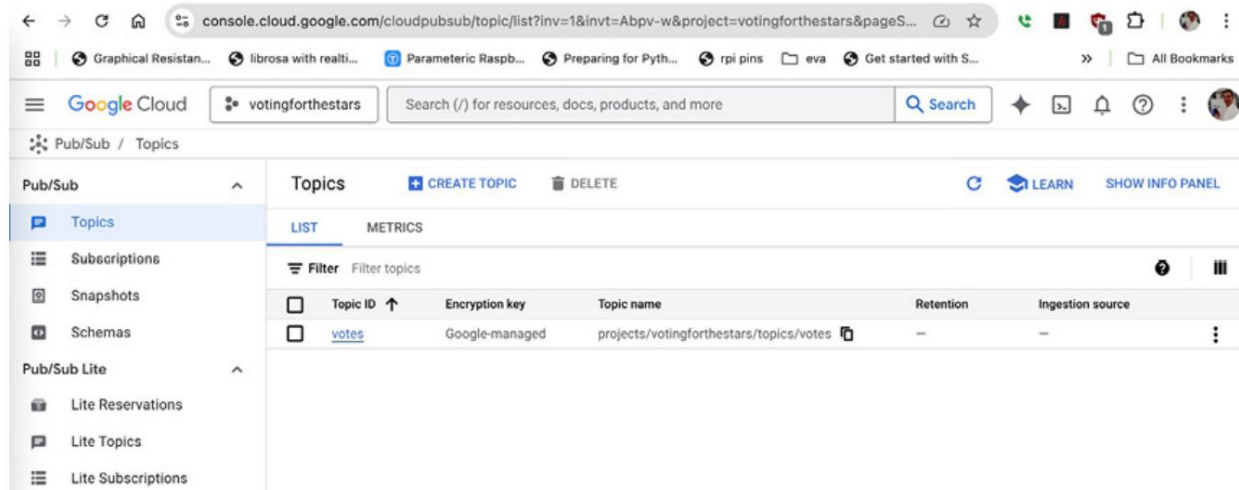
La función de Pub/Sub es desacoplar los requisitos de baja latencia de la
usuario de la operación de alta latencia de procesamiento (registro) de votos.
Pub/Sub para nuestros propósitos se materializa en el tema de votos en [la Figura 10.2](#)
con las operaciones de publicación y extracción que son la interfaz
entre los dos componentes desacoplados y el tema. Lo anterior
La sección ha descrito las opciones y la implementación del editor.
lado, ahora es el momento de examinar el lado del suscriptor, representado por el
Operación de extracción en [10.2](#) .
[la Figura](#) Los mensajes sobre un tema se reciben a través de una suscripción a ese tema.

Pub/Sub ofrece tres tipos de suscripciones:

- Suscripciones de extracción, en las que un suscriptor llama al tema para extraer los siguientes mensajes de la cola. Los mensajes permanecen en la cola hasta que se extraen explícitamente.
- Las suscripciones push son similares a las notificaciones push, que se describieron en el Capítulo 7, Frontends en el borde del cliente, en la sección "Manejadores de solicitudes del lado del cliente". Pub/Sub llama asincrónicamente a un suscriptor push, que envía un mensaje del tema cuando se publica.
- Las suscripciones de exportación reenvían automáticamente los mensajes publicados en su tema a una tabla de BigQuery o a un depósito de Cloud Storage.

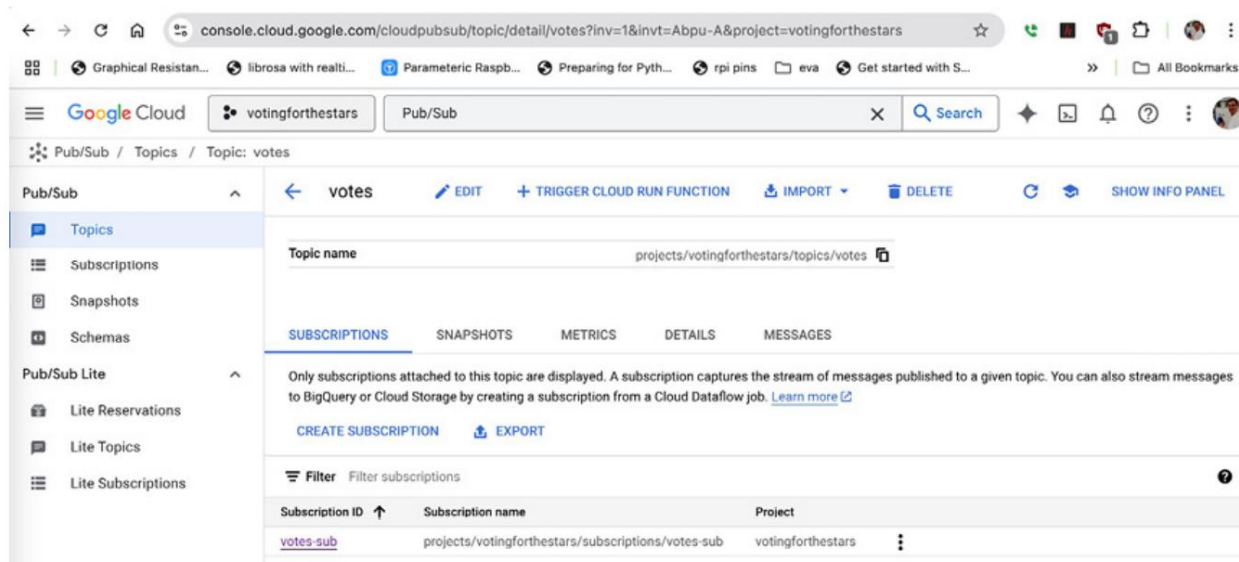
Cada uno de estos tiene sus propios usos y ventajas. En nuestro caso específico, ya hemos abordado el requisito de baja latencia en nuestro diseño, y esta parte de nuestra arquitectura se centra en la fiabilidad y la permanencia. Utilizaremos una suscripción push, que se adapta perfectamente a nuestro caso de uso: un servicio de Cloud Run que gestiona picos de tráfico intenso con largos periodos de inactividad. La razón de esta selección es que las suscripciones push permiten escalar automáticamente las instancias de Cloud Run para satisfacer la demanda, mientras que una suscripción pull tendría que gestionar su propio escalado de procesadores y requeriría que un trabajador estuviera siempre activo para sondear el tema durante los periodos de inactividad.

Configuraremos los recursos de Pub/Sub descritos para esto en la consola de desarrollador de Cloud. Un tema de Pub/Sub y, a continuación, una suscripción push predeterminada, que enviará automáticamente los mensajes publicados del tema a nuestro servicio Cloud Run como una solicitud HTTP POST. Los resúmenes de tema y suscripción en la consola de Google Cloud se muestran en las dos figuras siguientes:



Cifra 10.3 Pub/Subtema en la consola de Google Cloud

Consulte la siguiente figura para obtener más detalles:



Cifra 10.4 Pub/Suscripción en la consola de Google Cloud

Cuando el suscriptor detecta una publicación negativa reconocida (NACK) mensaje sobre el tema, coloca el mensaje en una lista pendiente estado y lo envía como el cuerpo de una solicitud POST al punto final de inserción definido para la suscripción.

Una instancia de un servidor HTTP en el punto final de inserción definido

Recibir el mensaje enviado por la suscripción. El protocolo HTTP

El código de estado devuelto por nuestro controlador le dice a la suscripción si

el mensaje es ACK o debe considerarse no procesado y reenviarse en una solicitud POST posterior.

En el siguiente fragmento de código se muestra una implementación mínima de este punto final de inserción. Los detalles completos y el código se incluyen en el laboratorio de código más adelante en este

```
capítulo: @app.route("/", methods=["POST"])
def index():
    """Recibe y analiza un mensaje de Pub/Sub que debe contener
    JSON válido."""
    mensaje = solicitud.datos

    print("¡Mensaje entrante!") if
    message:
        print("data: {}".format(message)) try:

    vote = json.loads(message)
    print("vote: {}".format(vote)) except
    ValueError as e: print("JSON
    no válido recibido: {}".format(e)) return ("JSON no
    válido", 400) except Exception as
    e: print("error: {}".format(e))
    return ("", 500)

    retorno ("", 204)
```

[Escalado del procesamiento backend](#). Lo que

hemos mostrado son el diseño general y los detalles de implementación de un sistema de votación que proporciona baja latencia al usuario y un almacenamiento transitorio de alta durabilidad de sus votos. El procesamiento real de los votos, considerado una tarea backend, puede realizarse sin necesidad de una baja latencia percibida por el usuario. La votación.

El procesador no está exento de consideraciones de latencia; simplemente no está vinculado directamente con ningún requisito del usuario o del cliente.

Algunas consideraciones que afectan el requisito de latencia del procesador de votos se basan en las limitaciones de la infraestructura y otras en los requisitos del usuario, además del acuse de recibo inmediato de un voto emitido. A continuación, enumeraremos algunas de estas consideraciones:

- Hay un número máximo de mensajes que un tema está configurado para contener al mismo tiempo.
- Un mensaje tiene un tiempo de vida máximo (TTL) que puede permanecer en un tema antes de ser consumido por un suscriptor.
- El público espera que los votos se cuenten y que los resultados de una ronda se anuncien dentro de un cierto período de tiempo después de concluida la votación de los actos de esa ronda.

Debemos satisfacer cada uno de estos requisitos; de lo contrario, el sistema o la empresa fracasarán. Un fallo del sistema erosionará el valor de la empresa y, con interrupciones repetidas, la empresa fracasará. Por lo tanto, es importante contar con un plan que garantice que el sistema cumpla con todos estos requisitos lo más cerca posible de la realidad.

Exploremos las consecuencias de no cumplir cada uno de los tres requisitos enumerados anteriormente:

- Si se dejan mensajes en el tema demasiado tiempo o se acumulan demasiados, los votos que confirmamos como registrados se descartarán. En ese caso, no hemos cumplido nuestro contrato con la comunidad de usuarios de contabilizar todos los votos. Tampoco hemos cumplido la garantía que dimos a cada usuario al emitir su voto individual, de que este se registró.
- Si tardamos cuatro días en anunciar a los ganadores de una ronda cuando habíamos prometido anunciarlos en dos días, hemos perdido credibilidad ante la audiencia. Cruzando otro umbral con consecuencias más graves, si tardamos una semana en determinar los resultados de una ronda, no podremos seleccionar a los participantes del siguiente nivel de votación si este comienza una semana después de la ronda cuya...

Los resultados se retrasaron.

Ninguna de las situaciones anteriores debería presentarse y podemos diseñar nuestro sistema para evitarlas. También podemos adherirnos a las mejores prácticas de confiabilidad del sitio e implementar un sistema de monitoreo para detectar y notificarnos cuando el estado del sistema presente alguna tendencia hacia alguno de los problemas mencionados, antes de que lleguemos a un punto crítico y podamos mitigar la situación.

La falla que podría llevar a cualquiera de estos escenarios es cuando nuestra tasa de procesamiento de votos es insuficiente para procesar los votos pendientes en el período de tiempo requerido.

[Aplicación del escalado de GCP](#). Existen

maneras de controlar la tasa de procesamiento de votos, la más directa de las cuales es aumentar el número de procesadores de votos que procesan los votos del tema. Este es un escalado horizontal simple, con varias ventajas que analizaremos a continuación.

El escalamiento horizontal es fácil de configurar, observar y controlar en GCP.

La mayoría de los productos computacionales de GCP se pueden configurar para aumentar o disminuir automáticamente la cantidad de trabajadores que se ejecutan en un servicio en respuesta a la demanda actual.

Pub/Sub admite y funciona bien con múltiples instancias de suscriptores push simultáneos que reciben mensajes de temas.

El efecto del escalamiento horizontal en el rendimiento o la tasa de procesamiento de votos es predecible y prácticamente lineal. Si un procesador de votos puede procesar 25 votos por minuto, dos procesadores de votos pueden procesar aproximadamente 50 votos por minuto, pero no exactamente. Aproximadamente, debido a que habrá cierta sobrecarga interna en Pub/Sub, al gestionar la semántica de bloqueo y el balanceo de carga entre varios clientes.

Otra forma de aumentar el rendimiento es mediante el escalamiento vertical, incrementando la velocidad de procesamiento de cada procesador de votos. Esto se puede lograr aumentando los recursos, como la memoria y los núcleos de CPU, utilizados para ejecutar cada servidor. Esta es una buena opción cuando los trabajadores que se escalan están limitados por alguno de estos recursos. La existencia de dicho recurso limitante se puede determinar mediante el perfilado de un trabajador cuando está...

Bajo carga. La creación de perfiles con una herramienta como Cloud Profiler puede revelar qué recurso, si lo hay, es el cuello de botella subyacente; por ejemplo, si el proceso ocupa toda la memoria disponible o utiliza el 100 % de la capacidad de sus núcleos de CPU. GCP se puede configurar para escalar verticalmente automáticamente las instancias del servidor al alcanzar ciertos umbrales de utilización de estos recursos.

Una ventaja del escalamiento vertical es que la cantidad de conexiones a recursos compartidos, como una tabla de base de datos de destino, permanece constante, a diferencia del escalamiento horizontal.

Con un diseño más complejo, ambas técnicas de escalado podrían emplearse simultáneamente. Por ejemplo, cuando la utilización de CPU en los trabajadores es alta, sus recursos de CPU pueden aumentarse hasta cierto límite para que puedan satisfacer la demanda. Si el grupo actual de trabajadores no logra mantener el ritmo de publicación de mensajes, incluso con núcleos de CPU y memoria adicionales, se pueden crear nuevas instancias para asumir la carga adicional. Esta técnica no es necesaria en la mayoría de los casos y, si se utiliza, traslada gran parte de la lógica de escalado de la configuración de GCP al código específico de la aplicación. Es mejor evitar la complejidad innecesaria y confiar en las funciones existentes y probadas siempre que sea posible.

Implementación del procesador de votos Los

requisitos funcionales del servidor backend del procesador de votos se han descrito con un nivel de detalle suficiente para que podamos usarlos para formar un conjunto de requisitos técnicos.

En resumen, algunos de los requisitos clave son: El

- servicio no debe ser accesible directamente desde la internet pública ni desde ningún punto final externo a nuestra aplicación. Esto se debe a un principio de buenas prácticas de ciberseguridad: el principio de mínima visibilidad. Este principio limita la visibilidad de datos y operaciones sensibles para minimizar la superficie potencial de un sistema vulnerable a ataques. Para implementar este principio de la forma más completa posible, dado que no es necesario que ningún punto final externo acceda al procesador de votos,

Ni siquiera lo hacen posible. Hacer que el servicio sea invisible para los endpoints externos no afecta nuestros casos de uso previstos, por lo que cualquier acceso potencial que hayamos eliminado sería involuntario o malicioso.

- El servicio debería escalar horizontalmente para adaptarse a los picos de demanda, seguidos de largas pausas de actividad. A medida que aumenta la demanda, se deben asignar automáticamente más recursos del sistema, y a medida que disminuye, se deben liberar automáticamente los recursos infrautilizados.
- El servicio Procesador de Votos no depende de servicios externos fuera de nuestra aplicación. Las únicas dependencias que hemos identificado son los Procesadores de Votos hermanos y nuestros servicios de persistencia de datos, todos ellos servicios de GCP dentro de la misma aplicación.

Usaremos Cloud Run para implementar el procesador de votos. Este puede conectarse a nuestros servicios internos, realizar un escalado horizontal automático de instancias bajo carga e interoperar con Pub/Sub y cualquier otro servicio de GCP.

[Monitoreo del rendimiento del backend](#) Además de escalar

recursos, se debe emplear monitoreo y notificación siempre que la demanda tienda hacia alguna situación crítica.

Por ejemplo, si el número máximo de mensajes pendientes en el tema está configurado en 10 000, sería útil una alerta cuando el recuento de mensajes supere el 85 % durante cinco minutos para que los operadores del sistema puedan verificar que el sistema funciona correctamente y que el escalado automático se activa y es eficaz. También es recomendable activar una alerta adicional si el escalado automático no logra reducir la acumulación de mensajes, por ejemplo, si el número de mensajes pendientes en el tema alcanza el 95 % o más de nuestro límite. Confiamos en que el escalado automático y el diseño de nuestra aplicación funcionen correctamente, pero también debemos estar preparados para lo peor.

Las revisiones periódicas del rendimiento histórico también son una buena práctica. Una revisión semanal de la demanda, el escalado automático y la utilización de los recursos del sistema revelará si las suposiciones sobre el rendimiento siguen siendo correctas.

Por ejemplo, si el escalado automático inicia más instancias nuevas que...
Como era de esperar, los trabajadores podrían estar hambrientos de CPU o de alguna otra fuente.
recurso, y podría ser eficaz aumentar la asignación de CPU a
aumentar la tasa de procesamiento de cada trabajador.

Envío de solicitudes de usuario de forma asincrónica a través de Laboratorio de código de

El laboratorio de código de este capítulo definirá la implementación de Vote
Interfaz de Ingestor y Procesador de Votos. El recuento de votos será...
se describe e implementa más adelante en [el Capítulo 11](#) Garantizar los datos
Permanencia .

En el frontend, este laboratorio de código agrega código para publicar votos válidos.
Pub/Sub, incorporando el éxito o el fracaso de esa operación en el
Respuesta del cliente.

El procesador de votos recibirá votos del tema Pub/Sub y
invocar un método de procesamiento de código auxiliar que se ampliará en
[Capítulo 11](#), Garantizar la permanencia de los datos El éxito o el fracaso
de ese método determinará si el voto es ACK y por lo tanto eliminado
del tema o dejado en el tema para recuperación y procesamiento repetidos.

Componentes de procesamiento de votos asincrónicos

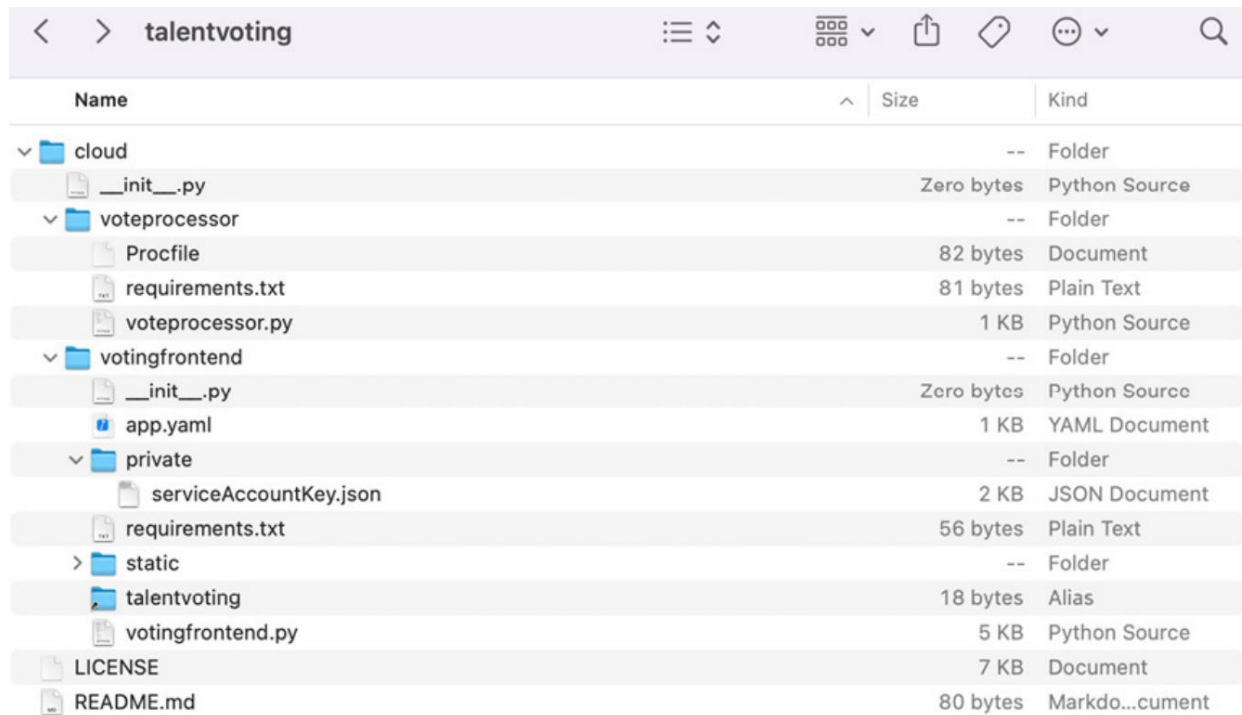
Los siguientes componentes de tiempo de ejecución se implementarán en este

laboratorio de código:

- **votingfrontend**: El servicio que se implementó en el anterior
[Capítulo 9](#) Identificando los puntos de tensión de nuestro sistema de votación ,
se puede ampliar para publicar votos en Pub/Sub e indicar el
éxito de esa operación en su respuesta al cliente.
- **RegionalVoteProcessor**: Este es un servicio que implementaremos usando
Cloud Run. Está escrito en Python. La implementación inicial...
recibir mensajes del tema y reconocerlos como procesados si
Se pueden analizar como JSON válido. Registrar el éxito y
El fallo de cada mensaje nos permitirá verificar posteriormente el Voto

Comportamiento del procesador.

- **Tema/de** Definimos el tema de votos anteriormente en este capítulo. Este tema es el conducto entre los servicios frontend y backend. Configuraremos el número máximo de mensajes pendientes y el tiempo de vida máximo de los mensajes.
- **Suscripción** una suscripción push para el tema de votos, llamada `votes_sub`, se creó anteriormente en este capítulo y se revisará aquí:



Name	Size	Kind
cloud	--	Folder
__init__.py	Zero bytes	Python Source
voteprocessor	--	Folder
Procfile	82 bytes	Document
requirements.txt	81 bytes	Plain Text
voteprocessor.py	1 KB	Python Source
votingfrontend	--	Folder
__init__.py	Zero bytes	Python Source
app.yaml	1 KB	YAML Document
private	--	Folder
serviceAccountKey.json	2 KB	JSON Document
requirements.txt	56 bytes	Plain Text
static	--	Folder
talentvoting	18 bytes	Alias
votingfrontend.py	5 KB	Python Source
LICENSE	7 KB	Document
README.md	80 bytes	Markdo...cument

Cifra 10.5 Estructura de carpetas y archivos de los paquetes en la nube

- **Servicio Cloud Run:** Se implementará un servicio Cloud Run llamado `vote_processor_service` en Cloud Run desde fuente.

[Directorios de nivel superior con nuevo código en la nube.](#) No hay archivos

nuevos ni eliminados en el paquete `votingfrontend`, pero el paquete en la nube ahora tiene un segundo subdirectorio: `voteprocessor`. Revisaremos el contenido de estos archivos más adelante.

laboratorio de código.

Adiciones de código al Gestor de Votos. El Gestor de Votos, implementado en el módulo `votingfrontend`, cuenta con una función llamada `vote` que analiza la solicitud del cliente, devuelve respuestas de error para varios posibles problemas y, a continuación, simplemente reproduce los detalles de la votación solicitada al cliente. Extenderemos este método y añadiremos un nuevo método auxiliar para publicar la votación en Pub/Sub.

El siguiente código contiene extractos del módulo `votingfrontend.py` actualizado. Hay pequeñas refactorizaciones y cambios de estilo en todo el módulo, pero destacaremos las partes relevantes para la funcionalidad de este capítulo en el siguiente código:

```
import json
import sys
```

```
from talentvoting.common.acts import Act, Acts
```

```
from talentvoting.common.policy.votingpolicyengine import VotingPolicyEngine
```

```
from talentvoting.common.interfaces.voteingester import VoteIngester
```

```
from talentvoting.common.interfaces.responses import FrontendError, IneligibleVote, InvalidUser, InvalidLogin, MalformedRequest, VoteCastError
```

```
from talentvoting.common.interfaces.servicelocations import VOTE_WEB_CLIENT_DOMAIN, VOTE_QUEUE_TOPIC, PROJECT_ID
```

```
from google.cloud import pubsub_v1
```

```
motor_de_políticas = VotingPolicyEngine()
```

```
editor = pubsub_v1.PublisherClient()
```

```
# Crea una ruta al tema de votaciones
```

```
topic_path = publisher.topic_path(PROJECT_ID,
```


TEMA DE LA COLA DE VOTACIÓN)

def _getActs() ->Actos:

devolver _policy_engine.getAllActs()

excepto auth.InvalidIdTokenError:

generar InvalidLogin(str(id_token))

def _recordVote(act:Act) ->cualquiera:

prueba: log("_recordVote({})".format(act))

Los datos del mensaje deben ser una

cadena de bytes message_data =

act.encode("utf-8") future = publisher.publish(topic_path,

_ message_data) = future.result()

excepto Excepción como

e: logError(e, "_recordVote()")

raise VoteCastError(str(act))

@app.route('/vote', métodos=['POST']) def

vote():

formulario =

solicitud.formulario uid = __validateUser(formulario)

prueba: actId = form['votedAct']

vote = {"usuario": uid, "act": actId}

vote = json.dumps(vote)

```

    log("vote({}).format(vote))
    _recordVote(vote)
    respuesta = make_response(vote, 200)
    _fixResponseHeaders(respuesta)
    log("datos de respuesta: {}".format(str(response.get_data())))
    log("encabezados de respuesta: {}".format(str(response.headers)))
    devolver

respuesta excepto BadRequestKeyError:
    error = {"error": str(MalformedRequest("votedAct").response()[0])}
error
    = json.dumps(error)
    logError(error, "error:vote()")
    respuesta = make_response(error, error.response()[1])
    _fixResponseHeaders(respuesta)
    log("datos de respuesta: {}".format(str(response.get_data())))
    log("encabezados de respuesta: {}".format(str(response.headers)))
    devolver respuesta

excepto FrontendError como
    e: error = {"error" : str(e.response()[0])}
    error = json.dumps(error)
    logError(error, "error:vote()")
    response = make_response(error, e.response()[1])
    _fixResponseHeaders(response)
    log("response data: {}".format(str(response.get_data())))
    log("response headers: {}".format(str(response.headers)))
    return response

```

Primero, al principio del archivo, ahora hay código para importar e inicializar la biblioteca Pub/Sub y crear un PublisherClient que se usará con nuestro tema Pub/Sub.

La función de voto ahora invoca un nuevo método, `_recordVote`. También añadimos la autenticación de usuario al método de voto, de forma idéntica al código del método `getActs` definido en el laboratorio de código del capítulo anterior.

La nueva función `_recordVote` es donde se lleva a cabo el trabajo real.

Esto publica el JSON de la solicitud del cliente en un nuevo mensaje en el tema de Pub/Sub. Si se produce algún error, se registra para su posterior análisis forense y se genera una nueva excepción definida por la aplicación: `VoteCastError`.

Esta nueva clase de error se agrega al final del módulo `responses.py` y sigue el mismo patrón que otras clases de error en ese módulo:

```
class VoteCastError(FrontendError): def
    __init__(self, act):
        self.__act = act

    def respuesta(self) -> cualquiera:

        return ("error:voto no registrado. "+ "act:
            {}".format(str(self.__act)), 500)
```

La nueva clase de error devuelve al cliente un código de estado HTTP 500, lo que indica un error interno del servidor. Los detalles del fallo no se envían al cliente para evitar la ingeniería inversa y proteger contra posibles ataques maliciosos. Nuestros ingenieros accederán a los registros del servidor mediante la monitorización de GCP para ver el error exacto detectado.

También hemos añadido nuevas definiciones de endpoints al módulo `servicelocations.py`. A continuación, se presenta el contenido completo de dicho módulo.

```
_DIRECCIÓN DEL INGESTOR DE
VOTACIONES = "https://votingforthestars.ue.r.appspot.com"
```

```
URL_DE_VOTACIÓN = _DIRECCIÓN_DE_VOTACIÓN_INGESTOR + "/voto"
```

```
GETACTS_URL = _DIRECCIÓN_DE_REGISTRO_DE_VOTACIÓN + "/getacts"
```

```
DIRECCIÓN_DEL_PROCESADOR_DE_VOTACIONES
```

```
= "https://votingforthestars.run.app/processor"
```

```
VOTE_QUEUE_TOPIC = "votos"
```

```
DOMINIO_CLIENTE_WEB_DE_VOTACIÓN = "https://votingforthestars.web.app"
```

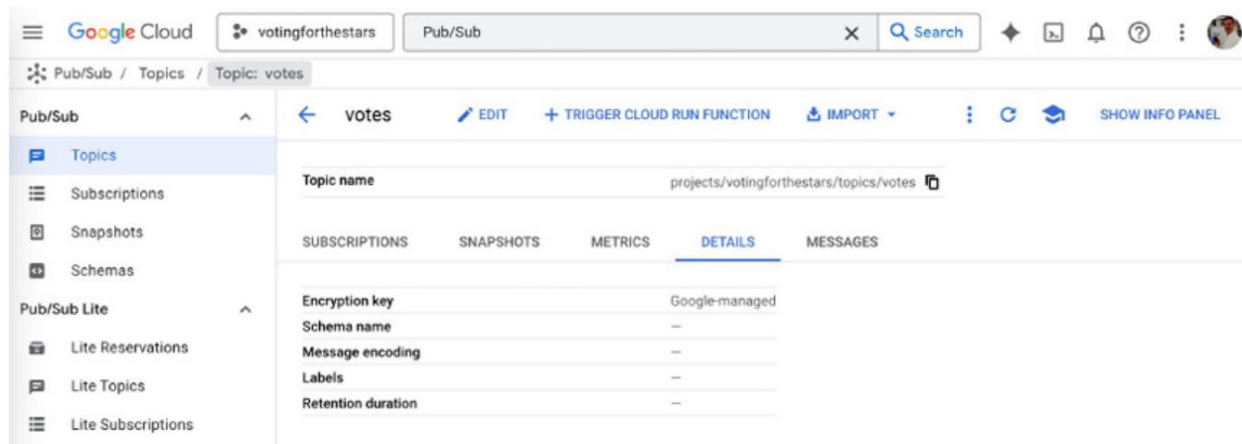
```
PROJECT_ID = "votandoporlasestrellas"
```

Configuración

Los

temas de Pub/Sub tienen varias opciones de configuración que deben entenderse al definir un tema, ya que influyen en su idoneidad para un patrón de uso específico. Para la mayoría de estas opciones, la configuración predeterminada es aplicable a la mayoría de las aplicaciones, mientras que otras se aplican a casos de uso específicos.

La siguiente figura muestra la pestaña DETALLES de la configuración del tema de votos en la consola de Google Cloud:



Cifra 10.6 Pub/Detalles del subtema

Los valores mostrados son los predeterminados al crear un tema. Las configuraciones más relevantes y que se modifican con más frecuencia son el nombre del esquema y la duración de la retención, cuyos valores predeterminados y su relevancia se detallarán a continuación.

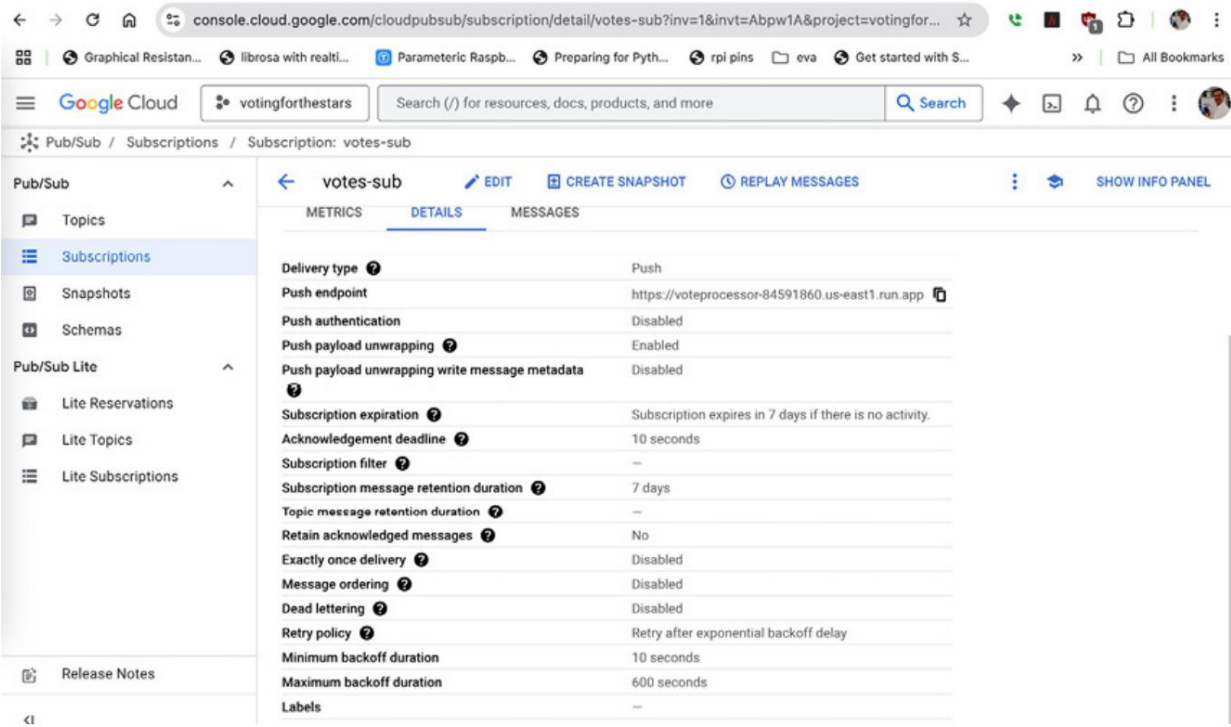
El esquema le permite definir un formato estricto que los editores de Pub/Sub

Se aplicará al publicar mensajes en el tema. Un esquema es una definición de registro que puede especificarse en formato JSON o de búfer de protocolo. La ventaja de aplicar un esquema es que garantiza que todos los mensajes tengan el formato esperado al ser consumidos desde un tema. Las desventajas de usar un esquema son el trabajo de configuración adicional y el coste de rendimiento, ya que cada operación de publicación debe invocar la lógica de validación para su mensaje. Para nuestra aplicación, con una sola fuente de mensajes y un alto requisito de latencia, la aplicación de un esquema no ofrece ninguna ventaja real.

La duración de retención es el tiempo máximo que un mensaje publicado se almacena en la cola de temas antes de ser potencialmente eliminado. Una vez publicado, un proceso lo consume regularmente: en tiempo real, periódico o bajo demanda. Dado que los mensajes pendientes ocupan espacio de almacenamiento, generan cargos. Si se permiten indefinidamente, los mensajes huérfanos seguirán acumulando espacio de disco y se les cobrará al propietario. Los mensajes de acuse de recibo (ACK) suelen eliminarse de inmediato, pero esta configuración define el tiempo máximo que un mensaje se almacenará en el tema, independientemente de si se ha entregado al suscriptor y al ACK.

El periodo de retención predeterminado es de 7 días, diseñado para los mensajes que se consumen en tiempo real o mediante flujos de procesamiento por lotes diarios o semanales. La duración de la retención puede variar entre 10 minutos y 31 días.

Además de un tema, se necesita una suscripción para consumir mensajes de Pub/Sub. La siguiente figura muestra la pestaña DETALLES de la configuración de la suscripción de votos-sub en la consola de Google Cloud:



Cifra 10.7 Pub/Detalles de la suscripción

Las configuraciones más relevantes para nuestra aplicación son Tipo de entrega, Punto final de push, Desenrollado de carga útil de push, Retener mensajes reconocidos, Política de reintentos y Letra muerta.

Un tipo de entrega Push especifica que la publicación de un mensaje activará una acción por parte de esta suscripción de Pub/Sub. La suscripción emitirá una solicitud POST al punto final de Push con los datos del mensaje.

El punto final de inserción es la URL donde se implementa el servicio Vote Processor Cloud Run. Esto se mostrará con el comando `gcloud` al implementar el servicio. Por ahora, puede introducir cualquier valor y editar la suscripción más tarde, o puede esperar a crear el tema y la suscripción hasta que haya implementado el servicio.

La descompresión de la carga útil de push indica a la suscripción que envíe los datos sin procesar del mensaje como cuerpo de la solicitud POST. Si esta opción está deshabilitada, el mensaje se encapsula en un objeto JSON específico de Pub/Sub con varios campos de metadatos. Esta opción está deshabilitada porque los datos del mensaje son el único contenido que necesitamos en este momento.

Un valor de No para los mensajes reconocidos de retención dirige a Pub/Sub a

Eliminar mensajes del tema en cuanto un suscriptor los haya leído y confirmar su entrega. Nuestro procesador de votos solo confirmará un mensaje una vez procesado correctamente y no tengamos ningún otro uso para ellos, por lo que lo configuramos como no para liberar el almacenamiento de mensajes lo antes posible.

La política de reintento controla cómo la suscripción gestiona los mensajes que no reciben confirmación de recepción (ACK) en su punto final. El suscriptor puede intentar reenviar un mensaje inmediatamente después de una respuesta de error o un tiempo de espera, o puede esperar un retraso exponencial. Esto último es útil para recuperarse de un problema transitorio que probablemente se solucione con el tiempo, como la indisponibilidad temporal de un servicio externo. La opción de mensajes no confirmados (DACK) de la política de reintento especifica una cola de mensajes alternativa a la que se transfieren los mensajes que permanecen como NACK después de un umbral especificado.

Código del procesador de votos. El

controlador de mensajes para nuestra suscripción push es un fragmento de código relativamente corto. La mayor parte del trabajo específico de Pub/Sub lo realiza la suscripción, dejando solo un punto final HTTP REST básico por implementar. El código completo para nuestro procesador de votos basado en Cloud Run es el siguiente: `import json from flask import Flask, request`

```
app =
```

```
Flask(__name__)
```

```
def __processVote(mensaje:json) -> bool:
```

Procesar un mensaje JSON que contiene un voto.

args:

mensaje: Un diccionario JSON

Devoluciones:

Verdadero si se procesó un mensaje, falso en caso contrario

```

"""
si no hay mensaje:
    devuelve Falso

si no es instancia(mensaje, dict): devuelve
    Falso

Si "usuario" no está en el mensaje o "acto" no está en el mensaje:
    devuelve Falso

devuelve Verdadero

@app.route("/", methods=["POST"]) def index():
    """Recibe un
        mensaje Pub/Sub que debe contener JSON válido
    y emitir una respuesta HTTP.""" mensaje
    = solicitud.datos

    print("¡Mensaje entrante!") if mensaje:
    print("datos:
        {}".format(mensaje)) try: voto =

    json.loads(mensaje) if _processVote(voto):
        print("Voto procesado:
            {}".format(voto)) else:

        print("Voto inválido: {}".format(voto)) return
        ("Voto inválido", 400) # Estado de error de contenido inválido excepto
    ValueError como e: print("JSON
        inválido recibido: {}".format(e)) return ("JSON inválido",
        400) excepto Exception como e:
    print("error: {}".format(e))

```



```
return ("", 500) # Estado de error interno del servidor
```

```
return ("", 204) # Sin contenido Estado OK
```

Esta aplicación básica de Flask tiene un controlador, la ruta raíz `"/"`. La solicitud se valida como que contiene JSON usando la función `json.loads()`, que generará una excepción `ValueError` si se encuentra JSON no válido.

Si el cuerpo de la solicitud contiene un objeto JSON, este se pasa a la función `_processVote`. Esta función garantiza que el JSON contenga los elementos esperados para un mensaje de voto y devuelve "Verdadero"; de lo contrario, "Falso". En función de este valor de retorno, el controlador confirma el mensaje con un código de estado "OK" o rechaza la entrega del mensaje con un estado de error.

En el próximo capítulo, la función `_processVote` se ampliará para contar votos, pero para el propósito del procesamiento asíncrono mediante Pub/Sub, esta función cuenta con todas las características.

Configuración e implementación de Cloud Run. Para implementar el servicio Vote Processor en Cloud Run, usaremos inicialmente una función llamada "implementar desde el código fuente". Esta función permite especificar el contenido del servicio Cloud Run con algunos parámetros de configuración y que el servicio `gcloud` build cree un contenedor Docker a partir de una imagen preexistente que incluye el paquete generado a partir de los archivos fuente y las especificaciones de configuración.

La configuración de nuestro servicio Cloud Run está contenida en dos archivos, `requirements.txt` y `.Profile`.

El archivo de requisitos tiene la misma finalidad que el utilizado para el servicio de frontend de votación de Google App Engine. Enumera los paquetes que se instalarán en el entorno de ejecución del servicio. El contenido de nuestro archivo `requirements.txt` se muestra a continuación: `gunicorn==23.0.0`

```
Flask==3.0.3
```

```
Werkzeug==3.0.3
```

```
google-cloud-pubsub
```

```
google-api-core
```

El segundo archivo, Procfile, especifica cómo se iniciará el servicio cuando Cloud Run haya terminado de configurar el entorno de ejecución y esté listo para que comience la ejecución. El contenido de nuestro Procfile es el siguiente:

```
web: gunicorn --bind :$PORT --workers 1 --threads 8 --timeout 0 voteprocessor:app
```

Estos dos archivos y el

archivo fuente de nuestro servicio, voteprocessor.py, son todo lo que se necesita para implementar un servicio Cloud Run desde la fuente.

En el directorio donde se encuentran estos archivos, el siguiente comando implementa el servicio: `gcloud run`

`deploy --allow-unauthenticated --source .`

El parámetro `--source .` especifica que se trata de una implementación de origen del directorio actual, denominado `.`

El comando confirmará el nombre del servicio y solicitará una región.

Una vez hecho esto, se invocará el servicio Cloud Build para crear un contenedor e implementarlo. Se mostrará la URL de los registros del servicio Cloud Build, donde se podrán consultar los mensajes de estado y error de la compilación.

Al finalizar una implementación, se muestra la URL del servicio implementado. Esta es la URL que se usará como punto final de la suscripción push, mencionada en la sección anterior "Configuración de PubSub " de este laboratorio de código.

Configuración del escalado automático del procesador de votos: Los

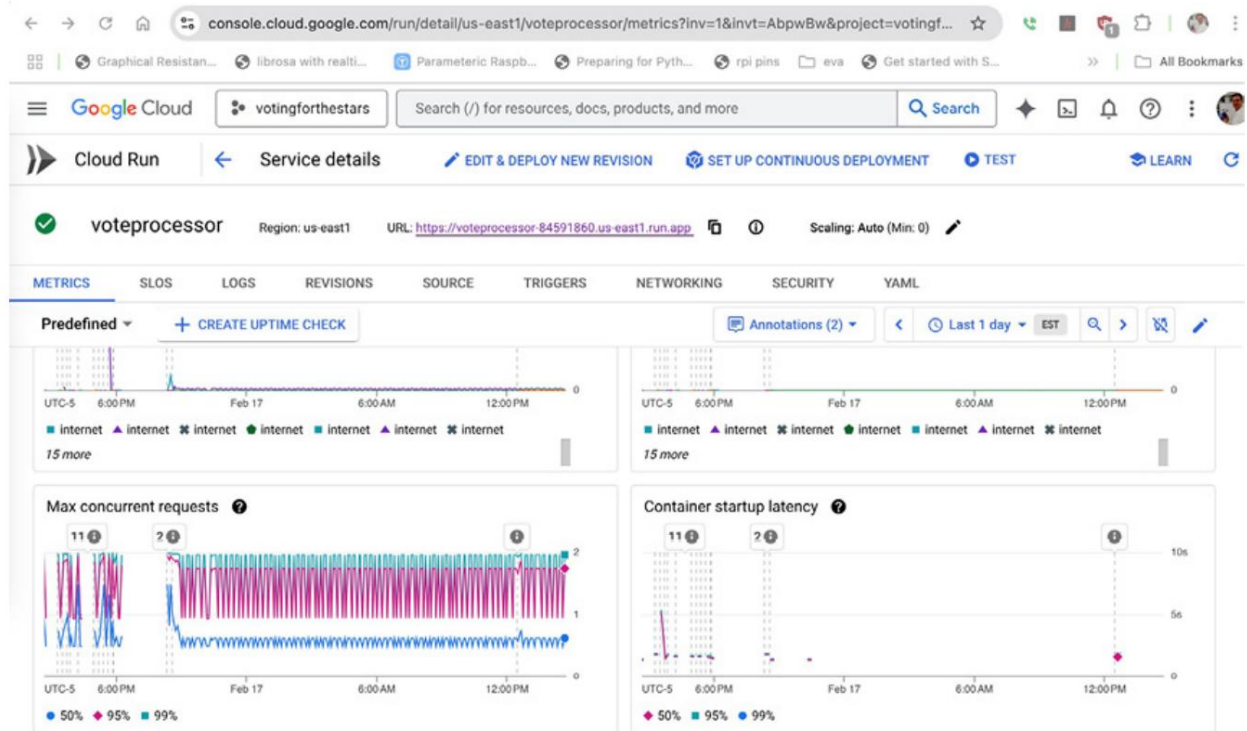
servicios implementados de Cloud Run están configurados para permitir el escalado automático horizontal de forma predeterminada. Al usar una suscripción push para enviar mensajes a un servicio de Cloud Run, este iniciará nuevas instancias si no hay ninguna disponible para recibir un mensaje. Cuando una instancia permanece inactiva durante un tiempo, Cloud Run la reducirá automáticamente; es decir, la cerrará para ahorrar recursos y costos.

Para evitar costos elevados inesperados durante picos de tráfico, existe un número máximo de instancias en las que se iniciará el escalado automático. De forma predeterminada, el número máximo de instancias es 100, pero se puede anular cuando

implementando una revisión del servicio agregando el indicador `--max-
instances=n` al comando `gcloud run deploy`, donde `n` es un entero entre 1 y el número permitido por su región y sus configuraciones de CPU y memoria.

Es importante tener en cuenta que cada solicitud concurrente no requiere su instancia del servicio. Los servicios de Cloud Run envían múltiples solicitudes concurrentes a cada instancia y solo inician nuevas instancias cuando el uso de CPU de las instancias existentes es alto o se alcanza el límite de solicitudes concurrentes por instancia. De forma predeterminada, Cloud Run establece un límite de 80 solicitudes concurrentes por instancia. Este valor se puede configurar con el indicador `--concurrent=n` del comando `gcloud run deploy`, donde `n` es un número entero entre 1 y 1000. Se recomienda comenzar con los valores predeterminados y supervisar el uso de CPU y memoria de las instancias de servicio, aumentando el límite si el uso de estos recursos por parte de las instancias se mantiene bajo durante la carga. La supervisión del uso de los recursos de las instancias se describe en la siguiente sección.

Monitoreo del uso de recursos del servicio. Los paneles proporcionados automáticamente para un servicio de Cloud Run muestran todas las métricas necesarias para verificar el aprovisionamiento de las instancias para gestionar su demanda. En la consola de Cloud, navegue a Cloud Run y seleccione el servicio (en nuestro caso, `voteprocessor`), cuyo nombre se proporcionó con el comando `"gcloud run deploy"`. En el panel que se abre, al desplazarse hacia abajo, se mostrarán gráficos de utilización de CPU y memoria, como se muestra en la siguiente figura:

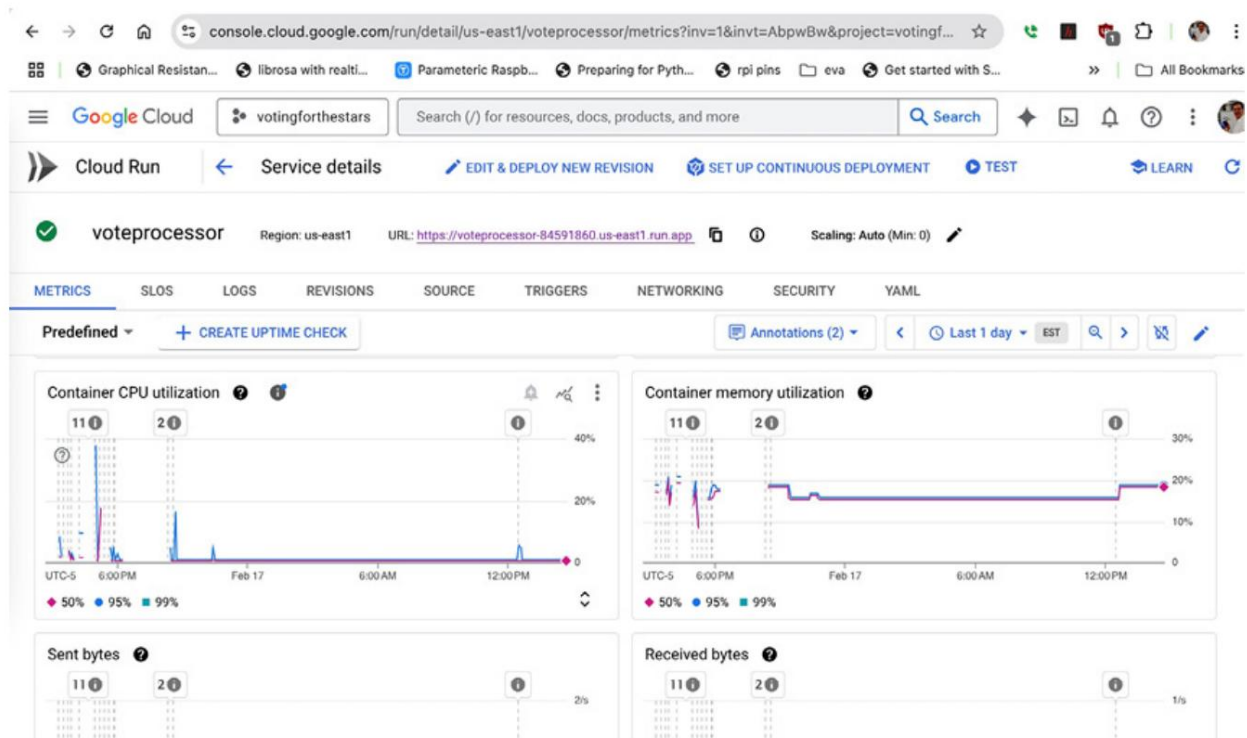


Cifra **10.8** Gráficos de recursos de instancias de ejecución en la nube

Esto demuestra que nuestra única instancia siempre utilizó menos del 20% de ambos Recursos de CPU y memoria durante la ventana mostrada. Idealmente, esto Se comprobará si hay una ventana que incluyó una gran carga de solicitudes.

El número de solicitudes simultáneas que manejó cada uno

La instancia se puede confirmar desplazándose para revelar un gráfico adicional en el mismo tablero, que se muestra en la siguiente figura:



Cifra **10.9** Gráficos de recursos de instancias de ejecución en la nube

Podemos ver que en la ventana mostrada, nunca hubo más de dos solicitudes simultáneas para nuestra instancia. Se pueden seleccionar diferentes rangos de tiempo hasta que se detecte un período de mayor carga. Una vez detectado un período de alta demanda, utilice los gráficos de recursos para medir los recursos utilizados por una instancia y proyectar cuántas solicitudes simultáneas podría atender.

Monitoreo de la acumulación de temas de

Además de los paneles, GCP ofrece alertas que cumplen una función complementaria. Un panel es una excelente manera de procesar una gran cantidad de información en intervalos de tiempo arbitrarios. Las alertas, por otro lado, proporcionan una sola pieza de información, un único punto de datos, en un momento específico: ahora mismo.

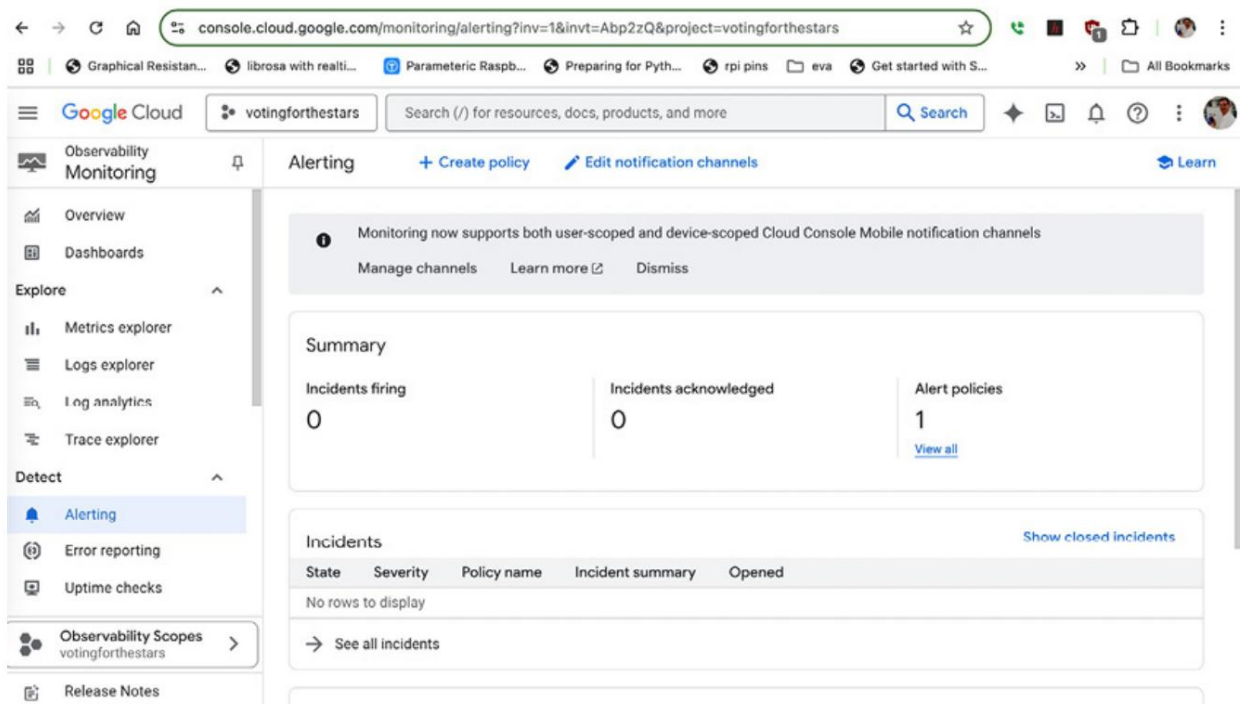
El uso típico es configurar alertas para condiciones que se pueden expresar como una sola declaración, como que hay más de 8500 votos en cola en el tema que es verdadero en este momento. La alerta debe ser una condición que requiera acción, y una respuesta recomendada es

Ver un panel relevante para ver el panorama general y comprender por qué se produjo la condición que activó la alerta.

Para configurar alertas se requieren dos definiciones: un canal de notificación (dónde enviar la alerta) y una política (cuál es la condición y la métrica que la activan).

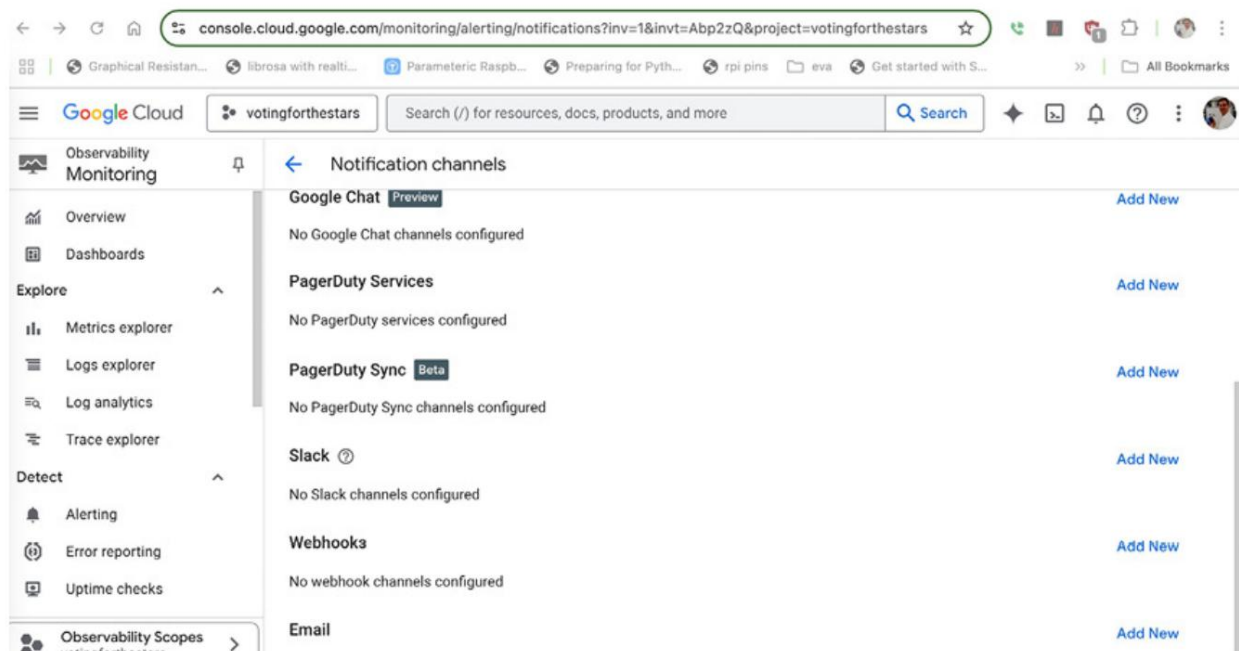
Primero, configure un canal con los siguientes pasos:

1. Desde la consola de Google Cloud | Observabilidad | Monitoreo, haga clic en Alertas en la sección Detectar de la barra de navegación izquierda, como se muestra en la siguiente figura:



Cifra **10.10** Página principal de alertas

2. Haga clic en Editar canales de notificación, lo que mostrará la página que se muestra en la siguiente figura:



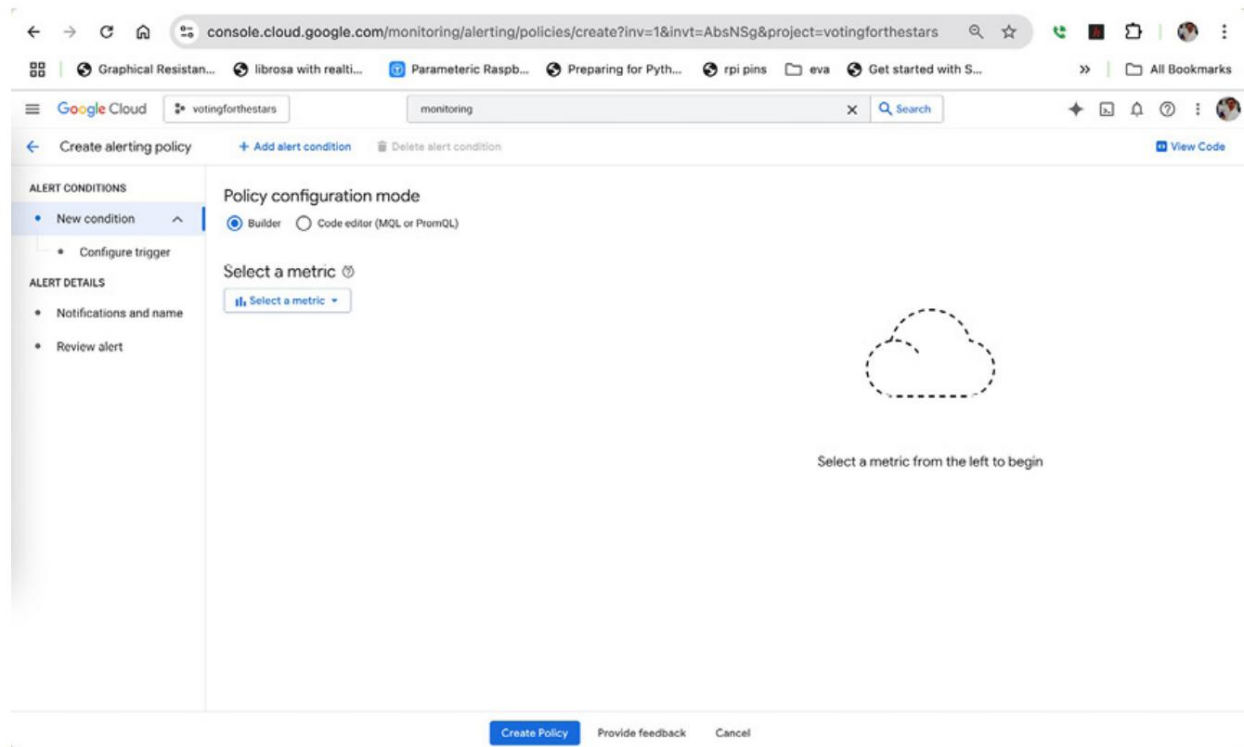
Cifra **10.11** Página de canales de notificación

Puedes agregar tantos canales de tantos tipos como desees.

El correo electrónico, los SMS y Slack son los más comunes según el preferencia de la parte notificada.

Se define al menos un canal para que puedas definir una alerta política. Esto normalmente implica tres componentes: una métrica, una recurso y un umbral.

3. En la página de Monitoreo, que se muestra en [la Figura 10.10](#), haga clic en Crear política que mostrará la página que se muestra en la siguiente figura:



Cifra 10.12 Página de política de alertas

4. Seleccione Builder como modo de configuración de política.
5. Primero, debemos especificar una métrica. Haga clic en el menú desplegable "Seleccionar una métrica" y seleccione Cloud Pub | Suscripción | Suscripción. A continuación, seleccione el elemento del submenú: Mensajes sin confirmar.
6. Haga clic en el botón Aplicar.
7. Ahora, restringimos la alerta a nuestro tema de votación. Esto se hace para preservar la integridad de esta alerta si añadimos otros temas al proyecto en el futuro.
8. Haga clic en "Agregar filtro", seleccione subscription_id = votes-sub en los controles y haga clic en "Listo". La expresión del filtro debería aparecer debajo del botón "Agregar filtros".
9. Haga clic en Siguiente.
10. Para definir el disparador de alerta, seleccione e ingrese estas configuraciones:
 - a. Tipo de condición = Umbral b.
 - Activador de alerta = Cualquier serie temporal que viole
 - c. Posición del umbral = Por encima del umbral

d. Valor umbral = **9999**

11. Haga clic en Siguiente.

12. Ahora en el paso final seleccionamos los canales de notificación que se utilizarán cuando se active esta alerta.

Seleccione al menos uno de los canales de notificación definidos e introduzca un asunto. A continuación, introduzca un nombre para la política de alerta, como "Gran cola de votos pendientes".

13. Finalmente haga clic en Crear política.

Se mostrará un gráfico que indica cuántas veces se activó esta alerta a lo largo del tiempo; actualmente ninguna.

[Monitoreo de mensajes antiguos](#) / [Otra alerta que vale la pena](#)

configurar es para los mensajes que permanecen NACK durante un tiempo excesivo. Un mensaje puede estar en este estado debido a un retraso en el procesamiento, un fallo transitorio en el procesador de votos o un problema con el mensaje en sí. Recuerde que nuestro procesador de votos aplica NACK a todos los mensajes con un formato de voto no válido, lo que los deja en el tema para posteriores intentos de reenvío. Esto es intencional, ya que un mensaje de este tipo indicaría un error en la interfaz de votación o que alguna otra fuente está publicando mensajes en el tema de la votación. Esta alerta se utilizará para llamar nuestra atención sobre el asunto.

Si un mensaje ha recibido un NACK tras varios reintentos, es probable que se deba a la tercera razón: un mensaje inválido o corrupto. Al recibir esta alerta, una buena solución es consultar los registros del servicio del Procesador de Votos para encontrar el mensaje que acompaña al mensaje rechazado.

El nombre de la métrica para esta política de alerta se define en las suscripciones, no en los temas, y se llama `earliest_unacked_message_age`. Utilice un valor de umbral de dos minutos, que equivale a 120 000 milisegundos, la unidad utilizada en la métrica.

Tras la investigación y el análisis del problema, se debe retirar el mensaje problemático y confirmarlo o eliminarlo. Existe una función de Pub/Sub que resulta útil en esta situación, llamada mensaje muerto.

que podría agregarse a la suscripción de votos si así se desea.

Monitoreo de la latencia del frontend. La tercera alerta

que se debe configurar se activa si la latencia del frontend de votación es alta, es decir, si tarda más de un tiempo especificado en devolver las respuestas a los usuarios. Un buen punto de partida para una latencia aceptable del frontend es de 300 milisegundos. La configuraremos de la misma manera que las políticas de alerta para Pub/Sub.

Realice las siguientes selecciones y entradas en el cuadro de diálogo del generador

- de alertas: Seleccione la métrica Aplicación GAE | Http | Latencia de respuesta.
- No se debe especificar ningún filtro.
- Establezca el disparador para que sea un valor superior a 300.

Responder a notificaciones. Cuando se activa

una alerta, se abre un incidente. Este es un sistema de seguimiento de problemas útil para supervisar la resolución, incluyendo la asignación de tareas a diferentes personas. Por ahora, se recomienda simplemente marcar un incidente como cerrado cuando corresponda. La gestión de incidentes es muy específica de cada equipo y producto, y la capacitación y el establecimiento de un protocolo deben completarse antes de implementar cualquier sistema crítico para el negocio.

Conclusión

Este capítulo exploró diversos aspectos del escalado de una aplicación para anticipar y atender la demanda de los usuarios, en cuanto a capacidad de respuesta y fiabilidad. Utilizando la sección de votación de nuestro sistema, se guió al lector en el ejercicio de modelado de la demanda. Con estas previsiones de carga del sistema, se describieron las opciones arquitectónicas que minimizan la parte del sistema que debe escalar según la demanda de los usuarios.

Se describieron técnicas de equilibrio de carga y esquemas de implementación para nuestra comunidad global.

El capítulo se centró luego en el uso de un procesamiento asíncrono.

Modelo para separar las respuestas de los usuarios de grandes porciones del procesamiento de votos. Se describió Pub/Sub en detalle y, a continuación, se identificaron las características y opciones específicas aplicables a nuestro caso de uso.

El laboratorio de código para esta sección desarrolló el conducto o canalización Pub/Sub desde el frontend de la aplicación hasta los procesadores de votación del backend.

Se agregó el monitoreo de puntos críticos del desempeño de la infraestructura, junto con el monitoreo de la capacidad de respuesta del usuario final.

En el siguiente capítulo, se analizará e implementará el procesamiento backend para almacenar los votos. Se compararán varios productos de GCP para determinar su idoneidad, junto con técnicas para sincronizar el trabajo entre regiones.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



CAPÍTULO 11

Garantizar la permanencia de los datos

Introducción

En este capítulo, se explorará e implementará el registro y la reproducción constantes y permanentes de la actividad del usuario. La integridad de los datos es la característica más importante de un sistema para preservar la confianza del usuario, y una falla en la misma es la forma más segura y duradera de perderla. La integridad de los datos tiene una breve definición: lo que escribes es lo que lees. Tras establecer la integridad de los datos, se requiere la disponibilidad de los datos, que se puede resumir en la siguiente declaración: puedes leer lo que escribiste cuando quieras.

Este capítulo describe los productos de almacenamiento de datos que se incluyen en GCP y los tipos de información que cada uno está especializado en contener.

A continuación, se describe el sistema de almacenamiento que se utilizará en el sistema de votación y las consideraciones que se tienen en cuenta para su selección. Una vez identificada la tecnología, se describen las características de fiabilidad de los datos de los sistemas de almacenamiento seleccionados y los modos de fallo que previenen y mitigan.

El laboratorio de código de este capítulo definirá los servicios de almacenamiento utilizados para persistir los votos y la actividad del usuario, y el código utilizado para implementar funciones de persistencia en el backend del sistema de votación, acompañado por código en el cliente web para reflejar la persistencia de la actividad del usuario.

La monitorización de estos servicios en términos de confiabilidad y rendimiento también se describirá y definirá en el laboratorio de código.

Estructura

En este capítulo se tratarán los siguientes temas:

- Tipos de datos
- Bases de datos relacionales y no relacionales
- Objetos binarios grandes
- Patrones de acceso
- Garantizar la disponibilidad de los datos
- Objetivos de permanencia de instantáneas y archivos
- Laboratorio de código sobre persistencia de votos y actividad electoral

Objetivos

El lector explorará los diferentes tipos de datos, aspectos de los datos que son independientes del contenido informativo. Estos se modelan según la forma de los datos, no su significado.

La idoneidad de ciertos servicios de almacenamiento para alojar un almacén de datos determinado se basa en estos aspectos estructurales de los datos, en lugar de basarse en la información que representan.

Tras estas facetas estáticas de los datos, los aspectos dinámicos, como el uso real en la práctica, desempeñan un papel fundamental a la hora de determinar el servicio de almacenamiento más adecuado para alojar datos. La frecuencia de acceso, lectura y escritura se puede examinar y categorizar en patrones de acceso típicos. Este capítulo demostrará esta categorización y guiará al lector en su aplicación.

El capítulo guiará al lector a través de la consideración de la necesidad de la permanencia de los datos y los diferentes escenarios a los que este término puede referirse. Se identificarán y explicarán las prácticas para garantizar cada uno de ellos.

El laboratorio de código del capítulo ilustrará la aplicación práctica de

Los principios analizados en el capítulo se basan en herramientas y funciones específicas de Google Cloud Platform (GCP) . El laboratorio de código se basará en la aplicación desarrollada en laboratorios de código anteriores, añadiendo actualizaciones a las votaciones y la actividad de los usuarios en el sistema.

Tipos de datos

Además de su función, los datos tienen forma. Definamos estos dos términos de uso frecuente en el contexto de los datos en un sistema de procesamiento de información.

La función de los datos es el contenido informativo puro, el significado que representan o pretenden transmitir. La temperatura actual en Londres, los asistentes a una conferencia y la imagen de la Mona Lisa son ejemplos de la función o contenido informativo.

Nota: No se mencionó la forma o representación como Celsius, lista o JPEG. , , .

La forma o estructura de los datos son los bits físicos y las estructuras utilizadas para representar el contenido de la información. Aquí es donde se encuentran los detalles específicos de la implementación, utilizando términos como un número decimal establecido en grados Celsius de la temperatura actual y un archivo JPEG a color de alta resolución de 24 10 bits de una pintura.

La forma de los datos es cómo se representan, mientras que la función o el contenido es lo que se representa.

La forma de los datos suele ser lo que guía la selección de un servicio y método de almacenamiento. Nos centramos en dos aspectos de la forma de los datos: su grado de estructura interna y la profundidad o niveles de dependencia y abstracción interna de los datos.

Datos estructurados y no estructurados. Alguna informació

se adhiere a un formato común y predefinido en muchas instancias. Dichos datos se organizan y organizan en componentes identificables y nombrables. Un ejemplo de esto es una fecha en el calendario gregoriano. Una fecha es un elemento de información, pero siempre consta de tres componentes que podemos nombrar: el año, el mes y el...

día del mes. Puedes examinar una fecha anterior al uso social.
de electricidad: 3 de abrilrd, 1832, y una fecha en el futuro lejano: 18 de febrero ,
2433, y el formato y el proceso para interpretar la información
sigue siendo el mismo

Una gran fortaleza de los datos estructurados es la previsibilidad en muchos aspectos.
dimensiones. Para cada fecha, siempre ocupará la misma cantidad de
espacio, se necesitará aproximadamente la misma cantidad de trabajo para interpretarlo
y el proceso y la importancia o cantidad de información que es
codificado es el mismo

Pasemos ahora a discutir los datos no estructurados.

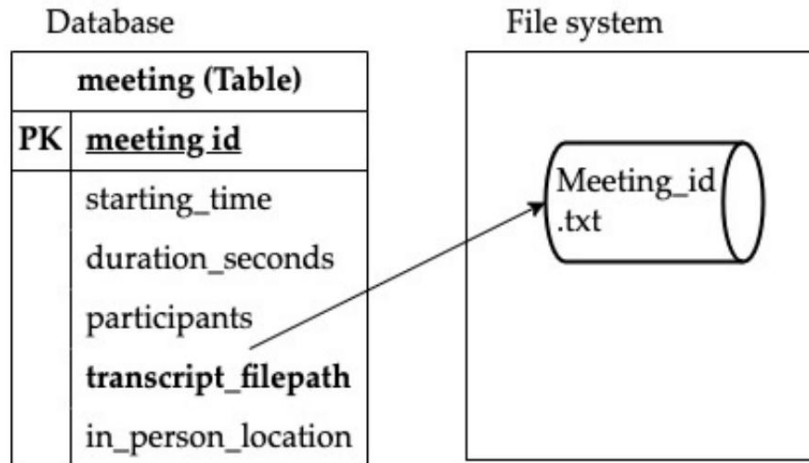
Los datos no estructurados son algo que es difícil de descomponer en
Partes constituyentes consistentemente relevantes. Una transcripción de voz audible
es un buen ejemplo. Los tres siguientes no son sorprendentes.

Ejemplos de discurso transcrito:

- ¿Qué quieres comer? SorpréndemeNo quiero elegir por
· Bueno, no importa lo que hagas, estaré,feliz de tenerlo.
Mientras esté contigo ·
- veo un árbol en el camino adelante !
- "bla, bla, bla"

Además de contener palabras, que es el elemento básico del habla,
Estos tres tienen poco en común. La longitud del texto, el número
de subdivisiones, oraciones y la cantidad de información contenida
varía mucho entre los tres ejemplos. Además, hay
inferencias que se pueden hacer, como la participación de dos
hablantes en la primera transcripción que no se puede hacer fácilmente sin
análisis del texto completo de la transcripción.

Ciertos tipos de sistemas de almacenamiento están optimizados para almacenar datos estructurados.
datos, mientras que otros son ideales para almacenar datos no estructurados.
La diferencia entre ambos es tan significativa que cuando la información
Contiene componentes estructurados y no estructurados, un componente común
La solución es guardarlos en sistemas de almacenamiento separados con una referencia
(enlace) a la ubicación de los datos no estructurados almacenados como parte de la
Datos estructurados. La siguiente figura ilustra un esquema de este tipo:



Cifra 11.1 Un esquema de almacenamiento de dos sistemas

Este es un caso común, que vale la pena describir en detalle ya que ocurre con suficiente frecuencia como para que los proveedores de bases de datos hayan agregado características para respaldarlo.

El caso de uso general es el de un conjunto de campos de datos que describen una reunión, algunos de los cuales se pueden definir como columnas de base de datos y otros no. En este caso, se trata de la transcripción de la base de datos. La transcripción es un único campo de texto de tamaño arbitrario; podría incluir las cuatro palabras "Esta reunión se cancela" o el texto de las intervenciones de 10 participantes en una cumbre de 3 horas. Esto es ideal para un archivo de texto. Los archivos están optimizados para almacenar eficientemente datos de tamaño muy variable. Los sistemas de archivos proporcionan acceso directo y eficiente a archivos individuales a través de su nombre de archivo. Por lo tanto, es sencillo almacenar el nombre de archivo de la transcripción de una reunión como proxy de la transcripción real y luego resolver una referencia adicional para abrir el archivo ubicado en ese nombre. Algunos sistemas de bases de datos ofrecen un tipo de dato de objeto binario grande (Blob) que se implementa mediante diferentes técnicas, algunas de las cuales son similares al archivo y la referencia separados descritos anteriormente. Cada enfoque tiene sus ventajas y desventajas: el uso de tipos de datos Blob simplifica el diseño de la aplicación, y la gestión de archivos almacenados por separado permite mayor libertad en cuanto al tamaño de los archivos y el uso del disco.

Datos [profundos](#) vs. [datos planos](#)

Un tipo específico y único de estructura que los datos pueden contener es la estructura profunda. Los datos profundos son aquellos que contienen una jerarquía interna o capas de datos. Los datos planos son aquellos en los que todos sus componentes tienen el mismo nivel de significancia y visibilidad. El ejemplo de fecha utilizado anteriormente en esta sección es un ejemplo de datos planos. Los tres componentes de una fecha (año, mes y día) tienen la misma visibilidad y el valor de uno de estos campos no está contenido dentro de otro. Se pueden determinar los días junio, 3 y 1976 independientemente uno del otro.

Un ejemplo ilustrativo de datos profundos es el de un departamento o equipo de una organización. Los datos departamentales pueden organizarse en estructuras anidadas; cada parte de un departamento tiene subcomponentes. Un departamento tiene, por ejemplo, un número arbitrario de miembros; cada miembro del equipo tiene un ID de empleado, un nombre y una fecha de inicio.

La siguiente figura ilustra una representación típica de esta estructura de datos. Esto no implica una implementación específica para cada plataforma, pero se utilizará para determinar la idoneidad de cada servicio de almacenamiento para almacenar estos datos:

department	
PK	<u>dept_id</u>
	manager +employee_id +email_address +name cost_center dept_email_address in_person_location team_members[] +employee_id +email_address +name

Cifra 11.2 Una definición profunda de un elemento de datos

Los elementos clave que indican que estos datos se clasifican como profundos son la presencia de una matriz, el conjunto de empleados denominados colectivamente `team_members` y la existencia de subcampos para los campos `manager` y `team_members`. Cada uno de estos datos no es un valor de empleado atómico, sino un conjunto compuesto de valores que, si bien tienen un significado independiente, contribuyen a la especificación completa de un empleado.

Definiciones de datos de votación. Hay

dos conjuntos de datos a los que la aplicación debe acceder en la función de votación que estamos desarrollando. El recuento de votos para cada acto es el más obvio, pero hay otro implícito en las reglas de la elección. Cada usuario tiene un presupuesto de votos que puede emitir en cada ronda y no debe excederlo. Una restricción adicional es que cada usuario solo puede votar una vez por un acto específico en cada ronda. Para ello, tendremos que mantener un registro duradero del número de votos emitidos por cada usuario durante la ronda.

permanece abierto para votación. Ya sea que mantengamos o no estos recuentos fuera La elegibilidad de cada ronda es irrelevante para la competición.

La siguiente figura muestra los datos que almacenaremos para las votaciones de actos:

Votes	
PK	<u>round_id</u>
PK	<u>act_number</u>
	voting_opened_at
	voting_closed_at
	total

Cifra **11.3** Datos almacenados de la votación de la Ley

La clave para una votación es un identificador de dos partes: un Act_ID que es el Identificador de la ronda, concatenado con el número de acto dentro de esa ronda. Este es un identificador único para el rendimiento, que es lo que

Se está emitiendo el voto. El mismo intérprete en diferentes rondas puede tienen diferentes números de acto, la asociación de artistas y su acto

Las identificaciones se realizan en otra parte del sistema y no son relevantes para la votación. proceso. Cabe destacar que esta clave compuesta se ha visto

Anteriormente, en el módulo acts.py incluido en el laboratorio de código del [capítulo 9](#), "Identificar los puntos de estrés de nuestro sistema de

votación", se escriben votos con mucha más frecuencia de la que se leen. Un ejemplo popular...

Cada espectador de una ronda determinada puede votar por un acto, por lo que un voto puede potencialmente se actualizará tantas veces. Aquí es donde el uso de

El procesamiento asíncrono por parte del procesador de votos muestra sus beneficios.

Las actualizaciones se pueden distribuir en un período de tiempo arbitrario, en lugar de... que intentarlo en el estrecho período durante el cual los espectadores proyectan sus votos.

La siguiente figura muestra la definición de datos para el seguimiento de la votación de los usuarios. actividad:

votebudget	
PK	<u>round_id</u>
PK	<u>user_id</u>
	last_voted_at
	total_votes_cast
	voted_acts[12]

Cifra **11.4** Datos presupuestarios de votación de usuarios elegibles

Estos datos se utilizan para rastrear la actividad de votación de cada usuario en una ronda específica y para aplicar su presupuesto de votación. El acceso a los almacenes de datos de presupuesto de votación y votos está estrechamente vinculado y sigue la siguiente secuencia de operaciones:

- Cada voto debe comprobarse primero con el presupuesto de votación del usuario.
- Si el presupuesto permite esta votación, el Votebudget se actualiza para incluir esta votación.
- Si se permitió la votación, el total de votos para esta ronda y acto se incrementa.

Las tres operaciones anteriores deben realizarse en una transacción. Si falla la actualización de Votes o Votebudget, se debe revertir cualquier actualización del otro almacén de datos.

Los presupuestos de votación se leen con la misma frecuencia con la que se actualizan. Si bien existe una posible actualización de un presupuesto de votación por cada posible actualización de votos, las operaciones en este presupuesto se distribuyen en muchos más registros o filas: cada usuario accede a su propio presupuesto de votación, mientras que todas las actualizaciones de votos convergen en un total de solo 12 registros.

Dado que existen diferentes ámbitos de concurrencia y bloqueo entre los dos almacenes de datos, minimizar el tiempo empleado en la transacción debe ser un objetivo fundamental de nuestro diseño. Esta optimización explica la presencia de `total_votes_cast` y del campo de matriz fijo `voted_acts`. Es más rápido comprobar el valor de `total_votes_cast` que calcular la longitud de una matriz de longitud variable de votos ya emitidos. En una optimización similar, dado que las reglas de show garantizan 12 actos por ronda, podemos usar una matriz directa.

acceso al elemento, indexado por número de acto, en el campo para verificar y establecer el estado de votación de un acto individual.

Bases de datos relacionales y no relacionales

Existen numerosos tipos de bases de datos disponibles para almacenar datos con distintos grados de estructura. Por tipo, no nos referimos a productos, sino a bases de datos que implementan un paradigma o modelo específico de organización y acceso a los datos. Las bases de datos más comunes y utilizadas son las que siguen un modelo relacional, conocidas como sistemas de gestión de bases de datos relacionales (SGBDR). Dado que estas bases de datos son la mayoría, describiremos qué diferencia a un SGBDR de las demás categorías, incluyendo qué hace que una base de datos relacional sea la mejor opción y qué aspectos de los datos justifican la investigación de alternativas.

En una base de datos relacional, los datos se organizan en una matriz: una matriz bidimensional de filas y columnas, similar a la estructura de una hoja de cálculo. Las columnas representan las declaraciones de los datos, los nombres de los campos, sus tipos de datos y su precisión. Las filas representan los valores reales de los campos asociados a la misma entidad. La matriz completa, las definiciones de las columnas y las filas de valores se denominan tabla.

La siguiente figura ilustra una pequeña tabla relacional de datos de empleados:

employee_id Integer NOT NULL	first_name String	last_name String NOT NULL	email_address String NOT NULL	start_date Date NOT NULL	end_date Date
103	Raymond	Blum	raymond@nosuch.com	09/18/2007	NULL
2	Tarana	Abdulla	taranaa@nosuch.com	03/16/2002	NULL
5	William	Wither	willw@nosuch.com	04/01/2002	05/30/2003

Cifra **11.5** Una mesa de empleados

La tabla anterior no ofrece ninguna característica distinta a la de un simple archivo de texto o una hoja de cálculo. Hasta el momento, no hay nada en la tabla que explique o justifique el término relacional.

La base de una base de datos relacional está en cómo se organizan los datos en una tabla.

relacionado con otros datos.

Agreguemos otro dato que deseamos almacenar: Las relaciones entre gerentes y subordinados en la organización. En nuestra empresa, un gerente puede tener cualquier número de subordinados directos y un empleado puede reportar a uno solo o a ninguno. Esta relación se describe como una relación de uno a muchos entre gerentes y sus subordinados directos, y una relación de uno a cero o de uno entre empleados y gerentes. Esto se puede codificar eficientemente en nuestra tabla de empleados añadiendo una columna que represente la relación. La siguiente figura muestra esta versión extendida de la tabla de empleados:

employee_id Integer NOT NULL	first_name String	last_name String NOT NULL	email_address String NOT NULL	start_date Date NOT NULL	end_date Date	manager_id Integer References(Employee.employee_id)
103	Raymond	Blum	raymond@nosuch.com	09/18/2007	NULL	2
2	Tarana	Abdulla	taranaa@nosuch.com	03/16/2002	NULL	NULL
5	William	Wither	willw@nosuch.com	04/01/2002	05/30/2003	2

Cifra **11.6** Una tabla de empleados relacionados

Esta relación es declarativa; la definimos para la columna `manager_id` y el sistema de base de datos la aplica con cada modificación de una fila en la tabla de empleados. El hecho de permitir valores nulos para `manager_id` permite el caso de uno a cero en el que un empleado no tiene gerente.

Se puede definir el mismo tipo de relaciones entre dos o más tablas, lo que permite que un esquema modele relaciones complejas directamente en las declaraciones de la base de datos.

Relacionar datos es algo en lo que una base de datos relacional se destaca específicamente. Existen otras estructuras y tipos de datos además de las relaciones, y es importante reconocer dónde otro tipo de base de datos es una opción más apropiada que una base de datos relacional.

Estructuras no relacionales. La compatibilidad

con campos repetitivos es una característica donde el modelo relacional no es el líder.

Consideremos el caso de una tabla de pruebas de motor donde cada fila representa una prueba de funcionamiento del motor de una motocicleta. Deseamos...

Registre las veces que se aplicó y luego se soltó el acelerador durante la prueba.

En el código, esto se declararía con una matriz como en el siguiente fragmento de C++: `typedef`

```
struct { std::time_t
    startTime; std::time_t
    endTime;
} Duración;

typedef struct
{ std::string driver;
    int max_speed;
    std::vector<Duration> throttles; }
engine_test; Esta
```

estructura de matriz es sencilla y eficiente en un lenguaje de programación, pero las columnas de una tabla relacional son escalares: solo contienen un valor. Para representar un conjunto de valores, como estos pares múltiples de tiempos de aceleración en una base de datos relacional, se requiere la introducción de una nueva tabla que contenga los valores de la matriz con una referencia a la clave de la fila de la prueba en la primera tabla.

Algunas bases de datos no relacionales, como Bigtable, admiten directamente campos repetidos en una columna, es decir, el almacenamiento de una matriz dentro de los datos de una columna. Este diseño es más simple y directo, y también más eficiente en tiempo de ejecución. Si sus datos contienen valores repetidos, una base de datos no relacional puede ser una opción más adecuada.

La búsqueda en grandes cantidades de texto no estructurado es una debilidad de las bases de datos relacionales y la especialidad de otro tipo de sistema de bases de datos: las bases de datos documentales como Firestore. Existen extensiones de proveedores y de terceros para varias bases de datos relacionales que solucionan esta deficiencia.

Existen otros tipos de bases de datos, cada una con sus propias ventajas.

Estos temas se discutieron con mayor amplitud en [el Capítulo Elección 4](#),

Entre las Opciones de GCP, en la sección Dónde almacenar sus datos, es posible que sus datos contengan elementos ideales para un tipo de base de datos y otros más adecuados para otro. En tal caso, puede optar por almacenar los datos en varios servicios o buscar una solución unificada que se adapte mejor a sus necesidades.

Modelos de almacenamiento para datos electorales.

Ahora que hemos descrito los tipos de almacenamiento de datos disponibles y algunas de las ventajas de cada uno, seleccionemos productos de almacenamiento para los dos almacenes de datos de este capítulo: Votos y Presupuesto Electoral. Recapitulemos las principales consideraciones discutidas anteriormente en esta sección sobre el acceso a estos dos almacenes de datos:

- Los votos están sujetos a actualizaciones únicas de muchas solicitudes separadas concurrentes.
- Los votos se identifican mediante una clave primaria compuesta.
- Un presupuesto electoral está sujeto a una lectura y potencialmente a una actualización.
- Se accede a muchos Votebudget simultáneamente.
- Un Votebudget se identifica mediante una clave primaria compuesta.
- Una transacción incluirá un presupuesto de votación y un voto.
- Los votos serán parte de muchas transacciones simultáneamente.
- Un presupuesto de votación generalmente será parte de una pero no más de seis transacciones simultáneamente.
- Las votaciones tienen alcance global. Es probable que las actualizaciones de una ley se produzcan en varias regiones simultáneamente, y todas deben integrarse en una única fuente de información de aplicación global.
- Los presupuestos electorales suelen ser locales. Si bien no se puede descartar, es improbable que un votante específico emita votos procesados en varias regiones.

Además, hay una consideración de diseño: Votebudget contiene un campo de matriz.

Considerando estos requisitos, seleccionamos Spanner como producto de almacenamiento para Votos y Votebudget. Spanner ofrece alta disponibilidad.

y garantías de consistencia eventual en bases de datos distribuidas. Spanner admite campos repetidos mediante el uso del tipo de datos matriz.

Si bien Bigtable parecería ser una opción ideal para Votebudget, no admite transacciones distribuidas en múltiples almacenes de datos.

Objetos binarios grandes

La palabra Blob es un sustantivo que describe la forma de los datos que suelen contener. Expliquemos cada uno de estos términos.

Un blob, como sustantivo, se define como una masa o gota de algo, o como algo informe. Esto concuerda con los datos almacenados en un blob, que pueden contener cualquier cosa, sin esperar estructura ni metadatos. Un blob puede almacenar datos de una imagen, una grabación de audio, un documento de Word o una hoja de cálculo. El sistema de almacenamiento de un blob no impone restricciones ni estructura. De esta manera, un blob es una masa de datos informe.

Blob, acrónimo:

- Binario: El contenido de un Blob se almacena como datos binarios, unos y ceros, sin requisito de que los datos estén codificados como Código estándar americano para el intercambio de información (ASCII), Unicode u otros formatos de datos de orden superior.
- Grande: Los blobs pueden contener grandes cantidades de datos; su tamaño es arbitrario. Grande no implica un requisito ni un tamaño mínimo, pero los datos pueden tener un tamaño indefinido y grande.
- Objeto: Los datos de un blob se consideran un solo objeto, a diferencia de una estructura, una matriz u otro conjunto definido de componentes. El blob es un único objeto y el sistema de almacenamiento no puede reconocer ni aislar sus subcomponentes.

Los sistemas de almacenamiento optimizados para el almacenamiento de blobs suelen basarse en sistemas de archivos. Ninguno de los siguientes requisitos es explícito para el almacenamiento de blobs, pero en la práctica son características de la mayoría de estos sistemas.

En un sistema de archivos, los blobs se denominan archivos. Los archivos se pueden direccionar directamente mediante nombres únicos. Cada archivo es independiente; se pueden crear nuevos archivos y los existentes se pueden modificar o eliminar sin afectar a los demás archivos del sistema.

El tamaño de un archivo no lo impone el sistema de archivos, pero suele haber un tamaño máximo que limita la implementación. Por ejemplo, la cantidad de bits utilizados para almacenar el tamaño de un archivo o la capacidad de los dispositivos compatibles en los que un sistema de archivos almacena datos de archivos.

El almacenamiento de blobs suele asociarse con aplicaciones que procesan datos internamente de forma compleja, como el análisis de texto, la manipulación de imágenes o la reproducción de audio. Esto se debe a que la cantidad de procesamiento de datos que realiza la infraestructura de almacenamiento y el código de la aplicación es complementaria. La razón por la que los sistemas de almacenamiento de blobs tienen pocos o ningún metadato sobre blobs es que la aplicación posee metadatos y procesamientos complejos y de gran tamaño. Por otro lado, una aplicación que utiliza un sistema de almacenamiento de datos estructurado, como una tabla de datos de empleados, requiere relativamente poco esfuerzo para interpretar los datos. El propio sistema de almacenamiento gestiona el análisis de la tabla en campos individuales, la garantía de las dependencias internas y la determinación de la cardinalidad de los datos, utilizando los metadatos codificados en las definiciones de tabla y columna.

El sistema de votación es, en gran medida, un sistema de contabilidad: los votos se acreditan a una actuación, se debitan de la cuenta del votante y, finalmente, todos los libros se concilian. El sistema se define por la estructura estática de sus datos, a la que se aplican operaciones atómicas muy simples. La aplicación no realiza operaciones complejas ni de alto consumo de recursos en datos muy específicos. En otras palabras, el sistema no cumple con ninguno de los requisitos de los blobs.

Patrones de acceso

Los patrones de acceso de un almacén de datos determinado son un factor importante en el diseño para cumplir con los requisitos de latencia. Los patrones de acceso son un...

Término usado, que en este contexto se refiere a la proporción y orden de lecturas y escrituras. Consideremos algunos ejemplos contrastantes: escriba un dato inmutable una sola vez, como un reconocimiento legal; lea este dato muchas veces, siempre que sea necesario para comprobar si una de las partes ha firmado un contrato.

Escribir datos transitorios o efímeros con frecuencia; leerlos muchas veces, sin estar vinculados a las operaciones de escritura. Un ejemplo es la caché del precio actual de las acciones, que se actualiza a medida que cambia el precio y se consulta constantemente para determinar el valor actual del símbolo.

El primer caso no tiene requisitos de latencia especiales ni en la operación de escritura ni en la de lectura, aparte de aquellos impuestos por los requisitos de rendimiento de las aplicaciones que acceden a los datos.

El segundo es más interesante: existe una necesidad crítica de baja latencia en la operación de escritura. Si un valor de seguridad cambia, esa información debe volverse canónica lo antes posible. Las operaciones de lectura no deberían tardar demasiado, pero el mayor impacto se produciría si se devolviera un precio obsoleto, por lo que la consideración crítica de latencia se centra en la escritura.

Al diseñar un almacén de datos, se deben considerar los patrones de acceso descritos anteriormente al elegir el servicio de persistencia que se utilizará y al diseñar el esquema físico. Algunos servicios pueden estar optimizados para escrituras rápidas o para muchas lecturas simultáneas. Algunos diseños de esquemas pueden estar optimizados para lecturas, cuando en realidad las escrituras son la operación más frecuente.

En el diseño de esquemas, la colocación de campos en un registro o fila se suele determinar mediante agrupaciones lógicas: los datos de una persona se ubican en una fila de la tabla Persona. Sin embargo, ¿qué ocurre si distintos subconjuntos de los campos se actualizan con frecuencia, mientras que otros suelen ser estáticos? Puede ser conveniente dividir el registro lógico en registros físicos separados para evitar bloquear y recargar filas grandes cuando solo cambia una cantidad relativamente pequeña de datos.

Garantizar la disponibilidad de los datos

La disponibilidad de datos es una medida de cuánto tiempo se puede leer la información dentro de lo que se considera un umbral de latencia aceptable.

Por ejemplo, el porcentaje de tiempo que la lectura de cualquier registro de Votos puede completarse en 50 milisegundos. Si los datos tardan demasiado en leerse, es prácticamente lo mismo que si no estuvieran disponibles.

Spanner incluye funciones integradas para aumentar la disponibilidad de datos en un sistema de aplicaciones distribuidas. El sistema Spanner escala los recursos de cómputo y almacenamiento en función del tamaño de los datos y de las solicitudes y patrones de acceso que se aplican a ellos.

Spanner crea automáticamente réplicas síncronas de una base de datos, eliminando la dependencia de cualquier recurso de la máquina. Además, al transferir las bases de datos de Spanner a varias regiones, Spanner utiliza esta redundancia para equilibrar la carga y permitir la conmutación por error de la base de datos.

La garantía de consistencia sólida de Spanner permite a nuestra aplicación utilizar actualizaciones transaccionales en almacenes de datos globales, como Votes, sin preocuparse por el orden de las operaciones ni por las actualizaciones de concurrencia entre réplicas. Spanner puede ordenar transacciones globales en tiempo real y realizar el bloqueo en un orden secuencial. Con esta garantía de un contexto transaccional global, la principal consideración de la aplicación es el rendimiento: minimizar el tiempo empleado en cada transacción de lectura y escritura. La gestión de estas transacciones y el bloqueo subyacente de los recursos se confía al servicio Spanner.

Objetivos de permanencia de instantáneas y archivos

La permanencia de los datos garantiza que lo leído sea igual a lo escrito. Existe otro aspecto de la permanencia: preservar el estado de un conjunto de datos en un momento específico. Ambas no son lo mismo, ya que la primera garantía implica que lo leído sea igual a lo escrito por última vez.

Existen dos técnicas para lograr la permanencia de los datos: instantáneas y archivos. Ambas se analizarán en esta sección, pero primero...

Describe una técnica más fundamental que se utiliza para garantizar el funcionamiento continuo del sistema: copias de seguridad y restauraciones.

Copias de seguridad para la recuperación

Todo sistema en funcionamiento debe tener un programa regular de copias de seguridad y un manual de recuperación probado.

La pérdida de datos es la interrupción más potencialmente catastrófica que una empresa puede experimentar. La mayoría de las interrupciones del servicio, como las relacionadas con la disponibilidad o la alta latencia, son transitorias y pasan por las siguientes etapas o pasos: 1. No se puede acceder al servicio

o tarda demasiado en responder.

2. Los usuarios están frustrados.

3. Se notifica a los operadores del servicio.

4. Se mitiga la interrupción.

5. El servicio se restablece.

6. Los usuarios reanudan el uso del servicio.

7. Las disculpas y garantías las emite el propietario del servicio.

La interrupción ha terminado y es probable que las empresas y los usuarios sigan cautelosos, pero a medida que pase el tiempo, la interrupción no tendrá un efecto duradero.

Esto no ocurre con una interrupción que provoque corrupción o pérdida de datos. Dicha interrupción tiene efectos secundarios que pueden durar años.

A continuación se muestran las etapas de uno de esos posibles escenarios:

1. El almacén de datos que contiene los documentos del usuario se ve afectado por un error de

software que vacía todos los archivos de usuario guardados antes del 1 de enero . 2. Algunos

usuarios encuentran que sus archivos recientes están vacíos y envían tickets de soporte.

3. Se informa a los operadores de servicios sobre el gran número de tales entradas.

4. Los administradores del sistema comprueban y encuentran muchos archivos de usuario vacíos.

5. Los desarrolladores son alertados e identifican el error del software.

6. Se implementa una solución para el error con prioridad urgente.

7. Se entrega al departamento legal de la empresa una lista de los archivos y usuarios afectados.

y personal de relaciones públicas.

8. Las disculpas y garantías las emite el propietario del servicio.

El error puede haberse solucionado, pero los archivos perdidos siguen desaparecidos. Las consecuencias de esta interrupción podrían extenderse durante años. ¿Qué sucedería si algunos de los archivos fueran necesarios para la declaración de impuestos o el cumplimiento legal?

Hay una forma de evitarlo: los operadores del sistema deben tener la capacidad de restaurar todos o parte de los datos. "Cualquiera" significa que se pueden restaurar elementos seleccionados, como una lista de archivos o filas de la base de datos. "Todo" significa que se puede restaurar el estado completo de un almacén de datos. Una operación de restauración debe poder completarse en un tiempo razonable, y no es aceptable informar a un usuario que sus correos electrónicos perdidos se restaurarán en su bandeja de entrada en dos semanas.

Restaurar datos es una función, y las copias de seguridad son una forma de habilitarla. No todas las restauraciones requieren copias de seguridad: por ejemplo, si un dato determinado se puede recrear de forma rápida y segura bajo demanda, hay menos necesidad de realizar una copia de seguridad. Esto es preferible a restaurar desde una copia de seguridad, ya que una restauración no recupera los datos tal como estaban cuando se escribieron por última vez, sino que solo los recupera tal como estaban cuando se realizó la última copia de seguridad.

Hay dos factores a tener en cuenta al planificar una estrategia de respaldo, conocidos como objetivos de punto de recuperación (RPO): cadencia de respaldo y retención de respaldo.

La cadencia de las copias de seguridad indica la frecuencia con la que se realizan. Esto, a su vez, determina la cantidad máxima de pérdida de datos que se puede experimentar. Si realizamos una copia de seguridad cada 12 horas y se produce una pérdida de datos 1 minuto antes de la siguiente, se perderán todos los datos escritos en las últimas 11 horas y 59 minutos, ya que no estaban presentes en la copia de seguridad más reciente.

La retención de copias de seguridad define el tiempo durante el cual las conservamos. Esto determina el tiempo que permitimos para detectar una pérdida de datos y para iniciar una restauración.

Supongamos que realizamos copias de seguridad de nuestros datos cada 12 horas y solo guardamos las dos últimas. Si se produce una pérdida de datos a las 3:00 a. m. del domingo y los usuarios la reportan a partir de las 8:00 a. m. del lunes, no hay nada que podamos restaurar. Las dos copias de seguridad que tenemos son copias de los datos que existían después de la pérdida. No guardamos copias de seguridad de más de un día.

Podríamos guardar las copias de seguridad durante más tiempo, quizás una semana. Sin embargo, esto...

Requeriría suficiente espacio para albergar 14 copias adicionales de todo nuestro almacén de datos y, además, ocuparía recursos del sistema, como memoria y ancho de banda, con esta copia constante de todo nuestro almacén de datos. Existe una manera de reducir ambos problemas: las copias de seguridad incrementales.

Para que el proceso de restauración sea oportuno y ocupe poco espacio, existen dos tipos de copias de seguridad: copias de seguridad completas y copias de seguridad incrementales.

Las copias de seguridad completas son copias completas del contenido de un almacén de datos. Su tamaño es igual al del original y su realización requiere un tiempo considerable. Es una garantía: existe otra copia completa de los datos si el original se corrompe.

Una copia de seguridad incremental es una copia únicamente de los cambios: adiciones, actualizaciones y eliminaciones, desde la última copia de seguridad completa. Dado que es muy probable que no todos los datos se actualicen durante el proceso de copia de seguridad, estas copias de seguridad incrementales serán mucho más pequeñas y, por lo tanto, mucho más rápidas de completar que una copia de seguridad completa.

Utilizando estas dos técnicas podríamos establecer una cadencia como:

- Una copia de seguridad completa semanalmente, los sábados a las 3:00 a. m., cuando el sistema está bastante inactivo
- Copias de seguridad incrementales los sábados a las 15:00 y todos los demás días de la semana tanto a las 15:00 como a las 15:00.

Ahora tenemos un límite de pérdida de datos de 12 horas y un plazo máximo de notificación de una semana. Si una copia de seguridad incremental promedio solo debe incluir el 5 % del almacén de datos, el espacio que necesitamos para almacenar las copias de seguridad de una semana equivale al tamaño total de nuestro almacén de datos. Por seguridad, conservaremos el conjunto completo de copias de seguridad hasta que la siguiente copia de seguridad completa se complete correctamente, por lo que necesitaremos el 265 % del tamaño de nuestro almacén de datos durante ese período, mientras se ejecuta la copia de seguridad completa y antes de que se verifique su finalización correcta.

Cloud Spanner permite al desarrollador configurar una programación de copias de seguridad para bases de datos, especificando la frecuencia y la duración de la retención. Lo describiremos en detalle en el laboratorio de código de este capítulo.

Instantáneas para continuidad

Si desea recuperar el estado de su sistema en un momento determinado, puede lograrlo tomando una instantánea.

Una instantánea es una copia puntual de sus datos, que captura su estado actual en el momento de la captura. Su objetivo es restaurar rápidamente los datos a ese estado, generalmente como parte de la recuperación de un estado corrupto. Una instantánea se basa en mecanismos internos del sistema de gestión de bases de datos (SGBD), como los registros de transacciones, y, por lo tanto, reside en el mismo entorno que la base de datos principal o activa. Un punto importante de las instantáneas es que representan un punto dentro de la base de datos activa, en lugar de una copia externa o independiente de su estado. Esto es deseable, ya que proporciona la manera más rápida de tomar una instantánea y restaurar el sistema a un punto temporal específico, sin tener que copiar ni transformar datos desde y hacia destinos y formatos externos.

Archivos para la posteridad.

A diferencia de las instantáneas y las copias de seguridad, un archivo es una copia portátil e independiente de los datos de un sistema. Un archivo está congelado en el tiempo y separado del sistema en funcionamiento; puede que ni siquiera esté en un formato que el sistema en vivo pueda utilizar.

Existen razones para preservar el estado de un sistema, además de la recuperación ante desastres. En un sector regulado, es posible que se requiera preservar el contenido de un sistema al final de cada período de informe.

En un sistema involucrado en litigios, podría requerirse el estado del sistema en un momento específico.

Un uso habitual de un archivo en las operaciones comerciales es la investigación.

Existen varias ventajas al realizar investigaciones en un archivo independiente en lugar de consultar la base de datos principal. Enumeremos algunas de ellas: Las consultas complejas para análisis de

- mercado o rendimiento afectarían el rendimiento del sistema en vivo.
- Las consultas utilizadas para dichos análisis podrían requerir una indexación diferente a la del sistema en vivo.

- Puede resultar difícil o imposible realizar un análisis para una ventana de tiempo específica con datos en vivo que siempre se modifican por las operaciones normales del sistema a medida que los consulta.

[Instantáneas y archivos de votaciones.](#) Todo sistema en

funcionamiento debe contar con un programa regular de copias de seguridad y recuperación comprobada. Las siguientes explicaciones se incluyen como complemento o adición a dichos planes de recuperación. El establecimiento de copias de seguridad periódicas se describe en las guías de inicio rápido y tutoriales de GCP Spanner.

El almacén de datos de Votos contiene el estado permanente de nuestro sistema, mientras que los datos de Votebudget son prácticamente irrelevantes tras el cierre de cada ronda. Estos datos pueden utilizarse para analizar el comportamiento del usuario, por ejemplo, agregando y correlacionando la frecuencia con la que se votaron dos actos. No es necesario recuperar el estado de Votebudget en el sistema en vivo tras el procesamiento de una ronda.

Por lo tanto, resulta útil tener un archivo de datos de Votebudget cuando se cierra cada ronda.

Las votaciones son continuas durante la vida útil del sistema; el contenido de las votaciones al final de una ronda es necesario para recrearlo posteriormente. Por esta razón, se debe tomar una instantánea de los datos de las votaciones cuando el procesador de votos haya terminado de procesar las colas de mensajes de las votaciones en Pub/Sub.

Cloud Spanner permite tomar instantáneas manualmente desde la consola de Cloud Spanner o la interfaz de línea de comandos de Google Cloud (gcloud CLI).

Los archivos de bases de datos de Spanner se crean comúnmente exportando la base de datos a un bucket de Cloud Storage, ya sea en formato de valores separados por comas (CSV) o Apache Avro. Esto también se puede hacer mediante la consola o el comando gcloud .

[Laboratorio de código sobre persistencia de votos y actividad electoral](#)

El laboratorio de código de este capítulo creará los artefactos Spanner y el código de aplicación para implementar el recuento real de y

Aplicación de los presupuestos de voto de los usuarios a la aplicación de votación. A continuación, se ofrece un resumen más detallado.

Componentes de Codelab

Estos recursos de base de datos se crearán en este laboratorio de código:

- Una instancia y una base de datos de Cloud Spanner
- Dos tablas de Spanner: Votos y Presupuesto de votos

El laboratorio de código implementa los siguientes componentes de tiempo de ejecución:

- Voteprocessor actualizará los votos y el presupuesto de votos de los usuarios.
- Votingfrontend marcará los actos como elegibles o no elegibles según el presupuesto de votación del usuario.
- La página del cliente web aplicará la política de presupuesto de votación a medida que se emiten los votos.

Además, el laboratorio de código describirá los siguientes procedimientos:

- Inspeccionando la programación de copias de seguridad de la base de datos de Cloud Spanner.
- Agregar una programación de copia de seguridad incremental de la base de datos de Cloud Spanner.

[Habilitación de la facturación para el proyecto](#): Antes de

habilitar Spanner en un proyecto, este debe estar asociado a una cuenta de facturación. Una cuenta de facturación especifica cómo se realizará el pago de los cargos generados por el uso de servicios de GCP que superen la cuota de uso gratuito incluida en la mayoría de los productos de GCP. Spanner es uno de los servicios que no se pueden usar sin una cuenta de facturación.

Para crear una cuenta de facturación, consulte los siguientes pasos: 1. Abra la consola de Google Cloud y seleccione Facturación en el menú de la izquierda.

Menú de navegación manual.

2. Seleccione su proyecto si aún no está seleccionado.
3. Cree una cuenta de facturación haciendo clic en Administrar facturación Cuentas y enlaces para crear cuentas.
4. Siga las instrucciones para crear una cuenta.

5. Copie el ID de la cuenta de facturación que se muestra para la nueva cuenta; deberá pegarlo en los comandos en sesiones posteriores.
6. Además, anote el ID de su proyecto. Suele coincidir con el nombre que especificó al crear el proyecto en la nube.

Para protegerse de cargos inesperados mientras desarrolla un proyecto, es recomendable habilitar la facturación para su proyecto cuando comienza una sesión de desarrollo y luego deshabilitarla cuando finaliza.

Los comandos para hacer esto se presentan más adelante en el laboratorio de código.

Interfaces de administración de GCP

La mayoría de las funciones administrativas de GCP están disponibles a través de tres mecanismos:

- Una API de servicio y bibliotecas de cliente.
- Una CLI: El comando `gcloud`.
- La interfaz de usuario de la página web de GCP Console, que hemos utilizado anteriormente. Para ejercitar una interfaz administrativa diferente a la que hemos utilizado hasta ahora, ahora utilizaremos el comando `gcloud` al definir nuestros recursos `Spanner`.

Definición de componentes de `Spanner` Antes de emitir

comandos de `Cloud Spanner`, se debe habilitar la facturación para el proyecto.

En una terminal de Linux o MacOS, los comandos y respuestas se verán así:

Nota: Las líneas que comienzan con `#` son comentarios para el lector.

```
# Asegúrate de haber iniciado sesión y de estar usando la cuenta y el proyecto correctos
```

```
$ gcloud auth
```

```
login
```

```
Su navegador ha sido abierto para visitar:      :
```

```
...
```

Ahora estás conectado como [XX@XX@gmail.com]. .].

Tu proyecto actual es [votingforthestars]. Puedes cambiar esta configuración ejecutando:

gcloud config establece el proyecto PROJECT_ID

Vincula tu cuenta de facturación al proyecto

Reemplace votingforthestars con el ID de su proyecto y

[BILLING_ACCOUNT_ID] con su cuenta de facturación \$ gcloud

alpha billing projects link votingforthestars \ --billing-

account=[BILLING_ACCOUNT_ID]

billingAccountName: billingAccounts/BILLING_ACCOUNT_ID billingEnabled:

true nombre: projects/

votingforthestars/billingInfo projectId: votingforthestars

:

Crear una instancia de llave inglesa

\$ gcloud spanner instances create talentvoting-instance --description=Talentvoting-

instance --config=regional-us-central1 --nodes=2

=

1

=2

La API [spanner.googleapis.com] no está habilitada en el proyecto

[votingforthestars]. ¿Desea

habilitarla y volver a intentarlo (esto tardará unos minutos)? (¿S/N?))? (/)?

Habilitando el servicio [spanner.googleapis.com] en el proyecto

[votingforthestars]...]...

Creando

... .

Crear una base de datos en talentvoting-instance \$ gcloud spanner databases

create talentvoting-database \ --instance=talentvoting-

instance Creando

... .

Crea dos tablas en esta base de datos

```
$ gcloud spanner databases ddl update talentvoting-database --
ddl='CREATE TABLE Votos (      INT64 NOT NULL,      ,
Act_number INT64 NOT NULL, Voting_opened_at TIMESTAMP,      ,
Voting_closed_at TIMESTAMP, Total      ) CLAVE PRIMARIA
(Round_id, Act_number)' --instance talentvoting-instance
Actualización del esquema...listo
```

```
$ gcloud spanner databases ddl update talentvoting-database --
ddl='CREATE TABLE Votebudget (      INT64 NOT NULL,      ,
Userid STRING(128) NOT NULL, Last_voted_at TIMESTAMP,      ,
Total_votes_cast INT64 NOT NULL,      ,      _
ARRAY<STRING(11)>) CLAVE PRINCIPAL (Round_id, Userid)'
-- instancia talentvoting-instance
Actualización del esquema...listo
```

Confirmar los recursos de Spanner en la consola

Ahora que hemos definido nuestras tablas, usemos otra interfaz de administración de GCP, la consola de Cloud, para ver los recursos que hemos definido. En la consola, seleccione el proyecto votingforthestars y, en el cuadro de búsqueda, ingrese talentvoting-database y podrá hacer clic en la siguiente página de descripción general de la base de datos:

The screenshot shows the Google Cloud Console interface for a Google Standard SQL Database instance. The left sidebar contains navigation options like 'Overview', 'Spanner Studio', 'Import/Export', 'Backup/Restore', 'Operations', 'Change streams', 'System insights', 'Query insights', 'Lock insights', 'Transaction insights', 'Hotspot insights', and 'Key Visualizer'. The main content area displays the 'Overview' page for the 'talentvoting-database' instance.

Overview

Database Name	talentvoting-database
Database Dialect	Google Standard SQL
Encryption type	Google-managed
Scheduled backups	default_daily_full_backup_schedule
Schema updates	No recent updates

Summary

Statistics are updated every 3-5 minutes, which may cause some delay in actual data.

CPU utilization (mean)	Operations	Throughput	Total database storage
1.13%	Read: ~/s Write: ~/s	Read: ~/s Write: ~/s	0 B

TABLES **VIEWS**

Tables are structured with rows, columns, and values, and they contain primary keys and indexes. Parent-child relationship between tables can be defined through table interleaving or foreign keys. [Learn more](#)

[CREATE TABLE](#)

Filter Filter tables

Name	Schema	Indexes	Interleaved in	Watched by	Table Size
Votebudget	Default	—	—		—
Votes	Default	—	—		—

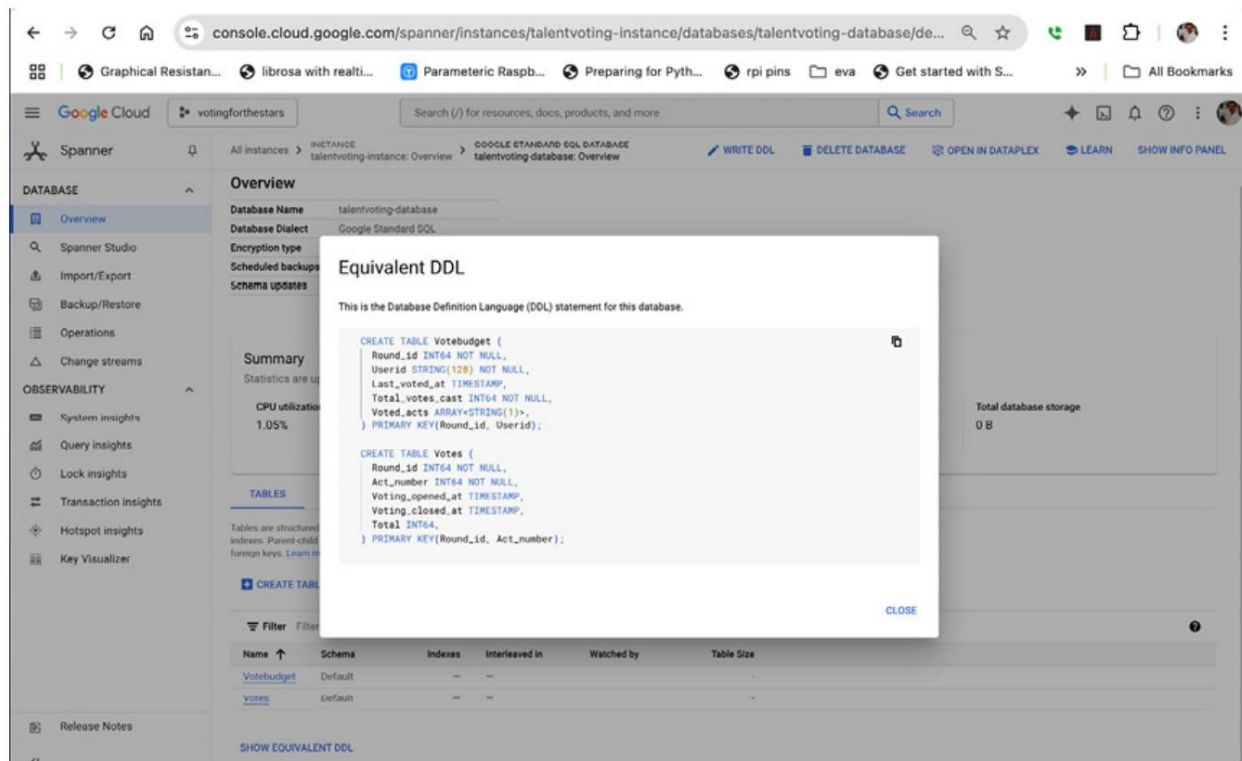
[SHOW EQUIVALENT DDL](#)

Cifra 11.7 Página de la base de datos de la consola de GCP

En esta página de base de datos, puede ver que cuando se creó la base de datos, GCP configuró una programación de respaldo predeterminada denominada descriptivamente `daily_full_backup`. Veremos cómo ver y cambiar la programación de respaldo más adelante en este laboratorio de código.

Tenga en cuenta el bajo uso de CPU y los 0 bytes de almacenamiento que se informan en esta página. Esto es apropiado, ya que los recursos computacionales de las instancias solo han ejecutado algunos comandos de lenguaje de definición de datos (DDL) y no hemos insertado ninguna fila en las tablas de la base de datos.

Al final de esta página, observe los nombres de las dos tablas que creamos en la base de datos: `Votebudget` y `Votes`. Haga clic en el enlace "Mostrar DDL Equivalente" para abrir la siguiente página:



Cifra **11.8** Tabla de consola de GCP DDL

Al revisar esto y compararlo con las instrucciones que ejecutamos para crear las tablas, se confirma que los comandos de gcloud spanner se ejecutaron correctamente. Nuestra base de datos y tablas están definidas.

Deshabilitar la facturación del proyecto

Cuando haya terminado de trabajar en el proyecto durante una sesión, deshabilite la facturación mediante la CLI de gcloud:

Desvincular la cuenta de facturación de este proyecto

Reemplace votingforthestars con el ID de su proyecto \$ gcloud billing projects unlink votingforthestars billingAccountName: "

billingEnabled: false name: :

projects/votingforthestars/

billingInfo projectId: /votingforthestars /
:

Código para conectarse a los recursos de Spanner

Una función común para devolver una conexión a nuestra base de datos Spanner se define en el módulo `talentvoting.cloud.common.voting.databaseutils.py`. Esto será utilizado por todos los componentes de la nube que requieren una conexión a la base de datos:

```
from google.cloud import spanner from
talentvoting.common.interfaces.servicelocations
import SPANNER_INSTANCE, \ SPANNER_DATABASE

def get_database():
    "Crear una conexión de base de datos."

    spanner_client = spanner.Client()
    instancia = spanner_client.instance(SPANNER_INSTANCE) base de
    datos = instancia.database(SPANNER_DATABASE) print("Conexión
    de base de datos creada") return database
```

La función `get_database()` devuelve un objeto de base de datos que proporciona una conexión abierta a la base de datos Spanner. Esta conexión se utiliza para ejecutar sentencias SQL (lenguaje de consulta estructurado) y generar transacciones que sirven como contenedores para múltiples sentencias SQL ejecutadas en la base de datos.

Se han agregado definiciones complementarias al módulo común `talentvoting.common.interfaces.servicelocations.py` :

```
SPANNER_INSTANCE =
"talentvoting-instance"
SPANNER_DATABASE = "base de datos de votación de talentos"
```

Adiciones de código al procesador de votos

El servicio backend del Procesador de Votos, implementado en el módulo `voteprocessor`, presenta varias novedades y refleja la mayor parte de los cambios relevantes para la persistencia. A continuación, se presenta el código fuente completo del módulo, junto con una descripción de algunas características de la implementación.

Primero, se resuelven todas nuestras dependencias y creamos el objeto de servicio web: `import json from flask`

```
import Flask,
```

```
request from
```

```
talentvoting.common.policy.votingpolicyengine import DefaultPolicyEngine from
```

```
talentvoting.cloud.common.votingdatabaseutils import get_database from
```

```
talentvoting.common.acts import parseAct
```

```
aplicación = Flask(__nombre__)
```

A continuación se definen varias funciones auxiliares internas:

Nota: La convención de visibilidad Pythonesque de prefijar los nombres de funciones con uno o dos guiones bajos para indicar que la función es respectivamente protegida o privada.

```
def __is_vote_in_budget(transacción, id_usuario, id_ronda, número_acto):
```

```
    """
    Obtenga el historial de votación de la ronda si el usuario tiene votos restantes
    en su
```

```
    presupuesto y no ha votado por esta ley.
```

```
    En una transacción, inserte un nuevo Votebudget si no existe ningún Votebudget.
```

```
    Devuelve: Matriz de cadenas si se puede emitir un voto para este acto, Falso en
    caso contrario
    """
```

```
    resultado_filas = transacción.ejecutar_sql(
```

```
        "SELECCIONAR Total_votos_emitidos, actos_votados de Votebudget "+
```

```
        "DONDE Round_id = {} Y Userid = '{}".format(round_id, user_id) ) result =
```

```
    result_rows.one_or_none()
```

```

    si resultado:

        prev_total = resultado[0]
        prev_votes = resultado[1]

        si DefaultPolicyEngine.is_eligible_vote(id_ronda, número_acto, prev_votes):
            devuelve
                prev_votes de lo
            contrario:
                devuelve Falso

        demás:

            new_votes = DefaultPolicyEngine.DEFAULT_VOTE_HISTORY print("creando presupuesto
            de votación para el usuario {} ronda {}".format(user_id, round_id))

print("nuevo_presupuesto_de_votación: {}".format(str(nuevos_votos))) sql =
    ("INSERTAR EN Presupuesto_de_votación "
    "(ID de usuario, ID de ronda, Última votación, Total de votos emitidos, Actos
    votados) "
    "VALORES('{}',{},{},NULL,0,ARRAY{})".format(id_usuario, id_ronda, str(nuevos_votos))

) fila_ct = transacción.ejecutar_actualización(sql)

si row_ct != 1:
    generar ValueError("Error al insertar, {} filas para la ronda {} act {}".
    formato(fila_ct, id_ronda, número_acto))

    devolver nuevos votos

def __update_vote(transacción, número_de_acto, id_de_ronda):
    "En una transacción, incrementa los votos para esta acción en 1." row_ct =
    transaction.execute_update(
        "ACTUALIZACIÓN Votos"

```

```

        "SET Total = Total + 1
    "

    "DONDE Número_de_acto = {}".format(número_de_acto) + "Y
    Id_de_redondeo = {}".format(id_de_redondeo)

    ) si row_ct != 1:

        genera ValueError("Se actualizaron {} filas inesperadas para la ronda {} act {}".

                                formato(fila_ct, id_ronda, número_acto))

def __update_votebudget(transacción, id_usuario, id_ronda, número_acto,
presupuesto_voto):
    """
    En una transacción, actualice el historial de votos para mostrar este acto como
    votado.

    El historial de votaciones está indexado en 0, los números de acto comienzan en 1, así que ajuste
    el índice

    en la matriz de historial.
    """

    presupuesto_voto[número_acto-1] = 'Y'

    print("presupuesto_voto establecido en: {}".format(str(presupuesto_voto))) sql=
    "ACTUALIZAR presupuesto_voto "+ \
    "ESTABLECER actos_votados = ARRAY{}, ".format(str(presupuesto_voto)) + \
    "Total_votos_emitidos = Total_votos_emitidos + 1, " +\
    "Última_votación_a las = MARCA_DE_TIEMPO_ACTUAL " +\ "DONDE
    Id_de_ronda = {} Y Id_de_usuario = {}".format(id_de_ronda, id_de_usuario)

    print("ACTUALIZAR '{}' ".format(sql)) fila_ct =

    transacción.execute_update(sql)

    si row_ct != 1:

        generar ValueError("Se actualizaron {} filas inesperadas para el usuario redondo {}

```

```

{}"
        .format(len(fila_ct), id_de_ronda, id_de_usuario)) def
__aplicar_voto_si_se_permite(transacción, id_de_usuario, id_de_ronda,
número_de_acto):
    """
    En una transacción, verifique la elegibilidad de este voto y
    actualice las tablas de votos y presupuesto de votos.
    """
    presupuesto_voto = __is_vote_in_budget(transacción, id_usuario, id_ronda, número_acto)
    si no
        presupuesto_voto:
            print("presupuesto_voto: {}".format(presupuesto_voto))
            print("voto fuera del presupuesto, usuario {} ronda {} acto {}".format(id_usuario,
                                                id_de_ronda, número_de_acto))
            devolver
    __update_vote(transacción, número_de_acto, id_de_ronda)
    __update_votebudget(transacción, id_de_usuario, id_de_ronda, número_de_acto,
id_de_voto) return

```

Luego, definimos la función de procesamiento de votos expandida y el controlador para los mensajes entrantes de Pub/Sub: **def**

```

__process_vote(message:json) -> bool:
    """
    Procesar un mensaje JSON que contiene un voto.
    Crea una transacción y actualiza la base de datos si el usuario y la acción están en el
    mensaje.
    args:
        mensaje: Un diccionario JSON
    Devoluciones:
        Verdadero si se procesó un mensaje, falso en caso contrario
    """

```

```

"""
si no hay mensaje:
    devuelve Falso

si no es instancia(mensaje, dict):
    devuelve Falso

Si "usuario" no está en el mensaje o "acto" no está en el mensaje:
    devuelve Falso

ronda, acto = parseAct(mensaje["acto"])
obtener_base_de_datos().ejecutar_en_transacción(_aplicar_voto_si_está_permitido,
                                                id_usuario=mensaje["usuario"], id_ronda=ronda,
                                                número_acto=acto)

    devuelve Verdadero
"""

@app.route("/", métodos=["POST"]) def índice():
    """
    Reciba un mensaje Pub/Sub que debe contener JSON y emita una respuesta
    HTTP.

    Procesar el voto si el mensaje contiene JSON válido.
    """

    mensaje = solicitud.datos

    print("¡Mensaje entrante!") if message:
    print("data:
        {}".format(message)) try: vote =

    json.loads(message) except ValueError
    as e: print("JSON no válido
        recibido: {}".format(e)) return ("JSON no válido", 400)

```

```

intentar: si _process_vote(voto):
    imprimir("Voto procesado: {}".format(voto)) de lo
    contrario:
        print("Voto inválido: {}".format(voto)) return
        ("Voto inválido", 400) # Estado de error de contenido inválido excepto
Excepción como e:
    print("error: {}".format(e)) return
    ("", 500) # Estado de error interno del servidor
return ("", 204) # Sin contenido Estado OK

```

Los puntos principales de la implementación del procesador de votos son la creación y el uso de una transacción de base de datos, que comienza en el método `_process_vote()`. Una base de datos proporciona un método, `run_in_transaction()`, que puede usarse como conexión para ejecutar SQL, al igual que una base de datos. El mecanismo es ligeramente diferente en su semántica: `run_in_transaction()` acepta una referencia a un método de devolución de llamada, junto con los parámetros para dicha devolución. Al método de devolución de llamada, en este caso `_apply_vote_if_allowed()`, se le pasa una transacción, que puede usarse como conexión para ejecutar sentencias SQL. _ _ _ ()

Si el método de devolución de llamada se completa y retorna correctamente, la transacción se confirma y el SQL ejecutado a través de ella se finaliza en la base de datos. Si la devolución de llamada no retorna, generalmente debido a una excepción, la transacción se revierte y los efectos de las sentencias SQL contenidas se deshacen y no se reflejan en la base de datos.

Durante una transacción, los efectos del SQL que se está ejecutando son aislados y no son visibles fuera de la transacción, por lo que el efecto de una transacción revertida es que nunca sucedió hasta donde cualquier proceso externo u observador puede detectar.

Adiciones de código a la interfaz de votación

El frontend de votación ahora obtiene una copia del presupuesto de votación de un usuario cuando el cliente solicita una lista de actos. Estos datos del presupuesto de votación se utilizan para establecer el indicador de elegibilidad para cada acto que se devuelve al cliente. A continuación se muestra el método que busca el historial de votaciones. Para obtener la fuente completa del módulo `talentvoting.cloud.votingfrontend.votingfrontend.py`, se remite al lector al repositorio de GitHub para este capítulo: `def`

```
__get_user_vote_history(db_connection, \
                        user_id:str, round_id:int)->Tuple[int,List[str]]: "Devuelve
la matriz del historial de votación de este usuario en esta ronda." result_rows
= db_connection.execute_sql( "SELECT
    Total_votes_cast, voters_acts from Votebudget "+ "WHERE Userid = '{}'
    and Round_id = {}".format(user_id, round_id) ) result =
result_rows.one_or_none() if result:
    log("votebudget(total,history):{} [1]:{}".format(result[0], result[1])) prev_total = result[0]
    prev_votes = result[1]
    else:
    devolver (Ninguno, Ninguno)
    devolver (prev_total, prev_votes)
```

Esta es la única sentencia SQL que se ejecuta en este módulo y es de solo lectura. Por lo tanto, no se requiere una transacción y esta función se ejecuta sin ninguna, mediante una conexión proporcionada directamente por una base de datos mediante las siguientes sentencias:

```
con get_database().snapshot() como db_connection:
    _, historial_de_votos_obtenidos = __obtener_historial_de_votos_del_usuario
(conexión_a_base_de_datos,
    uid, id_redondo)
```

[Código en el motor de políticas de votación.](#) El motor de políticas de votación ha cambiado sustancialmente. Codifica las reglas de un presupuesto de votación, el número máximo de votos que un usuario puede emitir por ronda y limita a un usuario a votar por un acto específico solo una vez por ronda.

La interfaz para el motor de políticas ahora se implementa a través de una instancia singleton `DefaultPolicyEngine` a través de la cual los clientes importan y acceden a constantes y métodos.

El código completo para `talentvoting.common.policy.votingpolicyengine.py` es el siguiente: módulo

```

de talentvoting.common.acts importar Act, Acts, exampleActs, parseAct de
typing
importar List

clase VotingPolicyEngine(objeto):
    "Esto está pensado para usarse a través de la instancia proporcionada (ver la parte inferior
    del archivo)".

    MÁXIMO_VOTOS_POR_RONDA = 6

    HISTORIAL_DE_VOTACIONES_PREDETERMINADO =
    ['N','N','N','N','N','N','N','N','N','N','N','N']

    def __init__(self):
        self._acts = ejemploActos()

    def get_all_acts(self) ->Acts: "Devuelve
        todos los actos actuales tal como fueron recuperados."

        devolver self._acts

    def get_current_round_id(self)->int: "Analizar
        el round_id del primer acto de la ronda actual."

```



```

    primer_acto = self._actos[0]["acto"] return
    parseAct(primer_acto)[0]

@metodoestático
def is_eligible_vote(id_ronda:int, número_acto:int,
                    votos_anteriores:Lista[str]) ->bool:
    """
    Determinar si esta votación está dentro del presupuesto de votación.

    El historial de votaciones está indexado en 0, los números de acto comienzan en 1, así que ajuste
    el índice

    en la matriz de historial.
    """

    si no es round_id o no es act_number o no es prev_votes:
        devuelve Falso

    prev_total = prev_votes.count('Y') si
    prev_votes[act_number-1] == 'Y': devuelve
        Falso

    si prev_total >=
    DefaultPolicyEngine.MAX_VOTES_PER_ROUND: devuelve
        Falso

    devuelve Verdadero

# Proporcionar un singleton para que los clientes lo utilicen
DefaultPolicyEngine = VotingPolicyEngine()

```

[Código en la página del cliente web.](#) El cliente web

ahora actualiza la tabla de actos para aplicar las reglas de presupuesto de votos codificadas en DefaultPolicyEngine. El historial de votos se extrae de la respuesta GetActs/ y se almacena en el documento. A medida que se emiten los votos correctamente, este historial de votos local se actualiza y las reglas se aplican a la tabla de actos. Esto es...

Se realiza localmente dentro de la página para evitar tráfico adicional del servidor con cada votación.

A continuación se detallan las funciones de JavaScript que establecen y aplican la política. Para consultar el código fuente completo de la página `talentvoting.webclient.public.index.html`, se remite al lector al repositorio de GitHub de este capítulo: // [Contar](#)

todos los votos "Y" en la matriz `vote_history` y almacenarlos como nuestro recuento de votos.

```
función countVoteHistory(vote_history) { sea vote_tally
```

```
  = 0; para (sea voteldx
```

```
    = 0; voteldx < vote_history.length; voteldx++) { si (vote_history[voteldx] == 'Y')
```

```
      { vote_tally = vote_tally+1; } }
```

```
  document.getElementById('voteTally').value = vote_tally; }
```

// Esto duplica el conocimiento en el motor de políticas de Python, // se mantiene local para mejorar el rendimiento.

// Si hemos alcanzado el límite de votos, deshabilitar todos los botones de votación. //

Después de que se haya emitido un voto para una ley, deshabilitar su botón de votación. `function enforceBudgetForVote(voted_act) {`

```
  deja que el
```

```
  conteo_de_votos = parseInt(documento.getElementById('voteTally').valor);
```

```
  conteo_de_votos = conteo_de_votos + 1;
```

```
  documento.getElementById('voteTally').valor = conteo_de_votos; deja que el
```

```
  límite_de_votos = documento.getElementById('voteLimit').valor; constante
```

```
  tablaDeActos = documento.getElementById('TablaDeActos'); constante filas
```

```
  = tablaDeActos.filas
```

```
  para (let i = 0; i < rows.length; i++) { const
```

```
    row = rows[i]; if (i > 0) { //
```

```
      La primera fila es una TH
```

```
const rowActId = row.cells[0].children[0].value; if (rowActId
== vote_act || voters_count >= vote_limit) { row.cells[1].innerHTML =
"; // Elimina todos los hijos row.cells[1].innerHTML = '--'; } } }
```

Implementación del servicio Cloud Run El módulo

voteprocessor.py ahora tiene dependencias en la jerarquía de paquetes talentvoting.common .

Para implementar este servicio desde la fuente, es necesario que hagamos una copia de las dependencias en el directorio base del servicio Cloud Run, similar al enlace que se creó en el directorio base del frontend de votación de Google App Engine. Tener que depender de estos dos trucos para corregir nuestras implementaciones indica una discrepancia entre nuestra jerarquía de paquetes y nuestros componentes implementados. Esto se abordará en los siguientes laboratorios de código cuando usemos los paquetes y repositorios de Python. Por ahora, usaremos los siguientes comandos para implementar el servicio Cloud Run desde el directorio base voteprocessor :

Crear directorios comunes locales y copiar los directorios comunes en ellos \$ mkdir

-p talentvoting/cloud/

```
common; \ cp -r ../.. /
# talentvoting/common/ talentvoting/common/; \ cp -r ../common/ talentvoting/
cloud/common/; \ gcloud run
```

```
deploy voteprocessor --allow-unauthenticated -- max-
instances=10 \ / / ;
```

```
../.. / / / / /;
../ / / / /;
```

=10

--fuente . .

Inspeccionar el programa de copias de seguridad

Usaremos la CLI de gcloud para ver la programación de copias de seguridad predeterminada creada para nuestra instancia. Recuerde que la facturación debe estar habilitada para poder acceder a los recursos de Spanner: Listar todas las copias de

#seguridad de la base de datos de Spanner \$ gcloud spanner

backup-schedules list --instance=talentvoting-instance \

--database=talentvoting-database NOMBRE

BACKUP_TYPE CRON

ENCRYPTION_TYPE KMS_KEY_NAME KMS_KEY_NAMES

default_daily_full_backup_schedule FULL

USE_DATABASE_ENCRYPTION Los principales campos de interés son

CRON

Un tipo FULL indica que se copia todo el contenido de las tablas de la base de datos. CRON es una programación en el formato de expresión CRON. El valor de indica 42 minutos después de la hora 22 (10:42 PM), y los tres asteriscos indican, respectivamente, todos los días de cada mes y todos los días de la semana. Para obtener más información sobre este formato, busque en Google la especificación cron o el formato de expresión cron.

indica cuánto tiempo se conservarán estas copias de seguridad en segundos; es 7 días.

604800

[Añadir una programación de copias de seguridad.](#)

Esta programación de copias de seguridad completas representa una pérdida máxima de datos de 24 horas y un plazo máximo de notificación de una semana. En nuestro sistema, asumiremos que perder hasta un día de actualizaciones es un riesgo excesivo y aceptaríamos un máximo de 12 horas. Para ello, inyectaremos una copia de seguridad incremental entre dos copias de seguridad completas.

Usando la CLI de gcloud nuevamente, este comando agrega un incremento a las 10:42 a. m. todos los días: Agregar un

#nuevo programa de respaldo para la base de datos de spanner \$ gcloud spanner

backup-schedules create

```
programación de copias de seguridad diarias
--proyecto=votandoporlasestrellas --instancia=instancia-de-votación-de-talentos \
--database=base-de-datos-de-votación-de-talentos --duración-de-retención=604800s \
```

```
--cron="42 42 *10" tipo-de-copia-de-seguridad=copia-de-seguridad-incremental \
--encryption-type=USAR_CIFRADO_DE_BASE_DE_DATOS
```

Este comando falla en nuestra base de datos porque estamos usando la versión inferior.
Nivel de servicio de Spanner, edición estándar. La copia de seguridad incremental
La función solo está disponible para los usuarios de los planes más altos y más caros.
Edición empresarial.

Por ahora, agregaremos una segunda copia de seguridad completa todos los días a las 10:42 a. m. Hasta el
La base de datos crece a un tamaño mucho mayor, el costo y el tiempo de una revisión completa
La copia de seguridad frente a la incremental no es prohibitiva:

```
$ gcloud spanner backup-schedules crear daily_incr_backup_am_schedule
\
--proyecto=votandoporlasestrellas --instancia=instancia-de-votación-de-talentos \
--database=base-de-datos-de-votación-de-talentos --duración-de-retención=604800s \
```

```
--cron="42 42 *10" backup-type=copia de seguridad completa \
--encryption-type=USAR_CIFRADO_DE_BASE_DE_DATOS
```

Al enumerar nuevamente los programas de copia de seguridad de la base de datos se muestran ambos programas:

#Listar todas las copias de seguridad de la base de datos de llave inglesa

```
$ gcloud spanner backup-schedules list --instance=talentvoting-instance \
```

--database=base de datos de votación de talentos			
NOMBRE	TIPO DE COPIA DE SEGURIDAD CRON		
DURACIÓN	TIPO DE CIFRADO		
NOMBRE DE CLAVE KMS	NOMBRES DE CLAVE KMS		
copia de seguridad diaria incr_am_schedule	COMPLETO	42 10	***
604800	USAR_CIFRADO_DE_BASE_DE_DATOS		
de copia de seguridad completa diaria predeterminada	COMPLETA	42 22	***
604800	USAR_CIFRADO_DE_BASE_DE_DATOS		

Ahora podemos estar seguros de que podemos recuperar el contenido de nuestra base de datos a cualquier estado dentro de los últimos 7 días con un máximo de 12 horas de pérdida de datos.

Conclusión

En este capítulo se analizaron diferentes formas de datos: estructurados y no estructurados, y jerarquías profundas y datos planos. Se describieron los tipos de datos más adecuados para bases de datos relacionales, otras bases de datos no relacionales y almacenes de datos para artefactos sin procesar (blobs).

Una vez identificadas estas categorías, se realizó la clasificación pertinente de nuestros datos de votación.

Luego se discutieron los objetivos de rendimiento de la persistencia de datos, como la disponibilidad y la permanencia, junto con las técnicas para medirlos y garantizarlos.

Finalmente, el laboratorio de código de este capítulo revisó la implementación de la persistencia de datos en la aplicación de votación. El laboratorio concluyó con la creación y copia de seguridad de los recursos de Spanner para el sistema, a fin de garantizar el soporte de esta funcionalidad.

En el siguiente capítulo, nos centraremos en la experiencia de usuario más inmediata: la arquitectura de la aplicación cliente. Se analizarán las decisiones sobre qué funcionalidades implementar en la plataforma cliente o en la nube, junto con las técnicas para lograr un equilibrio óptimo entre la experiencia de usuario y las exigencias del sistema distribuido. Desarrollaremos aún más el cliente web e introduciremos una aplicación móvil sencilla para el sistema de votación.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com>

.



CAPÍTULO 12

Desarrollo de interfaces de votación

Introducción

El backend y la arquitectura general pueden determinar la supervivencia a largo plazo de una aplicación en el mundo real, pero sin un frontend atractivo, nadie sabrá jamás el rendimiento de su sistema. Este capítulo describe las características de un buen frontend, uno que complemente las capacidades que hemos incorporado a nuestro backend basado en la nube. La experiencia en frontend abarca más que el diseño de la interfaz de usuario (UI) . La separación de responsabilidades entre las plataformas cliente y backend debería permitir que la aplicación cliente aproveche al máximo el backend para que los desarrolladores frontend puedan centrarse en la experiencia del usuario (UX) en lugar de diseñar considerando consideraciones de rendimiento distribuido.

Estructura

En este capítulo se tratarán los siguientes temas:

- Responsabilidades de la aplicación cliente
- Fuera del alcance
- Nuestra aplicación para clientes votantes

- Interfaz web del votante
- Aplicación móvil para votantes
- Refactorización del código base
- Codelab para la creación de múltiples aplicaciones cliente

Objetivos

Este capítulo examinará la composición de una función visible para el usuario y los detalles específicos de su implementación. Cada una de estas partes constituyentes del cuerpo de la función se categorizará en diversas dimensiones que contribuyen a las fortalezas y debilidades de una plataforma de cliente típica en comparación con un servicio en la nube. La afinidad entre estas partes es un factor a considerar, y esta fuerza de unión influye en la representación de estos componentes.

Con esta tabla o gráfico definido, se guiará al lector en la evaluación de la ubicación óptima para cada uno de estos componentes internos detallados. El resultado será un diseño detallado de las características visibles para el usuario que identifica claramente el alcance de las responsabilidades y dependencias de la aplicación cliente. El código base se refactorizará para facilitar el desarrollo independiente de múltiples productos de software relacionados.

El código de laboratorio de este capítulo ilustrará una implementación mínima de una aplicación cliente móvil que se basa en la metodología descrita en el capítulo.

Responsabilidades de la aplicación cliente

La aplicación cliente, independientemente de la plataforma, es la ventana o puerta de entrada del usuario a todo el sistema. Por plataforma, nos referimos a un vehículo independiente de la aplicación en el que reside nuestra aplicación, presumiblemente junto con otras. Esto podría referirse a un dispositivo de hardware específico, como un modelo determinado de teléfono inteligente, tableta o computadora portátil.

Las computadoras modernas para usuarios finales ofrecen una mayor independencia de plataforma, y para las aplicaciones cliente, no hay razón para distinguir entre modelos específicos de iPhone o teléfonos Android de diferentes fabricantes. Asimismo, no diseñamos para un modelo o marca de computadora portátil, sino para las capacidades que comparte un tipo de dispositivo.

Identificación de plataformas de aplicaciones cliente . Para los desarrolladores de aplicaciones, la plataforma cliente es la capa de software que define y respalda una experiencia de usuario (UX) consistente. Los iPhones 13, 10 y 15 no son plataformas cliente; iOS, el sistema operativo móvil que define las capacidades comunes de estos dispositivos, es la plataforma. Asimismo, las aplicaciones cliente basadas en navegador son una plataforma, en lugar de combinaciones individuales de navegador y sistema operativo host. Para completar el alcance de estos ejemplos, Linux y Windows son plataformas cliente de escritorio, en lugar de Ubuntu en las laptops Asus y Windows 11 Home en las computadoras de escritorio Hewlett Packard.

Para este capítulo, y en consonancia con los debates anteriores a lo largo del libro, consideramos que las plataformas de clientes se encuentran entre las categorías de la siguiente tabla:

Specific hardware	Client platform	Client metaplatform
iPhone 11	iOS	Mobile app
iPhone 15		
iPad Air	iPadOS	
iPad Pro		
Samsung Galaxy Tab S10	Android	
Google Pixel 9 Pro		
HP Desktop running Ubuntu	Linux desktop	Desktop client
Dell laptop running Windows 11	Windows	
Mac Pro running macOS	macOS	
Linux Chrome browser on Asus laptop	Web page	Web app
Safari browser on Macbook		

Aunque no es exhaustiva en sus ejemplos, la comunidad de dispositivos anterior proporciona ejemplos de todas las plataformas y metaplataformas que consideraremos en la taxonomía de plataformas de cliente. Por metaplataformas, nos referimos al conjunto de características que agrupa un conjunto de plataformas que los usuarios podrían considerar funcionalmente equivalentes. Los límites entre las capacidades y las expectativas del consumidor de un smartphone potente y una tableta son difusos, al igual que los de los diferentes sistemas operativos de escritorio.

[Responsabilidades de la aplicación cliente](#) . En resumen, la

aplicación cliente es responsable de interactuar con el usuario. Si bien esta descripción es práctica y precisa, también es demasiado amplia para ser medible o proporcionar pautas de acción al desarrollador. Analicemos sus expectativas con mayor claridad.

Una aplicación cliente revela el estado relevante del sistema al usuario.

La aplicación cliente mantiene al usuario informado sobre la información del sistema que le resulta relevante en su interacción con ella. Existen diferentes niveles de relevancia, por lo que ofreceremos y explicaremos varios ejemplos y contraejemplos a modo de explicación. Estos ejemplos se extraerán de la función de votación desarrollada en capítulos anteriores, y específicamente del cliente web implementado en [los capítulos "Asegurar todos los votos de los usuarios" y "Permanencia"](#).

10,

11, Garantizando los datos

- La lista de actos que compiten en la ronda abierta actualmente es un elemento de estado interno relevante para el usuario, quien, en este caso de uso, seleccionará entre dichos actos. Por lo tanto, el conjunto de actos que compiten actualmente es relevante para cada usuario y la presentación de dicha lista es responsabilidad de la aplicación cliente.
- La elegibilidad de un acto específico para que el usuario actual emita un voto es un estado muy específico que es relevante solo para el usuario actual y potencialmente cambia con cada acción que realiza el usuario.

La información está muy estrechamente vinculada al usuario individual y a la momento actual en el tiempo y es mejor delegarlo al cliente solicitud.

- El recuento actual de votos para cada acto, los resultados en vuelo de la competencia, es periféricamente relevante o quizás de interés para el usuario pero no es necesario para la función de votación ni es práctico proporcionarla a la aplicación cliente. Esto requeriría una transmisión a cada aplicación cliente, con cierta cadencia alta. El valor de esto La transmisión es dudosa ya que presumiblemente el votante ya está viéndola una transmisión en tiempo real de la competencia en sí. Presentación de esta La información se asigna mejor a algún componente centralizado y mecanismo de visualización, manejar esto en la aplicación cliente no agrega nada valor y supone un coste elevado.
- La ventana de tiempo durante la cual actúan las rondas actuales es El derecho a votar es un caso que no hemos explorado. Recordemos las reglas De la competición, los actos de una ronda son elegibles para votación de La primera transmisión de la ronda hasta cierto punto declarado en el tiempo Después de la emisión del último acto. Esto se vio reflejado en las votaciones. Tabla de llaves definida en el laboratorio de código del [Capítulo 11](#),¹Garantizando Permanencia de los datos, por las columnas de la tabla `Votingopenedat` y La votación está cerrada.

Si bien aún no hemos implementado ninguna función que refleje esto funcionalidad, podemos diseñarla y colocarla dentro de la aplicación ahora. El votante puede estar viendo una transmisión en vivo mientras vota, pero hay No había ninguna expectativa de eso. Una grabación de transmisión, un videoclip de un amigo, o simplemente una acción retrasada puede desvincular un voto potencial de el momento en que se abre una ronda. Así, la presentación de la votación ventana, el momento en que la ronda más reciente está abierta a votación es relevante al acto de votar y es bueno presentarlo en el cliente aplicación, independientemente de cualquier otra visualización de esta información, como Estas otras pantallas pueden ser grabaciones o no estar disponibles cuando el usuario desea votar.

Una aplicación cliente proporciona controles para que el usuario inicie una función.

comportamiento.

Nuestra función de votación es interactiva. Interactivo significa que dos elementos, en este caso el usuario y el sistema de la aplicación, se influyen mutuamente. El sistema informa al usuario sobre los actos que puede votar, y el usuario emite votos que modifican la clasificación de los actos en la competencia almacenada en el sistema.

No todas las aplicaciones son interactivas: Los paneles son aplicaciones completas que presentan el estado de la información que conoce el sistema, pero el usuario no puede usar la aplicación para influir en dicho estado. En un ejemplo extremo y obvio, un panel que muestra el tiempo local puede presentar diversas vistas de información meteorológica, pero no puede ofrecer al usuario un botón para detener la lluvia o hacer que sople el viento.

En la parte del sistema de votación que nos hemos centrado, hay una acción crítica del usuario: votar por un acto. El cliente web implementado en los laboratorios de código de capítulos anteriores también ofrece un botón para cargar la lista de actos, pero esto podría reubicarse en una función automática de la aplicación al abrir la página por primera vez.

Una vez identificada la acción del usuario de nuestra función (votar por un acto específico), podemos considerar la mejor ubicación de esta función. Los controles para actualizar el conjunto de actos candidatos e indicar el acto por el que el usuario desea votar requieren la entrada directa del usuario, dirigida a elementos de datos específicos. Esto implica que los controles para especificar el acto de votación deben estar en la aplicación cliente. ¿De qué otra manera podría el usuario indicar su solicitud? Como hemos visto, la ejecución de esa solicitud del usuario puede implicar muchos componentes del backend, pero la especificación de las opciones del usuario reside en el dominio de la aplicación cliente. El usuario debe poder indicar sus preferencias con la precisión necesaria tanto en el espacio (por cuál de los actos del conjunto va a votar) como en el tiempo (el momento en que desea emitir su voto). Esta precisión requiere comunicación e interacción directas con el usuario, solo disponibles desde la plataforma cliente.

Una aplicación cliente proporciona un medio para que el usuario se identifique ante el sistema de aplicación.

Esto es cada vez más difícil de implementar y es fundamental para que cualquier sistema empresarial funcione bien, ya que tanto los desafíos a la seguridad de las aplicaciones como las técnicas de ciberseguridad utilizadas para combatirlos se vuelven más sofisticados y conllevan mayores riesgos.

Durante varias décadas, el mecanismo estándar para establecer la identidad de un usuario fue la combinación de dos datos, uno público y otro privado: un nombre de usuario y una contraseña. A medida que aumentaba el incentivo para secuestrar o falsificar la identidad de un usuario, con el auge de las finanzas y la presencia del capital social en internet, el aumento de la ingeniería social (estafas) y de los recursos técnicos (robo de credenciales de sitios web, campañas de rociado de contraseñas) ha reducido la seguridad que ofrece el mecanismo de nombre de usuario y contraseña. La principal vulnerabilidad del mecanismo clásico reside en que un nombre de usuario y una contraseña no son tan diferentes: ambos son cadenas de caracteres que el propietario conoce.

Los sistemas de autenticación modernos complementan la confirmación del conocimiento del propietario con su presencia demostrada. Existen diferentes métodos populares para lograrlo: biometría como Touch ID o reconocimiento facial, y autenticación multifactor (MFA). Cada uno de estos métodos se basa en la presencia de una persona conocida o en la corroboración de la identidad de una persona mediante la prueba de algún otro elemento físico que se sepa que el usuario ha poseído en el pasado. La característica común es que ninguno de estos métodos se basa únicamente en el conocimiento del usuario; existe un aspecto adicional y diferente del usuario que se cuestiona y debe demostrarse; en estos ejemplos, uno vinculado al mundo físico.

La clave es que todos estos dispositivos de seguridad se encuentran muy cerca del usuario, por lo que son ideales, o incluso solo aptos, para su implementación en una plataforma cliente. Un servicio en la nube no dispone de medios directos para tomar la huella digital del usuario ni verificar que posee su clave de seguridad.

Hay otros reemplazos o complementos a la dependencia de las combinaciones de usuario y contraseña: los captchas y el análisis de riesgo de comportamiento se utilizan para demostrar aún más la presencia de un usuario, pero generalmente son

se considera tan eficaz en la autenticación de usuarios como los mecanismos físicos que hemos descrito anteriormente.

Una aplicación cliente adapta o permite al usuario personalizar sus funciones según las necesidades y preferencias del usuario individual.

Existen muchos atributos de una aplicación que contribuyen a su facilidad de uso. Algunos factores de usabilidad se centran en el sistema y sus datos, como la cantidad de información presentada simultáneamente y el nivel de detalle y orden adecuados de presentación. Otros factores que determinan la facilidad o dificultad de uso de la aplicación para el usuario se centran en el usuario y su entorno. Captar los atributos de la experiencia de usuario local y adaptarlos para que se ajusten a ellos es responsabilidad de una aplicación cliente bien diseñada.

La primera categoría de factores específicos del cliente es la ambiental. Los niveles de luz, el sonido ambiental y la hora del día se determinan fácilmente, y son consideraciones relevantes que deberían determinar el comportamiento y la apariencia de una aplicación. Un desarrollo popular reciente que refleja estas preocupaciones es la adopción del modo nocturno en muchas aplicaciones y navegadores. Es importante permitir que el usuario acepte o desactive estos cambios en la aplicación, en lugar de asumir que todos prefieren una pantalla oscura después de las 10 p. m.

Los requisitos de accesibilidad son aquellas consideraciones que se centran en el usuario individual, más que en su entorno, aunque un mismo entorno puede afectar a cada usuario de forma diferente. La posibilidad de ajustar el contraste y el brillo son dos características de usabilidad obvias y bien conocidas, pero la visión no es la única consideración. El uso de la retroalimentación háptica, la velocidad de desplazamiento, el tamaño y la velocidad de seguimiento del cursor, y los patrones de clic son características importantes y necesarias que deben adaptarse a las necesidades del usuario.

La gama de opciones que una aplicación puede soportar para ambos tipos de comportamiento adaptativo está determinada por la plataforma del cliente y el hardware subyacente y, por lo tanto, son apropiadas para implementar en la aplicación del cliente.

La siguiente tabla resume aquellas características de una aplicación que requieren más claramente las fortalezas y se encuentran dentro del dominio de un cliente.

solicitud:

Client application feature	Reasoning
Display currently relevant information	Irrelevant or not timely information distracts from immediate user action Fetching and holding irrelevant information consumes client resources without adding value for its locality
Provide direct controls for feature activation	Low latency and precise controls depends on direct user interaction with the application
Authenticate the users identity	Multi factor and biometric authentication are coupled to physical presence
Adapt to the environment	The ability to detect environmental factors most resides on the local platform
Adapt to users' accessibility requirements	The capability to detect and range of adaptive features are particular to the client and depend on specific client platform control

Cifra **12.2** Resumen de las responsabilidades de la aplicación del cliente

Fuera del alcance

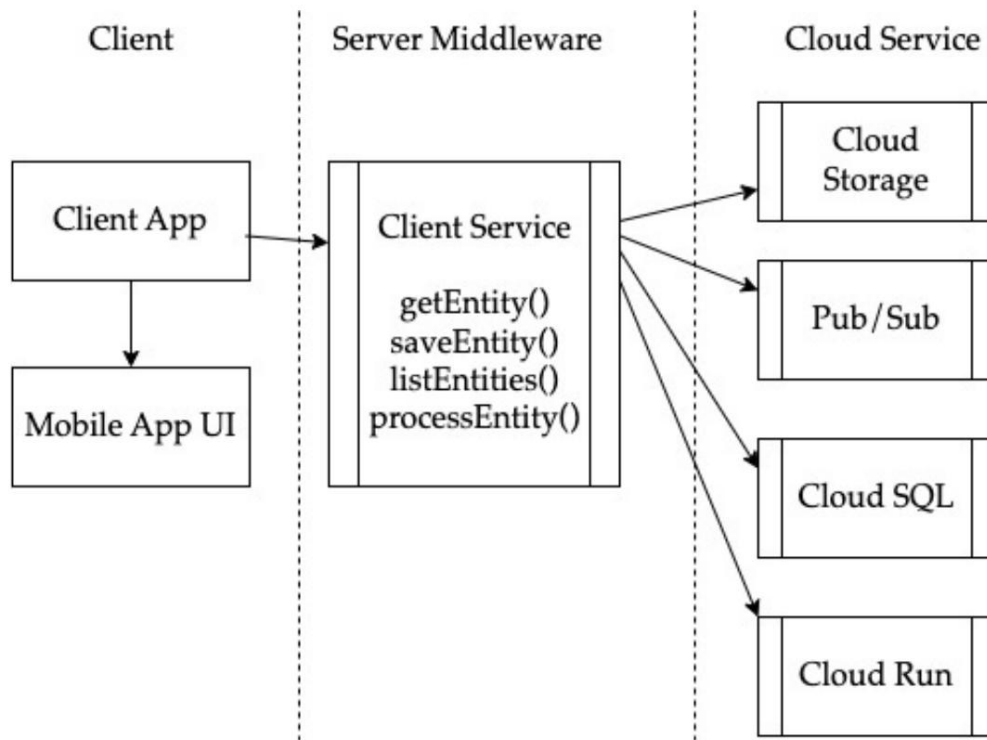
Hemos destacado ciertas características de una aplicación general que debe incluirse en los componentes locales de usuario que están orientados al cliente. el sistema. Ahora mencionaremos explícitamente los contracasos, aquellos funciones que son más adecuadas para residir y ser implementadas en El backend.

Primero, un breve resumen del alcance. La siguiente figura de un resumen...

La arquitectura de aplicación distribuida recrea una anterior vista en

[Capítulo 5](#) Servicios en la nube en la frontera del backend y servirá como

Referencia para esta sección:



Cifra 12.3 Arquitectura de una aplicación distribuida de n niveles

El backend se refiere a todos los componentes e infraestructura de soporte que no residen en la plataforma cliente frontend. En nuestro análisis, los componentes de los niveles de middleware del servidor y de servicio en la nube son componentes del backend.

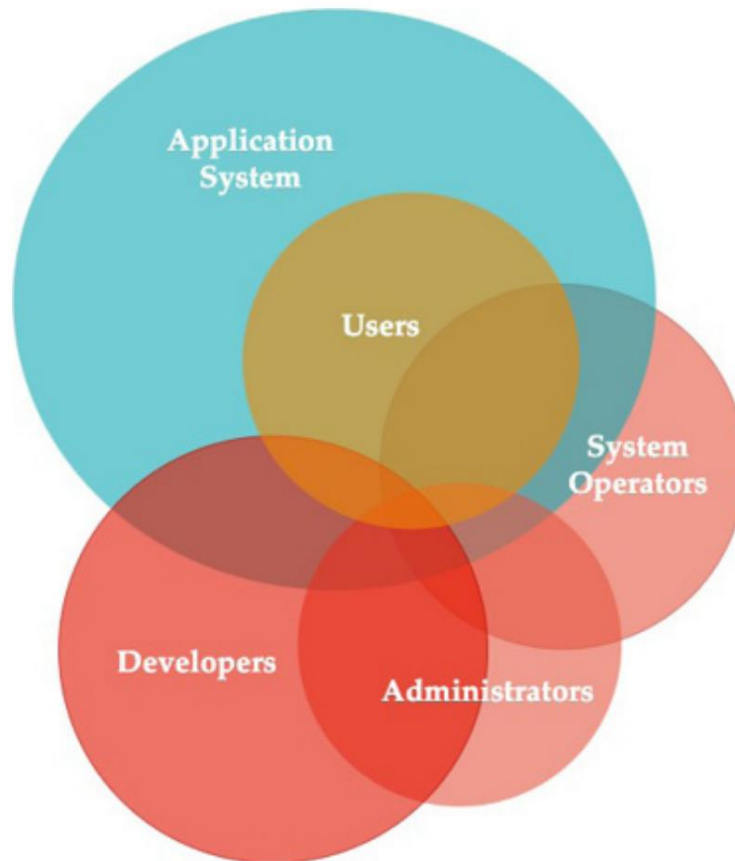
Sin embargo, existen otros componentes del sistema que son implícitos en el backend. La infraestructura y los servicios que respaldan las operaciones entre niveles, como las redes y los servicios de descubrimiento, también se consideran componentes de soporte del backend de la arquitectura.

El proceso seguido en la sección anterior para identificar los componentes del frontend funciona bien y se utilizará aquí para designar y describir la funcionalidad del backend de una aplicación.

Responsabilidades de los niveles backend

En primer lugar, el backend del sistema representa una parte del sistema como entidad independiente, que respalda la existencia de la aplicación, pero que existe más allá de la visibilidad o el cliente de cualquier usuario individual.

El alcance de la plataforma. La relación aproximada entre el sistema como entidad y sus diversos usuarios se representa en el siguiente diagrama de Venn:



Cifra 12.4 Uso de una aplicación distribuida

Los desarrolladores tienen la mayor superposición con el sistema y utilizan la mayor cantidad de herramientas y servicios externos en sus funciones, como compiladores, editores de código, depuradores y repositorios. Los administradores y operadores de sistemas se asemejan a los desarrolladores en que interactúan directamente con partes más extensas del sistema y utilizan algunas herramientas y servicios externos en sus funciones.

Los usuarios del sistema son únicos en el sentido de que su interacción con un sistema bien diseñado puede ser completamente a través de funciones proporcionadas por el propio sistema.

Lo importante es destacar el área del sistema de aplicación que no se superpone con ninguna de las otras funciones ilustradas. Hay partes

del sistema que existen independientemente de cualquier uso directo por parte de un desarrollador, operador o administrador, pero sin esta parte independiente del sistema, no existirían para interactuar con estas otras partes.

Hay dos tipos de funcionalidad que se incluyen en este ámbito. Existen características y artefactos del sistema que no son directamente visibles desde la perspectiva del usuario. También existe funcionalidad que soporta o materializa las características visibles para el usuario. Si bien las características visibles para el usuario dependen directamente de esta funcionalidad, no es viable su implementación en la plataforma del cliente.

Describiremos varias categorías de características del backend y usaremos ejemplos de nuestro sistema de votación previamente establecido para ilustrarlas.

El backend debe realizar funciones que fusionen o agreguen las acciones de muchos usuarios a lo largo del tiempo.

Una función como el recuento de votos de todos los usuarios es un claro ejemplo de una función del sistema de votación que cumple con este requisito. La recopilación y aplicación de votos a la base de datos centralizada se realiza para muchos usuarios durante toda la ventana de votación de una ronda. Ubicar esto con gran proximidad en la nube nos permite ubicar los componentes involucrados lo más cerca posible en términos de topología de red. Esto reduce el tráfico de red y, por lo tanto, la latencia, y potencialmente reduce los cargos por ancho de banda. Además, ubicar estos componentes en la nube nos permite controlar el escalado de recursos, algo que solo es posible desde una perspectiva que permite visualizar la demanda total de los usuarios, algo que un cliente individual no puede ver.

Las funciones cuyos efectos persisten en el sistema más allá del contexto de la solicitud inicial del usuario deben implementarse en el backend tanto como sea posible.

Esta declaración es algo extensa, pero tenemos un buen ejemplo que puede ayudar a aclarar su propósito. En primer lugar, la separación entre el registro del voto de un usuario y su procesamiento es un buen ejemplo. Cuando un usuario emite un voto, esta acción afecta el estado de la información almacenada en el sistema, incluso después de que el usuario haya realizado su siguiente acción. Esto es lo que ocurre en el Pub/Sub del procesador de votación.

El objetivo es trasladar los cambios permanentes o persistentes en el estado del sistema desde el cliente, que es un entorno relativamente efímero, al sistema más grande que reside en la nube, lo más rápido posible. Esta mayor confiabilidad se debe a que las tecnologías del lado del servidor son más robustas, lo que proporciona un mayor soporte para reintentos y conmutación por error de solicitudes en curso cuando falla un componente.

El preprocesamiento o posprocesamiento de datos para los clientes debe realizarse en el backend, con solo los niveles de detalle relevantes para el usuario presentes en la aplicación del cliente.

El preprocesamiento de datos para los clientes supone una sobrecarga y, muy probablemente, no implica mucha información del contexto del usuario. Un ejemplo de esto es el envío de actos desde el frontend de votación al cliente. No enviamos todos los actos de todas las rondas al cliente y luego dejamos que cada cliente filtre los actos de la ronda actual. Realizamos esta selección de datos relevantes para el usuario en los componentes de la nube y enviamos al cliente solo los datos que se le mostrarán. El enfoque alternativo, que deja el filtrado y la posible transformación de datos en manos de la aplicación cliente, aumentaría el tráfico de red hacia cada cliente conectado y trasladaría la ejecución de este filtrado al hardware del cliente, más lento y costoso. Además, la lógica de filtrado y transformación se ejecutaría una sola vez en una instancia del frontend que atiende potencialmente a cientos de clientes; en un diseño residente en el cliente, esta lógica se ejecutaría cientos de veces, repitiéndose en cada uno de esos clientes.

Nuestra aplicación para clientes votantes

Ahora que hemos descrito las partes de las características de un sistema de aplicación que son adecuadas para su implementación en plataformas cliente y algunos contraejemplos para contrastar, podemos diseñar la aplicación de votación, independiente de cualquier plataforma cliente específica.

El primer paso será definir los requisitos de nuestro cliente. Lo que sigue no es un documento de requisitos del producto (PRD) completo ni un diseño técnico completo, sino una declaración de intenciones suficiente para guiar nuestro desarrollo y verificar el producto.

Requisitos de la aplicación cliente para votación. Hay

aspectos de una aplicación cliente que reflejan la plataforma en la que se desarrolla, lo que muestra las fortalezas de ese entorno. También existe un conjunto de requisitos funcionales que cada implementación de cliente debe cumplir de alguna manera. Estos requisitos se establecen en términos independientes de la plataforma y centrados en el negocio.

Nuestros requisitos para el cliente votante son los siguientes:

- Resumen: La aplicación cliente de votación (en adelante cliente de votación) es el medio a través del cual un votante registrado puede seleccionar un conjunto de actos de la ronda actual de la competición por los que desea votar.

- Autenticación de usuario: El cliente de votación permite que un usuario inicie sesión como un usuario conocido. El establecimiento de una identidad de usuario no está contemplado.

El registro de usuarios se realiza a través de proveedores de identidad existentes. Para evitar que un mismo usuario vote con varias cuentas de diferentes proveedores, como su ID de Apple y su cuenta de Gmail, limitaremos el acceso a cuentas de Google.

Mientras no esté conectado, la única función disponible es el control para iniciar sesión.

Cuando un usuario inicia sesión, el control de inicio de sesión se reemplaza por un control de cierre de sesión.

El inicio y el cierre de sesión se gestionarán a través de las llamadas API del sistema de autorización especificado.

Un usuario que ha iniciado sesión permanece conectado hasta que el sistema de autorización subyacente pueda forzar un cambio en su estado de sesión.

El cliente votante debe reconocer los cambios asincrónicos en el estado de sesión cerrada y cambiar al estado de sesión cerrada descrito anteriormente, siendo la única función disponible el control para iniciar sesión.

- Visualización de la sesión iniciada: El usuario conectado ve una lista de actos en la ronda actual. Puede haber cualquier número de ellos y el cliente de votación ofrece una forma de recorrer la lista. El orden de los actos lo determina el servidor y la aplicación de votación debe mantenerlo.

Cualquier número de estos actos en la ronda actual es elegible para que el usuario actual emita un voto. Esta elegibilidad se determina mediante la política de votación, definida externamente. La elegibilidad es un estado booleano por acto; cada acto es elegible o no.

La elegibilidad de todos los actos debe reevaluarse después de cada acto de votación (a continuación).

- Votación: Para cada acto elegible, el cliente de votación proporciona un control para emitir un voto. No hay parámetros ni controles secundarios. Se emite un voto por un acto elegible sin que se requiera ninguna otra información.

Los votos podrán ser indicados en grupo o individualmente por el usuario.

Si un cliente con derecho a voto permite la selección de varios actos para votar, debe aplicar la política de votación al realizar cada selección. Esto se debe a la disposición de la política que establece que, si bien dos o más actos pueden ser elegibles para votación, la combinación puede no serlo.

API de frontend de votación. El

servicio de frontend de votación proporciona el único acceso al sistema de votación para las aplicaciones cliente. La interfaz de programación de aplicaciones (API) que presenta debe estar bien documentada y ser independiente de las preferencias de cada cliente, como su idioma.

Presentamos un conjunto de pautas generales de API y los detalles de cada llamada de método disponible en la siguiente sección.

Uso de la API del lado del servidor

El cliente votante debe utilizar los métodos proporcionados por la API REST del servicio frontend de votación.

Los URI del método frontend de votación se proporcionan en los siguientes símbolos de lenguaje del cliente y sus valores no deben modificarse mediante ingeniería inversa a ninguna URL base observada: La API solo acepta

- métodos POST.
- Se requiere HTTPS.

- Los parámetros del método se especifican por nombre del elemento en el formulario de solicitud.
- Los valores de los parámetros son todas cadenas.
- Los tipos de respuesta se especifican mediante el encabezado Content-Type.
- Se devuelven los códigos de estado HTTP 2XX, 4XX y 5XX.

La interfaz de votación proporciona 3 métodos para el cliente de votación:

- `getActsURL`:
 - Parámetros: `idToken`
 - Respuesta: `aplicación/json` (`acts_bundle: {acts: historial_de_votos: [], límite_de_votos: str}`, `_` :)
- URL de votación:
 - Parámetros: `idToken`, `voteAct:actId` :
 - Respuesta: `aplicación/json` / (`{acto:str}` , : }
- `obtenerURL` de reglas de política:
 - Parámetros: `token de identificación,` `clienteldioma:` :
(`'JavaScript'|'python'`) `texto/`)
 - Respuesta: `(JavaScript|python)` /(`|`)
(`función de ejecución presupuestaria_fuente`))

Rendimiento del cliente de votación. Cualquier

implementación del cliente de votación debe aspirar a los siguientes objetivos comunes de rendimiento. Esto es necesario para evitar que la instalación o el uso del cliente se desvíen hacia una plataforma específica, preservando el concepto de igualdad de acceso, una consideración importante en un sistema de votación: Conectividad: La conexión a internet, y por lo tanto a nuestros

- servicios en la nube, queda fuera del alcance del cliente de votación. Cualquier mecanismo de conectividad, ya sea nativo o de otra plataforma, debe ser el único proveedor de transporte de internet, y no se deben incorporar ni incluir en el cliente de votación controles o bibliotecas de conexión personalizados o de terceros.

- **Latencia:** Las respuestas al usuario son muy sensibles a la latencia. Una respuesta perceptible (es decir, visual, audible o háptica) debe presentarse en un plazo de 100 milisegundos (1/10 de segundo) tras la activación de cualquier acción por parte del usuario. Esta puede no ser la respuesta definitiva o completa a la solicitud del usuario. Por ejemplo, al hacer clic en un botón de votación, se podría generar un tono, un zumbido háptico o un destello en la pantalla para indicar el envío de una solicitud del backend, incluso si la respuesta pendiente es necesaria para mostrar la confirmación y actualizar el estado local.
- **Manejo de errores:** Los errores se pueden clasificar en dos dimensiones: esperados o inesperados y locales o originados por el servidor. La clasificación de un error específico al detectarse determina la utilidad y la descriptividad de la respuesta de la aplicación.

Siempre se recomienda ofrecer al usuario la opción de reintentar o abandonar/cancelar una operación fallida. Si el desarrollador de la aplicación previó el error, se pueden ofrecer instrucciones o medidas correctivas antes de reintentar.

En el caso de errores generados localmente, no se debe revelar al usuario la causa raíz o el origen técnico específico. Mostrar el nombre de un framework o componente en un mensaje, como un error devuelto por `<Document.body.scrollview>`, no informa al usuario de nada relevante ni procesable, y revela información que puede facilitar intentos de ingeniería inversa y vulnerabilidades de seguridad de la aplicación.

En el caso de errores generados por el servidor, como los códigos de estado HTTP de las series 400 o 500, se recomienda que el único nivel de detalle a revelar sea la ocurrencia de un error de servidor. Un error de la serie 400 en el backend puede deberse a un problema interno de permisos o acceso, pero no a uno que el usuario esté autorizado a modificar o solucionar. Se deben registrar todos los detalles del error para la depuración del servidor con suficiente información para vincular el error registrado con la solicitud fallida.

- **Accesibilidad:** Las funciones de accesibilidad son valiosas para lograr el objetivo de igualdad de acceso para todos los votantes. Estas funciones varían según la plataforma.

Su uso puede requerir la inclusión de ciertas opciones o prácticas de configuración en la aplicación. Algunas, como el texto alternativo, requieren la implementación activa por parte del desarrollador de la aplicación. El cliente que vota debe utilizar todas las funciones de accesibilidad disponibles.

Reglas de política del lado del cliente.

Las reglas utilizadas para determinar qué votos son elegibles son un caso que merece especial consideración. La política en sí puede encapsularse en una sola función, que opera sobre un conjunto de actos y su estado actual. Esto se observó en el laboratorio de código del [Capítulo](#) Garantizar la Permanencia de los Datos¹¹, en el método `is_eligible_vote()` definido en el módulo `voteingpolicyengine.py`.

Recuerde que la lógica de esta función se duplicó en el cliente web, en la función JavaScript `enforceBudgetForVote()` definida en el archivo `0`. Esto se debía a que, si dependíamos de una llamada al servidor para aplicar las reglas de la política, se duplicaría el número de llamadas al servicio frontend de votación desde un cliente y se incrementaría significativamente el número de bytes enviados a través de la red.

La desventaja del enfoque elegido es que hemos codificado la misma lógica en dos ubicaciones diferentes. Si las actualizaciones de una de ellas se publicaran de forma desincronizada con la otra, sería difícil garantizar el cumplimiento de las reglas y el resultado final probablemente sería incoherente con ambas definiciones.

Existe un punto medio. Podemos definir las reglas en un solo lugar y, aun así, evitar las constantes llamadas al servidor, ya que la política se aplica al cliente.

Obtendremos de forma segura una nueva función de política del servidor y la instalaremos en el cliente cuando un usuario inicie sesión. El cliente con derecho a voto ejecutará localmente el motor de políticas instalado durante la sesión del usuario. La ejecución de código remoto puede generar un tipo de vulnerabilidad conocido como ataque de inyección, por lo que el protocolo de autenticación y las medidas de seguridad en la configuración de nuestra aplicación y servicio son necesarios para evitar la explotación maliciosa de esta función.

Esta implementación la proporciona el método de interfaz de votación en

obtenerURLDeReglasDePolítica.

Resumen de requisitos. Este es el conjunto de requisitos que cumpliremos al implementar una aplicación cliente móvil. También refactorizaremos el cliente web para que converja con estos requisitos.

Interfaz web del votante

Se desarrolló una implementación web para el cliente de votación en los laboratorios de código de los capítulos anteriores, pero aún no cumple con los requisitos mencionados. Presentaremos las especificaciones de un cliente de votación web, identificando y abordando las diferencias entre la implementación actual del cliente web y dichas especificaciones.

Lenguaje y framework del cliente web. Debido a una combinación de consideraciones, decidimos implementar el cliente web en JavaScript simple. La elección de la plataforma se ve influenciada por diversos factores: técnicos, organizativos y de mercado. En este caso, decidimos que los factores organizativos: un equipo de ingeniería con experiencia en JavaScript y estudios de mercado que indican que la mayor comunidad potencial de usuarios se basa en la web/navegador, superan las ventajas técnicas del desarrollo multiplataforma o de un framework de alto nivel.

Inicio y cierre de sesión del cliente web El inicio

y cierre de sesión a través de una ventana emergente de Google, llena los requisitos de autenticación de usuario que se presentaron en la sección anterior.

Hay algunos cambios en el cliente web que son necesarios para que coincida con el comportamiento requerido de la pantalla de inicio de sesión.

La pantalla de inicio de sesión siempre debe mostrar el conjunto actual de actos, con controles de votación que reflejen la política de votación aplicada al estado actual del usuario. El botón Obtener actos no está incluido en los requisitos.

La información que presenta debe mostrarse siempre mientras el usuario esté conectado.

Ya existen partes del código donde realizamos acciones cuando un usuario inicia sesión. Se invoca la función `setView()` para intercambiar la visibilidad de las secciones HTML etiquetadas como iniciada y cerrada. También hay código que recupera el token de ID de Firebase Auth cuando el usuario se autentica. Invocaremos las mismas funciones que el botón "Obtener Actos" cuando el usuario se haya autenticado, ya que necesitamos el token de ID para realizar llamadas al frontend de votación.

La siguiente devolución de llamada en `firebase.auth().onAuthStateChanged` se invoca cuando cambia el estado de inicio de sesión del usuario y ya se utiliza para obtener el token de identificación: //

Escuche si el estado de autenticación cambia y almacene la información del usuario en el documento

```
firebase.auth().onAuthStateChanged(función asíncrona(usuario) {
  if (usuario)
    { // El usuario ha iniciado
      sesión. document.getElementById("userName").innerHTML = user.email;
      user.getIdToken().then((idToken) =>
        { document.getElementById("idToken").value=idToken; });
    }
```

Además, en `firebase.auth().onAuthStateChanged` hay un controlador de clics para el botón Obtener actos: //

Escuchar los clics en el botón Obtener actos \$

`( '#actsButton' ).click(() => getActs());` Podemos

eliminar el botón Obtener actos de la página y el controlador de clics anterior; agregándolo a la función de devolución de llamada de Firebase que ahora contiene esto:

```
firebase.auth().onAuthStateChanged(función asíncrona(usuario) {
  if (usuario)
    { // El usuario ha iniciado
      sesión. document.getElementById("userName").innerHTML = user.email;
      user.getIdToken().then((idToken) => {
```

```
documento.getElementById("idToken").valor=idToken;
```

```
obtenerActos(); });
```

Manejo de errores: El

manejo de errores en nuestro cliente web inicial era inconsistente. Los puntos de fallo esperados son códigos de respuesta HTTP distintos de 200 del frontend de votación. Las respuestas exitosas resultan en el efecto deseado evidente en la pantalla del cliente: al iniciar sesión se muestran las acciones actuales, al votar se muestra una alerta y se actualiza la lista de acciones elegibles. Las respuestas de error generan mensajes registrados en la consola del desarrollador del navegador y se evidencian por la falta del efecto esperado de la solicitud, pero hacemos muy poco para gestionar o informar al usuario de los fallos esperados. Las mejores prácticas recomiendan recopilar y reenviar dichos errores al servidor para detectar y analizar las tendencias en los errores del usuario.

Podemos adoptar una técnica consistente para informar al usuario sobre respuestas de error. El frontend de votación emite dos categorías de respuesta: un estado de respuesta de la serie 500, que indica un error interno del servidor, y un estado de la serie 400, que indica una solicitud incorrecta o no autorizada enviada desde el cliente.

En el caso de errores internos del servidor, el problema se encuentra en el servidor y es más probable que sea independiente del cliente. La naturaleza de estos hace que la causa raíz sea más probable que sea visible para la monitorización de GCP y, por lo tanto, reciba atención y mitigación. El estado del servidor también puede ser transitorio, como una escasez momentánea de recursos mientras se aplica el escalado automático. Por estos motivos, es adecuado un mensaje que informe al usuario de un error en el sistema y le sugiera reintentar su solicitud.

Es probable que la causa raíz de una respuesta de la serie 400 resida en el cliente o en un error en el código que gestiona la solicitud específica del cliente. Por ejemplo, la falta de un parámetro esperado en una solicitud, como el ID de acto de una solicitud de votación, indica un error en el código del cliente que no incluyó el campo ID de acto o, con menor probabilidad, un error en el controlador del servidor que extrae el ID de acto de los parámetros de la solicitud. En cualquier caso,

No es probable que volver a intentarlo produzca un resultado diferente y los datos del cliente deben considerarse dudosos o inconsistentes.

Existe otro tipo de fallo: el error inesperado. Si bien anticipamos que el servidor enviará una respuesta 400, no podemos prever fallos generados fuera de nuestro control. La separación de estos se puede ver en este fragmento omitido del código del cliente web actual: // **Enviar un formulario a un servicio remoto.**

```
// Pasa la respuesta json a una función de
devolución de llamada getResponse(form, url,
  callback) { console.log('getResponse URL: ',
    url); fetch(url,
      { method: form.method,
        body: new
          FormData(form) }) .then(function
            (response){ if(response.ok) {
              ...
            } de lo
              contrario { console.log('Error HTTP: ' + response.status);
                ...
              } }) .catch(función(error)
                { console.log('Error: ', error); }); }
```

El código para manejar una respuesta distinta de 200, los errores esperados, es el alcance que sigue a la declaración `console.log('HTTP-Error: ' + response.status);`. El código para manejar errores inesperados lanzados por los diferentes marcos que se utilizan para obtener la respuesta

La discriminación entre fallas esperadas e inesperadas y luego la distinción más precisa entre diferentes (clases de errores esperados) se implementarán y discutirán en el laboratorio de código de este capítulo .

Accesibilidad del cliente web. Existen

directrices bien definidas para la accesibilidad del contenido web, definidas por el Consorcio World Wide Web (W3C). Conocidas colectivamente como Pautas de Accesibilidad al Contenido Web (WCAG), estas directrices gozan de reconocimiento internacional para ayudar a los creadores y desarrolladores de contenido a hacer que el contenido web sea accesible para personas con discapacidad. La lista completa de directrices se publica en [http://www.w3.org/WAI/standards-guidance/wcag/](#)

Nuestra aplicación cliente web admite las pautas WCAG de varias maneras:

• proporcionamos etiquetas de texto para todos los botones.

-
- Nuestro diseño utiliza tipos de contenedores estándar anidados que garantizan que los elementos reciban el foco en un orden lógico al usar el teclado.
- El uso de controladores de clic de botón estándar garantiza que se admita la activación de los controles mediante el teclado.
- Todo el contenido visible se procesa utilizando elementos de documento HTML estándar, lo que proporciona compatibilidad con lectores de pantalla y preferencias de visibilidad como configuraciones de alto contraste.
- El funcionamiento y la interpretación de la interfaz de usuario son comprensibles, relevantes para el vocabulario de los usuarios y libres de jerga técnica.

Reglas de política del lado del cliente.

El módulo `votingpolicyengine.py` contiene reglas para determinar qué actos de la ronda actual son elegibles para recibir un voto de un usuario específico. Como se describió anteriormente en el libro, se recomienda encapsular la lógica de negocio, como la política de votación.

elegibilidad, a un único punto de autoridad en una solicitud.

Evitamos realizar llamadas al backend para evaluar las acciones de la ronda actual con las reglas de la política de votación después de cada votación, ya que esto duplicaría con creces el tráfico de red entre cada cliente y el frontend de votación. En su lugar, implementamos un equivalente local del motor de políticas en el JavaScript de nuestro cliente web: intercambiando la integridad potencial de la aplicación por rendimiento.

Esto se abordó en los requisitos del cliente de votación a través de la especificación de un motor de políticas local que se obtiene del backend.

Un punto final adicional en la interfaz de votación proporciona el código real que un cliente debe ejecutar al determinar la elegibilidad de los actos para recibir votos.

La implementación de este servicio en el backend se explicará en el laboratorio de código de este capítulo. También se describirán los mecanismos de JavaScript para añadir funciones dinámicamente a la página actual.

Aquí revisaremos cómo se incorpora esto al cliente web existente.

Primero, refactorizamos la función `markVoteCast`, que es la función de retorno de llamada que se invoca cuando se emite un voto correctamente a través del frontend de votación. Agregamos código a esta función para invocar una función llamada `updateEligibleVotes`, que toma parámetros para todos los valores necesarios para esta operación, almacenados en otra parte de la página: // **Procesar la**

respuesta de una llamada remota de "voto". // Confirmar al
usuario.

// Llamar al motor de políticas de votación descargado del servidor para //
actualizar la tabla de actos para aplicar la función

votebudget markVoteCast(responseJson) { const

act = responseJson['act']; alert('you

vote for ' + act + '!')

updateEligibleVotes(act, document.getElementById('voteTally'),

document.getElementById('voteLimit').value,

document.getElementById('actsTable')

);

```
}
```

La función que invocamos, `updateEligibleVotes()`, no está definida. En JavaScript, existen varias maneras de definir una función, una de las cuales es mediante una declaración dinámica mediante el constructor del objeto `Function()`. Así definimos la función `updateEligibleVotes`, mediante la ejecución de un constructor de funciones, antes de que se invoque la devolución de llamada `markVoteCast()`.

```
()
```

El código fuente del cuerpo de esta función se obtendrá del frontend de votación, en el nuevo endpoint especificado anteriormente en esta sección, en el documento de requisitos del cliente de votación. Dado que realizamos una llamada al frontend de votación, esto deberá realizarse después de que el usuario se haya autenticado, ya que debemos restringir el acceso al servicio a los usuarios autenticados.

Los requisitos de tiempo para la construcción del motor de políticas son similares a los de la carga de la lista de actos. Después de la autenticación del usuario y antes de cualquier votación, agregaremos código a la misma devolución de llamada de `firebase.auth` que se modificó para la función `getActs`. Esta devolución de llamada ahora contiene el siguiente

código: // Escuchar si cambia el estado de autenticación y almacenar la información del usuario en el documento.

```
firebase.auth().onAuthStateChanged(función asíncrona(usuario) {
  if (usuario)
    { // El usuario ha iniciado
      sesión. document.getElementById("userName").innerHTML = user.email;
      user.getIdToken().then((idToken) =>
        { document.getElementById("idToken").value=idToken; getActs();

        fetchPolicyEngine(); });
```

Nota: Almacenar el token de identificación en un elemento de documento de esta manera no es una práctica recomendada en una aplicación de producción y se hace aquí para simplificar. Las alternativas más seguras incluyen almacenar el token en el almacenamiento de sesión del navegador o en la memoria del navegador.

La función `fetchPolicyEngine` emite una llamada a la API REST de nuestro frontend utilizando las mismas técnicas que nuestras llamadas existentes a los puntos finales `/getActs` y `/`

```
vote : // Obtener la implementación del motor de
políticas function fetchPolicyEngine()
{ console.log("Obtener política", new Date().toLocaleString());
  getResponse(document.getElementById('actsForm'),
    getPolicyURL, setPolicyEngine);
}
```

En esta sección se han revisado las diferentes partes de la implementación del motor de políticas remoto y su sincronización en la ejecución de las funciones del cliente de votación. El manejo de la respuesta del servidor y el registro dinámico de la nueva función `updateEligibleVotes()` se describirán en el laboratorio de código de este capítulo.

Aplicación móvil para votantes

Además de un cliente web, tenemos el requisito de entregar una aplicación móvil para una plataforma, iOS o Android, y luego la otra plataforma.

A diferencia de la urgencia por entregar nuestro primer cliente, podemos dedicar tiempo a considerar el desarrollo multiplataforma. Además, contamos con un equipo que trabaja en ambas plataformas móviles, por lo que es muy deseable que se reutilicen al máximo los dos productos.

Herramientas de desarrollo de aplicaciones móviles En el

[capítulo 5](#), Opciones de tecnología del lado del cliente se abordan las ventajas y desventajas de las estrategias de desarrollo como plataforma nativa, plataforma móvil multiplataforma y plataforma multiplataforma, incluido el desarrollo de escritorio. Se revisaron varios ejemplos de productos populares para cada uno de estos, pero las alternativas discutidas no fueron exhaustivas y ninguna lista podría ser completa en un panorama en constante evolución y mejora.

Para reafirmar brevemente los parámetros de nuestra

- evaluación: el desarrollo móvil multiplataforma es ideal.

- La aplicación utilizará las API de GCP, con el intercambio de datos JSON asociado.
- Se requiere compatibilidad con autenticación de Firebase.
- El equipo de desarrollo está muy involucrado en Python.

Dos productos populares que cumplen estos requisitos son Kivy y BeeWare. La principal diferencia entre ambos radica en que Kivy produce aplicaciones Kivy, es decir, aplicaciones que utilizan las herramientas y funciones de Kivy, mientras que BeeWare tiende a integrar otras bibliotecas existentes como parte del producto.

Cualquiera de estos cumple con nuestros requisitos; utilizaremos BeeWare <https://beeware.org/> Dado que nuestra aplicación es bastante limitada, se adapta mejor al uso de bibliotecas y funciones de Python, tanto propias como de terceros. BeeWare no ofrece integración con Visual Studio Code, pero una serie de comandos repetidos en una sesión de terminal integrada funciona bien con las herramientas de BeeWare.

Aplicaciones móviles BeeWare: Implementaremos

una aplicación BeeWare estándar con una interfaz de usuario nativa que presenta pantallas y controles similares a los del cliente web. La compatibilidad con Firebase Authentication para la autenticación de usuarios se realizará mediante la API REST de Firebase Auth, independiente de la plataforma.

Las llamadas a nuestro servicio frontend de votación se realizarán utilizando el paquete httpx estándar de Python.

Una vez que tengamos una aplicación definida, la construiremos e implementaremos para iOS con la utilidad BeeWare Briefcase que construye e implementa aplicaciones BeeWare en plataformas específicas.

Reglas de política del lado del cliente móvil. La

distribución de módulos de tiempo de ejecución a un cliente es un patrón común, y la solución para una aplicación Python es similar a la solución implementada en JavaScript para el cliente web en la sección anterior de este capítulo. Los entornos de tiempo de ejecución móviles cuentan con salvaguardas para bloquear el reemplazo de código en una aplicación en ejecución, lo cual puede ser...

eludido en un caso de uso como el nuestro.

Sin embargo, la necesidad de esto en el cliente web surgió debido a la imposibilidad de que una aplicación JavaScript usara fácilmente un módulo de Python directamente. Con el uso adecuado de la abstracción y el empaquetado, el cliente móvil escrito en Python podrá invocar el mismo motor de políticas que utilizan nuestros componentes en la nube. Se puede evitar la complejidad adicional de la distribución de módulos del cliente en tiempo de ejecución, que es la decisión de diseño tomada en la arquitectura del cliente móvil.

Se incluye una implementación en Python de la instalación de métodos definidos remotamente en el repositorio de este capítulo, en el módulo `talentvoting_policy.ruleinstaller.py`. Este no se utiliza en la aplicación, pero se incluye para completar y comparar con su versión en JavaScript.

La aplicación cliente BeeWare completa y su implementación en iOS se describirán en el laboratorio de código de este capítulo.

Refactorización del código base

Inicialmente, la jerarquía de paquetes se definió desde una perspectiva puramente lógica, independiente de la implementación y del lenguaje. En la práctica, el lenguaje y el marco de trabajo que elegimos se consideran en la taxonomía de paquetes y módulos, y podrían no coincidir con esa definición abstracta previa. Se observaron indicios de esta discrepancia al añadir algunas mejoras ingeniosas a nuestras dos implementaciones de Python en la nube: la creación de enlaces simbólicos y la copia de subárboles de código fuente en los directorios base de nuestras implementaciones para el frontend de votación de Google App Engine y el procesador de votación de Cloud Run.

Portabilidad de paquetes en Python. La

jerarquía de paquetes en Python es descendente: un módulo fuente puede importar otros módulos fuente en relación con el directorio del módulo importador. No es posible referenciar un paquete de anclaje de nivel superior en un directorio fuera del árbol de subdirectorios o directorio principal de la ruta actual. Esto nos obligó a usar enlaces simbólicos y copias para...

Los paquetes comunes aparecen como subdirectorios debajo de los directorios de origen de nuestros módulos en la nube.

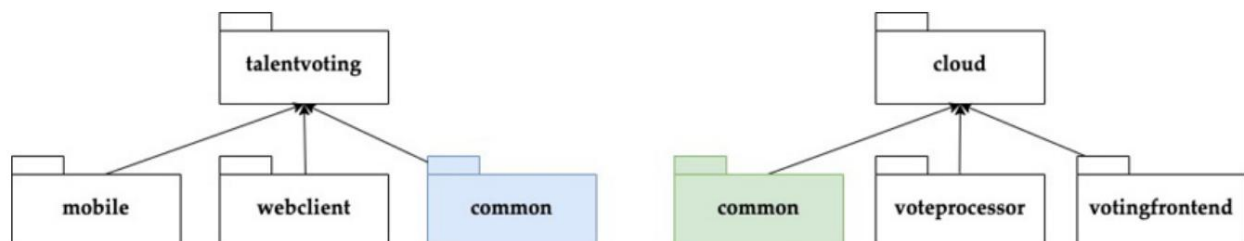
La forma típica de Python de gestionar esta situación es crear un paquete Python con los módulos necesarios y colocarlo en un directorio de paquetes conocido, visible para todo el código fuente del entorno. Así es como se puede acceder a los paquetes integrados, como `time` y `os`, así como a los paquetes de terceros instalados, como `firebase_admin` y `flask`, mediante las siguientes instrucciones de origen, que no se refieren a las rutas de directorio de los paquetes: `import time` `import os`

Desde `firebase_admin`, importar credenciales, autenticación desde `flask`, importar `Flask`, `solicitud`, `jsonify`, `Respuesta`, `make_response`. Existe un árbol de paquetes: `talentvoting.common`, que abarca los árboles de la nube y del cliente. Es ideal para publicarse como un paquete hermético, portátil, sellado y autónomo, que se puede importar con la misma facilidad que los paquetes `time` y `flask` del ejemplo fuente anterior.

El uso de un paquete de Python implica tres pasos: crearlo, distribuirlo e instalarlo. El lector interesado puede obtener más información en <https://packaging.python.org/>. Los pasos básicos para nuestro proyecto se describirán aquí y luego se implementarán en el laboratorio de código de este capítulo.

Aislar la fuente

La siguiente figura ilustra nuestro árbol de fuentes a un alto nivel:



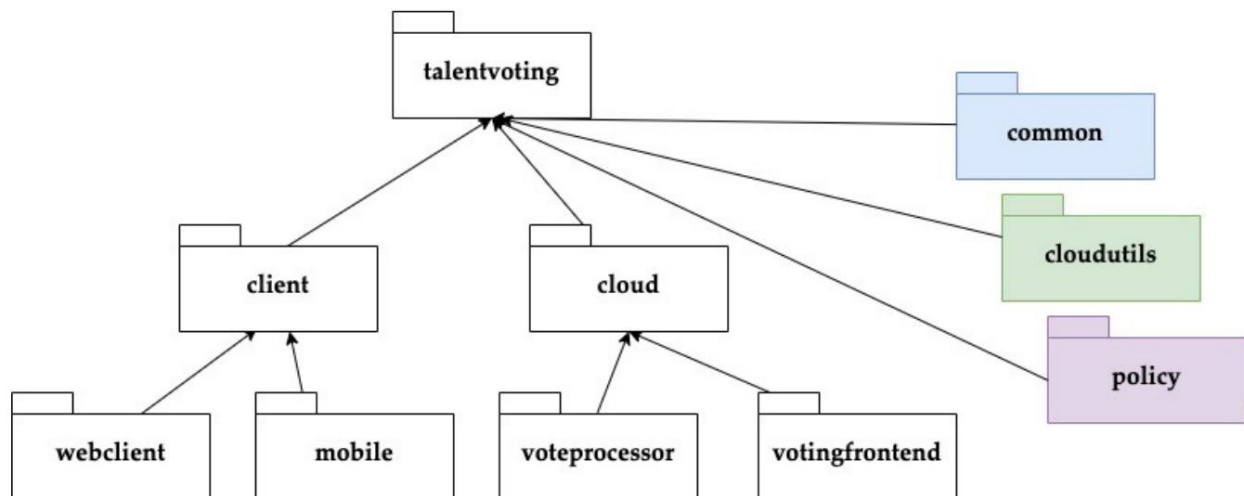
Cifra 12.5 Jerarquía de paquetes fuente de nivel superior

Hay tres relaciones problemáticas en esta jerarquía. La primera es que ambos paquetes comunes son hermanos de sus clientes. Una extensión de esto es que el paquete común de nivel superior, el `talentvoting`. En la jerarquía de paquetes en la nube se hace referencia a la votación de talentos. Si bien esto no es insuperable, la referencia, a un padre familiar

El directorio en Python es frágil y puede resultar confuso para el lector. Esto nos llevó a crear enlaces y copiar directorios de origen al implementar nuestros servicios en la nube.

La segunda cuestión es la del rango o equivalencia de los componentes en el Arquitectura del sistema. Los componentes de la nube: procesador de votos y Los frontend de votación se agrupan bajo la nube porque comparten una entorno común, pero las aplicaciones móviles y de cliente web son tan importante como cualquier componente de la nube, y tiene sentido para estos componentes desplegables que aparecen como hermanos o equivalentes.

Una taxonomía de paquetes alternativa que facilita el uso de Python El embalaje y las soluciones a estas preocupaciones se muestran a continuación. cifra:



Cifra 12.6 Jerarquía de paquetes revisada

Con la nueva estructura de directorios en su lugar, definiremos los tres paquetes sombreados comunes, política (anteriormente conocida como `common.policy`) y `cloudutils` (anteriormente conocido como

cloud:common) que se incluirán en paquetes de Python que construiremos y distribuiremos independientemente de los módulos que los utilicen.

Una vez que estos paquetes son portables (es decir, no tienen dependencias externas ni referencias relativas a sus propios límites o ubicación), cada uno puede considerarse una entidad independiente de nivel superior. Podemos importarlos y usarlos desde cualquier número de módulos, cada uno de los cuales puede ubicarse en cualquier punto de nuestro árbol de código fuente.

Definiendo el paquete

Un paquete Python, con el propósito de importar módulos fuente de otros módulos fuente en el mismo árbol de directorios, consta de un directorio cuyo nombre es el nombre del paquete y la presencia de un archivo vacío especial llamado `__init__.py` en ese directorio.

Al definir un paquete para su distribución e instalación independientes, existen requisitos adicionales que se definen según la herramienta de backend de compilación elegida. Utilizaremos el conjunto de requisitos centrado en una tabla [proyecto] para metadatos, un formato compatible con varios backends de compilación populares. Y lo más importante, utilizaremos GCP Cloud Build para almacenar nuestros paquetes en un Registro de Artefactos de Google Cloud, que utiliza las herramientas y el formato que hemos seleccionado.

El archivo `pyproject.toml` ubicado en la carpeta raíz de nuestro árbol de fuentes de paquetes le dice a las herramientas de compilación frontal qué backend usar para nuestro proyecto y qué dependencias externas tiene nuestro paquete.

Además, en este archivo se especifican los metadatos del proyecto, como la página de inicio del proyecto y el tipo de licencia.

Un archivo `README.md` es un archivo markdown que es el nivel raíz de la documentación de su paquete.

El archivo `LICENCIA` contiene el texto de la licencia elegida. Si se especifica la URL de una licencia conocida en `pyproject.toml`, las herramientas de backend obtendrán automáticamente este archivo al compilar el paquete.

Identificación de un repositorio de paquetes

Los paquetes de Python se pueden almacenar e instalar desde directorios locales o desde un repositorio remoto. El índice de paquetes de Python está disponible en <https://pypi.org>. Es el repositorio estándar o predeterminado para la distribución pública de paquetes. Para la aplicación de votación, utilizaremos un Registro de Artefactos de Google asociado a nuestro proyecto para almacenar nuestros paquetes de Python. Los pasos para ello se incluyen en el laboratorio de código. Utilizaremos la herramienta de instalación estándar de Python, pip, para instalar los paquetes en nuestro entorno de desarrollo local desde el Registro de Artifacts.

Las versiones implementadas de nuestros servicios en la nube: votingfrontend y voteprocessor se configurarán para instalar automáticamente las últimas versiones de nuestros paquetes publicados desde este registro.

Crear o construir un paquete

Los paquetes de distribución se crean a partir de los archivos fuente, incluidos los archivos de empaquetado descritos en "Definición del paquete". Para crear un paquete de distribución, se utiliza la utilidad de compilación de Python para crear archivos de paquetes. La compilación genera dos tipos de archivos: distribuciones fuente, cuya extensión de archivo es .tar.gz, y distribuciones compiladas o binarias, cuya extensión de archivo es .whl.

Distribución de paquetes

El Registro de Artefactos de Google es compatible con las API y los requisitos de las herramientas de distribución e instalación de Python existentes. La utilidad estándar de Python Twine carga los paquetes de Python en los registros compatibles, por lo que la usaremos para cargar los archivos de paquetes a nuestro Registro de Artefactos de Google proporcionando la URL del registro al comando.

Instalación de paquetes

Los paquetes de nuestro Registro de Artefactos se pueden instalar mediante la utilidad estándar de Python pip. Al igual que el comando twine, el comando pip install acepta la URL de nuestro registro como...

parámetro que especifica la ubicación de los paquetes que se están instalando.

Cuando implementamos nuestros servicios de Google App Engine y Cloud Run, agregaremos los nombres de nuestros paquetes relevantes al archivo `requirements.txt` para cada uno de esos servicios y especificaremos la URL del registro como un índice adicional en el mismo archivo `requirements.txt`.

Codelab para la creación de múltiples aplicaciones cliente

En el laboratorio de código de este capítulo, se describirán los distintos pasos necesarios para producir una aplicación de cliente móvil que sea coherente con los requisitos del cliente de votación.

Hay muchos pasos para esto, cada uno de los cuales se cubrirá. Aquí se incluirán secciones del código fuente relevantes para los puntos planteados; el código fuente completo está disponible en el GitHub de este capítulo.

La entrega de una aplicación cliente consistente se divide en las siguientes subtareas en este laboratorio de código:

- Refactorización de la jerarquía de paquetes.
- Creación y distribución de paquetes de Python a un registro de artefactos de Google.
- Modificar los descriptores de implementación para utilizar el Registro de artefactos.
- Configuración de un entorno de desarrollo BeeWare.
- Escribiendo la aplicación cliente BeeWare.
- Implementar la aplicación en un simulador de iOS.

El resultado será un ciclo de lanzamiento independiente para el código común y cada componente implementado. Nuestra aplicación cliente móvil se podrá implementar en dispositivos iOS y extender a Android y otras plataformas compatibles con BeeWare.

Refactorización de la jerarquía de paquetes. La reorganización

de la jerarquía de paquetes se realiza en tres pasos. Primero, simplemente renombramos y movemos los directorios y módulos a sus nuevas ubicaciones, independientemente de la consistencia referencial entre ellos. Después, podemos usar las funciones de nuestro IDE, Visual Studio.

Código para identificar problemas en el espacio de trabajo. Estos problemas incluirán declaraciones de importación que no se pueden resolver. Revisaremos y corregiremos cada uno de estos problemas, aunque las importaciones de nuestros tres paquetes portables no se resolverán hasta que hayamos terminado de compilarlos e instalarlos. El tercer paso para refactorizar la jerarquía será compilar los tres paquetes de Python, instalarlos localmente e implementarlos en un Registro de Artefactos en la Nube. Esto resolverá los problemas del espacio de trabajo y los problemas de implementación en la nube introducidos durante la refactorización.

La nueva estructura de directorios es la ilustrada en [la Figura 12.6](#). Para mover carpetas y archivos a sus nuevas ubicaciones, debemos usar el comando `git mv`, que garantiza que GitHub registre los movimientos y cambios de nombre de carpetas y archivos. Sin usar `git mv`, GitHub consideraría estos movimientos como eliminaciones de archivos antiguos y la adición de nuevos.

Nota: Asegúrese de cerrar su IDE o editores antes de mover archivos que puedan tener abiertos.

La siguiente tabla enumera las carpetas y archivos de origen y destino que se deben mover. Estos deben ejecutarse en el orden que se muestra en la siguiente figura para garantizar que las carpetas principales estén presentes cuando sea necesario:

Original folder or file	New folder or file	Note
cloud/__init__.py	talentvoting/cloud/__init__.py	Copy to the existing alternate location of the cloud directory
cloud/common	talentvoting/cloudutils	move and rename
cloud/votingfrontend/talentvoting	none	remove link
cloud/voteprocessor/talentvoting	none	remove directory and its contents
cloud/voteprocessor	talentvoting/cloud/voteprocessor	
cloud/votingfrontend	talentvoting/cloud/votingfrontend	
talentvoting/cloud/common	none	remove link
None	talentvoting/client	create this new directory
talentvoting/webclient	talentvoting/client/webclient	
None	talentvoting/client/mobile	create this new directory
talentvoting/common	talentvoting/common	no change
talentvoting/common/policy	talentvoting/policy	

Cifra 12.7 Movimiento de carpetas y archivos de paquetes

Una vez reorganizadas las carpetas y los archivos, es necesario actualizar las referencias a las ubicaciones de los paquetes fuente antiguos. Este mensaje, que aparece en la pestaña "Problemas de VS Code" del archivo, muestra un ejemplo.

votingfrontend.py: "

No se pudo resolver la importación de .

"talentvoting.cloud.common.votingdatabaseutils" [Ln Col

Para solucionar esto, actualice la declaración Import para hacer referencia a la nueva ubicación de la [12.7](#):

carpeta, como se ve en [la Figura](#) from

talentvoting.cloud.common.votingdatabaseutils

import get_database Lo anterior se reemplaza por lo siguiente: from

talentvoting.cloudutils.votingdatabaseutils import get_database El mismo cambio es necesar

Revise cada archivo .py para encontrar referencias rotas a módulos importados y actualícelos con los tres nuevos nombres de paquete.

[Creación y distribución de paquetes de Python](#). El código

fuelle de nuestro sistema se encuentra ahora en una estructura de directorios que refleja la compartimentación de los componentes por plataforma, como los subpaquetes de cloud, y la afinidad por rol, como los subpaquetes de client. Los árboles de directorios cuyo uso abarca diferentes subárboles de nuestra estructura de directorios (commonpolicy y cloudutils) ahora son independientes de sus clientes.

Este último punto es el enfoque de esta sección. Tras separar estos tres subárboles, podemos ponerlos a disposición de todos los clientes, con la misma visibilidad y utilizando los mismos mecanismos estándar.

Nombre del paquete

Crearemos tres paquetes de Python: talentvoting_common, talentvoting_policy y talentvoting_client. Existe una práctica de

empaquetado, el uso de paquetes de espacio de nombres, que podría utilizarse para permitir que los nombres de nuestros paquetes se agrupen lógicamente en un espacio de nombres común, por ejemplo, talentvoting.common, talentvoting.policy y talentvoting.client.

El uso de paquetes de espacio de nombres tiene varias desventajas y ofrece pocos beneficios.

Colección de paquetes como el nuestro. Podemos recomendar y usar el siguiente formato de importación para que el objeto de importación sea breve, si se

desea: `import talentvoting_common como common`

`import talentvoting_cloudutils como cloudutils import`

`talentvoting_policy como policy` La

instalación de paquetes de Python incluye no solo los archivos fuente del paquete, sino también sus dependencias. Esta es la principal razón práctica para dividir nuestros módulos en tres paquetes: el módulo `cloudutils` depende de los paquetes de Google Cloud, lo cual no tiene sentido en un cliente móvil como consecuencia de la instalación de un solo paquete cuando solo se usará el módulo común .

Otra razón es la dependencia tanto de la política como de `cloudutils` de lo común: separarlas nos permite indicar explícitamente dichas dependencias y controlar las versiones con mayor granularidad.

Metadatos y configuración del paquete

Con el código fuente reorganizado, creamos los metadatos del paquete que utilizan las herramientas de compilación e instalación. El contenido completo de los archivos de cada uno de nuestros tres paquetes se encuentra en el archivo del repositorio de este capítulo; aquí enumeraremos los archivos y su propósito.

Antes de emitir comandos de Cloud Spanner, se debe habilitar la facturación para el proyecto.

En una terminal de Linux o MacOS, los comandos y respuestas se verán así:

Nota: Las líneas que comienzan con `#` son comentarios para el lector. •

`# Asegúrate de estar en el directorio base del proyecto del paquete $ ls pyproject.toml`
`pyproject.toml #`

Instala el

comando de compilación de Python \$

`python3 -m pip install --upgrade build`

...

```
# Construye los paquetes, se crearán dos archivos en el directorio dist/ $ python3 -m
build
```

```
...
```

```
Se crearon correctamente talentvoting_common-0.0.0.tar.gz y
talentvoting_common-0.0.0-py3-none-any.whl $ ls dist
```

```
talentvoting_common-0.0.0-py3-none-any.whl
```

```
talentvoting_common-0.0.0.tar.gz. Las
```

versiones fuente y binaria de este paquete ya están compiladas y almacenadas localmente. El siguiente paso es publicarlas en una fuente de distribución.

Distribución de paquetes mediante un Registro de Artefactos. Para publicar nuestros paquetes en un Registro de Artefactos en la Nube, debemos habilitar la API de Registro de Artefactos para nuestro proyecto. Esta API requiere un proyecto de facturación activo al momento de su uso, por lo que también lo haremos. Estos son los mismos pasos que se realizaron en el laboratorio de código del [capítulo 11](#), "Garantizar la Permanencia de los Datos".

La siguiente sección utiliza una terminal en varias etapas del proceso. Se recomienda usar la misma sesión de terminal para preservar la continuidad de la ubicación y el estado de inicio de sesión.

En una terminal:

```
# Inicie sesión y configure las credenciales
```

```
correctas $ gcloud auth default-login
```

Su navegador ha sido abierto para visitar:

```
...
```

Ahora estás conectado como [raygeeknyc@gmail.com].

Tu proyecto actual es "votingforthestars". Puedes cambiar esta configuración ejecutando:

```
"gcloud config set project PROJECT_ID"
# Guarde el 2 en una variable $ export
O2TOKEN=$(gcloud auth print-access-token)
```

Vincula tu cuenta de facturación al proyecto

Reemplace votingforthestars con el ID de su proyecto y

[BILLING_ACCOUNT_ID] con su cuenta de facturación

Enlace de proyectos de facturación de gcloud alpha votingforthestars --billing-

account=[BILLING_ACCOUNT_ID] _]

nombreDeCuentaDeFacturación: CuentasDeFacturación/ID_DE_CUENTA_DE_FACTURACIÓN

facturación habilitada: verdadero

nombre: proyectos/votaciónparalasestrellas/informacióndefacturación

ID del proyecto: votando por las estrellas

A continuación, en la consola del desarrollador, busque Artifact Registry, haga clic en a través de las indicaciones para habilitar la API para su proyecto.

Ahora regrese a la Terminal y ejecute los siguientes comandos para crear un repositorio para sus paquetes de Python:

Crear un repositorio \$ gcloud artifacts repositories create

repositorio de Python para votación de talentos

--repository-format=python \ =

--ubicación=us-central1 \ 1

Repositorio de paquetes de Python para Talentvoting"

Solicitud de creación emitida para: [talentvoting-python-repo]]

...

Se creó el repositorio [talentvoting-python-repo].]

Enumere todos los repositorios del proyecto y confirme que este esté incluido en la lista

Lista de repositorios de artefactos de gcloud

Listado de elementos bajo el proyecto votingforthestars, en todos , Ubicaciones. .

REPOSITORIO	FORMATO	MODO	UBICACIÓN	DESCRIPCIÓN
ETIQUETAS	CIFRADO	TAMAÑO (MB)	HORA DE CREAR	
HORA DE	—	()		
ACTUALIZACIÓN	us.gcr.io	REPOSITORIO	ESTÁNDAR DE DOCKER	

Clave administrada por Google 2024 12 08 15:43:52 a nosotros- 2025 02 16 14:58:34 3340.152

Repositorio de Python para votación de talentos REPOSITORIO ESTÁNDAR DE PYTHON

Repositorio de paquetes de Python para la votación de talentos central1 Clave administrada por Google 2025 04 09 16:04:35 a nosotros- 2025 04 09 16:04:35 0

...

Establezca este repositorio como predeterminado para los siguientes comandos en esta terminal

\$ gcloud config set artefactos/repositorio/talentvoting-python-repo

Propiedad actualizada [artefactos/repositorio].

\$ gcloud config set /us-central1 1

Propiedad actualizada [/].

Finalmente, cargamos los archivos del paquete en nuestro directorio dist/ al Artifact Registro.

En la terminal:

Sube los dos paquetes en el directorio dist

\$ python3 -m twine upload --url del repositorio https://us-central1-python.pkg.dev/votingforthestars/votacióndetalentos-python-repo/ -- nombre de usuario o autenticación de acceso --contraseña \$O2TOKEN dist/* 1 /

Subiendo 1.0.0 3 .

200 OK

Subiendo talentvoting_common-1.0.0.tar.gz 1.0.0 .

200 OK

Listar los paquetes en el repositorio

\$ lista de paquetes de artefactos de gcloud --repository=talentvoting-python-repo

ANOTACIONES DEL PAQUETE	HORA DE CREAR	HORA DE ACTUALIZACIÓN
voto-de-talento-común	2025 04 09 16:45:00	2025 04 09 17:10:31

Ahora contamos con un Registro de artefactos de Google Cloud que distribuye nuestros paquete común tanto en formato fuente como binario.

Se deben seguir los mismos pasos para los paquetes cloutils y de políticas . Los archivos de configuración de los paquetes se encuentran en el archivo del repositorio de este capítulo.

Instalación de paquetes de Registro de artefactos local Ahora que los paquetes

están disponibles, podemos usar la utilidad pip estándar para instalarlos en nuestro entorno de desarrollo.

La autenticación de Google Cloud funciona con el comando pip a través de varias interfaces: autenticación HTTP y anillo de claves. Google Cloud recomienda usar el anillo de claves, que es el más seguro y cuenta con mayor compatibilidad. Para configurar el anillo de claves en su entorno, consulte la documentación para configurar la autenticación del repositorio de paquetes de Python en:

<https://cloud.google.com/artifact-registry/docs/python/authentication> De vuelta en la sesión de

Terminal:

```
# Instalar el paquete común desde GCP $ pip install --
index-url \ https://us-central1-
python.pkg.dev/ 1
votingforthestars/talentvoting-python-repo/ \ talentvoting-common Se instaló /
correctamente talentvoting-
common-1.0.0 Puede confirmar que el paquete está instalado 1.0.0
localmente en un intérprete interactivo de Python.
```

En la Terminal, ejecute los siguientes comandos que importan el paquete instalado, importan un símbolo de un subpaquete y muestran la definición de ese símbolo: \$ python Escriba "ayuda", "copyright", "créditos"

o "licencia"

para obtener más información. ,

.

```
>>>importar talentvoting-common desde
```

```
talentvoting_common.interfaces.servicelocations .
```

```

importar DOMINIO_CLIENTE_WEB_DE_VOTACIÓN
>>>DOMINIO_CLIENTE_WEB_DE_VOTACIÓN
'https://votingforthestars.web.app' .
>>>

```

Hemos instalado correctamente el paquete `talentvoting_common` desde nuestro Registro de Artefactos en la Nube. Siga los mismos pasos para instalar el paquete `talentvoting_cloudutils` .

[Implementación en la nube mediante paquetes de Artifact Registry](#). La instalación local

de paquetes nos permite compilar y compilar en nuestro entorno de desarrollo. Cuando una aplicación se implementa en los servicios de Google App Engine o Google Cloud Run, estas dependencias no se incluyen automáticamente con el código fuente de nuestro componente en la nube. Las dependencias de un componente de servicio de GCP deben especificarse explícitamente en el archivo `requirements.txt` del componente .

Cuando implementa estos servicios, las dependencias especificadas en el archivo `requirements.txt` se instalarán automáticamente con la aplicación implementada.

Dado que utilizamos nuestro propio repositorio de paquetes para los paquetes `talentvoting_common` y `talentvoting_cloudutils` , también especificaremos la URL de nuestro Registro de artefactos como una fuente adicional de paquetes.

El nuevo contenido de nuestra configuración del servicio Google App Engine en el archivo `talentvoting/cloud/votingfrontend/requirements.txt` es el siguiente: **--extra-**

```

index-url=https://us-central1-python.pkg.dev/
votingforthestars/talentvoting-python-repo/simple/

```

```
Matraz==3.0.3
```

```
gunicorn
```

```
firebase_admin
```

```
google-cloud-pubsub
```


`google-cloud-spanner``talentvoting_common``talentvoting_cloudutils``talentvoting_policy` El

nuevo contenido de nuestra configuración del servicio Google Cloud Run en el archivo `talentvoting/cloud/voteprocessor/requirements.txt` es el siguiente: `--extra-index-url=https://`

`us-central1-python.pkg.dev/votingforthestars/``talentvoting-python-repo/simple/``gunicorn==23.0.0``Frasco==3.0.3``Werkzeug==3.0.3``google-cloud-pubsub google-``api-core google-``cloud-spanner``talentvoting_common``talentvoting_cloudutils``talentvoting_policy`

Cuando implementamos el servicio Cloud Run y el frontend del motor de aplicaciones, las herramientas de implementación instalarán nuestros tres paquetes en los contenedores implementados de los servicios desde su repositorio, ubicado en `--extra-index-url`.

Nota: El comando de implementación del servicio `voteprocessor` ya no requiere los comandos `cp` que realizaban copias locales del código fuente de nuestros paquetes comunes. Estos se instalan correctamente desde el archivo `requirements.txt`. Por lo tanto, el comando para implementar el servicio `voteprocessor` ahora es simplemente el siguiente:

```

$ gcloud run deploy voteprocessor --allow-unauthenticated \ --max-instances=10 --
source . =10 .

```

Desarrollo con BeeWare

Nuestras aplicaciones cliente deben ser idénticas en funcionalidad y lo más similares posible en forma: su diseño y conjunto de características deben ser paralelos, y cualquier desviación debe ser impuesta o guiada por las pautas y requisitos específicos de la plataforma.

La implementación de las tres aplicaciones cliente planificadas está influenciada por esta expectativa de homogeneidad, pero también existe la consideración técnica de optimizar el desarrollo mediante la reutilización de código y procesos.

Un factor importante es la elección de la menor cantidad posible de lenguajes de programación, lo que tiene impacto de dos maneras.

La primera consideración es que la necesidad de que los ingenieros conozcan y cambien entre varios idiomas restringe la comunidad de desarrolladores e impone un impuesto o fricción a los desarrolladores a medida que cambian de contexto entre idiomas.

El segundo es el riesgo que conlleva el código redundante. El uso de un lenguaje nos permite definir valores como la URL de un punto final de servicio o el número de votos permitidos en una ronda, una sola vez. Esto se ilustró en el módulo `talentvoting_common.interfaces.servicelocations`, cuyo contenido parcial es el siguiente: `_VOTE_INGESTOR_ADDRESS = https://votingforthestars.ue.r.appspot.com` `VOTE_URL =`

```
_VOTE_INGESTOR_ADDRESS + "/vote"
://
```

```
PROJECT_ID = "votandoporlasestrellas"
```

```
SPANNER_INSTANCE = "instancia de votación de talentos"
```

```
SPANNER_DATABASE = "base de datos de votación de talentos"
```

Un ejemplo más sustancial es el objetivo de proporcionar implementaciones canónicas, únicas y autoritativas de lógica de negocio crítica. El método `is_eligible_vote` del módulo `talentvoting_common.interfaces` es un ejemplo de ello. Los métodos de este módulo pueden importarse y utilizarse en diferentes aplicaciones en la nube y cliente escritas en Python.

Nuestra concesión a este principio fue proporcionar un método JavaScript equivalente. Conteniendo los dos métodos dentro de un mismo módulo.

Nos permite ofrecer dos codificaciones separadas del mismo comportamiento y reduce, pero no elimina, el riesgo de que estas implementaciones diverjan.

El conjunto de bibliotecas y herramientas BeeWare permite el desarrollo de aplicaciones Python que pueden compilarse y ejecutarse en diferentes plataformas de destino, utilizando las bibliotecas y frameworks nativos de cada una. Desarrollaremos nuestra aplicación móvil con BeeWare y luego la compilaremos e implementaremos para iOS.

Descripción general de BeeWare

BeeWare es un conjunto de herramientas y bibliotecas que se utilizan juntas para definir e implementar aplicaciones multiplataforma con una interfaz gráfica de usuario (GUI) nativa en Python.

La suite BeeWare incluye:

- Toga, un kit de herramientas de widgets multiplataforma.
- Maletín, una herramienta para empaquetar proyectos de Python para plataformas de destino.
- Bibliotecas multiplataforma como Rubicon ObjC para acceder a bibliotecas nativas de la plataforma.
- Distribuciones de Python para plataformas donde no se suministran entornos Python.

BeeWare está disponible en macOS, Windows, Linux, clientes web, Android e iOS.

Instalación de BeeWare

BeeWare es una serie de herramientas, como Briefcase y sus dependencias, como versiones recientes de Python y Git. Todas se instalan mediante herramientas de instalación estándar como pip, brew y los paquetes de instalación de la plataforma.

Cómo escribir una aplicación BeeWare

Una aplicación GUI de BeeWare se construye en gran medida en torno a la interfaz gráfica del desarrollador.

Elección del kit de herramientas de interfaz de usuario. BeeWare proporciona el kit de herramientas de interfaz de usuario Toga, pero una aplicación BeeWare podría usar otros frameworks de interfaz de usuario nativos o de terceros. Con la aplicación básica definida para dicho framework, el resto de la aplicación es prácticamente independiente de BeeWare: las funciones estándar de Python y otras bibliotecas se pueden usar libremente en el cuerpo de la aplicación.

Un ejemplo del uso gratuito de las funciones estándar de Python se ilustra en estos fragmentos de código de una versión transitoria del módulo `talentvoting.client.mobile.src.voteclient.app`, el módulo principal de nuestra aplicación cliente BeeWare. El código fuente de la versión final se incluye en la instantánea del repositorio de GitHub de este capítulo: `from talentvoting_common.acts import`

```
example_acts
```

```
...
```

```
    para actuar en example_acts():
```

```
        act_row = toga.Box(style=Pack(direction=ROW))
```

```
        act_row.add(toga.Label(act["nombre"],
estilo=Pack(ancho=COL1_WIDTH_PX))) si
```

```
        acto["elegible para votar"]:
```

```
            acto_fila.add(toga.Button(icono=brutus_icon, id=acto["acto"],
```

```
                on_press=controlador_botón_de_votación)) de lo
```

```
            contrario:
```

```
                act_row.add(toga.Button(icon=icono_de_grillo, habilitado=Falso))
```

```
            acts_list.add(act_row)
```

```
...
```

Nota: El uso Pythonesque del dict que es devuelto por nuestro método `examplesActs()`.

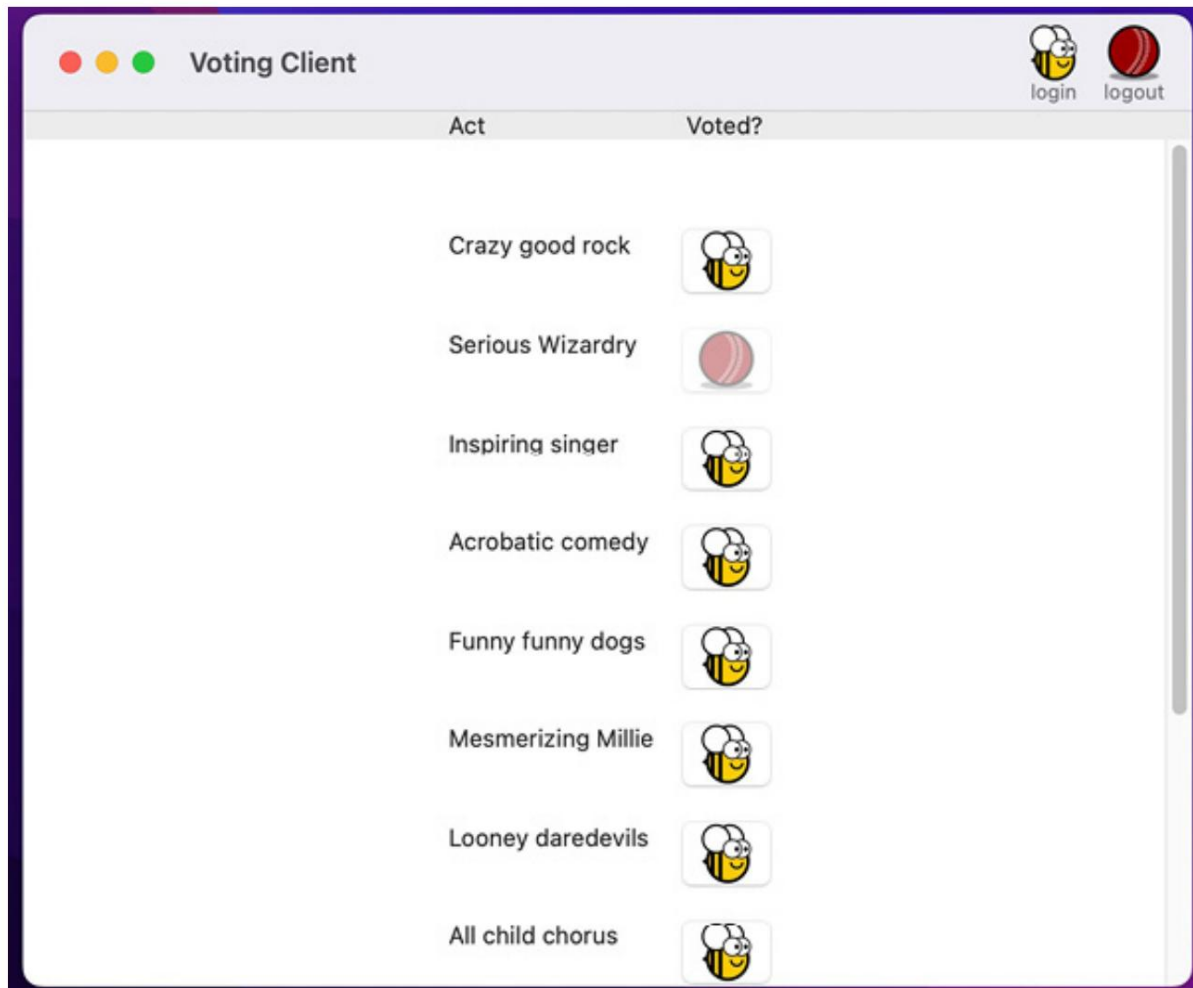
```
    .
```

El ciclo de desarrollo de una aplicación BeeWare se define mediante una secuencia de subcomandos y objetivos intermedios de la herramienta Briefcase. Puede encontrar una descripción completa del ciclo de desarrollo en el sitio web del tutorial de BeeWare:

<https://docs.beeware.org/en/latest/> . / / /

Briefcase puede ejecutarse en nuestra aplicación como cliente nativo en nuestro entorno de desarrollo. Esto suele ser la opción preferida y suficiente para probar cambios durante una sesión de desarrollo.

La siguiente figura muestra nuestra ventana principal ejecutándose en macOS:



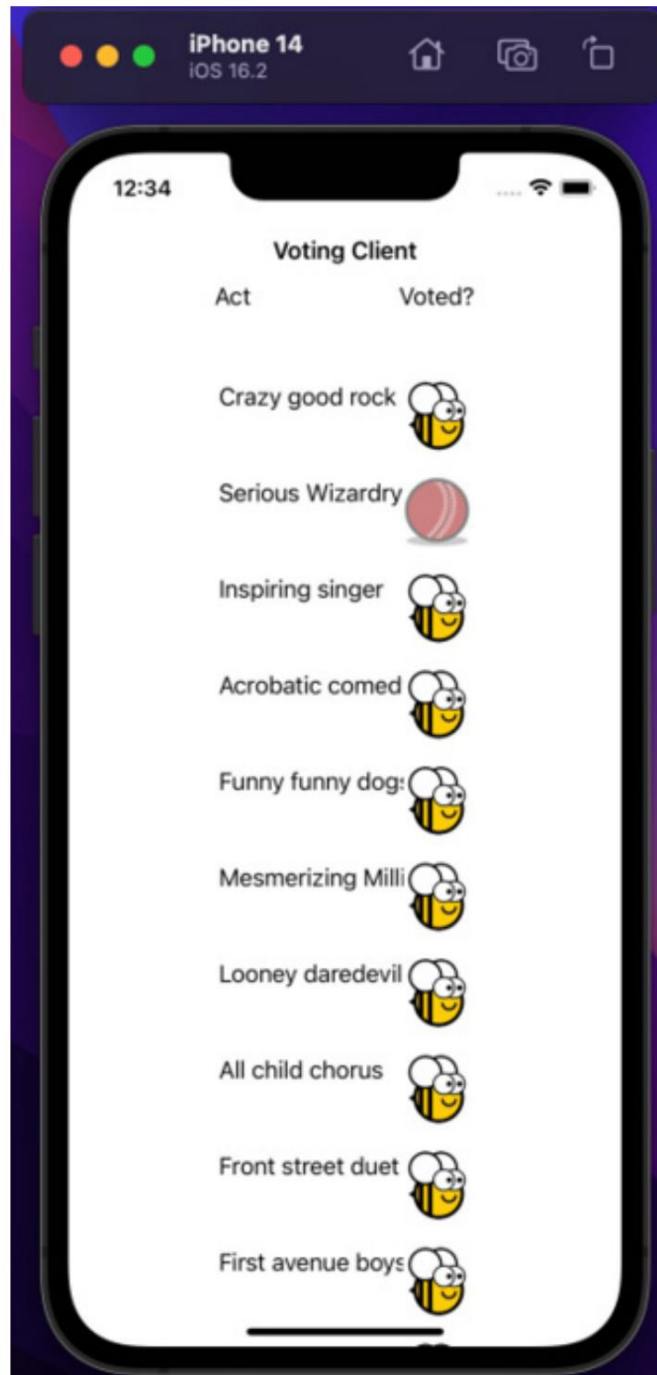
Cifra **12.8** El cliente de votación en macOS

El diseño básico y el comportamiento de la aplicación son consistentes en todas las plataformas de cliente y el ciclo de creación e implementación para la plataforma de desarrollo es mucho más rápido que el de empaquetar, implementar y ejecutar en otro cliente, como iOS o Android.

Implementación de la aplicación en iOS

Para publicar realmente una aplicación en la App Store se necesita una cuenta.

La creación e interacción con el Soporte para Desarrolladores de Apple queda fuera del alcance de este libro. Sin embargo, el desarrollo en iOS no se ve limitado por esto; las herramientas para desarrolladores de Apple incluyen simuladores para numerosos dispositivos iOS y iPadOS, que permiten depurar y probar aplicaciones iOS. La herramienta BeeWare Briefcase es totalmente compatible, y nuestra aplicación puede ejecutarse en un simulador de iPhone, como se muestra en la siguiente figura:



Cifra 12.9 El cliente de votación en un iPhone 14 simulador

La aplicación cliente de votación ahora se puede implementar en iOS y con algunos
Más configuración y ajustes a la aplicación, un cliente
También se pueden crear aplicaciones para entornos Android y de escritorio.
y desplegado.

Conclusión

Una aplicación cliente debe tener responsabilidades claramente definidas que complementen las fortalezas de la plataforma cliente: una conexión y configuración localizadas para cada usuario. Los límites de los niveles de cliente y backend deben definirse intencionalmente, y las implementaciones posteriores de cada uno deben cumplir con sus requisitos y, por lo tanto, confiar en que la implementación del otro nivel cumpla con los suyos. El código base se refactorizó para aislar varias jerarquías de módulos reutilizables en paquetes Python portables que posteriormente se implementaron en un servicio de Google Cloud Platform: Google Artifact Registry.

El desarrollo de componentes del lado del cliente y del servidor a veces puede introducir directivas conflictivas: los deseos de mantener la evaluación de la política de votación local y, sin embargo, compartir el código estaban en desacuerdo en nuestro cliente web. Se presentó al lector un patrón de diseño que aborda tales situaciones: la instalación remota de código en tiempo de ejecución.

La elección de una herramienta de desarrollo multiplataforma debe considerar diversos factores, lo que finalmente nos llevó a utilizar BeeWare. Se describió el desarrollo de una aplicación cliente mínima en BeeWare para ilustrar el uso de frameworks y bibliotecas independientes de la plataforma.

En el capítulo "Reemplazo de componentes en aislamiento", se profundiza en la portabilidad y se aborda la inevitable necesidad de que un sistema exitoso migre algunos componentes a una nueva infraestructura. El lector aprenderá prácticas innovadoras para facilitar dicho mantenimiento y se le guiará por los pasos para realizar una migración dentro de un sistema en funcionamiento.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com>



Sección 4:

Resumen: Aplicación del caso
Estudiar

CAPÍTULO 13

Reemplazo de componentes en Aislamiento

Introducción

Las plataformas en la nube están en constante evolución y crecimiento, agregando nuevas capacidades que a menudo reemplazan la base existente de su sistema. La migración a un nuevo servicio en la nube se justifica demostrando que el valor agregado por el nuevo servicio excede el costo de la migración.

Reconocer el valor añadido de un nuevo servicio requiere un análisis crítico de las afirmaciones del proveedor. El equipo de desarrollo influye en el coste de la migración en dos etapas del ciclo de vida del sistema: al diseñar la arquitectura del sistema y al planificar y ejecutar la migración propuesta.

En capítulos anteriores del libro se han abordado los principios de diseño que maximizan el aislamiento de los componentes de sus dependencias de infraestructura. Este capítulo describe las mejores prácticas para actualizar a servicios nuevos o equivalentes con mínimas refactorizaciones y modificaciones de ingeniería: el lector aprenderá a hacer que lo revolucionario sea evolutivo.

Estructura

En este capítulo se tratarán los siguientes temas:

- Diseño de abstracciones para responsabilidades de infraestructura
- Mapeo de la funcionalidad de la base en interfaces de infraestructura
- Contener dependencias mediante la convergencia a cajas negras
- Planificación de una migración
- Cambiar los neumáticos mientras el autobús está en movimiento
- Diseño de la migración de un laboratorio de código de almacén de datos

Objetivos

Los principios de ingeniería para maximizar la reutilización de software se han establecido durante décadas, quizás formalizados por primera vez por Brad Cox durante el desarrollo del lenguaje de programación Objective-C como vehículo para la creación de componentes de software. Es a partir de este trabajo que ha evolucionado el concepto de componentes de software listos para usar. No existe una solución completa y predefinida, pero en este capítulo describiremos las prácticas establecidas y los servicios disponibles de Google Cloud Platform (GCP) que maximizan la interoperabilidad de nuestro software con la mayor cantidad posible de componentes de nivel inferior. Al definir claramente los límites entre nuestra aplicación y la infraestructura dependiente, y luego definir con precisión los requisitos que imponemos a nuestros servicios de infraestructura, aumentamos la capacidad de evaluar alternativas adecuadas.

Realizaremos un estudio de caso del reemplazo de un componente de infraestructura, en este caso un servicio de persistencia, en un sistema en ejecución.

A través de esto, destacaremos las ventajas obtenidas de las prácticas de diseño pertinentes y los riesgos restantes. Para evitar perjuicios al negocio, elaboraremos planes para reducir y mitigar los riesgos identificados.

Diseño de abstracciones para responsabilidades de infraestructura

Un punto de partida beneficioso al diseñar las interfaces de la aplicación con la infraestructura para maximizar la portabilidad es partir de una perspectiva independiente del servicio o producto. En lugar de considerar las características de la

servicio de infraestructura, en lugar de centrarse en las necesidades que sus propios componentes de software tienen de cualquier infraestructura del sistema que los respalde.

Este es un ejemplo clásico de un enfoque basado en las necesidades para especificar la interfaz entre dos sistemas. En lugar de responder a las afirmaciones y capacidades publicadas por un proveedor de servicios, el consumidor o cliente especifica sus necesidades. El trabajo consiste entonces en identificar cómo un proveedor candidato puede satisfacerlas mejor, con algún método para medir el esfuerzo y la calidad de la solución. En definitiva, la clasificación de los proveedores candidatos es un resumen directo de las calificaciones de la solución, a menudo ponderadas según la criticidad de cada requisito individual.

[Evaluación de infraestructura basada en requisitos.](#) Este proceso se lleva a cabo durante las fases iniciales de nuestro proyecto de desarrollo, cuando se diseña la arquitectura para que se adapte óptimamente a las opciones de plataforma de infraestructura disponibles. Sin embargo, se repite posteriormente, cuando un proyecto se encuentra en mantenimiento o necesita un rediseño importante o cambios forzados.

Los cambios en la infraestructura podrían ser necesarios debido a cambios en el mercado o cambios disruptivos en la tecnología.

Los objetivos de este proceso de evaluación difieren entre el diseño inicial y las fases posteriores. Durante el diseño inicial, buscamos configurar tanto los componentes de la aplicación como la infraestructura elegida para encontrar la mejor opción; los requisitos pueden ser flexibles hasta cierto punto. Al realizar esta evaluación en un sistema en funcionamiento para evaluar proveedores de infraestructura nuevos o modificados, consideramos que los requisitos son más rígidos y evaluamos a los proveedores de infraestructura para ver qué tan bien pueden igualar o mejorar el soporte de los servicios existentes de nuestra aplicación.

[Plantilla para el análisis de requisitos.](#) Para aclarar este proceso, ejecutémoslo en un par de casos de uso de la aplicación de votación. El primer ejemplo es la selección de un servicio de persistencia, también conocido como sistema de almacenamiento, para el backend de votación.

Primero, identificaremos el componente de nuestra arquitectura que es cliente de un servicio de persistencia. Según nuestro diseño inicial, es el Procesador de Votos.

El subsistema que toma los votos de la cola de votos registrados y los registra permanentemente, tanto para el acto votado como para el usuario que emite el voto. Nuestro primer requisito para el sistema de almacenamiento se encuentra en la sentencia, tanto para el acto como para el usuario. El sistema de almacenamiento debe permitir la actualización de ambos como una única transacción atómica.

Otro punto relevante es que la cola de votos registrados está aislada; podría haber una o varias colas de este tipo sin ninguna diferencia para el procesador de votos. Esto se debe a que los votos se emiten en diferentes regiones, y mantenemos el servicio de cara al usuario (la interfaz de votación) local para los usuarios por razones de rendimiento.

Si bien las colas de votos registrados pueden estar aisladas, el registro permanente del historial de votos de usuarios y actos no lo está. Existe un único número de votos emitidos para un acto, y del mismo modo, un único número de votos emitidos por usuario durante una ronda. Nuestro segundo requisito es que el sistema de almacenamiento debe ofrecer un almacén de datos global a más de un cliente aislado regionalmente.

Otro requisito de nuestro diseño es la direccionabilidad de los datos. Recuperamos y actualizamos los actos mediante una clave única y a los usuarios mediante sus propios valores de clave únicos. Estos son nuestros principales métodos de acceso: operaciones directas sobre entidades individuales, pero también realizamos operaciones sobre rangos de entidades, como leer todos los actos de una ronda determinada para determinar su posición en la competición. Estos son nuestros requisitos para la direccionabilidad de los datos, el acceso directo rápido a entidades individuales y el acceso con menor latencia a subconjuntos de entidades mediante un índice alternativo. La forma de los datos que almacenamos es otro factor a considerar. Recordemos lo explicado en [el capítulo "Garantizar la Permanencia de los Datos"](#), donde se distingue entre datos estructurados y no estructurados, así como entre datos profundos y planos. Todos nuestros datos son estructurados y, en algunos casos, pueden clasificarse como profundos, concretamente por la existencia de conjuntos de votos por usuario y ronda. Un requisito de nuestro servicio de persistencia de datos es que admita datos estructurados por entidad y datos con cierta profundidad, es decir, conjuntos de valores dentro de una entidad direccionable.

Los requisitos que hemos identificado se resumen en la siguiente tabla, que utilizaremos como hoja de trabajo durante todo el proceso.

Evaluar varios productos de almacenamiento potenciales que ofrece GCP. Revisaremos y ampliaremos esta hoja de trabajo a lo largo de este capítulo:

Application	<Product one>		<Product two>	
Requirement	Effort	Notes	Effort	Notes
Multiple entity transactions				
Globally available datastore				
Fast single entity access				
Address a subset/range of entities				
Supports structured data				
Support for deep data				

Cifra **13.1** Tabla inicial de requisitos de persistencia

Con esta tabla definida, vamos a anotar algunos detalles de esta etapa del proceso.

El número de requisitos (filas, nombradas en la columna de la izquierda) se determina subjetivamente, sin un tamaño correcto o incorrecto. Los requisitos de la primera columna se definen por la naturaleza del componente en cuestión (en este caso, productos de persistencia de datos) y los casos de uso implementados, como se describió en los párrafos anteriores de esta sección. Una regla general podría ser que un requisito es insuficiente y más de siete es excesivo, pero incluso estos umbrales podrían ser razonablemente cuestionables en algunos casos.

El número de productos (pares de columnas a la derecha de la primera columna) lo determina el mercado. Al planificar una migración de infraestructura, es importante incluir el servicio actual como primer producto como punto de partida: la migración debería mejorar la aplicación o el flujo de trabajo general del equipo de desarrollo. Contar con el estado actual proporciona un punto de partida conveniente para analizar las ventajas de cada producto candidato a ser reemplazado.

Podemos hacer una declaración más contundente al recomendar la cantidad de columnas de producto a incluir que al indicar la cantidad de filas de requisitos.

Debe haber al menos dos productos en la evaluación: el producto actual y un candidato a reemplazo. El límite superior en etapas posteriores debe estar abierto a los productos que satisfacen los requisitos, pero limitado por la energía y el tiempo disponibles del equipo que realiza la investigación. Si se establece un límite superior, siete es un buen número como umbral de atención humana.

Por ahora, nos centramos en construir la abstracción de las responsabilidades de nuestra infraestructura, que se refleja en la primera columna y se dedujo en la discusión anterior. Por lo tanto, la identificación y evaluación de productos se dejará para una sección posterior de este capítulo. A continuación examinaremos los requisitos de un segundo rol de infraestructura, el de proveedor de autenticación y autorización de usuarios, al que denominaremos servicio de identidad.

El primer requisito de un servicio de identidad es la capacidad de trabajar con las cuentas existentes de los usuarios en plataformas conocidas y de confianza. Estas pueden incluir Apple, Google, Facebook y LinkedIn. Consideramos que Google es el requisito más importante: el servicio de identidad debe poder autenticar una cuenta existente de Google (también conocida como Gmail) . Un requisito secundario, menos exigente, es la capacidad de autenticarse con al menos otra plataforma popular.

El tercer requisito es la capacidad de autenticar a un usuario mediante su número de teléfono móvil. Si bien aún no lo hemos implementado en el sistema de votación, lo consideramos un requisito para un sistema de producción de uso global, especialmente a medida que lo implementamos en regiones donde los teléfonos móviles son la plataforma informática de cliente más popular.

El siguiente y más importante requisito para un servicio de identidad es que proporcione credenciales que permitan autorizar a un usuario a acceder a los servicios de GCP. Ahora que nuestro servicio se ha desarrollado para GCP, este es un requisito fundamental a menos que planeemos migrar todos nuestros servicios de backend a otro proveedor.

El último requisito es la compatibilidad con las plataformas de nuestros clientes. Estas incluyen Python en iOS y Android, y JavaScript nativo.

Utilizando la misma plantilla que establecimos para resumir y evaluar los requisitos del rol de infraestructura, producimos una tabla, en la siguiente figura, como nuestro segundo ejemplo:

Application	<Product one>		<Product two>	
Requirement	Effort	Notes	Effort	Notes
Authenticates Google accounts				
Authenticates one other social account type				
Authenticates using mobile phone #				
Provides GCP authorization				
Supports Python (iOS, Android) and JavaScript				

Cifra **13.2** Tabla inicial de requisitos de servicio de identificación

Habiendo especificado primero estos requisitos claramente, sin mencionar o considerar un producto específico, podemos considerar las necesidades reales del sistema, en lugar de las ofertas de los proveedores de servicios.

En el resto de este capítulo, continuaremos con el proceso de evaluación y migración del producto utilizando únicamente el primer caso de uso: un servicio de persistencia. El mismo proceso puede aplicarse a cualquier rol de infraestructura, como ilustra el segundo ejemplo.

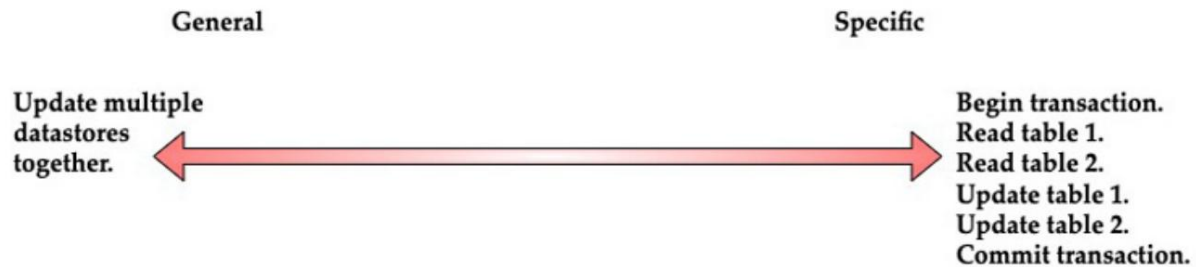
Mapeo de la funcionalidad de la base en interfaces de infraestructura

Tras identificar los requisitos funcionales de los proveedores de infraestructura, podríamos evaluar la idoneidad de un producto para nuestras necesidades. Sin embargo, esta evaluación se basaría principalmente en las afirmaciones del proveedor, sin datos concretos. El significado exacto de ofrecer un almacén de datos global puede tener significados muy diferentes para un gestor de productos de bases de datos y para un desarrollador de aplicaciones. Es mejor asignar los requisitos funcionales de alto nivel identificados a las operaciones discretas procedimentales de bajo nivel que nuestra aplicación espera realizar para satisfacer cada requisito.

Encontrar niveles adecuados de especificidad para los requisitos. En este punto, es importante

equilibrar la necesidad de identificar capacidades legítimas y concretas con la tendencia a depender de lo ya conocido y a pasar por alto o excluir soluciones nuevas y novedosas. Los requisitos funcionales no deben converger en el nivel de especificidad del código, que...

Es decir, los requisitos no son pseudocódigo, pero tampoco son simplemente objetivos o declaraciones de principios. La siguiente figura ilustra el rango de especificidad de los buenos requisitos funcionales y los extremos que deben evitarse:



Cifra **13.3** Rango de especificidad de los requisitos

Con los extremos explicados con tanta precisión, resulta útil contar con un contraejemplo de un nivel de especificación aceptable. Esto depende de cada caso de uso, pero para este ejemplo, un enunciado de requisito que se encuentra en el espectro entre ambos extremos es el siguiente: **Actualizaciones de múltiples entidades:**

El servicio debe ser capaz de aislar un conjunto de operaciones relacionadas, consistentes en operaciones de lectura de múltiples almacenes de datos agrupados con actualizaciones de dichos almacenes mediante valores derivados de dichas lecturas. Este conjunto de operaciones debe ser invisible para otros clientes de los almacenes de datos hasta que se completen correctamente como grupo.

Si el conjunto no se completa, se considera que nunca ocurrió y no tiene efecto en los almacenes de datos a los que se accede ni en otros clientes de los almacenes de datos.

Desde la ejecución de la primera operación del conjunto hasta su finalización, se debe bloquear el acceso de otros clientes a los elementos de datos que se están actualizando. Este bloqueo debe ser invisible para dichos clientes.

Este requisito normalmente se cumple con la implementación de uno

Tipo existente de semántica de transacción de base de datos: Transacciones multitabla con cerraduras exclusivas. Sin embargo, no hemos especificado cómo debería ser esto. implementado por el proveedor de servicios, ni hemos dejado ninguna ambigüedad en Nuestro requerimiento.

La falta de detalles de implementación es importante porque no nos limita a... en una característica existente y bien conocida. Considere un proveedor de bases de datos que ha implementado una forma novedosa de mirar hacia adelante que determina qué de las entidades leídas se pueden actualizar y define como un conjunto estrecho de bloqueos como sea posible para cada conjunto de lecturas y actualizaciones. Si tuviéramos que Especifique el pseudocódigo o la implementación comúnmente utilizada en el A la derecha de [la Figura 13.3](#), estaríamos excluyendo esta nueva y útil característica. de la consideración. Recuerde que el requisito no es multitabla. transacciones con bloqueos exclusivos, sino más bien las capacidades y efectos descrito en el fragmento de código anterior.

Este ejemplo presentó un requisito apropiado, pero el El equipo de ingeniería puede encontrar otros, más o menos específicos o expresado en términos diferentes. No hay una única solución correcta; si la Los ingenieros pueden comprender los términos utilizados y los requisitos. La especificación pasa la prueba de no ser una declaración de principios y tampoco Si es lo suficientemente preciso como para ser un código, lo más probable es que sea útil y requisito debidamente redactado.

Debería haber una expansión, como la que se vio anteriormente, para cada requisito enumerado en [la Figura 13.1](#).

Esta tabla y el documento adjunto siguen siendo del proveedor y del producto Agnóstico, estos son los requisitos de nuestra aplicación. Explorando cómo Cada producto en consideración puede satisfacer cada requisito. ejercicio que dará como resultado los valores de las columnas **Productos** y **Fuerza** de en la tabla de [la Figura 13.1](#).

Contener dependencias mediante la convergencia hacia el negro cajas

El término caja negra se refiere a un componente o sistema donde el interno Los trabajos son desconocidos y se encuentran dentro de una caja negra opaca metafórica. con solo los dispositivos de entrada y salida intencionales expuestos. El negro

El enfoque de caja se centra en las entradas y salidas de un sistema sin ahondar ni siquiera permitir la consideración de los mecanismos internos.

Definir las entradas esperadas y los resultados prometidos sin entender cómo se transforman las entradas en productos es la característica distintiva de este principio.

Adoptar este enfoque para especificar los componentes de nuestra infraestructura es ideal para permitirnos una movilidad fluida entre proveedores. Con los requisitos depurados hasta obtener un conjunto lo más preciso posible de capacidades necesarias y la interfaz definida en función del estado relevante disponible, podemos identificar el conjunto más amplio de candidatos de infraestructura que cumplan con nuestros requisitos.

Cualquier componente que pueda proporcionar lo que necesitamos (resultados) en función de lo que podemos proporcionar (insumos) es un proveedor de servicios potencial. Cómo lo haga el componente simplemente no nos incumbe, siempre y cuando se comprendan los efectos secundarios, como el coste de los recursos, el rendimiento y la sostenibilidad de la solución en la práctica.

Requisitos como especificaciones de caja negra El resumen de requisitos

en **la Figura 13.1** contiene los resultados o efectos deseados de un componente a través del contenido agregado de la columna más a la izquierda. Lo que falta para que esto sea un conjunto conciso de especificaciones de caja negra es un resumen de las entradas disponibles: qué botones podríamos presionar, o palancas podríamos configurar, en un componente de caja negra literal. Si bien este nivel de detalle no se proporciona normalmente, es un ejercicio útil para tener en cuenta al escribir requisitos: ¿Qué estado interno tiene un cliente que podría presentar como valores de parámetros a un servicio? Determinar los elementos de estado relevantes y resumirlos de manera consistente son los principales desafíos, pero no una tarea imposible.

Gramática para especificaciones de caja negra La propiedad

distintiva esencial de los requisitos de caja negra es que definen lo que un subsistema debe hacer desde la perspectiva del cliente, sin mencionar los detalles técnicos de cómo se hace.

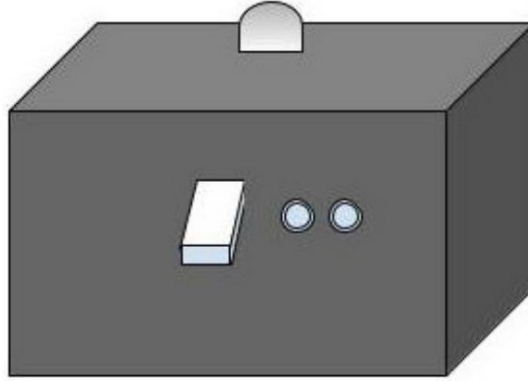
Los requisitos de caja negra bien escritos son lo suficientemente precisos como para usarse para escribir pruebas de caja negra: verificaciones en tiempo de ejecución de que un sistema está funcionando.

Como se esperaba. Estos requisitos se expresan en funciones matemáticas o en un lenguaje bien estructurado para reducir la posibilidad de interpretaciones erróneas debido a sesgos subjetivos o ambigüedad.

La siguiente lista de directrices debe tenerse en cuenta al redactar requisitos de caja negra y casos de prueba. También son útiles como lista de verificación para validar los requisitos una vez redactados: Enfoque en la funcionalidad:

- La especificación debe indicar claramente la funcionalidad esperada del subsistema, según los requisitos del usuario. El beneficio o valor añadido del componente debe ser claro.
- Desconocimiento de los mecanismos internos: La especificación no debe mencionar cómo el subsistema cumple el requisito ni qué estructura interna podría existir dentro de la caja negra. Una caja negra debe ser mágica desde la perspectiva del usuario, sin ningún respaldo científico evidente.
- Enfoque de entrada produce salida: El requisito debe especificar qué entradas se proporcionan a la caja negra y qué señales o salidas visibles produce como respuesta. Es útil visualizar una caja negra real con una serie de dispositivos de entrada discretos: botones, perillas e interruptores, y dispositivos de salida discretos, como lámparas, diales o zumbadores. Para esta caja negra simbólica, un requisito podría ser: cuando se presionan dos botones al unísono durante dos segundos, se enciende una luz roja para indicar que se están presionando varios botones. La luz permanece encendida hasta que se suelta uno o ambos botones, momento en el que se apaga.

Esta ilustración de un componente, literalmente una caja negra, nos recuerda el propósito de la designación de caja negra: la absoluta opacidad que confiere. Lo único que nosotros, el cliente, podemos ver son las características anunciadas, que están ahí para nuestro uso: un interruptor y dos pulsadores para que nuestro sistema los manipule, y una luz indicadora para que la caja negra se active. El resto del sistema carece de características y es privado para los proveedores, los fabricantes del componente.



Cifra **13.4** Un componente de caja negra figurativa

Ventajas de las especificaciones de caja negra. Los

beneficios de los requisitos y las pruebas de caja negra residen principalmente en la transferencia de la carga a las partes correspondientes. Comprender el funcionamiento interno y la semántica de un componente de software requiere tiempo y atención de los desarrolladores y el equipo de desarrollo del cliente.

En nuestro caso, el personal de producto e ingeniería que diseña e implementa la aplicación de votación tiene trabajo que realizar en ese ámbito y debería poder centrarse en resolver dichos problemas, no en diseñar o depurar un sistema de almacenamiento. Asimismo, el punto de vista complementario es igualmente convincente: los desarrolladores de un sistema de almacenamiento deberían comprender e implementar un servicio de almacenamiento bien definido y fiable, no familiarizarse con el dominio de problemas de cada sistema cliente potencial.

Los beneficios de la especificación de caja negra se extienden a lo largo del ciclo de vida del desarrollo de software, hasta la etapa de pruebas y verificación continua de un sistema. Resulta más rentable permitir que los ingenieros de pruebas apliquen su experiencia con las herramientas de su especialidad: marcos de pruebas unitarias y de integración, sistemas de pruebas continuas e inyectores de caos, para evaluar los resultados generados por una caja negra, sin necesidad de conocimientos ni investigación sobre el componente en sí.

Los beneficios de la especificación de caja negra se extienden a lo largo del ciclo de vida del desarrollo de software, hasta la etapa de pruebas y verificación continua de un sistema. Resulta más rentable permitir que los ingenieros de pruebas apliquen su experiencia con las herramientas de su especialidad: marcos de pruebas unitarias y de integración, sistemas de pruebas continuas e inyectores de caos, para evaluar los resultados generados por una caja negra, sin necesidad de conocimientos ni investigación sobre el componente en sí.

Planificación de una migración

Migrar una aplicación de una dependencia de infraestructura a otra debe considerarse altamente arriesgado y costoso. El riesgo es que se produzca una interrupción en las operaciones del sistema o incluso una interrupción importante, visible para el usuario.

Interrupción del servicio que afecta al negocio. El gasto consiste en el tiempo y el esfuerzo del desarrollador para familiarizarse con la nueva infraestructura y planificar y ejecutar la migración. Probablemente exista un costo adicional al pagar por los servicios de infraestructura antiguos y nuevos durante el período de migración.

Dado que existen costos reales, tanto conocidos como especulativos, la decisión de cambiar de componentes o proveedores de infraestructura es crucial y requiere una cuidadosa consideración y una planificación exhaustiva. El proyecto debe contar con las provisiones necesarias en cuanto a personal, presupuesto de gastos operativos (OpEx) y la autoridad de los responsables del proyecto. Esto último implica que un proyecto de migración debe contar con un patrocinador de alto nivel con la autoridad para, por ejemplo, impedir el lanzamiento de un producto que, a su juicio, aumenta significativamente el riesgo de fracaso del proyecto.

En la siguiente sección, repasaremos este proceso, utilizando el caso de uso de ejemplo de la migración propuesta del servicio de persistencia de datos de la aplicación de votación.

Determinar la necesidad de migrar. Una migración de

un entorno, una infraestructura conocida, a uno nuevo requiere cambios importantes en sus activos y planes. Cualquier cambio, por grande que sea, introduce caos y desorden, lo que aumenta la probabilidad de riesgo. La posición más segura suele ser permanecer donde se está: puede que no sea ideal, pero es familiar y su posición allí está establecida.

Para justificar el costo y el riesgo de una migración, debe existir una ventaja convincente, pero no suele ser una reducción del riesgo. A continuación, enumeraremos y describiremos varias razones comunes para migrar:

- Mejoras en la confiabilidad o reducción del costo de la confiabilidad: Una de las señales de una aplicación exitosa y bien recibida es la demanda de más nuevos. El término cinco nueves se entiende como 99.999% y se refiere a la alta disponibilidad de los servicios, cuando un servicio está disponible el 99.999% del tiempo, o alrededor de 5 minutos de inactividad al año. Si los usuarios pierden una aplicación durante su inactividad, es señal de que la aplicación es importante para la comunidad de usuarios. La confiabilidad de un componente de software depende en gran medida de la confiabilidad de sus dependencias. Si bien una aplicación puede compensar una

Un componente de infraestructura poco fiable, mediante una combinación de reintentos, conmutación por error y almacenamiento en caché local, supone una carga considerable y distrae de la misión principal de la aplicación. En este caso, una infraestructura más fiable reduce el coste de la fiabilidad de la aplicación. En los casos en que las debilidades de la infraestructura no se han compensado en la aplicación cliente, una infraestructura más fiable aumenta la fiabilidad de la aplicación.

- Nuevas capacidades que actualmente no son viables: En ocasiones, surge un producto revolucionario que destaca entre una categoría de competidores homogéneos. Una nueva característica con valor directo para los clientes de un servicio puede ser una razón convincente para cambiar. La primera plataforma informática que ofreció escalado automático o la primera base de datos relacional que ofreció reenvío de registros de transacciones, que permitía mantener servidores de reserva activos y réplicas locales de solo lectura, fueron lo suficientemente beneficiosas como para atraer a muchos sistemas de aplicaciones a sus productos de infraestructura.
- GCP reducido: La infraestructura de una aplicación tiene un precio. Este puede ser el coste de uso de un proveedor de servicios, el coste de los recursos de la máquina utilizados para ejecutar software de código abierto o gratuito, o la combinación de las tarifas de licencia de software y el coste de los recursos de la máquina.

Independientemente de la naturaleza exacta de los costes, en un entorno competitivo, es inevitable que algunos proveedores rebajen el precio habitual para ganar cuota de mercado. Si el ahorro en gastos operativos es significativo, conviene calcular el coste de la migración para comprobar si existe un ahorro neto y si es lo suficientemente grande como para justificar el esfuerzo de migración.

- Salida forzada: Un cliché bien conocido, « todo lo bueno se acaba », puede ser cierto tanto en los negocios como en cualquier otro ámbito. En el caso de los componentes de software, es bastante más fácil terminar la vida útil de un producto, ya que no hay artefactos físicos ni restos de los que preocuparse.

Un producto puede ser descontinuado por muchas razones: a veces, un software que es perfectamente bueno y útil para muchos ya no es un producto viable, o quizás la empresa propietaria ha cesado sus operaciones, dejando un producto en el limbo. En algunos casos, los organismos reguladores pueden ordenar el cese del uso de un producto en su jurisdicción. Independientemente del motivo, hay casos en los que el uso de una dependencia de infraestructura debe finalizar. Este es el más difícil.

En este escenario de migración, el cronograma suele imponerse al cliente como un edicto y puede que no exista una alternativa obvia y atractiva. En otras palabras, el servicio desde el que se migra es bien conocido, pero el servicio al que se migra no está identificado, y es posible que no exista una buena opción.

Supongamos que nuestra propuesta de migración es exploratoria, basada en el deseo de reducir los GCP. Spanner es muy eficaz, pero conlleva tarifas de uso relativamente altas, aunque adecuadas para un servicio premium. Investigaremos si se pueden obtener ahorros netos migrando a otro servicio de almacenamiento dentro de la suite GCP.

Recopilación de candidatos de reemplazo. La posibilidad

de una migración, o en caso de una salida forzada, la inminencia de una migración, desencadena una operación de sondeo. El conjunto inicial de candidatos de reemplazo se puede determinar mediante una investigación sencilla de las reivindicaciones y revisiones de características, con base en los requisitos enumerados en la columna de la izquierda.

La hoja de trabajo de requisitos, que se presentó por primera vez en [la Figura 13.1](#), ahora incluirá los nombres de posibles alternativas a nuestro servicio de persistencia actual, como se ve en la siguiente figura:

Application Requirement	Cloud SQL		AlloyDB		Bigtable	
	Effort	Notes	Effort	Notes	Effort	Notes
Multiple entity transactions						
Globally available datastore						
Fast single entity access						
Address a subset/range of entities						
Supports structured data						
Support for deep data						

Cifra **13.5** Hoja de trabajo para la migración de persistencia

Los tres conjuntos de columnas, uno para cada producto de almacenamiento en consideración, deben completarse sin compararlos con el producto actual, Spanner, sino en relación entre sí. Las columnas de Esfuerzo codifican una clasificación, y se puede usar una escala numérica, como del 1 al 5, o tallas de camiseta: pequeña, mediana y grande. Para esta migración, utilizaremos una clasificación de esfuerzo del 1 al 5, donde 1 es fácil y 5 el mayor grado de esfuerzo. Estas clasificaciones son puramente relativas, y la amplitud de la escala permite una distinción suficiente entre

Los candidatos. En ausencia de otra información o experiencia con los productos, una escala de 1 a 5 es una opción razonable y puede reducirse o ampliarse posteriormente si se considera demasiado burda o si nunca se necesitan 5 grados de precisión.

Evaluación de las alternativas. El objetivo de la

evolución es determinar el esfuerzo relativo para satisfacer cada requisito, normalizado tanto verticalmente, en relación con otras características, como horizontalmente, en relación con la solución de esa característica en otros productos. Esta consistencia en ambas dimensiones puede no ser posible con alta fidelidad, y el esfuerzo relativo de una característica específica en múltiples productos es una métrica más importante que el de las características dentro de la solución de un producto.

Como primer ejemplo, la capacidad de proporcionar transacciones que incluyan múltiples entidades está bien soportada tanto en Cloud SQL como en AlloyDB. El esfuerzo requerido no es trivial, sino que está bien comprendido, documentado y es ampliamente utilizado.

Asignaremos a este requisito un esfuerzo de 2 para ambos productos.

Las transacciones con múltiples entidades en Bigtable no son compatibles de forma nativa, pero pueden implementarse mediante bloqueos implementados por la aplicación y escrituras por lotes para agrupar actualizaciones en diferentes tablas. Esto supone un alto nivel de complejidad para la aplicación, aunque existe un estado de la técnica bien documentado.

Asignamos un esfuerzo de 5 al cumplimiento de este requisito por parte de Bigtable.

La hoja de trabajo de migración ahora tiene el siguiente contenido:

Application Requirement	Cloud SQL		AlloyDB		Bigtable	
	Effort	Notes	Effort	Notes	Effort	Notes
Multiple entity transactions	2	Small amounts of code, common	2	Small amounts of code, common	5	Application coordinating logic needed
Globally available datastore						
Fast single entity access						
Address a subset/range of entities						
Supports structured data						
Support for deep data						

Cifra **13.6** Hoja de trabajo de migración de persistencia parcialmente completada

El mismo proceso de investigación para cada fila de características restantes de la hoja de cálculo consiste en investigar cómo un cliente de cada uno de los tres servicios de almacenamiento propuestos cumpliría con un requisito. Se espera que cada clasificación requiera una investigación exhaustiva, y los investigadores deben mantener un registro exhaustivo de su trabajo para su posterior publicación. Si bien la hoja de cálculo es intencionadamente breve, esto no impide que...

La existencia de notas extensas y de fácil acceso en otros lugares. Estas notas serán valiosas para los ingenieros que implementen la solución elegida, como guía para determinar qué características y patrones de uso del producto de infraestructura elegido se consideraron apropiados para este caso de uso.

No repasaremos todas las iteraciones del proceso de evaluación, sino que proporcionaremos la hoja de trabajo final:

Application Requirement	Cloud SQL		AlloyDB		Bigtable	
	Effort	Notes	Effort	Notes	Effort	Notes
Multiple entity transactions	2	Small amounts of code, common	2	Small amounts of code, common	5	Application coordinating logic needed
Globally available datastore	4	Application syncs of regional data	4	Application syncs of regional data	3	Eventual consistency features can distribute regional updates, application logic aggregates
Fast single entity access	1		1		1	
Address a subset/range of entities	1	SQL like operator can use existing indices	1	SQL like operator can use existing indices	2	Alternate index creation support
Supports structured data	2		2		1	Protobufs as column types
Support for deep data	3	Sub-relation tables	3	Sub-relation tables	1	Repeated/ARRAY fields

Cifra **13.7** Hoja de trabajo final de migración de persistencia

Una observación inicial es que las puntuaciones de los dos primeros productos (Cloud SQL y AlloyDB) son idénticas. Esto es previsible, ya que ambos servicios son compatibles con PostgreSQL, cada uno con sus propias extensiones y fortalezas, que exceden los requisitos que hemos identificado.

Por lo tanto, la propiedad transitiva de igualdad se cumple, y estos obtienen la misma puntuación en nuestra hoja de cálculo. Otros aspectos de estos servicios, como el coste del producto y el rendimiento específico, pueden utilizarse para seleccionar entre ambos en una etapa posterior de la evaluación.

Con las características individuales de los tres productos clasificadas comparativamente, vale la pena examinarlas para ver si se presenta una decisión fácil: la presencia de un producto que requiera el menor esfuerzo en todas las características. Recuerde que las puntuaciones representan el esfuerzo, por lo que es preferible una puntuación baja.

Nuestra primera evaluación consiste en determinar si un candidato es el preferido para la mayoría de los requisitos. Al comparar AlloyDB y Bigtable, observamos que Bigtable requiere menos esfuerzo en 3 de los 6 requisitos, con un requisito equivalente. Esto no es una victoria clara ni evidente, ya que la distribución entre ambos es casi igual, por lo que procedemos a una evaluación más exhaustiva.

Esta siguiente etapa asume que los esfuerzos relativos entre las filas de requisitos son similares; es decir, la diferencia entre un esfuerzo de 5 y un esfuerzo de 2 es aproximadamente la misma en las diferentes filas. Si no es así,

En este caso, el equipo evaluador debe intentar volver a calificar los esfuerzos para normalizarlos tanto como sea posible.

Con las puntuaciones relativas normalizadas, podemos ver qué producto requiere el mayor esfuerzo relativo. La diferencia entre las puntuaciones de esfuerzo se puede calcular a partir de la hoja de cálculo y genera una tabla como la que se muestra en la siguiente figura:

Application	AlloyDB	Bigtable	Effort delta	
Requirement	Effort	Effort	(AlloyDB - Bigtable)	(Bigtable - AlloyDB)
Multiple entity transactions	2	5		3
Globally available datastore	4	3	1	
Fast single entity access	1	1		
Address a subset/range of entities	1	2		1
Supports structured data	2	1	1	
Support for deep data	3	1	2	

Cifra **13.8** Tabla de deltas del esfuerzo de migración de persistencia

La figura anterior muestra el coste relativo del candidato más caro para cada requisito. Podemos observar que la implementación de transacciones de múltiples entidades en Bigtable presenta el mayor coste para cualquiera de los candidatos. Sin embargo, el delta del coste total es el mismo para cada candidato:

$$(1+1+2) == 4 \quad == \quad (3+1)$$

Aquí es donde se puede aplicar el conocimiento de la propia aplicación. Los ingenieros que trabajan en la aplicación pueden determinar si la ventaja de la compatibilidad de Bigtable con campos repetidos supera la implementación más sencilla de transacciones de múltiples entidades que ofrece AlloyDB, siendo estas las dos mayores diferencias en cuanto a esfuerzo relativo.

Tras analizar el uso de la base de datos, el equipo de ingeniería determina que reemplazar los campos repetidos con una tabla relacionada independiente es una función relativamente sencilla de implementar, mientras que implementar la semántica de transacciones requerida requiere un mayor esfuerzo y es más difícil de probar y verificar. Por lo tanto, se decide explorar la migración a AlloyDB o Cloud SQL, teniendo en cuenta que estos productos obtuvieron puntuaciones equivalentes en la hoja de cálculo de requisitos de migración.

La elección entre los dos productos, que hasta ahora han sido equivalentes, requiere examinar aspectos de los productos que los distinguen.

El uno del otro. Dado que se trata de dos productos vendidos por el mismo proveedor, podemos basarnos en las recomendaciones de cada uno para los casos de uso ideales. Algunos de los puntos de comparación que ofrece Google son los siguientes: Ambos

- productos se describen como totalmente administrados, y podemos inferir que ninguno genera una carga operativa significativamente mayor que el otro.
- AlloyDB ofrece mayor rendimiento y escalabilidad que Cloud SQL, pero no está claro si la carga de trabajo de nuestra aplicación excede la zona de confort de Cloud SQL.
- AlloyDB ofrece capacidades analíticas avanzadas, que no son relevantes para nuestro uso de la base de datos.
- Se afirma que Cloud SQL es rentable. Esto significa que los costos de uso de Cloud SQL son significativamente menores.

Esta lista nos deja con dos preguntas para responder:

- ¿Cuánto más nos costará utilizar AlloyDB?
- ¿Es nuestra carga de trabajo lo suficientemente grande como para beneficiarnos del rendimiento superior de AlloyDB?

Para la primera pregunta, el costo relativo se puede responder mediante la aplicación de una calculadora de precios para cada servicio según nuestro uso esperado.

El almacenamiento total, la cantidad de instancias y las operaciones de lectura y escritura son valores que podemos modelar para una migración, en función del uso observado, la demanda proyectada y la arquitectura de servicio propuesta.

En cuanto a la segunda pregunta, el beneficio de AlloyDB se compensa con el de la primera: el costo relativo. Es un hecho que el servicio AlloyDB cuesta más que el mismo uso de Cloud SQL, pero ¿compensa ese costo adicional las ventajas que ofrece AlloyDB?

[Cálculo del costo del servicio de base de datos](#)

Trabajaremos en los cálculos de costos tanto para AlloyDB como para Cloud SQL como una forma de desarrollar una comprensión de los factores y procesos.

Google Cloud ofrece calculadoras de precios en línea que se mantienen actualizadas y deben considerarse la fuente de información veraz sobre las tarifas reales asociadas al uso de los productos. Cada uno de estos productos ofrece funciones adicionales, como copias de seguridad automáticas e instantáneas, que pueden generar tarifas.

debe considerarse en el costo final de cada producto.

Cálculo de valores para los factores de costo de AlloyDB

El primer factor de costo a determinar es el tamaño de la base de datos, es decir, el número de gigabytes almacenados al mes. Podemos calcular los costos por hora.

Pero nuestros datos no son muy grandes y los más complejos por hora

Los cálculos no ahorrarán mucho en costos de almacenamiento. Le proporcionaremos

Estimaciones de recursos para el tamaño de un clúster de base de datos de votación, de los cuales

Habrà una instancia por clúster y luego una instancia adicional

para nuestros resultados globales de votación. Dado que ni AlloyDB ni Cloud

SQL proporciona una base de datos global unificada como Spanner, nuestra base de datos de ingeniería.

El equipo ha diseñado una serie de bases de datos regionales que se actualizan

Procesadores de votos y una instancia de base de datos global adicional que

Contienen los votos agregados de las bases de datos regionales.

El tamaño de todas las instancias de la base de datos será casi idéntico, ya que el mismo

Se representan los actos, sólo que con diferentes recuentos de votos.

El tamaño de una fila de la base de datos en la tabla de votos es de poco menos de 128 bytes con una entrada de índice. Una fila en la tabla de presupuesto de votación y filas asociadas en un

Se estima que la nueva subtabla que contiene el detalle de los actos votados, con índices, ocupa menos de 1.024 bytes.

El número de filas en la tabla de votos está limitado al número de actos.

y rondas, lo cual fue ilustrado por la Figura en el Capítulo 8.1. Hacer un 8,

Sistema de votación disponible y confiable. Con base en esa discusión, el

El número de filas en la tabla de votos para una competencia completa será 712^{*}
o 84 filas.

La tabla de presupuesto de votación tendrá una fila para cada usuario y ronda de votación.

Hay siete rondas, y figura en el capítulo 8.3. Cómo hacer una votación 8,

Sistema Disponible y Confiable estima que nuestro número total de usuarios es de 57

millones. Supongamos que el uso ha mostrado 40 millones de usuarios hasta la fecha,

pero con un conteo que aumenta lentamente semana tras semana. Se decide que 50

Un millón de usuarios es una expectativa realista para la comunidad de usuarios de esta temporada.

El cálculo del tamaño de la base de datos utilizando los factores anteriores informa que

Se espera que cada instancia contenga 330 gigabytes de datos al mismo tiempo.

fin de una temporada.

El siguiente factor de costo que consideraremos es el tráfico de red que entra y sale de la base de datos.

La transferencia de datos a AlloyDB es gratuita o, mejor dicho, no genera ningún cargo adicional. La transferencia de datos desde AlloyDB entre regiones se factura según las regiones de origen y destino. Nuestra arquitectura propuesta incluye la sincronización de cada clúster regional de bases de datos con un clúster global después de cada ronda. Se planean cinco implementaciones regionales, una de las cuales se realizará en la misma región que la base de datos global. Esto implica que los datos de votos de cada ronda se enviarán desde cuatro regiones a una quinta región a lo largo de la temporada. En otras palabras, cada una de las cuatro regiones enviará el contenido completo de su base de datos a la quinta región una vez durante la temporada.

Nuestro tráfico de red facturable por temporada es de $336 \text{ GB} \times 4 = 1344 \text{ GB}$, enviado entre regiones.

El tercer y último factor de costo de AlloyDB corresponde a los recursos de cómputo de CPU y memoria. Las instancias de AlloyDB cuentan con una cantidad específica de CPU y memoria, que es lo que se factura. AlloyDB ofrece descuentos por uso de cómputo a largo plazo para compromisos de 1 y 3 años, de los cuales nos comprometemos a 1 año. Según las sugerencias de la documentación de dimensionamiento de AlloyDB, configuraremos inicialmente cada instancia regional con un nodo de cómputo de 64 GB de RAM y 8 CPU, y la instancia global con un nodo con las mismas características. Esta configuración de implementación incluye un total de 6 nodos de 64 GB de RAM y 8 CPU en varias regiones.

Calcular los costos de todas las alternativas

Para completar el análisis de costos, debemos realizar el mismo proceso que se describió anteriormente para cada producto competitivo en el caso de uso actual, es decir, Cloud SQL con un backend PostgreSQL.

Al calcular el costo de las alternativas, se debe examinar cada producto y calcular su costo por separado. El modelo de costos de Cloud SQL no ofrece una correlación directa de los factores de costo, por lo que es necesario examinar la calculadora de precios y la tabla de tarifas de Cloud SQL. Además, nuestra configuración de implementación de Cloud SQL no es idéntica a la arquitectura propuesta para AlloyDB.

Solución. Por ejemplo, nuestra arquitectura Cloud SQL 3 incluye réplicas adicionales para el reenvío de registros y la agregación de votos.

Cálculo de los costes de todas las alternativas

Tras analizar a fondo el coste de nuestra arquitectura Cloud SQL, observamos que el coste de la solución AlloyDB propuesta, según las tarifas vigentes al momento de redactar este documento, es de 11 000 USD para un período de votación de un mes. Esto incluye la aplicación de descuentos por compromiso de uso en el modelo de precios.

El costo de nuestra propuesta de Cloud SQL es de \$6,000 (USD) y un estimado de 2 semanas adicionales de tiempo de ingeniería.

Se decidió que contamos con \$5,000 (USD) adicionales y que los equipos de desarrollo y operaciones ya enfrentarían una carga de trabajo desafiante sin las 2 semanas adicionales de trabajo.

AlloyDB es el reemplazo de migración más rentable de los dos propuestos para Spanner.

Reevaluación de la decisión de migración. Al iniciarse este ejercicio, se declaró exploratorio debido al presunto costo relativamente alto del servicio Spanner. Nuestra implementación de Spanner es una base de datos global multirregional para la cual nos suscribimos a la edición Spanner Enterprise.

El costo de operar nuestra base de datos Spanner, incluyendo la replicación interregional, es de \$8,700 (USD) durante el período de votación de un mes. Si bien Spanner puede tener costos más altos, duplicar sus funciones distribuidas globalmente en un producto regional como AlloyDB es mayor que usar Spanner para lograr los mismos resultados. Podríamos ahorrar \$2,700 por temporada migrando a Cloud SQL, pero a costa de dos semanas de tiempo de ingeniería. Nuestros directivos señalan que, con los costos actuales de personal, se necesitarían seis temporadas para que la reducción de GCP compensara dos semanas de tiempo de nuestro personal de ingeniería y operaciones.

La migración no se realizará por razones bien entendidas y documentadas. Spanner es el producto de almacenamiento más adecuado y rentable para la aplicación de votación.

Construcción de un plan de migración. Si

bien esta propuesta de migración no tuvo éxito, diseñaremos, programaremos y realizaremos un ensayo de la migración de la infraestructura operativa propuesta. Este ejercicio es valioso, ya que es casi inevitable que surja otra situación donde los factores motivadores sean más críticos o quizás el ahorro de costos sea más significativo.

Una forma eficaz de ver una migración de infraestructura es como una refactorización. La refactorización de código consiste en reorganizar la implementación para lograr una estructura interna más eficiente o deseable (según alguna métrica) sin alterar el comportamiento del módulo refactorizado. Esta definición se aplica casi directamente al tipo de migración de servicios o dependencias que describimos. El objetivo es ofrecer el mismo producto final con una estructura interna modificada, superior según criterios invisibles para los usuarios finales del sistema.

Con base en esta definición, podemos afirmar que una migración de infraestructura exitosa es aquella que pasa desapercibida para quienes no pertenecen al equipo de ingeniería, mantenimiento y operación del sistema. Los usuarios finales, e incluso los equipos internos de producto y marketing, no deberían ver ningún cambio en el comportamiento, las características ni los contratos de usuario del sistema de aplicaciones. A continuación se presentan algunas pautas estratégicas para ejecutar una migración exitosa, que se ampliarán en el resto de esta sección:

- Espera lo mejor, planifica para lo peor
- Elija el momento y lugar de migración menos disruptivo
- Implemente y observe su nueva configuración gradualmente
- No abandones el antiguo sistema prematuramente
- Tenga un plan de contingencia completo

El primer punto, "esperar lo mejor y prepararse para lo peor", es un principio general que impregna todo el ejercicio, incluyendo los puntos siguientes. Espera y anhela lo mejor. Si planificas bien, sin dejar misterios ni incógnitas sin resolver, y proporcionas suficientes recursos para realizar correctamente cada tarea de tu lista, puedes esperar que todo salga según lo previsto.

Sin embargo, el mundo es complejo y nuestra atención, conciencia y previsión son limitadas. Los sistemas complejos se comportan de maneras inesperadas y

El entorno a menudo presenta situaciones e insumos que ni siquiera el mejor plan pudo anticipar. Este es un desafío mayor que cualquier factor negativo que abordemos en nuestro plan. ¿Cómo planificar una reacción ante un imprevisto? La respuesta es sencilla, aunque la respuesta real puede no serlo en absoluto. Prepárese para lo peor. A continuación, se presentan algunas preguntas que su plan de migración debe responder: ¿Qué hará si el sistema genera errores

- genéricos de servidor sin detalles útiles una vez implementados los nuevos componentes?
- ¿Cómo detectaremos lo antes posible que se están produciendo tales errores?
- ¿Qué hará si su respuesta a los errores no soluciona el problema?

A menudo, una respuesta responsable ante estas incógnitas incluye una primera acción de notificación o escalamiento. Si se esperaba que la migración de su base de datos no tuviera más efectos que una hora de servicio degradado un domingo por la noche, pero se está produciendo o se prevé una interrupción total o un efecto prolongado, asegúrese de informar a la cadena de mando. Si bien es perjudicial que un servicio visible públicamente esté fuera de servicio, es mucho peor que el director ejecutivo de la empresa se entere de la interrupción en redes sociales o por un periodista. Asegúrese de que el responsable de operaciones de la migración tenga una lista de números de teléfono y direcciones de correo electrónico a los que enviar el mensaje de celebración "Todo listo, funcionando correctamente" o "Tenemos un problema". La siguiente sección describirá los últimos cuatro elementos de la lista anterior de pautas estratégicas en esta sección, las pautas que minimizan el impacto potencial y proporcionan un camino para que el equipo de migración siga.

Cambiar los neumáticos mientras el autobús está en movimiento

El proceso de modificar un sistema en uso se compara coloquialmente con cambiar las llantas de un autobús en movimiento. La analogía es tan relevante como obvia. Imagine un autobús con pasajeros que se dirige a su destino. Sin interrumpir el servicio ni perder su horario, debe cambiar una llanta. Esto suena idéntico, en demanda e impacto, a cambiar un proveedor de base de datos mientras un sistema está en funcionamiento y procesando las solicitudes de los usuarios.

Seguiremos utilizando esta analogía en las discusiones sobre las tres directrices estratégicas para un plan de migración que se enumeraron en la sección anterior:

- Elija el momento y lugar de migración menos disruptivos: Si pudiera elegir el momento y el lugar para cambiar las llantas de su autobús, sería cuando haya menos gente en el autobús, las carreteras estén en buen estado y haya menos distracciones, como semáforos e intersecciones. En nuestro caso práctico, esto ocurriría cuando la actividad del usuario sea relativamente baja y haya pocos eventos que compitan, como mantenimiento del sistema, actividad del canal de procesamiento fuera de línea o mensajes instantáneos del equipo de producto. Esto se debe a dos razones. En primer lugar, podemos suponer que, durante el proceso de transición de la infraestructura antigua a la nueva, se iniciarán tareas y el responsable de operaciones querrá recopilar registros y seguimientos de actividad para supervisar el proceso. El inicio de la nueva tarea funcionará mejor si la demanda aumenta gradualmente para que funciones como el escalado automático puedan surtir efecto. Aumentar la demanda implica un punto de partida relativamente bajo. La segunda razón es la mitigación de riesgos. En resumen, es mejor fallar 10 solicitudes que 1000.

También será mucho más fácil reconocer y abordar problemas emergentes si hay menos otras entradas, como otros procesos de mantenimiento ejecutándose o compitiendo por la atención del equipo de operaciones.

- Implemente y observe su nueva configuración gradualmente: esta es una práctica recomendada para todas las versiones de software, pero con un cambio tan potencialmente impactante como un reemplazo de infraestructura, esta práctica debería elevarse de sugerencia a obligatoria. Conocido como canario.

Nota: El lector puede buscar el término en Google para conocer su desagradable origen.

Esta práctica implementa el cambio en una pequeña subdivisión del sistema implementado, como una región o un procesador de votos entre muchos. La parte modificada del sistema se deja integrar el cambio durante un período y se observa de cerca. Si los nuevos componentes funcionan bien, se amplía el alcance del cambio y se mantiene un período de espera similar. A medida que los nuevos componentes...

Si los nuevos componentes siguen funcionando según lo previsto, el cambio se implementa en todo el sistema. Si surgen imprevistos con los nuevos componentes, los balanceadores de carga pueden redirigir el tráfico de usuarios desde los componentes afectados hacia sus homólogos que ejecutan el software anterior, y el cambio se puede examinar y mitigar en las partes afectadas del sistema. En un caso como el nuestro, donde la migración da como resultado diferentes almacenes de datos, se necesita un plan de conciliación posterior a la migración para fusionar las actualizaciones del almacén de datos anterior con el nuevo.

- No abandone el sistema antiguo prematuramente: No tome la primera señal de éxito como una señal inequívoca de que el nuevo sistema está completamente listo y de que los componentes del sistema anterior nunca volverán a ser necesarios. Los pernos que sujetaban las llantas viejas han resistido el paso del tiempo. Las tuercas y los pernos que vienen con la llanta nueva no están probados y pueden aflojarse con el tiempo debido a las vibraciones del uso, por lo que deberá trasplantar las tuercas de sujeción antiguas a las llantas nuevas. En el caso del servicio de almacenamiento, es posible que se hayan pasado por alto detalles de configuración, registros de transacciones con información útil posterior, o que desee usar el componente anterior como parte de una prueba A/B si es necesario depurar un comportamiento inesperado en el nuevo sistema. En otro caso más grave, podría arrepentirse de desechar el componente antiguo si descubre que lo necesita de nuevo, como se describe en el siguiente punto.
- Disponga de un plan de contingencia completo: El mantra principal de un responsable de incidentes durante una interrupción de un sistema de producción es muy directo: salvar al usuario; un sistema en servicio no debe funcionar también como laboratorio. Si una nueva implementación no funciona, la gestión responsable de incidentes prioriza la puesta en marcha del sistema para los usuarios. Si las causas del fallo no son evidentes ni se solucionan rápidamente, revertir y depurar posteriormente. La capacidad de revertir se basa en preservar la configuración y las dependencias del sistema anterior en un estado que permita volver a estar en servicio de forma rápida y sencilla.

Imagine que reemplaza una llanta desgastada, pero que aún soporta el peso del autobús, por una nueva y brillante de material de última generación, y después de unos kilómetros, la nueva empieza a silbar al perder aire rápidamente, o peor aún, explota. La llanta vieja era mejor que ninguna, y prepararse para esta situación requiere haber dejado de lado la vieja.

Neumático y todos sus accesorios necesarios, sin desinflarlo. Puede encontrar la fuga en el neumático nuevo y repararla más tarde, no mientras esté montado en el eje, y la seguridad de sus pasajeros depende de ello.

Diseño de la migración de un laboratorio de código de almacén de datos

El laboratorio de código de este capítulo será una guía sobre un plan de migración para cambiar la aplicación de votación de Spanner a AlloyDB. Este plan no detalla los cambios en el código ni en los archivos de configuración, ni examina los detalles de implementación de las herramientas de migración diseñadas para este procedimiento. Todos estos cambios se habrán revisado, probado y aprobado previamente. Este plan es una guía para el operador, un script para quienes ejecutan la migración.

El script de migración

La primera parte del plan es una declaración de intenciones y la asignación de roles:

- **Objetivo de la operación:** Convertir los frontends y backends de las aplicaciones de votación para utilizar un conjunto de bases de datos regionales AlloyDB talentvoting-local y global talentvoting-global en lugar de la instancia Spanner talentvoting-instance.
- **Plan de canarying:** Primero, se actualizarán las tareas regionales de Sudamérica . Se observará un periodo de canarying de 3 horas, tras el cual se tomará una decisión sobre si se debe actualizar o no las tareas regionales de Europa Occidental . Tras otra hora de canarying de las tareas de Europa Occidental, se tomará una decisión final sobre si se debe actualizar o no el despliegue global.
- **Plan de reversión:** Si se detectan errores durante cualquiera de los períodos de canary, se eliminará el tráfico de usuarios de ambas regiones. Las solicitudes de Sudamérica se redirigirán a los frontends de Norteamérica , y las de Europa Occidental, a los de Europa del Este .
- **Equipo de migración:** La migración será ejecutada por dos ingenieros que trabajarán en conjunto. El responsable de operaciones es el principal responsable.

para ejecutar los pasos de la migración, y el líder de comunicaciones es responsable de actualizar los canales de comunicación identificados y las partes interesadas con actualizaciones de progreso y estar disponible para consultas.

- Líder de operaciones: Joëlle Lin jlin@nonesuch.com, +1555-555-5555 Responsable de
- comunicaciones: Raymond Blum raymundo@nonesuch.com, +1555-555-3333

La siguiente sección es una lista de verificación de recursos que deben identificarse antes de la

La operación comienza:

- Prerrequisitos:

Scripts: ☐

Script para cambiar el procesador de votos de Sudamérica y votar
interfaz para nuevas versiones

☐ Script para configurar redirecciones de frontends de Sudamérica a
Frontends de América del Norte

☐ Script para sincronizar almacenes de datos de AlloyDB desde los datos de Spanner después
de una transición del procesador de votos

☐ Script para cambiar el procesador de votos de Europa Occidental y la votación
interfaz para nuevas versiones

☐ Script para configurar redirecciones de frontends de Europa Occidental a
Interfaces de Europa del Este

Escucha:

☐ Panel que muestra las solicitudes de votación del frontend por versión y estado de respuesta
para América del Sur y América del Norte

☐ Panel que muestra las solicitudes de votación del frontend por versión y estado de respuesta
para América del Sur y América del Norte

☐ Panel que muestra las solicitudes de votación del frontend por versión y estado de respuesta
para todas las regiones

☐ Panel que muestra las solicitudes del procesador de votos por versión y estado de respuesta
para América del Sur y América del Norte

☐ Panel de control que muestra las solicitudes del procesador de votos por versión y

Estado de respuesta para América del Sur y América del Norte

- ☐ Panel de control que muestra las solicitudes del procesador de votos por versión y Estado de respuesta para todas las regiones
- ☐ Alerta por aumento en la tasa de error de la interfaz de votación para Sudamérica y América del Norte
- ☐ Alerta por aumento en la tasa de error del procesador de votos para Sudamérica y América del Norte
- ☐ Alerta por aumento de la tasa de error en la interfaz de votación para Europa Occidental y Europa del Este
- ☐ Alerta por aumento en la tasa de error del procesador de votos para Europa Occidental y Europa del Este

Contactos (rellenar lista de direcciones): _____)

- ☐ Lista de correo electrónico y SMS _____ para _____ corte _____ notificación:

- ☐ Lista de correo electrónico para actualizaciones periódicas sobre el progreso: _____

Versiones (rellenar las etiquetas de las versiones):

- ☐ Etiqueta de versión antigua para la interfaz de votación: _____
- ☐ Nueva etiqueta de versión para la interfaz de votación: _____
- ☐ Etiqueta de versión anterior para el procesador de votos: _____
- ☐ Nueva etiqueta de versión para el procesador de votos: _____

- Permisos: Confirmar que la cuenta principal de operaciones tenga el rol de administrador para lo siguiente:

- ☐ Procesador de votos en las regiones de Sudáfrica, NA, Europa Oriental y Europa Occidental
- ☐ Interfaz de votación en las regiones de Sudáfrica, Norteamérica, Europa del Este y Europa Occidental
- ☐ vota tema de Pub/Sub, votos-sub suscripción de Pub/Sub en SA, NA, Regiones de la UE oriental y occidental

La siguiente sección es la secuencia de operaciones a seguir y marcado con tiempos como completados:

- Script de migración:

o Tiempo realizado/Acción:

- _____/recorrido completo del documento por parte del líder de operaciones y comunicaciones.
- _____/lista de verificación de prerequisites por parte del líder de operaciones y comunicaciones.
- _____/enviar mensaje de inicio por el líder de comunicaciones. _____/
- cambiar tareas de SA a nueva versión por el líder de operaciones. _____/verificar panel de
- control para la interfaz de votación de SA/NA para confirmar solicitud de cambio a nueva versión por el líder de operaciones. _____/verificar panel de control para procesador de
- votación de SA/NA para confirmar solicitud de cambio a nueva versión por el líder de operaciones. _____/confirmar cambio exitoso de tráfico, inicio del período canario inicial por el
- líder de operaciones, líder de comunicaciones. _____/confirmar sincronización exitosa de datos de Spanner a AlloyDB por el líder de operaciones, líder de comunicaciones.
-

_____/enviar mensaje de progreso canario por parte del líder de comunicaciones.

- _____/verifique el panel frontal de votación SA/NA para una tasa de error plana a la hora de iniciado el canario por parte del líder de operaciones. _____/verifique el panel
- del procesador de votación SA/NA para una tasa de error plana a la hora de iniciado el canario por parte del líder de operaciones.

o Espere hasta que finalice el período canario inicial de 3 horas.

- _____/verificar el panel del procesador de votos SA/NA para ver si el líder de operaciones tiene una tasa de error fija y una tasa de respuesta fija al final del canario inicial. _____/verificar el panel del
- procesador de votos SA/NA para ver si el líder de operaciones tiene una tasa de error fija y una tasa de respuesta fija al final del canario inicial. _____/declarar la decisión de seguir adelante o no para el segundo canario
- por parte del líder de operaciones y el líder de comunicaciones. _____/enviar mensaje de progreso por parte del líder de comunicaciones.
-

El guión continúa de manera similar hasta el final del segundo canario y

Un lanzamiento global.

Estar preparado para lo peor

La siguiente sección es donde abordamos el plan para el peor principio discutido anteriormente:

- Modos de fallo:
 - Errores elevados en la interfaz de votación o en el procesador de votos durante el primer canario
 - Tiempo realizado/acción:
 - _____/establecer un temporizador de vigilancia de 10 minutos por parte del líder de
 - comunicaciones. _____/enviar una notificación de falla a la lista de escalamiento y a la lista de progreso por parte del líder de
 - comunicaciones. _____/verificar los registros para detectar errores obvios y remediarlos por parte del líder de operaciones y del líder de
 - comunicaciones. _____/a los 10 minutos del temporizador de vigilancia por parte del líder de comunicaciones (o antes por parte del líder de operaciones).
 - Ejecutar la reversión del Procesador de Votos a la versión anterior por parte del líder de operaciones.
 - Ejecutar la reversión de la interfaz de votación a la versión anterior por el líder de operaciones. _____/enviar
 - notificación de reversión a la lista de escalamiento y a la lista de progreso por el líder de comunicaciones. _____/verificar el
 - panel de control de la interfaz de votación de SA/NA para ver si el líder de operaciones volvió a la tasa de error plana y si la versión anterior atendió las solicitudes luego de la reversión. _____/verificar el panel de control
 - del procesador de votación de SA/NA para ver si el líder de operaciones volvió a la tasa de error plana y si la versión anterior atendió las solicitudes luego de la reversión. _____/ enviar notificación de reversión exitosa
 - o no a la lista de escalamiento y a la lista de progreso por el líder de comunicaciones.

Habrán secciones similares adicionales para abordar las fallas en el segundo canario, las fallas en la implementación global posterior al canario y las fallas en el

reversiones.

Un concepto importante es el de una lista de verificación. Al igual que los pilotos y cirujanos en su labor crucial, el personal de migración debe ser lo más explícito y preciso posible en sus funciones y acciones. Se requiere la confirmación de cada paso mediante una verificación, iniciales o algún otro acuse de recibo.

Conclusión

La idoneidad de un determinado servicio de infraestructura puede cambiar con el tiempo.

Lo que antes era la mejor opción para las necesidades de un sistema de aplicaciones puede verse eclipsado por la nueva competencia, obsoleto por cambios en los requisitos o descartado por el proveedor. El cambio es inevitable, y podemos planificarlo y facilitar la adaptación de nuestro sistema siguiendo los principios de diseño descritos a lo largo de este libro.

En este capítulo se analizaron las ventajas de ver las dependencias como una caja negra, compuesta únicamente por los efectos requeridos y las entradas disponibles.

Se ejercitó una hoja de trabajo y un proceso para evaluar dependencias a través de dos ejemplos de nuestra aplicación de votación.

Finalmente, se describieron los principios y una plantilla para ejecutar una migración de servicio de almacenamiento, libre de sorpresas, y luego se ilustró en el laboratorio de código de este capítulo.

El siguiente capítulo presenta la aplicación de votación como una manifestación específica de sistemas comerciales más genéricos y luego examina la adaptación de gran parte de la aplicación de votación a otra instancia específica de esa abstracción.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



CAPÍTULO 14

El sistema de votación como referencia Implementación

Introducción

En este capítulo, la votación es una aplicación más general que

La mayoría de la gente piensa que muchas aplicaciones son una variación de las básicas.

modelo de una solicitud para registrar interés en opciones y $\langle \text{metro} \rangle$ $\langle \text{norte} \rangle$

Luego, clasifica la popularidad de las opciones para aplicar a la siguiente.

etapa de procesamiento.

Más allá de la aplicabilidad de un sistema de votación, gracias a nuestro aislamiento de responsabilidades y dependencias que contienen, nuestra base

Los servicios se pueden utilizar como modelo para niveles equivalentes en aplicaciones cuyos productos no se parecen en nada a una votación.

Estructura

En este capítulo se tratarán los siguientes temas:

- Sistemas de votación en las empresas
- Estudio de caso de un procesador de solicitudes altamente escalable
- Reflexiones finales sobre los roles genéricos

Objetivos

El capítulo final examina estas dos extensiones de nuestro sistema de votación: la similitud con otra aplicación empresarial y la aplicación de uno de nuestros niveles fundamentales a un producto no relacionado con requisitos de infraestructura similares. Se presentará al lector una perspectiva de cómo se conectan problemas generales menores para resolver un problema empresarial específico. Aprenderá cómo se puede aplicar un mecanismo técnico genérico para implementar características muy dispares en diversas aplicaciones.

Al final de este capítulo y al concluir el libro, se mostrará al lector la posibilidad de reutilizar tanto la función como la forma de la aplicación de votación. Por reutilización de la función se entiende que la solución puede aplicarse a diferentes problemas. La forma de la aplicación que se reutiliza se refiere a la extracción y el uso de subsistemas y componentes discretos individuales como partes de soluciones que no guardan ninguna similitud ni relevancia con una aplicación de votación.

Sistemas de votación en las empresas

El primer tipo de reutilización que analizaremos es la aplicación de la forma de nuestra solución a problemas similares. Existen numerosos ejemplos de sistemas con diferentes nombres, pero que pueden considerarse sistemas de votación, o que comparten en gran medida los mismos objetivos y enfrentan los mismos desafíos que un sistema de votación.

Primero, comenzaremos con los principios básicos, qué significa la palabra voto.

Un diccionario popular da dos definiciones, como diferentes partes del discurso:

- Sustantivo: Una indicación formal de una elección entre dos o más candidatos o cursos de acción.
- Verbo: Dar o registrar un voto.

Un sistema de votación es el proceso y los mecanismos utilizados para realizar el acto de votar.

Los sistemas que pueden clasificarse como sistemas de votación se identifican fácilmente por su aplicación. Los sistemas políticos democráticos se basan en alguna forma de votación para determinar la voluntad popular. Los concursos de popularidad, como el sistema de votación de talentos en línea que se centra en este libro, son claramente sistemas de votación. Sin embargo, se trata de aplicaciones específicas de la votación, donde la suma de los votos es el objetivo final. Al examinar la definición de voto, no se estipula que el objetivo sea determinar la popularidad de los candidatos, solo que se registra la elección entre ellos.

Aplicaciones de votación no reconocidas. Teniendo

en cuenta la definición anterior de votación, existen otros casos de uso que podrían clasificarse como aplicaciones de votación. A continuación, se presentarán algunos de ellos, con la justificación para que el lector considere su clasificación como ejercicio de votación.

La práctica de pedir a la carta del menú de un restaurante indica qué aperitivo, plato principal, guarniciones y postre prefiere cada comensal. Esto es directamente análogo a la indicación de un alcalde, un consejo municipal y un ejecutivo del condado preferidos en una elección política local.

En este ejemplo hay consideraciones adicionales, la más importante de las cuales es el presupuesto. Si bien es una victoria para un candidato obtener una abrumadora mayoría de votos en unas elecciones políticas, es muy probable que sea problemático que todos los comensales pidan el mismo plato principal en un restaurante. Los ingredientes necesarios para que los votos de esos comensales surtan efecto (es decir, los pedidos) podrían no ser suficientes para alimentar a todos los clientes a la vez. Por lo tanto, si bien no hay un costo por voto para un candidato a la alcaldía, un restaurante debe considerar un inventario por pedido. Esto generalmente se gestiona mediante decisiones y reglas exclusivas del sistema; por ejemplo, declarar cerrada la votación para un candidato específico cuando la popularidad supera cierto umbral. Otra aplicación común que se superpone con la votación es la inscripción a un evento en línea o virtual. Los usuarios registran su interés en participar y proporcionan una dirección de correo electrónico a la que se les enviará...

Recibir una invitación para unirse al evento. Este ejemplo se asemeja más a la clásica aplicación de votación política, ya que no hay un suministro de ingredientes que agotar, pero se diferencia de los dos ejemplos anteriores, ya que los candidatos son las direcciones de correo electrónico de los usuarios: los candidatos son los propios votantes.

Aplicaciones de votación comunes. No todos los casos de patrones de sistemas de votación son tan sutiles como los ejemplos anteriores. En el ámbito empresarial y social, a menudo se recopila la distribución de preferencias para utilizarla como factor a la hora de elegir un curso de acción.

Las encuestas en línea están disponibles como aplicaciones independientes como SurveyMonkey, como herramientas separadas acopladas a otras herramientas de colaboración, por ejemplo, el descontinuado Google Moderator, o como funciones integradas como la aplicación Poll en Google Chat. Las personas a menudo realizan votaciones sin usar una aplicación de votación dedicada, como un mensaje de texto grupal, Hola a todos, pizzadosa u onigiri para el almuerzo. Esto puede funcionar porque con un grupo lo suficientemente pequeño, hay menos necesidad de un sistema de votación estructurado: las respuestas de cinco personas con respecto a sus preferencias de almuerzo son fáciles de ver y contabilizar, pero si la llamada para la preferencia de comida se dirigiera a 50 o 200 personas, necesitaría una aplicación para realizar un seguimiento de los votos.

En muchos asuntos de protocolo y regulación empresarial, se requiere que la votación sea controlada y auditable. Las resoluciones de los accionistas en empresas que cotizan en bolsa, la elección del consejo de administración y la votación consultiva sobre la remuneración de los ejecutivos son cuestiones sujetas a distintos grados de regulación en los distintos sectores empresariales, pero todas exigen que la votación se realice de forma autorizada y segura.

Derivando una aplicación de votación similar.

Examinemos la adaptación de nuestro sistema de votación a una de las aplicaciones más disímiles descritas anteriormente: el pedido de platos en un restaurante. El sistema de votación de talentos en línea hizo visibles varias entidades para el usuario final, y con la adaptación mínima de nuestro sistema,

Estas entidades tienen contrapartes directas. La siguiente tabla presenta entidades análogas en los dos dominios del problema:

Online talent voting		A la carte ordering	
Entity	Description	Entity	Description
Round	The set of acts that can be selected from for voting	Meal	The set of dishes that can be ordered in the same window of time: i.e. Breakfast, brunch, dinner
Act	An act that can be selected as preferred by a voter	Dish	An item that can be selected as preferred by a diner
Vote	Selection of an act in a round by a voter	Order	Selection of a dish in a meal by a diner
Vote budget	The maximum number of acts that a voter may vote for in the context of a round	Preparation capacity	The maximum number of orders for a given dish that can be accepted in the context of a meal
Voter	A participant in the voting process - one who creates votes	Diner	A presumed consumer of a meal - one who creates orders

Cifra **14.1** Paralelismos en la votación y el ordenamiento

Si bien existen analogías para las entidades significativas en ambas aplicaciones, las descripciones indican que no se trata de una correspondencia directa y perfecta. La diferencia más evidente radica en el presupuesto de votos y la capacidad de preparación. Si bien son similares, ya que imponen una restricción al número de pedidos que se pueden realizar, dichos límites se centran en diferentes partes del pedido. Un presupuesto de votos está asociado a un votante; limitamos el número de votos que un votante puede emitir en cada ronda. Además, la política de votación exige que un votante solo pueda emitir un voto por un acto específico en una ronda. Por otro lado, un comensal puede pedir tantos platos como desee, pero existe un límite en la cantidad de pedidos de ese plato (que es la contraparte de un acto) que se pueden preparar durante una comida debido a la cantidad limitada de ingredientes y al equipo y personal de preparación disponibles. Estas diferencias no rompen la analogía, pero sí resaltan la importancia de limitar la lógica de la política a componentes específicos. Podemos crear un clon de la implementación de nuestro sistema de votación y luego aplicar una refactorización relativamente sencilla para mapear las entidades y relaciones de la aplicación original a la nueva. Por otro lado, la lógica de negocio que requiere mayor atención en la nueva...

La aplicación puede estar sujeta a su propio ciclo de requisitos, diseño e implementación. Dado que conocemos la ubicación de esta lógica en el código base, en el paquete del motor de políticas, podemos abordar estos dos subproyectos de forma independiente con relativa confianza en su aislamiento.

Reutilización de componentes del sistema de votación.

Además de adaptar nuestro sistema de votación a otros casos de uso con un alto grado de similitud, como ya hemos comentado, existe otro nivel de granularidad en la reutilización que se puede aprovechar. El principio de especificación y diseño de caja negra produce módulos con límites y contratos bien definidos con sus socios, ya sean clientes o proveedores. Al combinarse con el aislamiento de la lógica de negocio en módulos independientes, esto produce componentes con un alto potencial de reutilización.

La reutilización de un sistema puede ser de mayor nivel que la implementación real. En el ejemplo anterior, que deriva un sistema de pedidos de menú de nuestra aplicación de votación en línea, la reutilización se centra en la arquitectura y el diseño, en lugar de en artefactos específicos de la implementación, como el código o las opciones de configuración. La reutilización a estos niveles superiores puede ser tan rentable o incluso más que la reutilización de un fragmento de código JavaScript o una clase de Python.

Así como la reutilización de un sistema puede realizarse en muchas etapas de desarrollo, la reutilización de componentes fuera de su función en el sistema original produce grandes beneficios en diferentes etapas de desarrollo.

Reutilización de patrones de

diseño. La aplicación de un patrón de diseño se facilita cuando se dispone de una implementación de referencia relevante. Muchas implementaciones de ejemplo son excelentes para ilustrar los conceptos que subyacen a un patrón de diseño, pero pueden ser difíciles de usar como punto de partida para la implementación. Las razones de esta dificultad varían considerablemente, pero la mayoría de las veces se debe a una divergencia entre el entorno y las dependencias presuntos del ejemplo y los que utiliza el desarrollador local. Si existe una implementación de referencia para un patrón de diseño en

La base de código local del desarrollador está en uso y puede implementarse, las dificultades que presenta el entorno desaparecen.

Analicemos un caso específico de reutilización de componentes, donde la aplicación de votación proporciona una excelente implementación de referencia a partir de la cual se podrían crear fácilmente otras implementaciones. Comenzaremos con una nueva definición del problema y luego analizaremos el componente del sistema de votación existente cuyo patrón de diseño se aplica al nuevo problema.

Necesidad de clasificadores y procesadores específicos de clase

Discutiremos el diseño de un modelo ideal o hipotético para una aplicación de procesamiento de voz que sea altamente sensible a la latencia.

En concreto, diseñaremos una aplicación de transcripción de voz en tiempo real para dispositivos móviles: una que captura el audio de un micrófono y convierte el habla escuchada en texto con muy baja latencia.

Un requisito fundamental es la necesidad de una capacidad de respuesta constante; es decir, la aplicación siempre debe completar la transcripción dentro de un intervalo de tiempo constante tras la escucha del discurso. Al no permitir pausas intermitentes ni retrasos en la transcripción, se descarta la posibilidad de enviar el audio de ida y vuelta a un servicio en la nube, ya que el ancho de banda de una conexión móvil es inestable y poco fiable.

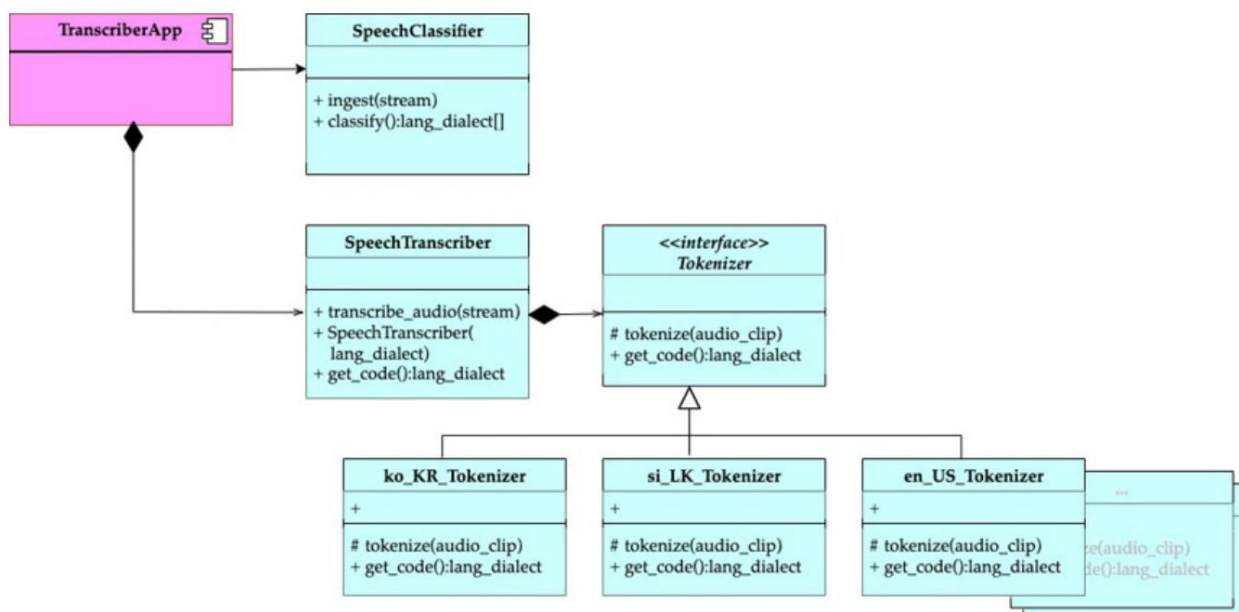
El diseño inicial de nuestra aplicación requiere un módulo específico para cada idioma y dialecto que permita etiquetar conjuntos de fonemas como palabras. Disponemos de una biblioteca que incluye varias docenas de módulos Tokenizer, cada uno específico para un dialecto regional o cultural. Todos estos módulos implementan la misma interfaz de programación de aplicaciones (API), por lo que podemos llamarlos indistintamente.

Un segundo requisito de nuestra aplicación es la capacidad de identificar automáticamente el idioma que se escucha. La misma biblioteca que proporciona los tokenizadores específicos también incluye un clasificador, un módulo que puede ingerir una secuencia de audio y generar un conjunto de idiomas y dialectos que probablemente se usen, con índices de confianza para cada uno. Su uso típico es alimentar el audio al clasificador hasta que identifique un candidato.

Idioma/dialecto con alta probabilidad. Una vez hecho esto, el mismo audio y los subsiguientes se envían al transcriptor, que está configurado para usar el Tokenizador para ese idioma/dialecto.

Este modelo introduce un retraso inicial en la transcripción mientras el clasificador funciona, pero se considera aceptable un retraso de inicio que genere un retraso potencial por sesión que exceda nuestro objetivo de latencia de 200 ms.

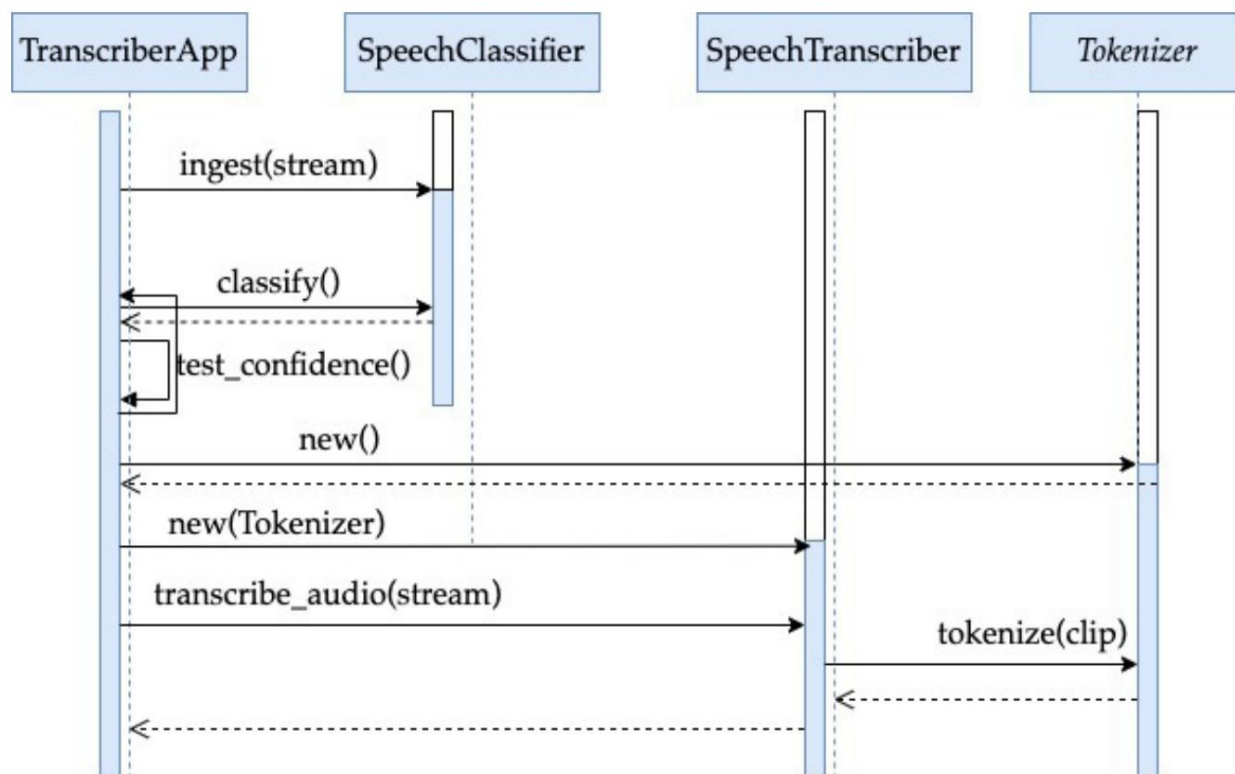
El diseño parece completo y robusto. Los siguientes diagramas de clases y secuencias ilustran los módulos involucrados y su interacción:



Cifra 14.2 Diagrama de clases de transcripción de voz

Este diagrama de clases ilustra los módulos y las relaciones relevantes de la explicación anterior. Nuestra aplicación **TranscriberApp** utiliza directamente un **SpeechClassifier** y un **SpeechTranscriber**. Un **SpeechTranscriber** requiere una instancia de la subclase **Tokenizer**. El diagrama muestra algunas clases específicas de **Tokenizer** con una referencia elidida a muchas más.

Con dichos componentes identificados, la siguiente figura ilustra sus interacciones:



Cifra 14.3 Diagrama de secuencia de transcripción de voz

La figura ilustra que se utiliza y se completa un `SpeechClassifier` antes de crear un `SpeechTranscriber`. Esto es necesario, ya que una dependencia de la creación de un `SpeechTranscriber` (un código de idioma y dialecto) se obtiene del `SpeechClassifier`.

Como se mencionó anteriormente, el diseño parece consistente y satisface nuestros requisitos. Esta es una conclusión prematura, ya que veremos en la siguiente sección que uno de nuestros requisitos no se puede cumplir.

Abordemos la necesidad de tener muchas alternativas disponibles cuando sólo se necesita una.

El problema con este diseño reside en el tamaño del espacio de ejecución de toda la implementación. Como ya hemos mencionado, la aplicación podría tener que soportar decenas o incluso cientos de combinaciones específicas de idiomas y dialectos. Esto implica tener un número igualmente grande de implementaciones de `Tokenizer` disponibles en tiempo de ejecución, cuando en realidad solo se necesita una a la vez. Si asumimos que una implementación de `Tokenizer` ocupa un gran número de bytes, el espacio de ejecución de nuestra aplicación supera con creces el tamaño viable.

Una solución a este problema es instanciar dinámicamente sólo el Tokenizer que se necesita para el lenguaje que se ha identificado.

Hemos implementado una función en la aplicación de votación que puede adaptarse para resolver este problema. La aplicación cliente de votación obtuvo el cuerpo de una función ejecutable del servidor para ejecutar localmente la política de votación. El mismo mecanismo general que obtiene e instala la lógica local de una fuente confiable puede usarse para obtener e instalar el tokenizador específico para nuestro idioma y dialecto actuales. En el caso de una plataforma que impide la ejecución de código remoto, la lógica puede codificarse en una gramática interpretada o un lenguaje de reglas y evaluarse mediante un intérprete integrado que carga las reglas del tokenizador específico desde una fuente remota. Hemos reutilizado un patrón de diseño, que es su naturaleza, y mediante una refactorización sencilla, podemos generalizar y reutilizar la implementación de ese patrón de diseño en nuestro código base.

Optimización del diseño para el rendimiento

Esta instalación de código dinámico generará un mayor retraso en el inicio, lo cual se acepta dentro de los límites de nuestros requisitos.

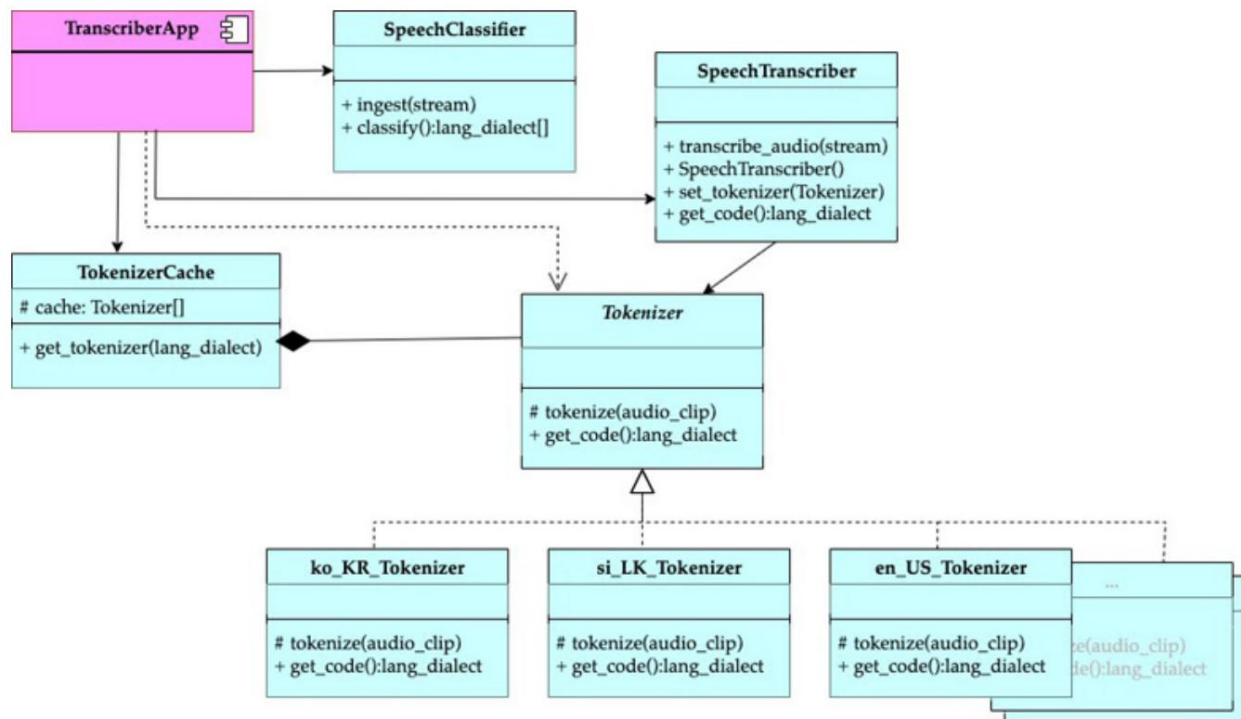
También podemos aprovechar algunos aspectos conocidos de este caso de uso para mejorar el rendimiento.

1. La mayoría de los clientes están acostumbrados a transcribir sólo una pequeña cantidad (1 o 2) de idiomas/dialectos.
2. El idioma/dialecto encontrado más recientemente en un cliente es

Es muy probable que sea el próximo también.

Teniendo en cuenta estos patrones de uso, podemos introducir una caché local en el cliente de los dos tokenizadores específicos utilizados más recientemente y comprobarla antes de obtener un tokenizador del servidor. Con esto, nuestro transcriptor puede gestionar cualquier idioma y dialecto compatible y solo transfiere un tokenizador una o dos veces en la mayoría de los clientes.

El diagrama de clases y secuencias para este diseño modificado se muestran en la siguiente figura:

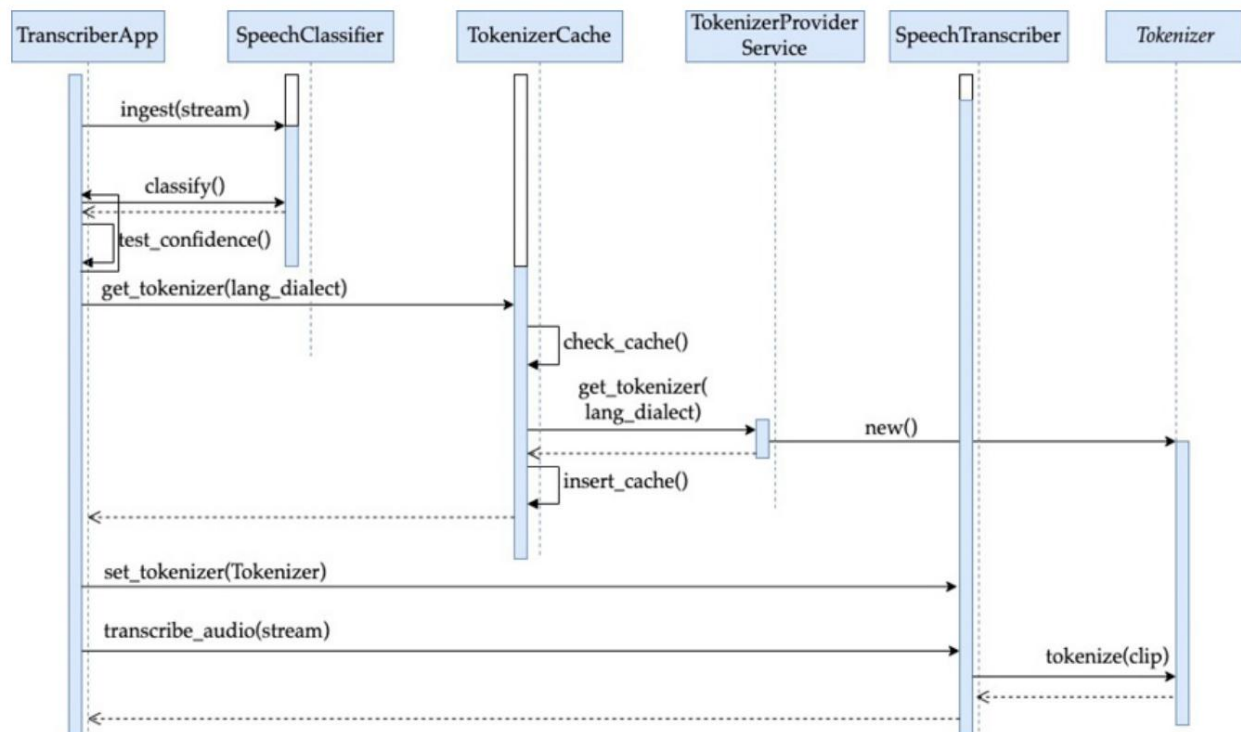


Cifra 14.4 Diagrama de clases de transcripción de voz en caché

Este diagrama de clases actualizado muestra la introducción de un **TokenizerCache** entre **TranscriberApp** y **Tokenizer** jerarquía de clases. En lugar de contener cualquier número de definiciones de clases y definiciones estáticas de subclases de **Tokenizer**, **TranscriberApp** ahora contiene una referencia a un único **TokenizerCache** que administra las clases **Tokenizer** en el cliente.

No se incluye en el diagrama de clases un servicio basado en la nube que devuelve una implementación específica de un Tokenizador, de manera similar a cómo **PolicyEngine** entrega una función ejecutable a un cliente remoto. Esto es un método de una API REST y se muestra en la secuencia adjunta diagrama.

La siguiente figura es el diagrama de secuencia de nuestro nuevo diseño:



Cifra 14.5 Diagrama de secuencia de transcripción de voz en caché

La figura de secuencia anterior ilustra el peor escenario de uso de TokenizerCache, en el que no existe un tokenizador para la combinación de idioma y dialecto solicitada en la caché. En la mayoría de los clientes, esto ocurrirá una sola vez, pero se muestra aquí como la ilustración más completa de los componentes involucrados.

Hemos explorado la reutilización de los recursos del sistema de votación de diversas maneras y con diferentes niveles. La aplicación en sí puede considerarse de forma más general y refactorizarse para permitir su uso compartido entre aplicaciones derivadas, o clonarse y modificarse para crear nuevas aplicaciones. Cuando vemos la oportunidad de aplicar un patrón de diseño implementado en el sistema de votación a una nueva aplicación, la implementación de votación de dicho patrón puede utilizarse como plantilla o generalizarse para cubrir el nuevo caso de uso. La reutilización no se limita a la reutilización de código; la aplicación de arquitecturas o conocimientos de diseño existentes es igualmente valiosa. Cualquier sistema de software bien diseñado constituye un recurso valioso como estado de la técnica para nuevos proyectos si se reconocen las similitudes.

Estudio de caso de un procesador de solicitudes altamente escalable

Un tema de controversia que surgió de los requisitos de la aplicación de votación fue la necesidad de respuestas de baja latencia a las solicitudes de votación del cliente, junto con la necesidad de aplicar actualizaciones coordinadas a los almacenes de datos globales para satisfacer dichas solicitudes. La tensión surgió al imponerse estos requisitos desde dos perspectivas muy diferentes: el cliente solo ve su propia acción, el voto emitido, mientras que el sistema no distingue a ningún cliente en particular; solo ve un conjunto de transacciones de bases de datos de diversas fuentes, todas las cuales deben completarse con la misma prioridad y fiabilidad.

La solución diseñada se centró en que la urgencia del cliente era que su acción quedara registrada, es decir, que la solicitud en sí fuera duradera, no que se completara. El punto de vista complementario es que el sistema, en general, no atribuye una urgencia inherente a ninguna solicitud individual, sino que simplemente exige cumplir con todas.

Este es un conjunto común de requisitos: la necesidad de garantías inmediatas de consistencia final. Existen sistemas donde la necesidad de una resolución consistente y definitiva del estado del sistema es urgente, pero estos son relativamente raros y suelen ocurrir en dominios altamente especializados, como aplicaciones médicas, control de sistemas en tiempo real y sistemas de gestión y respuesta a crisis. El principio de Eisenhower de que lo urgente no es importante y lo importante nunca es urgente se aplica a la mayoría de las situaciones empresariales y sociales.

Patrón de diseño de trabajador genérico. El diseño

de nuestro backend de procesador de votos y su fuente de trabajo, la suscripción de Pub/Sub votes-sub, es una variación de un patrón de programación concurrente bien conocido: el patrón de diseño de trabajador. Este patrón está diseñado para abordar problemas en los que:

- El tiempo de respuesta de una interfaz de usuario (UI) o API debe ser

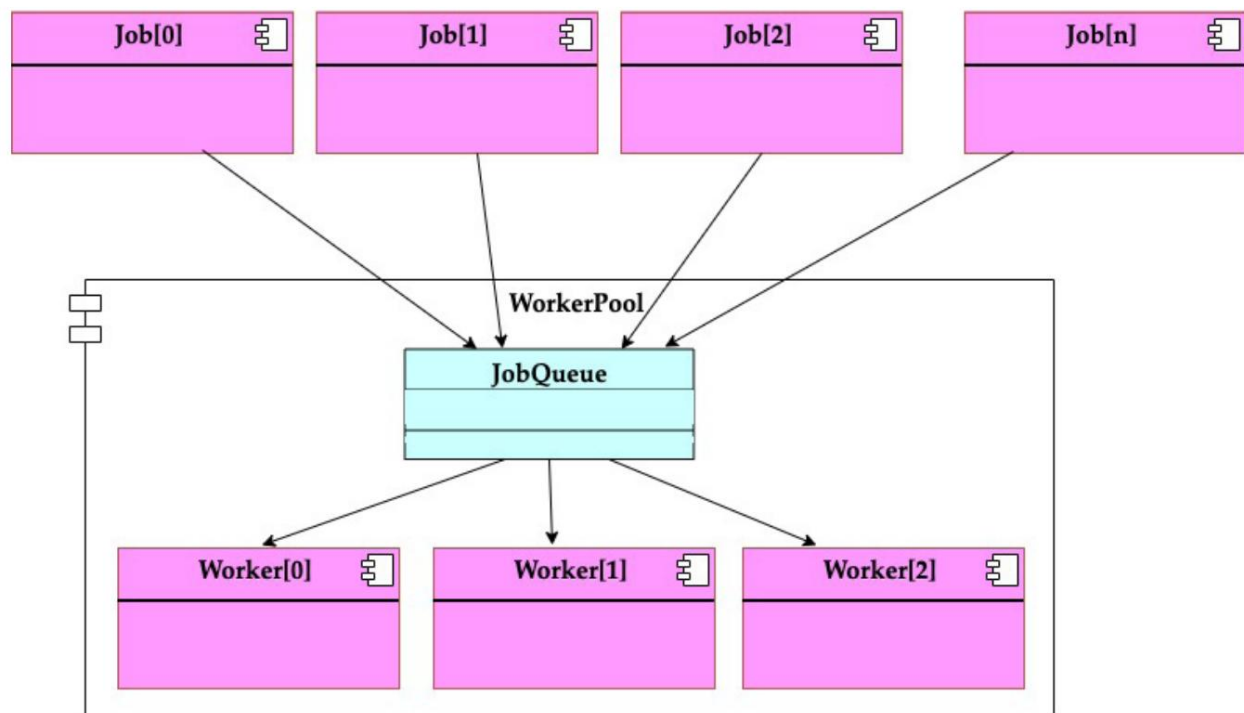
minimizado.

- El procesamiento en segundo plano sin bloqueo se puede ajustar para lograr un equilibrio entre el espacio (la cantidad de trabajadores que procesan solicitudes) y el tiempo (la latencia del cumplimiento de una solicitud).

Una forma especializada del patrón de diseño de trabajadores es el patrón de diseño de grupo de trabajadores, que introduce la gestión central de una cantidad de trabajadores y desacopla las solicitudes del frontend de trabajadores específicos.

El procesador de votos y sus recursos asociados son una implementación sencilla del patrón de diseño del grupo de trabajadores, pero que aprovecha los servicios de Google Cloud Platform (GCP) . Aquí analizaremos algunas de estas soluciones específicas de GCP.

Primero, presentaremos un diagrama de arquitectura de alto nivel del patrón de grupo de trabajadores genérico:



Cifra **14.6** Patrón de diseño de grupo de trabajadores

En referencia al diagrama anterior, a continuación se presentan breves descripciones de las entidades ilustradas:

- Un trabajo es un originador de solicitudes, como un cliente de UI o un cliente-

frente al punto final de la API.

- Un trabajador es una tarea asincrónica que procesa las solicitudes.
- JobQueue es la cola de solicitudes que han sido emitidas por los Jobs y que aún no han sido consumidas por los Workers.
- WorkerPool es la entidad que administra los recursos de JobQueue y las instancias Worker.

Con esos roles definidos, discutiremos los servicios específicos de GCP que utilizamos para cumplir cada uno de ellos en la aplicación de votación.

Cloud Pub/Sub gestiona correctamente la función de una cola de trabajos. El conjunto de un tema y una suscripción se diseñó para una cola de trabajos o tareas de baja latencia.

Garantizar que el reconocimiento de un voto sea confiable o digno de confianza depende de las garantías del servicio Pub/Sub de que un mensaje publicado sea duradero.

El WorkerPool se encarga de gestionar el ciclo de vida y la cantidad de trabajadores. La aplicación de votación delega esta tarea en la combinación de una suscripción push y el escalado automático del punto final de la suscripción.

Un trabajo en el sistema de votación está representado por una instancia de votingfrontend que valida y registra el voto de un usuario.

El rol de Trabajador lo desempeña el servicio voteprocessor , que recibe solicitudes de la cola, las procesa y devuelve el estado de ese procesamiento.

[Aplicaciones de procesamiento de solicitudes de alto rendimiento.](#) En

este capítulo se

analizaron aplicaciones con problemas similares, como el caso de uso de pedidos a la carta. Estos problemas tenían una función similar, mientras que esta sección explora aplicaciones con una forma o topología similares, así como desafíos tecnológicos y logísticos.

El problema general que representa la aplicación de votación se puede plantear de la siguiente manera: Un sistema que procesa transacciones que

Se originan de muchas fuentes independientes a lo largo del tiempo contra una cuenta común. El sistema de

votación implicaba una complejidad adicional, ya que también procesaba transacciones contra un conjunto secundario de cuentas: el presupuesto de votación de los usuarios. Sin embargo, esto no representaba un desafío en términos de escalabilidad y confiabilidad, por lo que centraremos este análisis en la acumulación de votos en el almacén de datos global.

Si buscamos sistemas que tengan una participación amplia o generalizada, que aporten información a un estado o conjunto de datos común, varios nos vienen a la mente. Enumeraremos algunos a continuación para

- ampliar: Sistemas de concursos y registro en tiempo real: Son aplicaciones que se utilizan para registrar el interés en un premio, como artículos o asistencia a eventos. Se trata de una actualización moderna del paradigma clásico de los concursos de radio, para los primeros 100 participantes o, en ocasiones, para cada décimo participante. En estos sistemas, se recompensa a los participantes por responder a tiempo. La entrega del premio a los ganadores puede ser un proceso de seguimiento, iniciado por un proceso de backend que consume asincrónicamente la cola de inscripciones.
- Sistemas de recolección de telemetría: Estos sistemas recopilan y registran lecturas de algún tipo de instrumento en tiempo real. Un uso específico es la recopilación de datos meteorológicos y climáticos de numerosos sensores meteorológicos locales desplegados en una ciudad o país. Las lecturas de temperatura, presión barométrica, velocidad y dirección del viento deben registrarse, pero la agregación e interpretación de los datos se realiza en segundo plano con cierta regularidad, como las actualizaciones del pronóstico meteorológico cada hora. Vale la pena destacar la escala de algunos de estos sistemas. En EE. UU., el Servicio Meteorológico Nacional (NWS) recopila y analiza más de 6.300 millones de observaciones al día (4,4 millones por minuto o 72.900 por segundo, de forma continua) y publica alrededor de 1,5 millones de pronósticos al año. Si bien no todas estas observaciones provienen de lecturas directas de sensores, la magnitud es impresionante, especialmente para un sistema que debe estar siempre activo. Este es un caso de uso donde la escala y la disponibilidad...

Los requisitos no tienen parangón en ningún sector empresarial, con excepción de la búsqueda web y la publicación de anuncios.

[Evolución de los sistemas de procesamiento de grandes solicitudes.](#) La

necesidad de una comunicación fiable entre las diferentes partes de un sistema es común y un caso de uso tan convincente que ha dado lugar a toda una clase de productos conocidos como sistemas de cola de mensajes, de los cuales Cloud Pub/Sub es la oferta principal de GCP. La necesidad de comunicación entre procesos fuera de la memoria local o los sistemas de archivos surgió a medida que los sistemas distribuidos se generalizaron. Sin infraestructura de sistemas distribuidos ni experiencia en la industria, los primeros sistemas contenían soluciones únicas de desarrollo propio que, a menudo, eran prácticamente suficientes para gran parte de su ciclo de vida. Ahora, los proveedores de infraestructura en la nube, especializados en abordar estos requisitos de infraestructura, proporcionan estas capacidades tan accesibles y fiables como las del sistema operativo o las de los marcos de aplicación básicos.

La disponibilidad de una comunicación distribuida y fiable entre procesos permitió a los desarrolladores de aplicaciones idear nuevas maneras de resolver problemas empresariales, como el procesamiento de un gran número de actualizaciones de un conjunto de datos. Las transacciones complejas de varias etapas que propagan actualizaciones a través de una jerarquía de concentración mediante una serie sucesiva de trabajos por lotes eran un patrón común en los sistemas financieros y de inventario antes de que la arquitectura de software distribuida actual se generalizara. En muchos de estos sistemas, los resultados finales de una actualización no se conocían hasta días o semanas después, y en algunos casos era imposible conocer el valor real de una cuenta.

[Ventajas de la infraestructura en la nube.](#) Hemos visto que

la aplicación de votación es una instancia de un patrón de diseño bien conocido: el grupo de trabajadores. Este patrón, como todos los patrones de diseño, se escribe de forma independiente de la implementación y, por lo tanto, no tiene en cuenta lo que podría proporcionar un servicio en la nube en comparación con lo que escribe el...

Desarrolladores implementadores.

El examen de los detalles de la solicitud de votación muestra que los dos

Los componentes más amplios del patrón, JobQueue y

Se implementaron WorkerPool en el sistema de votación con configuración

Opciones de dos servicios de GCP: Cloud Run y Cloud Pub/Sub.

Diseñado para incluir un caso de uso como este patrón y ofrecido

Soluciones que requerían solo una pequeña cantidad de configuración. Además

Proporcionando esos dos roles en el patrón, la combinación de Cloud Run

y Pub/Sub definen la API para el rol de trabajador que se implementa

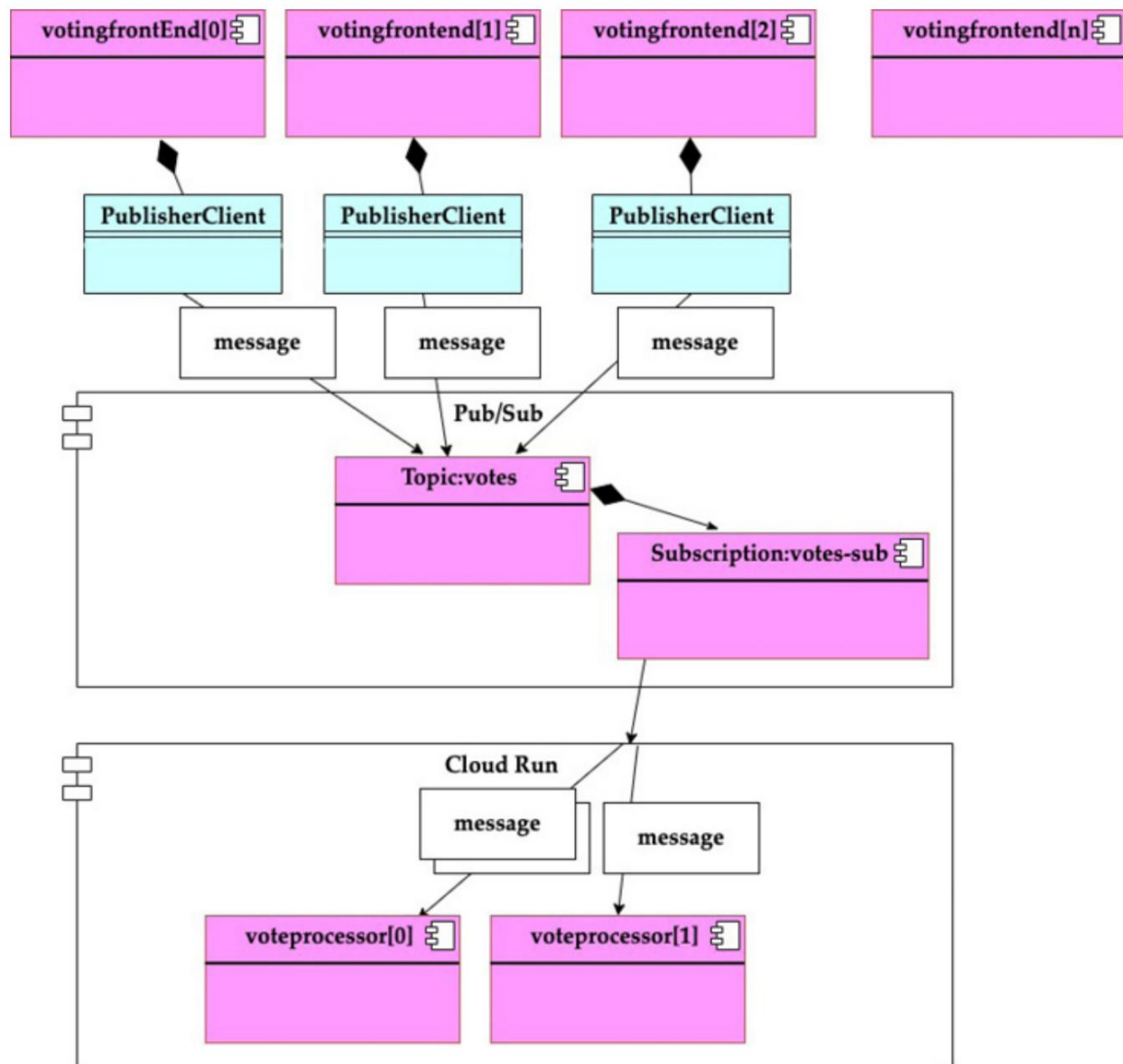
por el servicio de procesamiento de votos de Cloud Run ; y Pub/Sub definió el

API y proporciona componentes para permitir que el rol laboral sea cubierto por nuestro

Servicio de interfaz de votación .

El siguiente diagrama revisa la [Figura 14.6](#) del diseño del grupo de trabajadores.

patrón, pero con las implementaciones específicas respaldadas por GCP utilizadas en el solicitud de votación:



Cifra 14.7 Patrón de diseño de grupo de trabajadores con GCP

Implementar el patrón de diseño del grupo de trabajadores habría sido una tarea mucho más compleja sin los servicios de GCP que hemos aprovechado. Las ventajas adicionales de Google Cloud Monitoring, la gestión de recursos como el escalado automático y funciones como la redistribución automática y las garantías de durabilidad hacen que nuestra aplicación de votación sea de mayor calidad que la que nuestro propio equipo de desarrollo podría haber producido sin invertir mucho más tiempo.

Reflexiones finales sobre los roles genéricos

La aplicación de votación que se ha creado e ilustrado en este libro es solo una parte de un sistema más amplio. Las demás características de la aplicación son importantes; no podríamos lanzar ni ejecutar una aplicación de votación sin ellas, pero el enfoque del libro se limitó con una intención específica.

A continuación, describiremos brevemente una visión más completa del sistema general de votación de talentos. A continuación, compararemos la aplicación de votación, tema central del libro, con las demás funciones para destacar cómo se seleccionaron las características exploradas y qué las distingue para merecer especial atención como caso práctico para el desarrollo de GCP.

El sistema de votación completo. La

emisión de votos es el momento o la porción para la que existe el sistema, pero es solo una pequeña parte del sistema total que debe existir para habilitarlo. Sin funciones para configurar y dar soporte a la aplicación de votación, sería imposible gestionar el proceso de votación. La porción relativamente pequeña del sistema total que se utiliza directamente en el proceso de votación es compatible con muchos sistemas en los que las funciones críticas se ven superadas por el ecosistema de soporte.

Una analogía, en un ejemplo muy diferente pero relevante, es el cerebro humano. El cerebro es el centro de la conciencia, del yo; es donde reside el yo y es la parte de nuestro cuerpo con la que más nos identificamos. El cerebro humano adulto promedio pesa 1,3 kilogramos, de un peso corporal total promedio de 70 kilogramos. El cerebro, el centro del ser de una persona, representa solo el 1,8 % de su masa corporal, y otros sistemas y partes del cuerpo lo superan en un factor de 50. De igual manera, la aplicación de votación puede tener 2000 líneas de código e información de configuración, mientras que las funciones que la respaldan pueden tener 10 000 líneas en varias docenas de módulos.

El equipo de apoyo del cerebro humano incluye el esqueleto, la musculatura, el sistema circulatorio y el sistema linfático, entre muchos otros.

Otros. Enumeraremos y analizaremos los principales subsistemas análogos necesarios para la aplicación de votación: Entrada y edición de

- **actos:** Los artistas están representados en la aplicación de votación únicamente por su identificador único, el ID del acto, y un nombre corto en la ronda actual. Estos ID y nombres son referencias a una lista maestra de actos, que se utiliza para identificar a un artista en las rondas en las que compite. La entrada de estos datos, junto con la información de contacto y calificación asociada, debe realizarse mediante una aplicación de entrada de datos.
- **Administración de usuarios:** Si bien la identificación y autenticación de usuarios se gestionan mediante un producto de infraestructura de Gestión de Identidad y Acceso (IAM) , como Firebase Authentication, será necesario actualizar la información de cada usuario que el sistema genera y mantiene. Por ejemplo, podría ser necesario restablecer el historial de votaciones del usuario para mitigar un error del sistema o del usuario.
- **Definición de ronda y asignación de actos:** La aplicación de votación presentada solo incluía una definición de ronda predefinida con una lista de actos predefinidos. En la práctica, el sistema generará los actos para una nueva ronda basándose en los resultados de las rondas anteriores y debería incluir mecanismos para que un operador humano modifique los datos generados.
- **Gestión de la ventana de votación de rondas:** Al comenzar una ronda, también comienza la votación. La elegibilidad de los artistas para votar se determina mediante reglas codificadas en el motor de políticas. El sistema de producción incluirá un método para establecer la elegibilidad de cada artista en la ronda actual a medida que esta avanza. Al finalizar una ronda, la ventana de votación se cierra y los artistas ya no son elegibles para votar. El sistema completo proporcionará todas estas funciones.
- **Posprocesamiento de los resultados de la votación:** Una vez emitidos todos los votos y vacías las colas de mensajes, debe haber una forma de informar los resultados y activar las acciones bloqueadas por la apertura de la ronda de votación. Debería haber una

Una serie de paneles que ofrecen diversas vistas de los resultados, como resultados intermedios regionales y temporales. Los resultados podrían quedar en estado pendiente de revisión, lo que requiere la finalización por parte de una persona o un comité. Estas funciones deben implementarse y funcionar de forma fiable antes de que se complete el primer ciclo de votación.

- Paneles y controles de soporte de operaciones: no se puede asumir que el sistema siempre funcione como se espera. Deben estar disponibles paneles que revelen el estado actual del sistema para confirmar su correcto funcionamiento o identificar problemas. Se deben proporcionar algunos controles, como la posibilidad de silenciar temporalmente el sistema, lo que permite que rechace las solicitudes de los usuarios de forma controlada e informativa.

Al leer esta lista de subsistemas de soporte, es probable que algunos sean al menos tan grandes y complejos como la propia aplicación de votación. Si multiplicamos esto por el número de estos subsistemas (seis), resulta evidente que la aplicación de votación principal solo representará una pequeña parte del sistema total, aunque crucial. Al igual que el cerebro humano, el sistema de votación no podría funcionar, ni siquiera existir, sin su ecosistema de subsistemas hermanos que lo apoyan.

[Comparación de aplicaciones de votación.](#) Estos seis

sistemas identificados son críticos, y su calidad y rendimiento son necesarios para el funcionamiento completo del sistema. La aplicación de votación se distingue de sus similares en una característica, razón por la cual fue elegida como caso de estudio de este libro.

La combinación de escalabilidad y capacidad de respuesta que requiere la aplicación de votación no tiene parangón en ningún otro sistema. La razón de esta diferencia radica en el factor de demanda, la fuente de tráfico para esta función.

La solicitud de votación está sujeta a un fuerte aumento de la demanda, como se proyectó en [el Capítulo "Cómo crear un sistema de votación disponible y confiable"](#). Estas solicitudes provienen de millones de votantes y espectadores de nuestro programa de talentos. Además del tamaño de la audiencia,

Su comportamiento es algo impredecible. No podemos asegurar con certeza que la votación se distribuirá equitativamente entre los 12 actos de una ronda; un acto puede provocar que todos los espectadores emitan un voto. Precisamente esta imprevisibilidad es la razón por la que se vota. Si pudiéramos predecir qué actos generarían actividad de votación, podríamos predecir al ganador con gran certeza.

Comparemos esto con otro sistema: Entrada y edición de actas. Se trata de una función de actualización de la base de datos que ejecuta un administrador o funcionario de la empresa en un momento determinado. No hay consideraciones de escalabilidad: un solo usuario ejecuta una serie de transacciones en unas pocas filas de datos a la vez. Los requisitos de latencia tampoco son un problema: aunque se espera que la aplicación funcione bien, si el funcionario ve un cursor activo durante uno o dos segundos después de cada actualización, no afectará a nuestro negocio.

La escala de las dos características: la votación y la entrada de actas son de magnitudes muy diferentes, por lo que examinaremos otro caso con una mayor carga de trabajo que la entrada de información de actas: el posprocesamiento de los resultados de las votaciones. Se trata de un conjunto de análisis y acciones de la base de datos impulsados por un conjunto de consultas a los resultados de las votaciones y la actividad de votación de los usuarios. La entrada es la actividad de votación agregada, por lo que el tamaño del conjunto de entrada se basa en el número de actas y el número de votantes.

Esto puede ser considerable, ya que el número de votantes que participan en una ronda puede ser elevado, pero la naturaleza de la carga de trabajo es diferente a la de la aplicación de votación. Esto puede visualizarse como una canalización que ejecuta un pequeño número de consultas complejas y costosas, mientras que la aplicación de votación consiste en un gran número de transacciones simultáneas relativamente pequeñas. La canalización puede considerarse estrecha y profunda, mientras que la aplicación de votación es relativamente amplia y superficial. En teoría, podríamos acelerar la canalización de posprocesamiento aumentando la potencia de procesamiento, mientras que el escalado horizontal es una forma más eficaz de mejorar el rendimiento de la aplicación de votación.

Las medidas que podríamos tomar para reducir el tiempo de ejecución de nuestra canalización de posprocesamiento son bien conocidas y no son específicas de un producto de infraestructura en la nube: en efecto, conseguir una computadora más rápida. El sistema automático...

El escalamiento horizontal de los procesadores frontend y backend según la demanda es una ventaja clave de un servicio de infraestructura en la nube.

[Google Cloud Platform para aplicaciones distribuidas generales](#). Las dos comparaciones

anteriores ilustran que, si bien GCP, o cualquier plataforma integral en la nube, ofrece ventajas para un conjunto completo de funciones de aplicaciones distribuidas, a menudo habrá uno o dos requisitos del sistema de la aplicación que hagan necesario el uso de la infraestructura en la nube. En muchos casos, un uso bien planificado de una plataforma en la nube permite capacidades de aplicación que de otro modo serían inviables.

Además de satisfacer los requisitos esenciales de la aplicación de votación, GCP aporta valor al resto del sistema. La monitorización automática o generada, el despliegue de versiones mediante Cloud Build y el balanceo de carga inherente de los frontends reducen el trabajo y los artefactos repetitivos necesarios para desarrollar, desplegar y operar un sistema utilizando una infraestructura de servicios clásica e independiente. Si bien GCP puede ser la única forma viable de satisfacer algunos requisitos, su adopción mejora el desarrollo y el rendimiento general de la aplicación. Existe una curva de aprendizaje, y GCP no es tan práctico ni conciso como algunos frameworks y bibliotecas de aplicaciones especializadas, pero a medida que crece el alcance de una aplicación, aumenta la ventaja de una plataforma integral.

Conclusión

Este capítulo concluye el libro con un análisis de nuestra aplicación de votación como caso práctico en el desarrollo de sistemas de aplicaciones distribuidas. Se mostró al lector cómo visualizar la aplicación en su forma más general, tanto como sistema de negocio como conjunto de características técnicas y su implementación.

Se describió el sistema de votación más amplio que rodea e incluye la aplicación de votación, y se distinguió la aplicación de votación de los otros subsistemas por su dependencia de

capacidades específicas de GCP.

El capítulo revela que nuestra aplicación implementa un patrón general de diseño de sistemas distribuidos. Con estos ejemplos, el lector está bien preparado para encontrar oportunidades para reutilizar el trabajo específico y, aún más importante, las metodologías empleadas a lo largo del libro para desarrollar con mayor rapidez aplicaciones de software fiables y eficaces que aprovechen al máximo la infraestructura de la nube moderna.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



Índice

A

Abstracciones [298](#)

Abstracciones, configuración [298](#), [299](#)

Abstracciones, ejecutando [299](#)

Accesibilidad [160](#)

Accesibilidad, enfoques

 Recursos de aplicación [161](#)

 Solicitud de rediseño [162](#), [163](#)

 Proceso de reubicación estratégica [160](#)

Patrones de acceso [237](#)

Cientes agnósticos [110](#)

Cientes agnósticos, asegurando [116-118](#)

Cientes agnósticos, términos

 Interfaces en la nube [113](#)

 Diseño general de API [110](#)

IA/ML [22](#)

IA/ML, artefactos

 Géminis [23](#)

 Google Cloud Atención médica [23](#)

 Inteligencia artificial en lenguaje natural [23](#)

 Texto de voz [22](#)

 Vértice AI [23](#), [24](#)

Antipatrones [8](#)

Antipatrones, ilustrando [10](#)

Antipatrones, razones [9](#)

Evaluación del impacto [174](#)

Evaluación de impacto, términos

 Costo a largo plazo [175](#)

 Costo a corto plazo [174](#)

Mensaje asincrónico [86](#)

Mensaje asincrónico, arquitectura [87](#)

Mensaje asincrónico, operaciones

Tareas en la nube [90, 91](#)

Escalabilidad de mensajes [88, 89](#)

Pub/Sub [88](#)

Modelo asíncrono [138](#)

Modelo asincrónico, determinando [138, 139](#)

Modelo asincrónico, implementando [139, 140](#)

Procesamiento asincrónico [5, 6](#)

Procesamiento asincrónico, objetivos

Monitoreo de backend [209](#)

Procesamiento de backend [206](#)

Escalado de GCP [207, 208](#)

Procesador de votos [208](#)

Votos publicados [203, 204](#)

B

Patrón Backends/Frontends, implementación [137, 138](#)

Mesa grande [77](#)

Bigtable, configurando [77](#)

Objetos binarios grandes (Blob) [236](#)

Especificaciones de la caja negra [303](#)

Especificaciones de la caja negra, ventajas [305](#)

Especificaciones de caja negra, arquitectura [304](#)

Especificaciones de caja negra, implementando [304, 305](#)

Blob, acrónimo

Binario [236](#)

Grande [236](#)

Objeto [236](#)

Blob, arquitectura [236](#)

do

Chatbot [106](#)

Chatbot, interfaz

GCP [106](#)

Interfaces de voz [107](#)

Aplicación cliente [260](#)

Aplicación del cliente, identificando [260, 261](#)

Aplicación cliente, implementando [261-264](#)

Solicitud de cliente, condiciones

Registro de artefactos [285](#)

Desarrollo de BeeWare [289](#)

Implementación en la nube [288](#)

Jerarquía de paquetes [282](#), [283](#)

Paquetes de Python [283](#)

Paquete de registro [287](#)

Solicitudes de clientes [136](#), [137](#)

Lógica del lado del cliente [130](#)

Lógica del lado del cliente, términos

Código Frecuente [135](#), [136](#)

Lógica sensible/Observación [134](#), [135](#)

Comportamiento significativo [130](#), [131](#)

Entrada del usuario, capturando [132](#), [133](#)

Interfaz de usuario [131](#), [132](#)

Cloud Firestore [74](#)

Cloud Firestore, configuración [74](#), [75](#)

Interfaces en la nube [113](#)

Interfaces en la nube, configuración 113-115

Interfaces en la nube, parte [115](#)

Servicios en la nube [123](#)

Servicios en la nube, dependencias [123](#), [124](#)

Servicios en la nube, aislando [125](#)

SQL en la nube [76](#)

Ventajas de Cloud SQL

Gestión de bases de datos [76](#)

Aplicación independiente de la plataforma [76](#)

Almacenamiento en la nube [80](#)

Almacenamiento en la nube, configuración [80](#), [81](#)

Almacenamiento en la nube, facetas

Rendimiento/Confiabilidad [81](#)

Opciones de servicio [81](#)

Compute Engine [71](#)

Plataforma de cómputo [60](#)

Plataforma de cómputo, características

Mesa grande [77](#)

Cloud Firestore [74](#), [75](#)

SQL en la nube [76](#)

Almacenamiento en la nube [80](#)

Llave inglesa [78](#)

Plataforma de cómputo, configuración 60-62

Plataforma de cómputo, términos

Motor de aplicaciones de Google [63](#)

Google Cloud Run [67](#)

Gestión de recursos informáticos [72](#)

Gestión de recursos informáticos, formas

Configurar [72](#)

Monitoreo [73](#)

Aprovisionamiento/Implementación [72](#)

Recurso, desplegando [72](#)

Terminación/Respuesta [73](#)

Potencia de cálculo [12](#)

Poder de cómputo, servicios

App Engine [13](#)

Lote [13](#)

Carrera en la nube [12](#)

Funciones de Cloud Run [13](#)

Multiplataforma [103](#)

Compatibilidad entre plataformas [99](#), [100](#)

Compatibilidad entre plataformas, términos [101](#)

Aleto [100](#)

Kotlin Multiplataforma (KMP) [100](#)

React Native [100](#)

Multiplataforma, proveedores

Flutter

[103](#) .NET MAUI [103](#)

Ciberseguridad [55](#)

Ciberseguridad, desafíos

Corrupción [55](#)

Servicio de denegación [56](#)

Robo [55](#)

Operación no autorizada [55](#)

D

Datos [228](#)

Agregación de datos [157](#)

Agregación de datos, implementación [158](#), [159](#)

Análisis de datos [17](#)

Análisis de datos, arquitectura [17](#), [18](#)

Análisis de datos, términos

BigQuery [18](#)

Fusión de datos en la nube [18](#)

Flujo de datos [19](#)

Pub/Sub [19](#)

Disponibilidad de datos [238](#)

Bases de datos [14](#)

Bases de datos, paradigmas

Mesa grande [14](#)

SQL en la nube [14](#)

Firestore [14](#)

Llave inglesa [15](#)

Permanencia de datos [6](#), [238](#)

Permanencia de datos, fundamentos

Copia de seguridad/Recuperación [239](#), [240](#)

Archivos de la Posteridad [241](#)

Instantáneas Continuidad [241](#)

Instantáneas Votación [242](#)

Laboratorio de código [316](#) del almacén de datos

Laboratorio de código de almacén de datos, requisitos previos [317-319](#)

Laboratorio de código de almacén de datos, roles [317](#)

Datos, tipos

Profundo/Plano [230](#)

Estructurado/No estructurado [229](#)

Datos de votación [231](#)

Modelo de demanda [196](#)

Modelo de demanda, configuración [196-198](#)

Modelo de demanda, factores [196](#)

Cliente de escritorio [101](#)

Cliente de escritorio, soluciones

Multiplataforma [103](#)

Escritorio nativo [102](#)

Entorno de desarrollo [21](#)

Entorno de desarrollo, herramientas

Versión [22](#) de Cloud Build

SDK de nube [21](#)

Estaciones de trabajo Clud [22](#)

Integración de GitLab [22](#)

Directorio Codelab [163](#)

Directorio Codelab, términos

Backends/Frontends [165](#)

Archivos iniciales [164](#), [165](#)

API de votación [166](#)

Sistema distribuido [176](#)

Sistema Distribuido, términos

Control de observación [176](#)

Aplicaciones de GCP [178-180](#)

Métricas clave de confiabilidad [177](#)

Métrica [177](#)

Terminología de monitoreo [176](#)

Puntos de observación del rendimiento [181](#)

Sistema de votación [178](#)

mi

Mitigación eficaz [184](#)

Mitigación efectiva, términos

Drenaje/Conmutación por error [185](#)

Reversión [185](#)

Recursos de escala [185](#)

Indisponibilidad temporal [185](#)

Enumeración de dependencias [28](#)

Enumeración de dependencias, aspectos

Clases [31](#)

Demanda [31](#)

Afinidad de características [31](#)

Proximidad topológica [31](#)

Duración del flujo de trabajo [31](#)

Enumeración de dependencias, configuración [28-30](#)

Enumeración de dependencias, características [32](#)

Enumeración de dependencias, operaciones

Inmediato [32](#)

[32](#) de larga duración

Tubería multietapa [32](#)

Transacción [32](#)

Sesión de usuario [32](#)

Enumeración de dependencias, términos

Gasto computacional [30](#)

Latencia [30](#)

Paralelizable [30](#)

Sensibilidad [30](#)

Consistencia eventual [48](#)

Consistencia eventual, configuración [48-52](#)

F

Fallos [168](#)

Fallos, configuración [169-171](#)

Fracasos, ilustrando [173](#)

Fallos, alcance [168](#)

Afinidad de características [34](#)

Afinidad de características, arquitectura [34](#)

Afinidad de características, puntos [34](#)

Composición de largometrajes [37](#)

Composición de características, pasos

Costo Describe [38](#)

Enumerar [37](#)

Rendimiento, determinante [38](#)

Rendimiento de funciones [35](#)

Requisitos de funciones [35](#)

Requisitos de funciones, configuración [35-37](#)

Autenticación de Firebase [83](#)

Autenticación de Firebase, configuración [83](#)

GRAMO

GCP, arquitectura [12](#)

GCP, categorías

IA/ML [22](#)

Potencia de cálculo [12](#)

Análisis de datos [17](#)

Bases de datos [14](#)

Entorno de desarrollo [21](#)

Redes [16](#)

Seguridad/Identidad [20](#)

Almacenamiento [15](#)

GCP, componentes

Código [24](#) del lado del cliente

Industria/Sistemas regulatorios [25](#)

GCP, recursos

Permanencia de los datos [57](#)

Privacidad de datos [56, 57](#)

Diseño general de API [110](#)

Diseño general de API, configuración 110-113

Motor de aplicaciones de Google [63](#)

Motor de aplicaciones de Google, configuración [63, 64](#)

Motor de aplicaciones de Google, implementando [64](#)

Google Application Engine, evitando [65, 66](#)

Google Cloud IAM [83](#)

Google Cloud IAM, configuración [84, 85](#)

Plataforma de Google Cloud (GCP) [11, 12](#)

Google Cloud Run [67](#)

Google Cloud Run, configuración 67-69



IAM, partes

Autenticación de Firebase [83](#)

Google Cloud IAM [83](#)

Plataforma de identidad [85](#)

Identidad/Autenticación [5](#)

Gestión de autenticación de identidad (IAM) [82](#)

Plataforma de identidad, tipos

Autorización de aduana [86](#)

Correo electrónico/Contraseña [86](#)

Lenguaje de marcado de aserción de seguridad (SAML) [86](#)

Redes sociales [86](#)

Cliente en el navegador [104](#)

Cliente en el navegador, aplicaciones

Angular [105](#)

GCP [106](#)

Reaccionar [105](#)

Aplicación web [106](#)

Programación de aplicaciones web [104](#)

Respuesta a incidentes [183](#)

Respuesta a incidentes, componentes [183, 184](#)

Consideraciones de infraestructura [4, 5](#)

Consideraciones de infraestructura, proceso

Enumerar [41](#)

Característica, coincidencia [42](#)

Cualidades Consistencia [41](#)

Consideraciones de infraestructura, secciones

Antipatrones [8](#)

Responsabilidades de la plataforma [7](#), [8](#)

L

Equilibrio de carga [198](#)

Equilibrio de carga, asegurando [198](#), [199](#)

Equilibrio de carga, implementación [199](#)

Equilibrio de carga, tipos

Nube [198](#)

Ferretería [198](#)

Software [198](#)

METRO

Colas de mensajes [201](#)

Colas de mensajes, términos

Recurso de publicación y suscripción [201](#)

Publicación de votos [202](#)

Métrica [177](#)

Métricas, tipos

Contadores [177](#)

Indicadores [177](#)

Resúmenes [177](#)

Terminología de monitoreo, dominios

Alerta [177](#)

Tablero [176](#)

Métrica [176](#)

Monitoreo [176](#)

Nodo [177](#)

Empuja [177](#)

Causa raíz [177](#)

norte

Escritorio nativo [102](#)

Native Desktop, herramientas

Interfaz gráfica de usuario

de Linux [102](#) macOS [102](#)

Ventanas [102](#)

Aplicaciones móviles nativas [96](#), [97](#)

Aplicaciones móviles nativas, categorías

Desarrollo de Android [97](#), [98](#) Desarrollo
de iOS [98](#), [99](#)

Redes [16](#)

Redes, servicios

Cortafuegos en la nube [17](#)

Equilibrio de carga en la nube [16](#)

Red de distribución de contenido (CDN) [16](#)

Bases de datos no relacionales [234](#), [235](#)
[142](#) no iniciado por el usuario

Acciones no iniciadas por el usuario

Interfaces en la nube [145](#), [146](#)

Tráfico generado por el servidor [143](#)

Iniciado por el servidor [142](#)

Oh

Concurso de talentos en línea [150](#)

Concurso de talentos en línea, niveles

Requisitos de escalabilidad [152](#)

Proceso de votación [151](#)

PAG

Jerarquía de paquetes [278](#)

Jerarquía de paquetes, puntos

Repositorio de paquetes [281](#)

Distribución de paquetes [281](#)

Paquete, importando [280](#)

Distribuciones de fuentes [281](#)

Cualidades pertinentes [39](#)

Cualidades pertinentes, dimensiones

Costo [40](#)

Escalabilidad horizontal/vertical [40](#)

Apoyo a largo plazo [40](#)

Rendimiento [39](#)

Planificación de la migración [119](#), [306](#)

Planificación de la migración, secciones

Evaluación de alternativas [308](#)

Sustituciones de candidatos [307](#)

Costo del servicio de base de datos [311](#)

- Infraestructura Determinante [306](#)
- Decisión [313](#) sobre migración
- Diseño de migración [120-122](#)
- Plano de construcción [313](#), [314](#)
- Obsolescencia del producto [120](#)
- Bloqueo de proveedor [119](#)

R

- Bases de datos relacionales [233](#), [234](#)
- Procesador de solicitudes [332](#)
- Procesador de solicitudes, caso práctico
 - Infraestructura en la nube [335](#)
 - Publicación en la nube/subscripción [335](#)
 - Diseño de trabajador genérico [332](#), [333](#)
 - Procesamiento de solicitudes [334](#)
- Entrega de respuesta [6](#)

S

- Escalabilidad [5](#)
- Seguridad/Identidad [20](#)
- Seguridad/Identidad, términos
 - Gestión de identidad y acceso [20](#)
 - Gerente Secreto [20](#), [21](#)
- Llave inglesa [78](#)
- Llave inglesa, cajas [80](#)
- Llave inglesa, configurando [78](#), [79](#)
- Almacenamiento [15](#)
- Almacenamiento, secciones
 - Almacenamiento en la nube [15](#)
 - Firebase [15](#)
 - Disco persistente [15](#)

T

- Proximidad topológica [33](#)
- Modelo de transacción [53](#), [54](#)
- Modelo de transacción, términos
 - Atomicidad [54](#)
 - Consistencia [54](#)
 - Durabilidad [54](#)

Aislamiento [54](#)

U

Respuesta inesperada [140](#)

Respuesta inesperada, configuración [140](#), [141](#)

Urgencia Importancia [46](#)

Urgencia Importancia, configurando [46-48](#)

Capacidad de respuesta visible para el usuario [52](#)

Capacidad de respuesta visible para el usuario, técnicas

- Escalado de Pub/Sub [53](#)

- Modelo de transacción [53](#)

V

Interfaces de voz [107](#)

Frontends de voz, tipos

- Texto a voz [107](#)

- Transcripción [107](#)

API de ingesta de votos [185](#)

API de ingesta de votos, repositorio

- Código basado en la nube [188](#), [189](#)

- Proyecto [191](#) de Firebase

- Configuración de GCP [190](#)

- Monitoreo de GCP [192](#)

- Cliente web/Código común [186](#), [187](#)

Procesador de votos [209](#)

Procesador de votos, términos

- Votación asincrónica [209](#)

- Escalado automático [218](#)

- Código de nube [210](#)

- Cloud Run, implementando [217](#)

- Latencia de frontend [224](#)

- Mensaje antiguo, monitoreo [224](#)

- Configuración de Pub/Sub [214](#)

- Recurso de servicio, monitoreo [219](#)

- Atraso de temas [220](#)

- Ingestor de votos [211](#)

- Código de procesador de votos [216](#)

Codelab [242](#) sobre actividad electoral

Codelab de actividad electoral, términos

Programación de copias de seguridad [256](#)

Servicio de ejecución en la nube [255](#)

Llave de nubes [244](#)

Adiciones al Código [252](#)

Deshabilitar facturación [246](#)

Habilitación de la facturación [243](#)

Interfaces de administración de GCP [243](#)

Recurso de llave inglesa [247](#)

Recursos de llave inglesa [245](#)

Procesador de votos [247](#)

Motor de política de votación [253](#)

Página [254](#) del cliente web

Solicitud de cliente votante [267](#)

Solicitud de Cliente Electoral, aspectos [268](#)

Solicitud de cliente votante, términos

Política del lado del cliente [271](#)

Rendimiento del cliente [270](#)

API de interfaz de votación [269](#)

Aplicación móvil para votantes [277](#)

Aplicación móvil para votantes, términos

BeeWare Móvil [277](#)

Reglas de política del lado del cliente [278](#)

Aplicación móvil [277](#)

Interfaz web para votantes [271](#)

Interfaz web para votantes, términos

Inicio/cierre de sesión del cliente [272](#)

Política del lado del cliente [275](#), [276](#)

Manejo de errores [273](#)

Accesibilidad del cliente web [274](#)

Idioma del cliente web [271](#)

Solicitud de votación [336](#)

Solicitud de Votación, análoga a [337](#), [338](#)

Aplicación de votación, arquitectura [337](#)

Aplicación de votación, optimizando [338](#), [339](#)

Solicitud de votación con GCP, que ilustra [339](#)

Sistema de votación, objetivos

Especificación de caja negra [327](#)

Patrón de diseño [327](#)

Ordenamiento de paralelos [326](#)

Aplicación independiente [325](#)

Voto no reconocido [324](#), [325](#)

Sistema de votación, modelos

Grupo Continente [156](#)

Solicitud de votación [156](#), [157](#)

Rendimiento del sistema de votación [153](#)

Rendimiento del sistema de votación, aspectos

Tráfico de distribución [155](#)

Tasa de solicitud inactiva [153](#)

Tasa máxima de solicitud [153](#)

Tasa de turba sostenida [155](#)

Pendiente máxima de tráfico [154](#)

Desencadenante/Pico de tráfico [154](#)

O

Aplicación de cliente web [101](#)